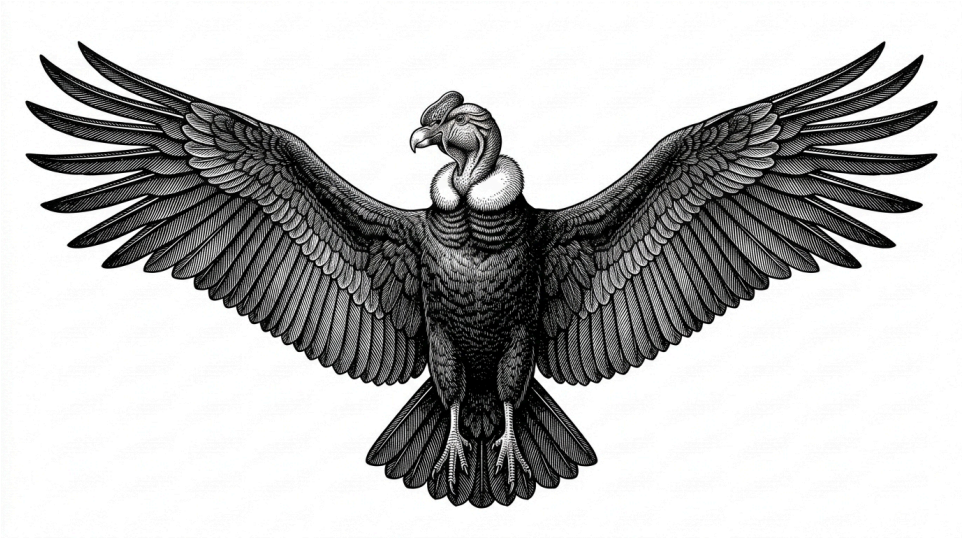
EDICIÓN NÚCLEO · VÍA RÁPIDA A EMPLEABILIDAD

DATA ENGINEER

Los 23 capítulos imprescindibles para conseguir empleo como Data Engineer Junior



Luis Galvis

2026

EDICIÓN NÚCLEO ESENCIAL

DATA ENGINEER

Guía condensada de alta densidad técnica para inserción laboral

Luis Galvis

2026

“A mis padres, por ser el cimiento inquebrantable, el sacrificio silencioso y la luz que impulsó cada uno de mis pasos. A mi pareja, por su amor incondicional, su infinita paciencia y su aliento constante en cada noche de estudio y creación. Y a todos los ingenieros y científicos que buscan comprender la esencia profunda de los datos más allá de las herramientas efímeras.”

— Luis Galvis

Cómo ser un Data Engineer — Edición Núcleo

Extracto de 23 Capítulos Esenciales para Empleo

© 2026 Luis Galvis. Todos los derechos reservados.

Ninguna parte de esta obra puede ser reproducida, almacenada en sistemas de recuperación ni transmitida, total o parcialmente, por ningún medio, sin la autorización previa y por escrito del autor.

Los fragmentos de código fuente (PySpark, SQL, Python, HCL) se distribuyen bajo la licencia de código abierto MIT y pueden reutilizarse libremente en entornos productivos, conservando el aviso de licencia.

Escrito y compilado en Ibagué, Tolima, Colombia · 2026

Índice de Capítulos Núcleo

Cap. 0	Data Engineering Roadmap y Panorama de la Profesión	6
Cap. 0A	Cimientos de Plataforma: Linux, Git, Docker y Redes	30
Cap. 0B	Bases de Datos Relacionales (OLTP) y Modelado Transaccional	43
Cap. 0C	Python para Ingeniería de Datos: Tipado, APIs y Resiliencia	54
Cap. 0D	Entorno de Laboratorio Reproducible y Generación de Datos	72
Cap. 1	Fundamentos de SQL Declarativo y Computación Analítica	82
Cap. 2	Agrupaciones Multidimensionales, Window Functions y Optimización	102
Cap. 2A	DuckDB y Cómputo In-Process: Cuándo No Usar Computación Distribuida	114
Cap. 3	De Hadoop y MapReduce al Cómputo en Memoria	124
Cap. 4	Arquitectura de Clúster: Driver, Workers y Ejecutores	129
Cap. 5	Entorno de Configuración y SparkSession	135
Cap. 7	Ingesta de Datos: Archivos, JDBC Batch y Fuentes Incrementales	140
Cap. 8	Formatos de Archivo: Parquet, ORC y Avro	148
Cap. 9	Tipos de Datos y Casting Riguroso en Memoria	156
Cap. 10	Filtrado y Expresiones Condicionales	162
Cap. 11	Transformaciones de Columnas y Operaciones con Esquemas	167
Cap. 12	Joins Distribuidos y Estrategias de Cruce	173
Cap. 17	Arquitectura Medallón y Modelado Dimensional	183
Cap. 18	Lakehouse y Formatos de Tabla Abiertos: Delta Lake y Apache Iceberg	197
Cap. 23	Orquestación de Flujos: Apache Airflow y Azure Data Factory	208
Cap. 24	Transformación Modular con dbt, Snapshots e Idempotencia	220
Cap. 34	Proyecto Integrador: Plataforma Lakehouse y Portafolio de Producción	231
Cap. 36	Apéndice B — El Manual Definitivo de Carrera, Empleo y Preparación de Entrevistas	248

Propósito de la Edición Núcleo

Este extracto contiene únicamente los 23 capítulos catalogados como **[NÚCLEO EMPLEABILIDAD]** del tratado completo de 50 capítulos. Representa la ruta crítica indispensable que exige el mercado laboral para superar pruebas de código en vivo y entrevistas de arquitectura de nivel Junior a Mid-Level.

Si dispone de tiempo acotado o busca validar rápidamente sus competencias mínimas viables antes de postularse, comience por esta selección.

CAPÍTULO 0

Data Engineering Roadmap y Panorama de la Profesión

> *"El rol del Data Engineer no es simplemente escribir consultas SQL; es diseñar sistemas de distribución robustos, resilientes e idempotentes capaces de mover y transformar petabytes con latencia predecible."*

0.1 Intuición Inicial: El Acueducto Invisible de los Datos

Imagina abrir el grifo de tu cocina y recibir agua potable limpia, a presión constante y sin interrupciones. No piensas en los kilómetros de tuberías subterráneas, las plantas de tratamiento químico, los tanques de decantación ni las bombas de presión que evitan que la red colapse.

En una empresa contemporánea, el analista de negocio y los tableros de Power BI son como ese grifo: esperan abrir una herramienta analítica y observar números limpios, coherentes y actualizados al segundo. Sin embargo, detrás de esa interfaz existe un torrente de datos caótico: bases de datos transaccionales saturadas, archivos JSON semiestructurados con formatos rotos, registros de clientes duplicados y millones de eventos por segundo llegando desde aplicaciones móviles.

El Ingeniero de Datos (Data Engineer) es el profesional que diseña, construye, monitorea y repara esa gigantesca red de acueductos digitales. Su misión no es contemplar el agua en el vaso, sino garantizar que la información fluya desde los sistemas de origen hacia los destinos analíticos de forma pura, oportuna y sin inundar los sistemas operativos. Si un pipeline falla a las 3:00 de la madrugada, es el Data Engineer quien recibe la alerta y subsana el problema antes de que los tomadores de decisiones inicien su jornada laboral.

> **Repositorio Oficial de Laboratorio en GitHub:**
> Todo el código fuente, los entornos de Docker Compose, los scripts de generación de datos y las soluciones completas a los ejercicios de esta obra se encuentran alojados y mantenidos en el repositorio oficial de código abierto:
> **https://github.com/luisgalvis/como-ser-un-data-engineer**
> *(Consulte el Capítulo 0D para las instrucciones detalladas de clonación y despliegue local).*

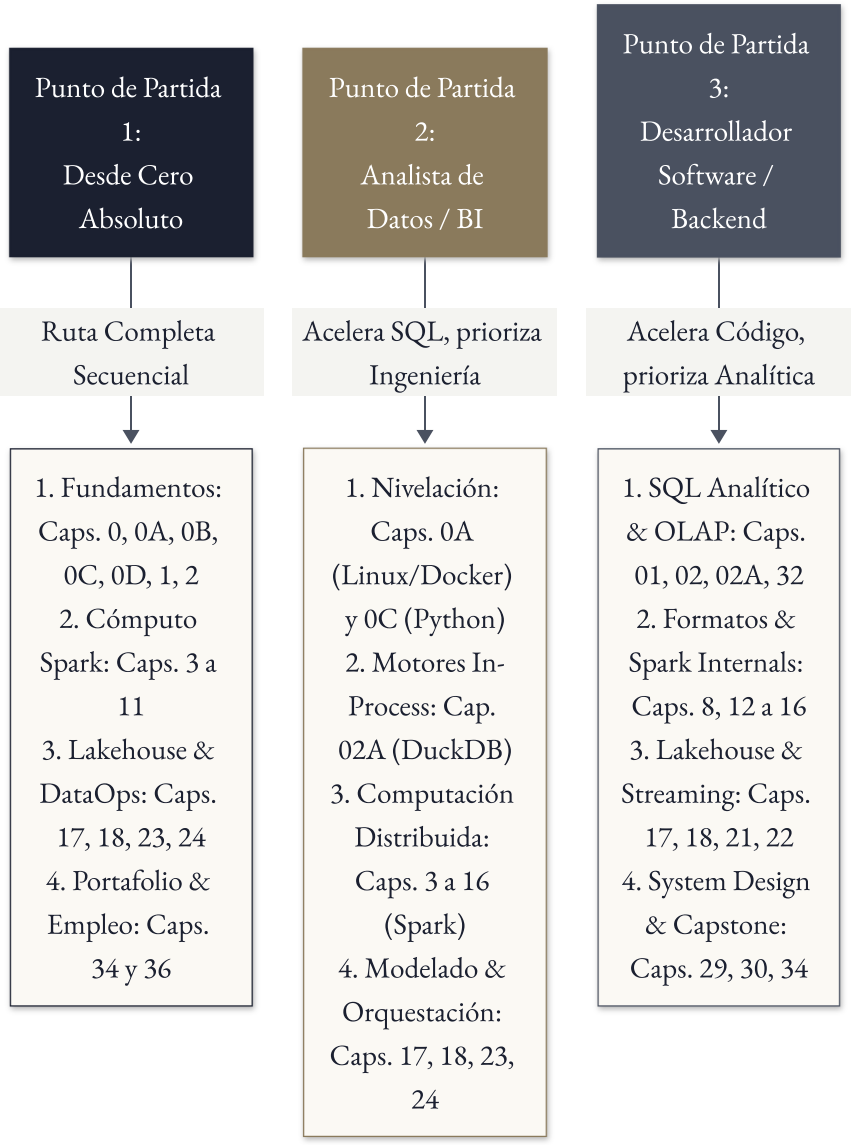
Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución y fronteras del rol de Data Engineer frente a analistas de datos, científicos de datos e ingenieros de machine learning.
- Identificar las tres rutas de lectura recomendadas según su punto de partida técnico (desde cero absoluto, analista en transición o programador backend).
- Distinguir qué capítulos constituyen el **Núcleo de Empleabilidad** para obtener un primer trabajo frente a los módulos recomendados y avanzados.
- Conocer la jerarquía física de memoria, los fundamentos web HTTP/APIs y la estadística operativa mínima necesaria para la disciplina.
- Trazar un plan de estudio deliberado de 9 a 12 meses con dedicación horaria honesta y una estrategia pragmática de certificaciones oficiales.

Rutas de Lectura según tu Punto de Partida

Para maximizar el aprovechamiento de esta obra, identifique su perfil inicial y siga la secuencia curricular adaptada a sus fortalezas previas:



Cuestionario de Auto-Diagnóstico Inicial: ¿Dónde me encuentro hoy?

Responda con honestidad a las siguientes 10 preguntas para confirmar cuál es su ruta de estudio óptima:

1. ¿Sabe clonar un repositorio con Git, crear ramas (*branches*) y resolver conflictos básicos en terminal? `[ ] Sí [ ] No`
2. ¿Puede escribir un archivo `docker-compose.yml` y levantar servicios aislados (como PostgreSQL o MinIO) en su máquina? `[ ] Sí [ ] No`

3. ¿Escribe funciones en Python con tipado estático (`typing`), manejo estructurado de excepciones y pruebas en `pytest`? `[ ] Sí [ ] No`
4. ¿Ha consumido APIs REST con librerías HTTP gestionando respuestas con paginación por cursor y códigos de estado (200, 429, 500)? `[ ] Sí [ ] No`
5. ¿Domina consultas SQL complejas empleando Expresiones de Tabla Comunes (CTEs) y funciones de ventana (`ROW_NUMBER`, `LEAD`, `LAG`)? `[ ] Sí [ ] No`
6. ¿Comprende la diferencia fundamental entre una base relacional normalizada en 3NF y un esquema analítico en estrella de Kimball? `[ ] Sí [ ] No`
7. ¿Ha ejecutado transformaciones sobre DataFrames de Apache Spark (PySpark) y analizado planes de ejecución en la interfaz Spark UI? `[ ] Sí [ ] No`
8. ¿Conoce los mecanismos de consistencia transaccional ACID y control de versiones (*Time Travel*) de Delta Lake o Apache Iceberg? `[ ] Sí [ ] No`
9. ¿Ha programado y orquestado flujos de trabajo (*DAGs*) productivos en Apache Airflow bajo el paradigma TaskFlow? `[ ] Sí [ ] No`
10. ¿Ha modelado canalizaciones de transformación en dbt aplicando pruebas automáticas de unicidad y no-nulos? `[ ] Sí [ ] No`

Interpretación de Resultados:

- **0 a 3 respuestas afirmativas: Ruta 1 (Cero Absoluto).** Su prioridad es avanzar de forma estrictamente lineal por los capítulos marcados como `[NÚCLEO EMPLEABILIDAD]`, sin saltarse los fundamentos de terminal, Linux, Python básico y SQL inicial.
- **4 a 6 respuestas afirmativas (con fortaleza en SQL/BI): Ruta 2 (Analista en Transición).** Puede avanzar rápido en SQL básico y centrar el 80% de su energía en Python modular, Docker, computación distribuida con Spark y orquestación con Airflow.
- **4 a 6 respuestas afirmativas (con fortaleza en código/DevOps): Ruta 3 (Desarrollador Backend en Transición).** Puede obviar la sintaxis básica de Python y terminal. Su foco crítico debe ser el SQL analítico avanzado, el modelado dimensional de Kimball, el tuning de memoria en Spark y la arquitectura Lakehouse.
- **7 o más respuestas afirmativas:** Diríjase de inmediato a las partes de optimización física avanzada (Partes V a X), al Proyecto Integrador Capstone y al Apéndice B para preparar entrevistas de nivel Mid-Level o Senior.

Clasificación de Capítulos por Nivel de Prioridad

A lo largo del libro, cada capítulo incluye una insignia editorial que delimita su alcance curricular:

- `[NÚCLEO EMPLEABILIDAD]`: Temas indispensables para superar entrevistas técnicas junior y desempeñarse en el primer trabajo (Linux, Git, Docker, Python, SQL, DuckDB, Spark básico,

Delta Lake, Airflow, dbt y los Proyectos de Portafolio).

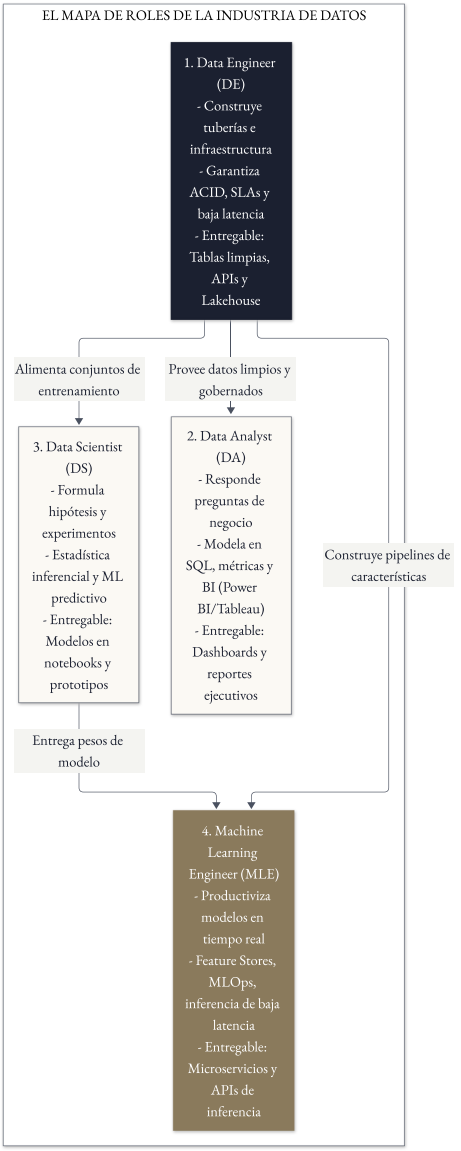
- `[RECOMENDADO]`: Herramientas que diferencian su perfil y aceleran la transición a puestos intermedios (Kafka, Terraform, Great Expectations, nubes públicas AWS/Azure/GCP, Databricks y Snowflake).
- `[AVANZADO]`: Conceptos de optimización física profunda, arquitectura distribuida y liderazgo técnico (Catalyst, Shuffling a fondo, Data Vault 2.0, Flink, MLOps, RAG y System Design senior).

Profundiza: El Fundamento Formal de los Sistemas de Datos

> **Definición 0.1 (Ingeniería de Datos como Disciplina de Sistemas Distribuidos)**
> La ingeniería de datos contemporánea es la rama de la ingeniería de software responsable del diseño, implementación, escalabilidad y observabilidad de plataformas para la recolección, transformación, almacenamiento y gobierno de activos de información a gran escala.
>
> Su objetivo fundamental es garantizar que los datos fluyan desde sistemas operacionales heterogéneos hacia capas analíticas con alta disponibilidad, latencia predecible, consistencia transaccional (semántica ACID) e integridad estructural verificable.

0.2 El Mapa de Roles de la Industria Analítica

Para evitar la habitual confusión en las ofertas de empleo, delimitamos formalmente las fronteras entre los cuatro roles principales del ecosistema:



- **Data Engineer (DE):** Arquitecto y constructor de las plataformas. Si el pipeline se detiene a la medianoche, si los datos contienen duplicados o si la consulta tarda 4 horas en vez de 3 minutos, es su responsabilidad directa.
- **Data Analyst (DA):** Traduce datos limpios a decisiones corporativas. Depende de que el DE le entregue dimensiones conformadas y tablas de hechos agregadas en la capa Gold.
- **Data Scientist (DS):** Descubre patrones probabilísticos y correlaciones para predecir comportamientos futuros.
- **Machine Learning Engineer (MLE):** Toma el modelo matemático validado por el científico y lo despliega como un microservicio escalable con monitoreo de deriva (*drift*).

0.3 Fundamentos Previos: Computadores, Redes y Estadística Mínima

Antes de manipular millones de registros en clústeres distribuidos, todo ingeniero debe comprender las restricciones físicas impuestas por el hardware y la red.

1. La Jerarquía de Memoria y la Física de la Latencia

En sistemas de datos, el tiempo requerido para acceder a un dato delimita el rendimiento máximo de cualquier algoritmo. A continuación se presentan las latencias de hardware normalizadas a escala humana (suponiendo que un ciclo de CPU equivalga a 1 segundo):

Nivel de Memoria / Operación	Latencia Física Real	Tiempo Equivalente Humano
Ciclo de reloj de CPU (Registros)	0.3 ns	1 segundo
Caché L1	0.9 ns	3 segundos
Caché L2	2.8 ns	9 segundos
Caché L3	12.9 ns	43 segundos
Memoria RAM Principal	100 ns	5.5 minutos
Lectura de Disco SSD NVMe (Bloque 4 KB)	10.000 ns (10 µs)	9.2 horas
Lectura de Disco Duro Magnético (HDD Seek)	10.000.000 ns (10 ms)	1.05 años
Transferencia en Red LAN (Mismo Datacenter)	500.000 ns (0.5 ms)	19.3 días
Llamada de Red WAN (Transcontinental)	60.000.000 ns (60 ms)	6.3 años

> **Lección Fundamental de Ingeniería:** Acceder a la memoria RAM es aproximadamente **100.000 veces más veloz** que buscar en un disco magnético, y transferir datos por la red es la operación más costosa de un clúster. Cuando Spark ejecuta un *Shuffle*, debe serializar datos en memoria, escribirlos a disco local y transmitirlos por la red a otros nodos. **Optimizar un pipeline consiste esencialmente en evitar transferencias de red y serializaciones innecesarias.**

2. Fundamentos de la Web: HTTP, REST y APIs

Gran parte de la ingesta de datos transaccionales se produce consumiendo interfaces de programación de aplicaciones (**APIs REST**) sobre el protocolo **HTTP/HTTPS**:

- **Métodos HTTP:** `GET` (lectura idempotente), `POST` (creación no idempotente), `PUT` (reemplazo idempotente), `PATCH` (modificación parcial).
- **Códigos de Estado Críticos en Pipelines:**
  - `200 OK` / `201 Created`: Solicitud exitosa.
  - `400 Bad Request` / `422 Unprocessable`: Datos con formato o contrato inválido (error en el payload).
  - `401 Unauthorized` / `403 Forbidden`: Problemas de token o permisos de acceso.
  - `429 Too Many Requests`: Se ha excedido la cuota de peticiones (*Rate Limit*). Requiere retroceso exponencial con variación aleatoria (*Exponential Backoff con Jitter*).
  - `500 Internal Error` / `502 Bad Gateway` / `503 Service Unavailable`: Falla transitoria del servidor de origen (permite reintentos automáticos).
- **Formato JSON:** El estándar dominante en eventos web. Formato semiestructurado con tipado débil que requiere validación estricta de esquemas antes de incorporarse al almacenamiento columnar.

3. Estadística Mínima para el Data Engineer

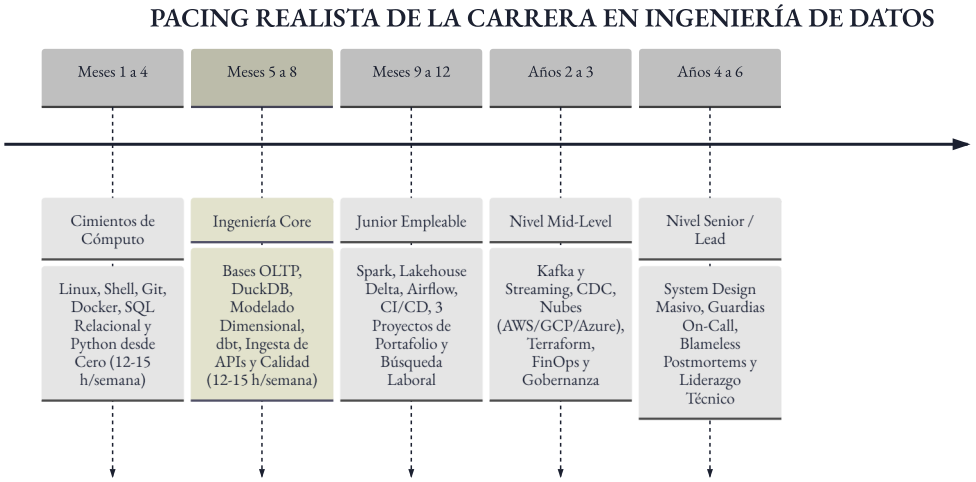
El Data Engineer no formula modelos econométricos, pero debe dominar tres conceptos estadísticos indispensables para no corromper datos de negocio ni diseñar alertas erróneas:

1. **La Trampa de la Media frente a la Mediana:**
   En distribuciones con asimetría severa (*skewness* o colas pesadas, como duración de consultas o volumen de compras), la media aritmética se ve fuertemente distorsionada por valores atípicos (*outliers*). En ingeniería de datos, **los acuerdos de nivel de servicio (SLAs) nunca se miden por el promedio, sino mediante percentiles**.
2. **Percentiles Operativos ($p50$, $p95$, $p99$):**
   - $p50$ (Mediana): El 50% de las ejecuciones concluyen por debajo de este umbral.
   - $p95$ y $p99$: El 95% o 99% de las ejecuciones terminan en este límite. El percentil 99 expone el peor escenario sufrido por los usuarios en momentos de saturación.
3. **Detección de Anomalías Volumétricas:**
   Para detectar caídas abruptas en la ingesta o duplicaciones accidentales, se monitorean la **Desviación Estándar (σ)** y el **Rango Intercuartílico ($IQR = Q_3 - Q_1$)**. Si el volumen diario de registros se desvía en más de 3σ respecto al promedio móvil mensual, el pipeline debe emitir una alerta preventiva antes de consolidar la capa Silver.

0.4 Pacing Profesional Realista: De Cero a Empleabilidad

Prometer que una persona sin formación técnica previa alcanzará el nivel de "Senior Data Engineer" en 12 meses es una afirmación irreal. La ingeniería de sistemas distribuidos exige asimilar abstracciones complejas, depurar fallos en memoria y acumular experiencia ante incidentes reales.

El camino profesional honesto y probado en la industria se estructura en tres hitos:



Plan de Estudio Mensual con Entregables Semanales (Hito 1: Junior Empleable)

Dedicación sugerida: **12 a 15 horas semanales** de estudio deliberado y práctica en laboratorio. Cada mes concluye con entregables verificables en código:

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 1	**Cimientos de Entorno y Linux** Cap. 00A (Linux/Docker/Git)	• **Semana 1:** Terminal Linux configurada con `zsh`/`bash`, scripts shell con pipes (`grep`, `awk`, `sed`) y control de versiones con Git (ramas, commits convencionales). • **Semana 2:** Contenerización con Docker: Dockerfile multi-stage, volumes y redes bridge. • **Semana 3:** Orquestación multicontenedor con `docker compose` levantando Postgres y MinIO con variables de entorno en `.env`. • **Semana 4:** Entregable Mes 1: Repositorio con un pipeline en Shell/Docker que descarga un dataset público, valida checksum SHA-256 y lo carga en PostgreSQL.
Mes 2	**Bases Relacionales y Python Moderno** Cap. 00B (OLTP) y 00C (Python Cero)	• **Semana 5:** Modelado relacional normalizado (3NF), índices B-Tree, constraints (`CHECK`, `FK`) y análisis de `EXPLAIN ANALYZE` en PostgreSQL. • **Semana 6:** Python moderno: entorno virtual aislado con `uv`, tipado estático estricto (`mypy`), estructuras inmutables y decoradores. • **Semana 7:** Manejo de datos estructurados: parseo robusto de JSON con Pydantic v2, lectura de CSVs y llamadas HTTP con `httpx` implementando backoff exponencial. • **Semana 8:** Entregable Mes 2: Paquete Python probado con `pytest` al 90% de cobertura que extrae datos de una API pública con paginación y valida esquemas con Pydantic.
Mes 3	**SQL Analítico Avanzado y DuckDB** Cap. 01, 02 y 02A (DuckDB)	• **Semana 9:** SQL declarativo: agregaciones avanzadas (`GROUPING SETS`, `ROLLUP`, `CUBE`) y expresiones de tabla comunes (`WITH RECURSIVE`). • **Semana 10:** Funciones de ventana analíticas: `ROW_NUMBER()` para deduplicación, `RANK()`, `DENSE_RANK()`, `LEAD()`, `LAG()` y marcos móviles (`ROWS BETWEEN`). • **Semana 11:** Motor in-process DuckDB: lectura directa de Parquet, inferencia de esquemas, exportación columnar y persistencia de catálogo local. • **Semana 12:** Entregable Mes 3: Script analítico en DuckDB + SQL que procesa 5 millones de registros de e-commerce en menos de 2 segundos generando un reporte de cohortes de retención.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 4	**Estructuras de Datos y APIs para DE** Cap. 02B (Estructuras de Datos)	• **Semana 13:** Algoritmos clave en DE: hashing, sets y diccionarios para uniones $O(N)$ en memoria sin bibliotecas pesadas. • **Semana 14:** Generadores y streams en Python: procesamiento perezoso (*lazy evaluation*) con `yield` para procesar archivos de 10 GB en máquinas con 2 GB de RAM. • **Semana 15:** Concurrencia controlada con `asyncio` y `ThreadPoolExecutor` para descargas paralelas masivas respetando límites de tasa (*rate limits*). • **Semana 16:** Entregable Mes 4: Extractor asíncrono con pool de hilos que descarga 50.000 registros mediante lotes de 100 sin desbordar memoria ni recibir errores 429.
Mes 5	**Fundamentos de Spark y Arquitectura** Cap. 03 a 06 (Arquitectura Clúster)	• **Semana 17:** Arquitectura distribuida: Driver, Cluster Manager, Executors, memoria JVM (Storage vs. Execution memory) y ciclo de vida de jobs. • **Semana 18:** RDDs y evaluación perezosa: transformaciones estrechas vs. anchas (*Narrow vs. Wide dependencies*) y costo físico del Shuffle. • **Semana 19:** Entorno PySpark local con Docker Compose: configuración de master/workers e interacción con PySpark CLI. • **Semana 20:** Entregable Mes 5: Benchmark comparativo de rendimiento entre un job con shuffle masivo (`groupByKey`) frente a una reducción local (`reduceByKey`).
Mes 6	**PySpark DataFrames y Formatos Columnar** Cap. 07 a 12 (Ingesta y Joins)	• **Semana 21:** Formato Apache Parquet: compresión Snappy/Zstandard, estadísticas de columnas (*min/max pushdown*) y particionamiento físico. • **Semana 22:** DataFrames API: transformaciones optimizadas por Catalyst Optimizer y Tungsten Engine. • **Semana 23:** Joins distribuidos: Shuffle Hash Join, Sort Merge Join y optimización con Broadcast Join (`broadcast()`). • **Semana 24:** Entregable Mes 6: Pipeline en PySpark que ingesta logs crudos, aplica broadcast join con una tabla dimensional y persiste en Parquet particionado por fecha.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 7	**Lakehouse, Delta Lake y Modelado** Cap. 17 (Medallón) y 18 (Delta Lake)	• **Semana 25:** Arquitectura Medallón: definición formal de capas Bronze (crudo), Silver (conformado/limpio) y Gold (agregado de negocio). • **Semana 26:** Modelado dimensional Kimball: tablas de hechos (*Facts*), dimensiones conformadas, claves subrogadas y dimensiones que cambian lentamente (SCD Tipo 1 y 2). • **Semana 27:** Delta Lake transaccional: registro `_delta_log`, ACID con `MERGE INTO`, Time Travel y optimización física con `OPTIMIZE` y `Z-ORDER`. • **Semana 28:** Entregable Mes 7: Pipeline Lakehouse completo Bronze → Silver → Gold implementando SCD Tipo 2 idempotente con `MERGE INTO` en Delta Lake.
Mes 8	**Orquestación con Airflow y Modelado con dbt** Cap. 23, 23A (Airflow) y 24 (dbt)	• **Semana 29:** Apache Airflow: arquitectura (Scheduler, Webserver, Triggerer, Workers), operadores básicos y diseño de DAGs idempotentes. • **Semana 30:** Manejo avanzado de dependencias: `TaskGroups`, reintentos automáticos, SLAs, conexiones seguras y variables en Airflow. • **Semana 31:** dbt Core: estructura de proyecto, modelos staging, intermediate y marts; documentación interactiva y linaje automático. • **Semana 32:** Entregable Mes 8: DAG de Airflow orquestando un proyecto dbt que ejecuta tests de unicidad y no-nulidad antes de actualizar los marts de analítica.
Mes 9	**DataOps, Calidad y Contratos** Cap. 26, 26A (CI/CD) y 28 (GX/Contratos)	• **Semana 33:** Contratos de datos (*Data Contracts*): definición de especificaciones semánticas y cuarentena de registros inválidos (*Dead-Letter Queue*). • **Semana 34:** Validación automatizada con Great Expectations / Soda / Pydantic Contracts: assertions de negocio y generación de reportes de calidad. • **Semana 35:** CI/CD para datos: GitHub Actions ejecutando linters (`ruff`, `sqlfluff`) y pruebas unitarias con Testcontainers en cada Pull Request. • **Semana 36:** Entregable Mes 9: Pipeline con GitHub Action que bloquea el merge si los esquemas de datos entrantes violan el contrato establecido.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 10	**Infraestructura Cloud Básica e IaC** Cap. 13 (AWS), 25 (Terraform)	• **Semana 37:** Fundamentos de AWS para datos: S3, IAM (principio de menor privilegio), Glue Data Catalog y Athena. • **Semana 38:** Infraestructura como Código: aprovisionamiento de buckets S3, roles IAM y bases de datos con Terraform modular. • **Semana 39:** Gestión de costos (FinOps): alertas de presupuesto de $5 USD en AWS, ciclos de vida de objetos en S3 y destrucción con `terraform destroy`. • **Semana 40:** Entregable Mes 10: Módulo Terraform reutilizable que levanta un Data Lakehouse en S3 con políticas IAM estrictas y alerta de costo configurada.
Mes 11	**Construcción de los 3 Proyectos de Portafolio** Cap. 34 (Proyectos Portafolio)	• **Semana 41:** Proyecto 1 (Local Zero-Cost): DuckDB + dbt + Airflow + Streamlit documentado de punta a punta. • **Semana 42:** Proyecto 2 (API Ingestion & Lakehouse): Extractor API con backoff → S3/MinIO → PySpark → Delta Lake con cuarentena de datos. • **Semana 43:** Proyecto 3 (Streaming o Cloud): Kafka básico o pipeline serverless en AWS/Databricks con métricas de rendimiento y costos. • **Semana 44:** Entregable Mes 11: Tres repositorios públicos en GitHub con README profesional, diagramas Mermaid, instrucciones `docker compose up` y videos demo.
Mes 12	**Tramo de Empleo y Procesos de Selección** Cap. 36 (Estrategia Laboral)	• **Semana 45:** Optimización de CV en formato X-Y-Z bilingüe (español/inglés) adaptado a sistemas ATS, perfil de LinkedIn y presencia en GitHub. • **Semana 46:** Práctica intensiva de Live-Coding: 50 problemas de SQL analítico (LeetCode/StrataScratch) y 30 ejercicios de algoritmos en Python. • **Semana 47:** Simulacros de entrevista de System Design de datos y resolución de pruebas para llevar a casa (*Take-Home Assignments*). • **Semana 48:** Entregable Mes 12: Estrategia de postulación activa (10 postulaciones semanales con mensajes personalizados y búsqueda de referidos internos).

Listas de Verificación: «Ya Puedo Hacer Esto» por Hito

Utilice estas listas de control como su estándar de autoevaluación. No avance al siguiente bloque formativo hasta haber marcado afirmativamente cada competencia mediante evidencia en su propio código:

Hito 1: Cimientos y Junior Empleable (Checklist de 12 Competencias)

- ☐ **1. Control de Entorno:** Sé clonar repositorios, resolver conflictos de ramas en Git y estructurar proyectos en Python usando `uv` y entornos virtuales limpios.
- ☐ **2. Contenedores:** Sé escribir un `Dockerfile` optimizado y un `docker-compose.yml` para levantar PostgreSQL, MinIO y servicios locales con variables de entorno protegidas en `.env`.
- ☐ **3. Ingesta Robusta:** Sé construir un script en Python que consuma una API REST con paginación, maneje códigos HTTP 429 con retroceso exponencial (*exponential backoff*) y valide tipos con Pydantic.
- ☐ **4. SQL Analítico:** Sé escribir consultas usando funciones de ventana (`ROW_NUMBER()`, `LEAD()`, `LAG()`, `DENSE_RANK()`) para deduplicar registros y calcular métricas acumuladas.
- ☐ **5. Motores Modernos:** Sé utilizar DuckDB para consultar archivos Parquet directamente desde disco o S3 sin requerir una base de datos externa.
- ☐ **6. Computación Distribuida:** Comprendo la diferencia entre transformaciones estrechas y anchas en Apache Spark, y sé explicar cómo y cuándo ocurre un *Shuffle*.
- ☐ **7. PySpark en Práctica:** Sé cargar datos en PySpark, aplicar esquemas estrictos (`StructType`), realizar transformaciones limpias y ejecutar optimizaciones mediante *Broadcast Joins*.
- ☐ **8. Almacenamiento Lakehouse:** Sé operar tablas Delta Lake, ejecutar operaciones `MERGE INTO` para actualización idempotente (upserts) y consultar versiones históricas con Time Travel.
- ☐ **9. Modelado Dimensional:** Sé diseñar un esquema en estrella (*Star Schema*) con tablas de hechos transaccionales y dimensiones con manejo de SCD Tipo 2.
- ☐ **10. Orquestación:** Sé crear un DAG en Apache Airflow con dependencias claras, reintentos configurados y tareas completamente idempotentes.
- ☐ **11. Calidad y Contratos:** Sé definir reglas de calidad que aíslen registros con errores en una cola de cuarentena (*Dead-Letter Queue*) sin detener el pipeline.
- ☐ **12. Portafolio Demostrable:** Cuento con un repositorio público en GitHub con CI/CD automatizado, tests con `pytest` y documentación clara que cualquier reclutador puede clonar y ejecutar.

Hito 2: Especialización Mid-Level (Checklist de 10 Competencias)

- ☐ **1. Optimización Spark Avanzada:** Sé diagnosticar problemas de sesgo de datos (*Data Skew*) mediante el Spark UI y resolverlos aplicando técnicas de *Salting* o *Liquid Clustering*.
- ☐ **2. Event Streaming:** Sé configurar un productor y consumidor en Apache Kafka, diseñar esquemas con Schema Registry y explicar la semántica *At-Least-Once* vs. *Exactly-Once*.

- ☐ **3. Spark Streaming:** Sé procesar flujos continuos en PySpark Structured Streaming aplicando *Watermarking* para gestionar datos desordenados o tardíos.
- ☐ **4. Infraestructura como Código:** Sé escribir plantillas de Terraform modulares para aprovisionar buckets, políticas IAM de menor privilegio y clústeres analíticos.
- ☐ **5. CI/CD Empresarial:** Sé construir pipelines en GitHub Actions que ejecuten pruebas de integración usando Testcontainers y validen calidad de código con linters antes del despliegue.
- ☐ **6. Modelado con dbt:** Sé estructurar proyectos dbt con Jinja macros, paquetes reutilizables, snapshots para SCD2 y pruebas genéricas/singulares.
- ☐ **7. Nube Pública:** Domino los servicios esenciales de al menos una nube líder (AWS S3/Glue/Athena o Azure Data Lake/Data Factory o GCP GCS/BigQuery).
- ☐ **8. FinOps y Costos:** Sé implementar políticas de retención, particionado eficiente y monitoreo de costos para garantizar la sostenibilidad financiera de la plataforma.
- ☐ **9. Contratos de Datos:** Sé acordar e implementar data contracts semánticos entre equipos de software transaccional y el equipo de datos.
- ☐ **10. Resolución de Incidentes:** Sé interpretar trazas de error complejas (OOM, Garbage Collection pauses, Deadlocks) y redactar análisis post-incidente (*postmortems*).

Hito 3: Nivel Senior y Liderazgo Técnico (Checklist de 8 Competencias)

- ☐ **1. System Design Masivo:** Sé liderar entrevistas de arquitectura de datos diseñando sistemas capaces de procesar decenas de terabytes diarios bajo estrictos SLAs de latencia.
- ☐ **2. Trade-offs de Arquitectura:** Sé justificar técnicamente la elección entre Lambda vs. Kappa, Delta vs. Iceberg, y Batch vs. Micro-batch según costos y requisitos del negocio.
- ☐ **3. Gobierno y Seguridad:** Sé diseñar políticas de cifrado en reposo y en tránsito (mTLS, KMS), control de acceso basado en roles (RBAC/ABAC) y cumplimiento normativo (GDPR / Ley 1581).
- ☐ **4. Observabilidad de Plataforma:** Sé instrumentar pipelines con OpenLineage, métricas Prometheus y trazas distribuidas para monitorear la salud de extremo a extremo.
- ☐ **5. Gestión de Guardias On-Call:** Sé diseñar esquemas de rotación on-call, manuales de respuesta rápida (*runbooks*) y matrices de escalamiento de severidad.
- ☐ **6. Datos para Inteligencia Artificial:** Sé diseñar canalizaciones para ingestión masiva de embeddings, integración con bases vectoriales y feature stores para inferencia en tiempo real.
- ☐ **7. Mentoría y Cultura de Ingeniería:** Sé guiar a ingenieros junior y mid-level, realizar revisiones de código de alto estándar y promover la cultura de *blameless postmortems*.
- ☐ **8. Alineación de Negocio:** Sé traducir necesidades estratégicas de directores ejecutivos en hojas de ruta técnicas viables y cuantificables en retorno de inversión (ROI).

Rúbrica de Autoevaluación de Proyectos de Portafolio

Todo proyecto incluido en su portafolio de GitHub debe evaluarse con la siguiente matriz. Para considerar un proyecto listo para postulación laboral, **debe alcanzar un puntaje mínimo de Nivel Producción (4 o 5 puntos) en las cinco dimensiones**:

Dimensión Evaluada	Insuficiente (1–2 Puntos)	Aceptable (3 Puntos)	Nivel Producción Senior (4–5 Puntos)
1. Idempotencia y Reejecución	El pipeline falla si se corre dos veces seguidas o duplica filas en la tabla de destino (`INSERT` ciego).	El pipeline usa `TRUNCATE` y recarga todo, o requiere borrar archivos manualmente antes de volver a correr.	El pipeline usa `MERGE INTO`, particionamiento determinista o reemplazo dinámico. Se puede reejecutar N veces con idéntico resultado sin duplicados.
2. Pruebas Automatizadas y Calidad	No hay pruebas unitarias ni validación de esquemas. El código se prueba corriendo el script completo a mano.	Tiene 2 o 3 pruebas con `pytest` para funciones básicas, pero no valida el esquema ni la calidad de los datos de entrada.	Suite completa con `pytest` (>85% cobertura), tipado estático (`mypy`), validación con Pydantic/Great Expectations y aislamiento a cuarentena (*Dead-Letter Queue*).
3. Infraestructura y Reproducibilidad	«Funciona en mi máquina». Requiere instalar múltiples herramientas en el host sin versiones fijas.	Incluye un `Dockerfile` básico, pero faltan servicios auxiliares o requiere configurar credenciales manuales.	Reproducible en un solo comando: `docker compose up -d`. Usa `.env.example`, versiones fijadas, dependencias bloqueadas con `uv.lock` y scripts de generación de datos.
4. Documentación y Arquitectura (README)	README vacío o generado por plantilla. Solo contiene el título del proyecto y dos líneas de texto.	Describe qué hace el código y lista las tecnologías usadas, pero carece de diagramas de arquitectura y métricas.	Diagrama de flujo en Mermaid, justificación técnica de decisiones de diseño, instrucciones paso a paso probadas y métricas cuantificables de rendimiento y costos.
5. Manejo de Errores y Observabilidad	El script aborta inmediatamente con excepciones no controladas; no hay logs ni alertas.	Usa bloques genéricos `try/except: pass` y mensajes `print()` simples por consola.	Logging estructurado (`loguru`/`logging` estándar con formato JSON), captura de errores con retries automáticos, alertas de fallo y aislamiento de registros corruptos.

0.5 Mapa Pragmático de Certificaciones Oficiales

Las certificaciones no sustituyen a los proyectos demostrables en código, pero actúan como un filtro clave en plataformas de reclutamiento automatizado (ATS) para validar el conocimiento formal de un estándar.

> **Estrategia Recomendada para Principiantes:**
> **Concéntrese en un máximo de UNA certificación oficial para obtener su primer empleo.** Intentar aprobar tres o cuatro certificaciones simultáneas mientras se parte de cero dispersa el esfuerzo y genera un costo económico innecesario. Obtenga una credencial clave y reserve las certificaciones especializadas para cuando su empleador financie los exámenes.

Certificación Oficial	Proveedor	Costo Oficial	Nivel Sugerido	Enfoque Principal
AWS Certified Data Engineer – Associate (DEA-C01)	AWS	$150 USD	Junior - Mid	*(Recomendada para primer empleo).* Ingesta, S3, Glue, EMR, Athena, Redshift y seguridad KMS/IAM.
Databricks Certified Data Engineer Associate	Databricks	$200 USD	Junior - Mid	*(Recomendada para primer empleo).* Apache Spark, PySpark, Delta Lake, Lakehouse y Lakeflow básico.
dbt Certified Developer	dbt Labs	$200 USD	Junior - Mid	Modelado dimensional Kimball, Jinja, macros, tests genéricos y snapshots.
Microsoft Fabric Analytics Engineer Associate (DP-600)	Microsoft	$165 USD	Junior - Mid	OneLake, modelos semánticos de Power BI, Delta Lake y T-SQL analítico.
Microsoft Fabric Data Engineer Associate (DP-700)	Microsoft	$165 USD	Junior - Mid	Ingesta Fabric, Data Factory, Lakehouse Spark y orquestación empresarial. *(Sustituye al retirado DP-203).*
SnowPro Core Certification	Snowflake	$175 USD	Junior - Mid	Almacenamiento columnar en micro-particiones, virtual warehouses, Time Travel y Roles.
HashiCorp Certified: Terraform Associate (003)	HashiCorp	$70.50 USD	Mid	Infraestructura como Código (IaC), State Management, módulos y proveedores cloud.
Databricks Certified Data Engineer Professional	Databricks	$200 USD	Mid - Senior	Optimización física de Spark UI, Unity Catalog empresarial, streaming y CI/CD avanzado.
Google Cloud Professional Data Engineer (GCP PDE)	Google Cloud	$200 USD	Mid - Senior	BigQuery a escala masiva, Apache Beam en Cloud Dataflow y Cloud Composer.

0.6 Construcción del Repositorio de Portafolio en GitHub

Para captar el interés de comités técnicos y líderes de ingeniería, el portafolio en GitHub debe alejarse radicalmente de notebooks aislados de Kaggle o scripts lineales sin pruebas. Debe reflejar la estructura de un repositorio de producción corporativo:

```
repositorio-lakehouse-produccion/
├── .github/
```

```
|   └── workflows/
|       └── ci_cd.yml                # Pipeline CI/CD: linter, tests y validación
IaC
├── infrastructure/
|   └── terraform/                    # Infraestructura como código (S3, IAM,
Clústeres)
|       ├── main.tf
|       └── variables.tf
├── src/
|   ├── ingestion/                    # Clientes API con backoff o productores
Kafka
|   |   ├── api_client.py
|   |   └── schemas.py
|   ├── transformations/              # Pipelines modulares en PySpark o DuckDB
|   |   ├── bronze_to_silver.py
|   |   └── silver_to_gold.py
|   └── quality/                      # Suites de validación y aislamiento de
cuarentena
|       └── data_contracts.py
├── dbt_project/                      # Modelado dimensional de capa Gold
|   ├── models/
|   └── dbt_project.yml
├── orchestrator/                     # DAGs de Apache Airflow
|   └── dags/
|       └── lakehouse_daily_dag.py
├── tests/                            # Pruebas unitarias e integración con pytest
|   ├── test_transformations.py
|   └── test_quality_rules.py
├── scripts/
|   └── data_generator.py             # Generador de datos sintéticos reproducible
├── docker-compose.yml                # Entorno local reproducible (Postgres,
MinIO, Spark)
├── pyproject.toml                    # Gestión moderna de dependencias con uv
└── README.md                         # Documentación técnica ejecutiva y
arquitectura
```

Elementos Críticos del `README.md` de Alto Impacto

1. **Diagrama de Arquitectura Claro:** Diagramas en Mermaid o esquemas gráficos que ilustren el flujo de datos desde el origen operacional hasta el consumo analítico.
2. **Justificación Técnica de Decisiones:** Explicar con rigor *por qué* se eligió Delta Lake frente a Parquet plano, o por qué se empleó una ventana móvil frente a un re-cálculo total.
3. **Métricas Tangibles de Rendimiento y Costo:** Demostrar el impacto de la optimización (ej. *«Se redujo el volumen de shuffle en un 65% mediante broadcast joins y salting, recortando el tiempo de ejecución de 40 a 14 minutos»*).
4. **Reproducibilidad en un Solo Comando:** Cualquier ingeniero debe poder clonar el repositorio, ejecutar `docker compose up -d` y reproducir los pipelines localmente sin configuraciones manuales complejas.

0.7 Estrategia de Preparación para Entrevistas Técnicas

Los procesos de selección en empresas internacionales se articulan habitualmente en cuatro fases técnicas diferenciadas:

1. Prueba de SQL en Vivo (Live-Coding)

- **Temas evaluados:** Funciones de ventana avanzadas (`ROW_NUMBER()` para deduplicación, `DENSE_RANK()`, `LEAD/LAG`), agregaciones condicionales, uniones relacionales complejas y expresiones de tabla comunes (CTEs) legibles.
- **Criterio de evaluación:** Corrección del resultado, manejo impecable de valores `NULL`, legibilidad y consideración del costo computacional del plan de ejecución.

2. Algoritmos y Manipulación de Datos en Python

- **Temas evaluados:** Procesamiento en memoria mediante generadores (`yield`), manejo de estructuras JSON anidadas, operaciones en $O(N)$ o $O(N \log N)$ y agrupaciones sin bibliotecas externas.
- **Criterio de evaluación:** Selección eficiente de estructuras de datos (diccionarios y conjuntos para búsquedas en tiempo constante $O(1)$), tipado estático y cobertura de casos límite (*edge cases*).

3. Computación Distribuida y Fundamentos de Arquitectura

- **Preguntas clásicas:** *«¿Qué etapas componen exactamente una fase de Shuffle en Spark?»*, *«¿Cómo diagnosticarías y resolverías un fallo OutOfMemory por sesgo de datos (Data Skew)?»*, *«¿Cómo se garantiza la semántica exactly-once al persistir eventos de Kafka en Delta Lake?»*.
- **Criterio de evaluación:** Claridad conceptual sobre la relación entre memoria RAM, discos locales, red y recolectores de basura en la JVM.

4. Entrevista de System Design de Datos (Nivel Senior)

- Ejercicio de 45 a 60 minutos donde el candidato lidera el diseño de una plataforma completa a escala (ej. *«Diseñe la plataforma analítica de telemetría para 500 millones de dispositivos IoT»*).
- **Estructura para responder:**
  1. *Aclaración de Requisitos y Alcance (5 min):* Delimitar requerimientos funcionales y no funcionales (latencia, frescura, throughput).
  2. *Estimación Numérica de Escala (5 min):* Calcular volumen de almacenamiento diario/anual, ancho de banda de red y memoria RAM.

3. *Diseño de Alto Nivel (15 min):* Trazar el diagrama modular (Ingesta → Mensajería → Procesamiento → Almacén → Consumo).
4. *Inmersión Profunda en Componentes Críticos (15 min):* Justificar la selección de tecnologías, estrategias de particionado y políticas de tolerancia a fallos.
5. *Manejo de Errores y Cuellos de Botella (5 min):* Diseñar recuperación ante caídas de nodos y monitoreo de salud del sistema.

0.8 El Inglés Técnico como Multiplicador Profesional

En el mercado global de datos, el dominio del **inglés técnico profesional** es el principal multiplicador salarial y el puente de acceso a posiciones remotas internacionales pagadas en dólares estadounidenses:

- **Acceso a la Verdad Técnica:** La documentación de proyectos de la Apache Software Foundation, especificaciones RFCs, discusiones de arquitectura en GitHub y literatura fundacional se publican exclusivamente en inglés.
- **Vocabulario Operativo Estandarizado:** Los debates cotidianos de producción giran en torno a términos en inglés (*shuffling, stragglers, backpressure, idempotency, data drift, watermarking, tombstoning, dead-letter queues*). Dominar esta terminología con soltura y precisión es un indicador inmediato de madurez profesional.

0.9 Resumen y Puntos Clave

- **De consumidores a constructores:** El Data Engineer no consume datos para armar gráficos ejecutivos; construye las plataformas escalables, idempotentes y observables sobre las cuales opera la corporación.
- **Rutas personalizadas:** El aprendizaje debe adaptarse al punto de partida técnico (desde cero absoluto, analista en transición o desarrollador backend) priorizando los capítulos clasificados como `[NÚCLEO EMPLEABILIDAD]`.
- **Cimientos físicos:** La física del hardware impone que la memoria RAM sea 100.000 veces más rápida que el disco magnético y que la transferencia por red sea el factor más costoso. Todo diseño distribuido debe minimizar transferencias innecesarias.
- **Pacing honesto:** La transición desde cero hasta la empleabilidad junior requiere entre 9 y 12 meses de estudio deliberado (12–15 horas/semana); el nivel senior consolidado exige entre 4 y 6 años de experiencia productiva real.

- **Estrategia pragmática:** Enfóquese en dominar los cimientos y obtener un máximo de una certificación inicial clave respaldada por un portafolio reproducible en GitHub con Docker Compose, CI/CD y pruebas unitarias.

Ejercicios de Revisión

Ejercicio 0.1: Estimación de Transferencia y Física de Latencia

Suponga que una consulta en Apache Spark sobre un clúster sin optimizar provoca un *Shuffle* no intencionado que redistribuye 120 GB de datos intermedios a través de una red local con un ancho de banda efectivo disponible de 1 Gbps (gigabit por segundo).

1. Calcule el tiempo teórico mínimo que consumirá únicamente la transferencia de red de dicho Shuffle (sin contar serialización ni escritura a disco).
2. Compare dicho tiempo con el acceso a 120 GB si los datos ya estuvieran en memoria RAM local con un ancho de banda de bus de 40 GB/s.

Solución Paso a Paso:

1. **Conversión de unidades y tiempo de red:**
   $120\ \text{GB} = 120 \times 8\ \text{gigabits} = 960\ \text{Gigabits}$.
   A un ancho de banda de 1 Gbps:

$$\text{Tiempo de red} = \frac{960\ \text{Gb}}{1\ \text{Gbps}} = 960\ \text{segundos} = 16\ \text{minutos}$$

2. **Tiempo de acceso en bus de memoria RAM:**

$$\text{Tiempo en RAM} = \frac{120\ \text{GB}}{40\ \text{GB/s}} = 3\ \text{segundos}$$

3. **Conclusión de ingeniería:** La transferencia de red tarda 960 segundos frente a 3 segundos en bus local (un factor de más de $300\times$ más lento). Esta disparidad ilustra por qué un broadcast join o un particionamiento adecuado transforma consultas de 20 minutos en procesos de pocos segundos.

Lecturas Recomendadas

1. **Reis, J. & Housley, M. (2022).** *Fundamentals of Data Engineering: Plan and Build Robust Data Systems.* O'Reilly Media.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems.* O'Reilly Media.
3. **AWS Training and Certification (2024).** *AWS Certified Data Engineer – Associate (DEA-C01) Exam Guide.* Amazon Web Services.
4. **Databricks Learning (2024).** *Databricks Certified Data Engineer Associate Exam Study Guide.* Databricks Inc.

CAPÍTULO 0A

Cimientos de Plataforma: Linux, Git, Docker y Redes NÚCLEO

> *"Un ingeniero de datos que no domina la terminal de Linux, el aislamiento de contenedores en Docker y el control de versiones con Git es como un cirujano que desconoce la anatomía básica: puede saber qué operación desea realizar, pero carece de la destreza instrumental para manipular el sistema sin provocar un desastre en producción."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Operar con fluidez en entornos **Linux** mediante la línea de comandos, dominando tuberías, redirecciones de flujo y herramientas de procesamiento de texto (`grep`, `sed`, `awk`, `jq`).
- Automatizar tareas periódicas en el sistema operativo mediante el demonio `cron` y gestionar procesos y memoria (`ps`, `top`, `kill`, `systemd`).
- Implementar flujos de trabajo profesionales con **Git** (estrategias de ramificación, Conventional Commits, `rebase` interactivo frente a `merge` y resolución limpia de conflictos).
- Diseñar contenedores reproducibles con **Docker**, optimizando imágenes mediante construcciones multi-etapa (*multi-stage builds*), volúmenes persistentes y redes aisladas.
- Orquestar entornos locales de ingeniería de datos compuestos por múltiples servicios mediante **Docker Compose**.
- Comprender los conceptos esenciales de **redes**: direccionamiento CIDR, subredes, puertos, resolución DNS, cortafuegos (*Security Groups*), VPCs y cifrado TLS/mTLS en tránsito.

> **Definición 0A.1 (La Pila de Ejecución del Data Engineer)** En la arquitectura de sistemas distribuidos, la infraestructura no es un detalle abstracto delegado exclusivamente al equipo de operaciones. Todo pipeline de datos —sea un clúster de Spark sobre Kubernetes, un broker de Kafka o un contenedor de Airflow— se ejecuta sobre núcleos **Linux**, se versiona en **Git**, se empaqueta en imágenes de **Docker** y se comunica a través de **redes TCP/IP**.
>
> La estabilidad y el rendimiento de un pipeline dependen directamente de cómo interactúa el código de datos con el sistema operativo anfitrión y los límites de recursos de CPU, memoria y sockets de red.

0A.1 Linux y Terminal para Ingeniería de Datos

El 99% de las plataformas de datos distribuidas se ejecutan sobre distribuciones Linux (Ubuntu, Debian, Red Hat Enterprise Linux, Alpine). La terminal no es una alternativa a la interfaz gráfica: es la interfaz nativa de automatización.

1. Flujos Estándar, Redirecciones y Tuberías (*Pipes*)

En Linux, todo proceso interactúa con tres descriptores de archivo estándar:

- `stdin` (Descriptor 0): Entrada estándar de datos.
- `stdout` (Descriptor 1): Salida estándar de resultados.
- `stderr` (Descriptor 2): Salida estándar de errores y diagnósticos.

```bash
# Redirigir stdout a un archivo (sobrescribiendo)
python pipeline.py > salida.log

# Redirigir stdout y stderr a un archivo combinado (modo append)
python pipeline.py >> ejecucion.log 2>&1

# Conectar la salida de un proceso con la entrada de otro mediante Pipe (|)
cat transacciones_raw.csv | grep "COMPLETADO" | cut -d',' -f1,4 >
ventas_filtradas.csv
```

2. Procesamiento de Texto de Alto Rendimiento en Terminal

Antes de cargar gigabytes en Spark o Python, la terminal permite inspeccionar y transformar datos crudos en milisegundos:

- **`grep` (Búsqueda por patrones regex):**

```bash
# Buscar errores en logs ignorando mayúsculas/minúsculas y contando ocurrencias
grep -i "OutOfMemory" /var/log/spark/*.log | wc -l
```

- `sed` **(Editor de flujos para transformaciones y sustituciones):**

```bash
# Reemplazar delimitadores de punto y coma por comas en archivos CSV masivos
sed 's/;/,'/g transacciones.txt > transacciones_limpias.csv
```

- `awk` **(Procesador analítico de columnas estructuradas):**

```bash
# Sumar el valor de la columna 3 para registros cuyo país (columna 2) sea 'CO'
awk -F',' '$2 == "CO" { suma += $3 } END { print "Total Ventas Colombia: " suma }' ventas.csv
```

- `jq` **(El estándar de oro para manipular JSON en línea de comandos):**

```bash
# Extraer campos anidados y filtrar eventos de telemetría provenientes de una API
curl -s https://api.empresa.com/eventos | jq '.data[] | select(.monto > 500) | {id: .transaccion_id, valor: .monto}'
```

3. Permisos de Archivos y Administración de Procesos

- **Permisos POSIX:** El comando `chmod` altera los permisos para Usuario (u), Grupo (g) y Otros (o). En scripts ejecutables de producción:

```bash
chmod 750 run_ingest.sh   # rwxr-x--- (Lectura, escritura y ejecución para dueño; lectura y ejecución para grupo)
```

- **Procesos y Memoria:** Identificar y finalizar procesos colgados en servidores:

```bash
# Localizar el identificador de proceso (PID) de un driver de PySpark zombi
ps aux | grep "pyspark"

# Matar el proceso de forma forzada enviando la señal SIGKILL (9)
kill -9 <PID>

# Monitorear consumo de RAM, CPU y Swap en tiempo real
htop
```

4. Automatización con `cron` y Acceso Remoto con SSH

El demonio `cron` programa tareas periódicas mediante una sintaxis de 5 campos temporales (`minuto hora día-mes mes día-semana`):

```bash
# Editar el crontab del usuario actual
crontab -e

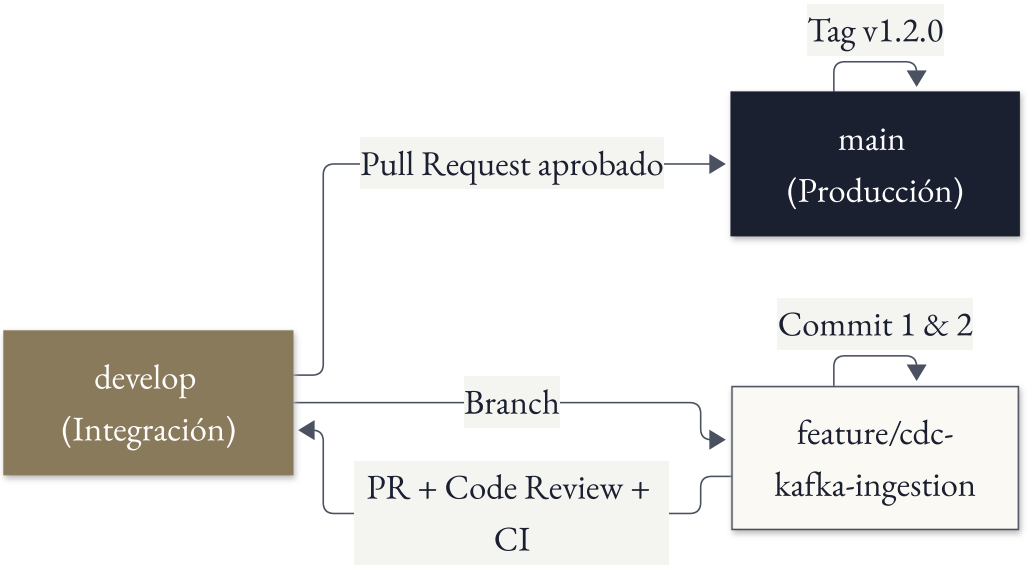
# Ejecutar el script de ingesta todos los días a las 02:30 AM UTC
30 2 * * * /usr/bin/bash /opt/pipelines/ingesta_diaria.sh >> /var/log/ingesta.log
2>&1
```

Para conectarse a instancias de cómputo en la nube sin exponer credenciales inseguras se utilizan pares de claves criptográficas SSH:

```bash
# Conexión segura usando clave privada RSA/Ed25519
ssh -i ~/.ssh/data_cluster_key.pem ubuntu@10.0.1.45
```

0A.2 Git Profesional y Estrategias de Colaboración

En la ingeniería de datos moderna, los pipelines son código fuente (*Pipelines as Code*). Toda alteración debe ser trazable, reproducible y sujeta a revisión por pares.



1. Estrategia de Ramas y Pull Requests (Trunk-Based vs GitFlow)

- **Trunk-Based Development:** Es la práctica recomendada para equipos ágiles de ingeniería de datos. Los ingenieros crean ramas de vida corta (`feat/nueva-fuente-ventas`) a partir de la rama principal (`main`), ejecutan pruebas automáticas y fusionan en cuestión de horas o días tras aprobación de un Pull Request (PR).
- **Conventional Commits:** Estandarizar los mensajes de confirmación permite generar changelogs automáticos y versionado semántico:
  - `feat: agregar particionado dinámico en carga de capa Silver`
  - `fix: corregir manejo de valores nulos en deduplicación de clientes`
  - `perf: optimizar broadcast join en cruce de catálogo de productos`
  - `test: agregar validaciones unitarias de esquema con Chispa`

2. `git rebase` interactivo frente a `git merge`

- `git merge`: Crea un commit de unión explícito que une dos historiales. Puede generar historiales enmarañados si se abusa en ramas locales.
- `git rebase`: Reescribe el historial aplicando los commits locales encima de la última versión de la rama base, manteniendo un historial lineal y limpio.

```bash
# Actualizar la rama local con los últimos cambios de main de forma lineal
git checkout feat/nueva-fuente
git fetch origin
git rebase origin/main

# Limpiar y condensar (squash) commits locales antes de abrir el PR
git rebase -i HEAD~3
```

0A.3 Docker y Contenedores para Plataformas de Datos

La clásica excusa *"en mi máquina local sí funcionaba"* es inaceptable en producción. Un contenedor encapsula la aplicación junto con sus bibliotecas de sistema, intérprete de Python, dependencias de Java (JARs de Spark/Hadoop) y variables de entorno, garantizando ejecución idéntica en local, en Kubernetes y en la nube.

1. Dockerfile Multi-Etapa (*Multi-Stage Build*)

Las construcciones multi-etapa permiten compilar dependencias en una imagen temporal pesada y copiar únicamente los binarios limpios a la imagen final ligera, reduciendo el tamaño de la imagen de 1.5 GB a menos de 200 MB y mitigando vulnerabilidades de seguridad:

```dockerfile
# Etapa 1: Constructor (Builder)
FROM python:3.11-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Instalar uv para resolución ultrarrápida de dependencias
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

COPY pyproject.toml ./
RUN uv pip install --system --no-cache -r pyproject.toml

# Etapa 2: Imagen Final de Producción (Runtime)
FROM python:3.11-slim AS runtime

WORKDIR /app

# Instalar Java OpenJDK para controladores locales de PySpark
RUN apt-get update && apt-get install -y --no-install-recommends \
    openjdk-17-jre-headless \
    curl \
```

```
    && rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
ENV PATH="${JAVA_HOME}/bin:${PATH}"

# Copiar paquetes instalados desde el builder
COPY --from=builder /usr/local/lib/python3.11/site-packages
/usr/local/lib/python3.11/site-packages
COPY src/ /app/src/

# Usuario sin privilegios por seguridad
RUN useradd -m -u 1001 datauser
USER datauser

ENTRYPOINT ["python", "-m", "src.pipelines.main"]
```

2. Volúmenes Persistentes y Redes Aisladas

- **Volúmenes (`-v` o `volumes:`):** Los contenedores son efímeros por diseño; si un contenedor de PostgreSQL se destruye, los datos almacenados en su interior desaparecen. Los volúmenes montan directorios del host o almacenamiento administrado por Docker para persistir datos.
- **Redes Bridge (`networks:`):** Permiten la comunicación interna entre contenedores por nombre de servicio mediante DNS interno de Docker, sin necesidad de exponer puertos al exterior de la máquina física.

0A.4 Composición de Infraestructura Local con Docker Compose

Para reproducir localmente un laboratorio de ingeniería de datos (con una base transaccional PostgreSQL y un almacenamiento de objetos S3 local compatible mediante MinIO), se utiliza **Docker Compose**:

YAML

```
version: '3.8'

services:
  # Base de datos relacional operacional (OLTP)
  postgres_oltp:
    image: postgres:16-alpine
    container_name: lab_postgres
    environment:
      POSTGRES_DB: transaccional_db
      POSTGRES_USER: ${POSTGRES_USER:-admin_datos}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD} # Inyectado de forma segura desde
archivo .env local
    env_file:
      - .env
```

```
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - red_datos

  # Almacenamiento de objetos local S3-compatible (Data Lake)
  minio_s3:
    image: minio/minio:latest
    container_name: lab_minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minio_admin
      MINIO_ROOT_PASSWORD: MinioSecretKey2026!
    ports:
      - "9000:9000"   # Puerto API S3
      - "9001:9001"   # Consola Web UI
    volumes:
      - miniodata:/data
    networks:
      - red_datos

volumes:
  pgdata:
  miniodata:

networks:
  red_datos:
    driver: bridge
```

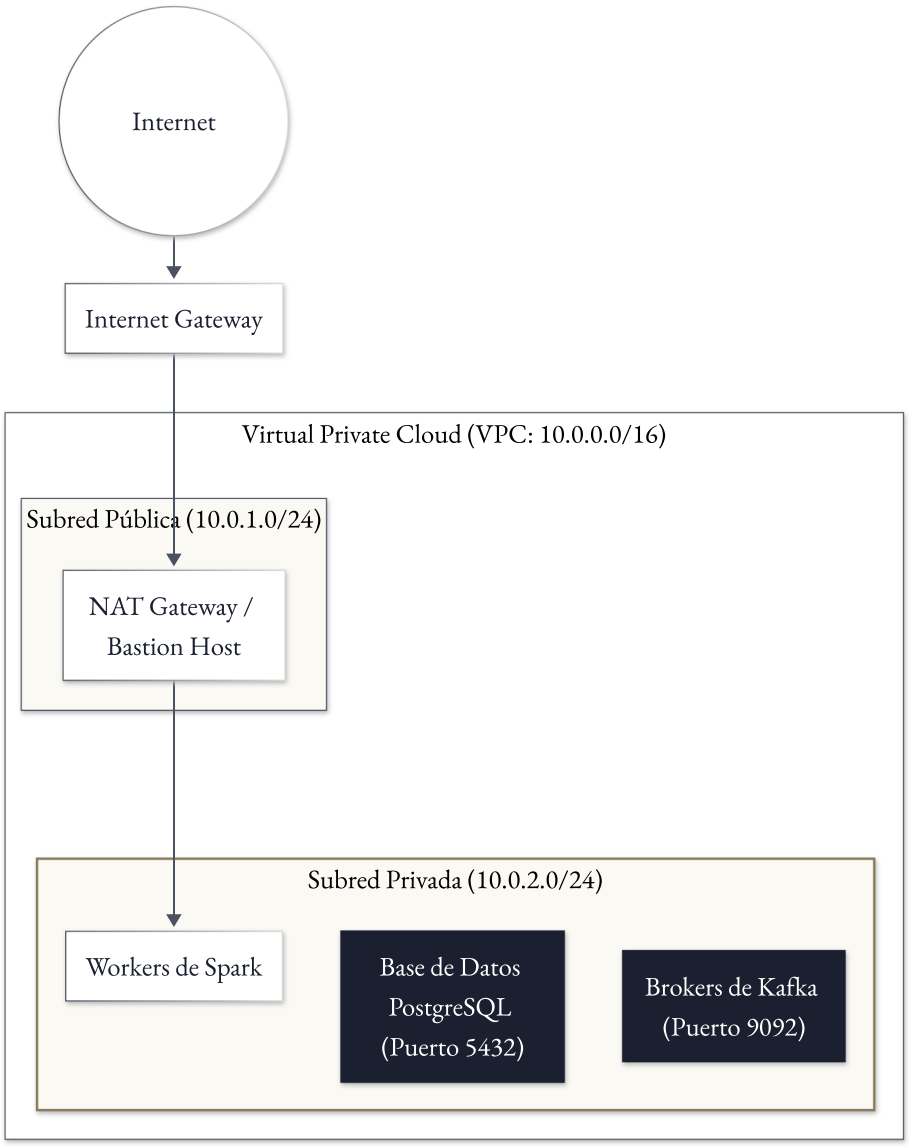
Para levantar todo el ecosistema de laboratorio en segundo plano, basta ejecutar un único comando en la terminal:

```
BASH
```

```
docker compose up -d
```

0A.5 Redes Básicas y Seguridad para Plataformas de Datos

Todo sistema distribuido opera sobre redes de comunicaciones. Conocer la arquitectura de redes previene fallos de conectividad comunes en la nube:



1. Direccionamiento CIDR y Subredes

- **CIDR (*Classless Inter-Domain Routing*):** Representa bloques de direcciones IP mediante la notación `IP/máscara`.
  - `10.0.0.0/16`: Red privada con $2^{32-16} = 65.536$ direcciones IP disponibles (espacio típico para una VPC).
  - `10.0.1.0/24`: Subred con $2^{32-24} = 256$ direcciones IP.
- **Aislamiento en Subredes Privadas:** Los clústeres de Spark, bases de datos y brokers de Kafka **nunca deben tener direcciones IP públicas**. Deben residir en **subredes privadas**,

comunicándose hacia internet exclusivamente a través de un **NAT Gateway** y recibiendo tráfico administrativo mediante un *Bastion Host* (Jump Server) o túneles VPN.

2. Cortafuegos de Red (*Security Groups*) y Puertos Estándar

Los cortafuegos definen reglas de entrada (*Inbound*) y salida (*Outbound*) basadas en protocolos y puertos:

- `Puerto 22`: SSH (Acceso a terminal segura).
- `Puerto 443`: HTTPS / TLS (Consumo de APIs y transferencia de objetos S3/ADLS).
- `Puerto 5432`: PostgreSQL.
- `Puerto 9092`: Apache Kafka.
- `Puerto 7077 / 4040`: Apache Spark Master y Spark Web UI.
- **Regla de Mínimo Privilegio:** Jamás abra un puerto de base de datos a `0.0.0.0/0` (todo internet). Restrinja la regla al bloque CIDR exacto de los nodos ejecutores de Spark.

3. Cifrado en Tránsito: TLS y mTLS

- **TLS (*Transport Layer Security*):** Cifra los datos transmitidos entre el cliente y el servidor mediante certificados criptográficos, impidiendo ataques de interceptación (*Man-In-The-Middle*).
- **mTLS (*Mutual TLS*):** Exige que **tanto el servidor como el cliente** presenten certificados válidos firmados por una Autoridad de Certificación (CA) de confianza. Es el estándar de oro para comunicar microservicios y productores con clústeres de Apache Kafka en arquitecturas de cero confianza (*Zero Trust*).

0A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutar Contenedores de Producción como Usuario `root`
Dejar que los procesos dentro del contenedor corran como `root` representa un riesgo grave de seguridad: un atacante que explote una vulnerabilidad en una biblioteca de Python podría escapar del contenedor y obtener privilegios totales sobre la máquina anfitriona (*host breakout*).
Corrección: Declare siempre `USER datauser` en su `Dockerfile`.

Anti-patrón 2: Almacenar Credenciales Secretas en Imágenes Docker o Repositorios Git
Escribir contraseñas de bases de datos o claves de acceso a AWS directamente en un `Dockerfile`, script o archivo de configuración versionado en Git es una de las principales causas de brechas de seguridad.

Corrección: Inyecte secretos en tiempo de ejecución mediante variables de entorno, gestores de secretos (AWS Secrets Manager, HashiCorp Vault) o herramientas de montaje seguro (`docker run --env-file .env`).

Anti-patrón 3: Ignorar el Archivo `.dockerignore` y `.gitignore`
Construir una imagen Docker sin un archivo `.dockerignore` copia accidentalmente entornos virtuales locales pesados (`.venv`), cachés de Python (`__pycache__`) y claves privadas a la imagen, aumentando el tiempo de construcción y exponiendo datos sensibles.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0A.1 (Filtrado y Agregación Eficiente con Herramientas de Terminal):
Se tiene un archivo de registro crudo de 20 GB denominado `conexiones_db.log` con el siguiente formato separado por espacios:
`TIMESTAMP IP_ORIGEN USUARIO ESTADO_CONEXION LATENCIA_MS`
Escriba un comando de terminal Linux de una sola línea (*one-liner*) que:

1. Filtre únicamente las conexiones fallidas (`ESTADO_CONEXION = 'FAILED'`).
2. Extraiga la dirección IP de origen (columna 2).
3. Calcule las 5 direcciones IP con mayor cantidad de intentos fallidos, ordenadas de mayor a menor.

Solución 0A.1:

```BASH
grep "FAILED" conexiones_db.log | awk '{print $2}' | sort | uniq -c | sort -nr | head -n 5
```

Explicación Paso a Paso:

- `grep "FAILED"` descarta de inmediato millones de registros exitosos sin cargarlos en memoria.
- `awk '{print $2}'` proyecta únicamente la columna con la dirección IP.
- `sort` agrupa las IPs consecutivamente para que `uniq` pueda operar.
- `uniq -c` contabiliza las ocurrencias de cada IP única.
- `sort -nr` realiza un ordenamiento numérico (`-n`) en orden descendente (`-r`).
- `head -n 5` trunca la salida a las 5 principales fuentes de ataques o errores de conexión.

Ejercicio 0A.2 (Resolución de Conflictos en Git):
Dos ingenieros de datos modificaron simultáneamente el archivo `pipeline_ventas.py`. Al intentar hacer `git rebase origin/main`, Git detiene la ejecución notificando un conflicto en la línea donde se define la ruta de salida de almacenamiento. Explique el procedimiento exacto paso a paso para resolver el conflicto y completar el rebase sin perder cambios.

Solución 0A.2:

1. Identificar el archivo en conflicto ejecutando `git status`.
2. Abrir `pipeline_ventas.py` y localizar los marcadores de conflicto insertados por Git:

```python
<<<<<<< HEAD
RUTA_SALIDA = "s3://bucket-analitica/silver/ventas_v2/"
=======
RUTA_SALIDA = "s3://bucket-analitica/silver/ventas_particionadas/"
>>>>>>> feat/nueva-ruta
```

3. Editar el archivo manualmente conservando la lógica arquitectónica correcta y eliminando los marcadores `<<<<<<<`, `=======` y `>>>>>>>`.
4. Marcar el archivo como resuelto en el área de preparación (*staging*): `git add pipeline_ventas.py`.
5. Continuar el proceso de rebase ejecutando: `git rebase --continue` (¡Nunca ejecutar `git commit` durante un rebase activo!).

0A.7 Resumen y Puntos Clave

- **Linux es el sustrato universal:** Las herramientas nativas de terminal (`grep`, `sed`, `awk`, `jq`) permiten realizar transformaciones exploratorias y diagnósticos de fallos órdenes de magnitud más rápido que escribir scripts ad-hoc.
- **Git profesional:** Flujos basados en ramas cortas, Conventional Commits y `rebase` interactivo mantienen historiales limpios y trazables para auditorías de código.
- **Contenedores reproducibles:** Las imágenes multi-etapa de Docker minimizan la superficie de ataque y garantizan que los pipelines se ejecuten exactamente igual en cualquier entorno.
- **Redes y seguridad de datos:** Las plataformas analíticas deben residir en subredes privadas aisladas con reglas de cortafuegos de mínimo privilegio y canales cifrados mediante TLS/mTLS.

Lecturas Recomendadas

1. **Nemeth, E., Snyder, G., Hein, T. R., Whaley, B. & Mackin, D. (2017).** *UNIX and Linux System Administration Handbook (5th Edition).* Addison-Wesley.
2. **Chacon, S. & Straub, B. (2014).** *Pro Git (2nd Edition).* Apress (Disponible libre en línea).
3. **Mouat, A. (2015).** *Using Docker: Developing and Deploying Software with Containers.* O'Reilly Media.
4. **Kurose, J. F. & Ross, K. W. (2021).** *Computer Networking: A Top-Down Approach (8th Edition).* Pearson.

CAPÍTULO 08

Bases de Datos Relacionales (OLTP) y Modelado Transaccional NÚCLEO

> *"Casi la totalidad de los datos analíticos que fluyen a través de un Lakehouse nacieron originalmente en una base de datos relacional orientada a transacciones. Si un ingeniero de datos no comprende los principios de normalización, la mecánica de los árboles B-Tree y los niveles de aislamiento transaccional, es incapaz de diseñar procesos de extracción e ingesta que no degraden o bloqueen los sistemas operacionales de la empresa."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la fundamentación matemática del modelo relacional y las formas normales canónicas (**1NF**, **2NF**, **3NF**).
- Distinguir las razones arquitectónicas por las cuales los sistemas **OLTP** normalizan rigurosamente sus esquemas mientras que los sistemas **OLAP** desnormalizan intencionadamente.
- Analizar la estructura interna de los índices **B-Tree** frente a índices **Hash**, comprendiendo su costo de mantenimiento en operaciones de escritura.
- Desglosar las cuatro propiedades **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad) y su implementación física en discos y diarios de transacciones (*Write-Ahead Logging / WAL*).
- Dominar los cuatro niveles de aislamiento transaccional estándar y diagnosticar las anomalías de concurrencia: *Dirty Reads*, *Non-Repeatable Reads*, *Phantom Reads* y *Write Skew*.
- Explicar el funcionamiento del Control de Concurrencia Multiversión (**MVCC**) en PostgreSQL y la necesidad de mantenimiento mediante `VACUUM`.

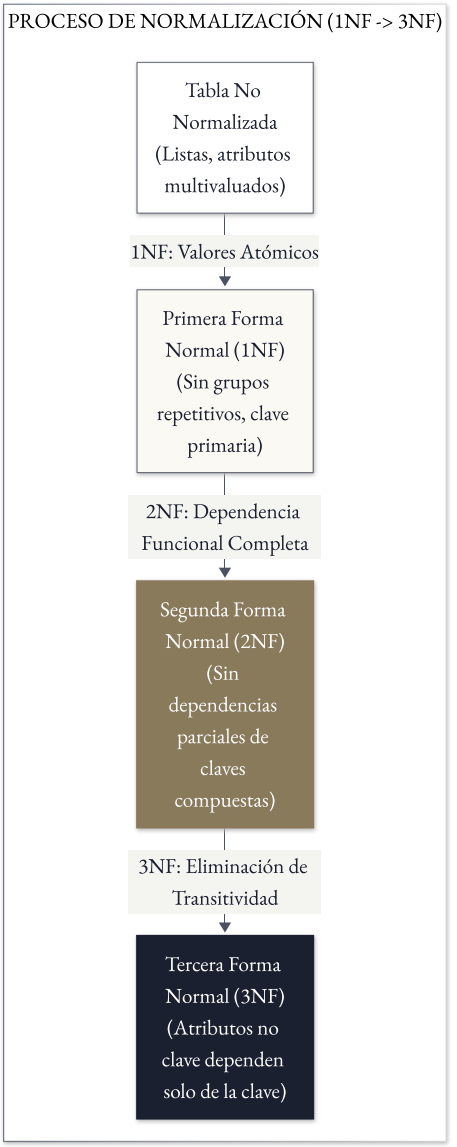
Definición 0B.1 (Sistemas OLTP frente a Sistemas OLAP)

- **OLTP (*Online Transaction Processing*):** Sistemas diseñados para procesar cientos o miles de operaciones simultáneas por segundo de baja latencia (altas, modificaciones, lecturas puntuales de tuplas individuales). Su prioridad absoluta es la integridad transaccional inmediata, la no redundancia de datos y la concurrencia masiva mediante bloqueos o versiones.
- **OLAP (*Online Analytical Processing*):** Sistemas diseñados para ejecutar consultas complejas que agregan y analizan millones de registros a lo largo de amplios rangos temporales. Su prioridad es el rendimiento en lectura masiva, el escaneo secuencial en disco y la compresión columnar.

0B.1 Normalización Relacional y Eliminación de Anomalías

En los sistemas transaccionales, el objetivo principal del diseño de esquemas es la **eliminación de redundancias** para impedir anomalías operacionales:

- *Anomalía de Inserción:* Imposibilidad de registrar datos de un cliente hasta que este realice una compra.
- *Anomalía de Actualización:* Modificar la dirección de una sucursal en 10.000 filas de facturación, corriendo el riesgo de dejar registros inconsistentes si el proceso falla a la mitad.
- *Anomalía de Eliminación:* Borrar el último pedido de un usuario provocando la pérdida accidental de todos los datos de contacto de dicho usuario.



1. Primera Forma Normal (1NF)

Exige que cada celda de la tabla contenga un único valor **atómico** indivisible y que no existan grupos repetitivos (como columnas `telefono_1`, `telefono_2` o listas serializadas en texto). Cada fila debe poseer una clave primaria única.

2. Segunda Forma Normal (2NF)

Aplica a tablas con claves primarias compuestas: exige cumplir 1NF y que ningún atributo no clave dependa funcionalmente de una **parte** de la clave primaria (*dependencia parcial*). Todo atributo debe depender de la totalidad de la clave.

3. Tercera Forma Normal (3NF)

Exige cumplir 2NF y que ningún atributo no clave dependa transitivamente de la clave primaria a través de otro atributo no clave (*dependencia transitiva*).

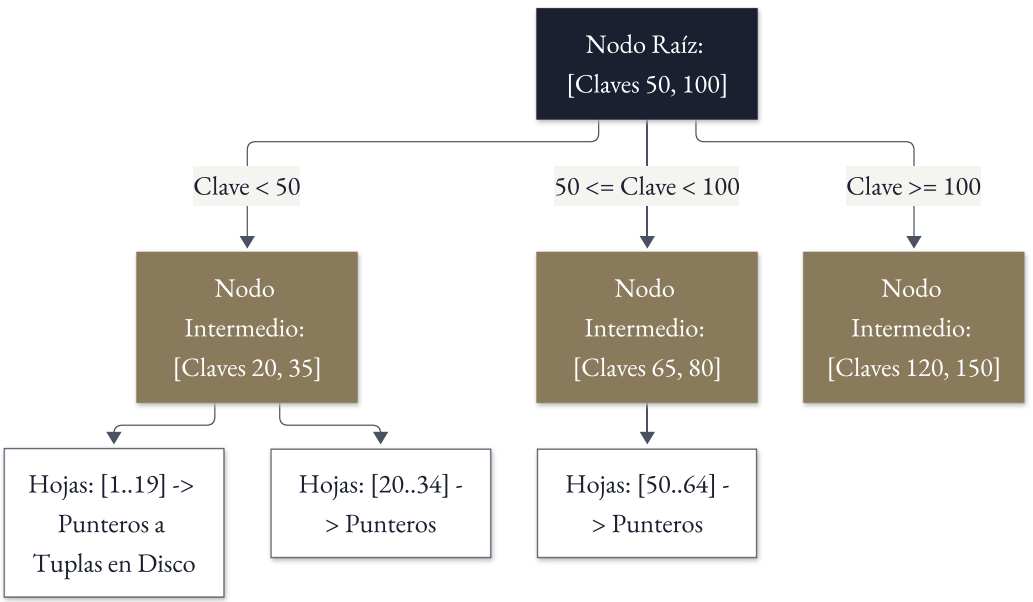
- *Ejemplo:* Si una tabla tiene `(factura_id, cliente_id, ciudad_cliente)`, `ciudad_cliente` depende de `cliente_id`, y este de `factura_id`. Para alcanzar 3NF, la información del cliente se traslada a su propia tabla normalizada `clientes`.

> **¿Por qué OLTP normaliza y OLAP desnormaliza?**
> Normalizar en OLTP garantiza que una actualización modifique exactamente **un solo registro físico en disco**, eliminando inconsistencias. Sin embargo, consultar una tabla en 3NF requiere múltiples `JOINs` costosos. En OLAP (Data Warehouses / Lakehouses), el espacio en disco es económico y las lecturas analíticas son masivas: desnormalizar en esquemas de estrella (*Star Schema*) optimiza drásticamente el escaneo columnar sin bloqueos.

---

oB.2 Estructura Física e Índices: B-Trees frente a Hash

Una base de datos relacional almacena las filas en páginas de disco (típicamente de 8 KB en PostgreSQL). Buscar un registro sin índices fuerza un escaneo secuencial completo de la tabla (*Sequential Scan* o $O(N)$).



1. El Árbol B-Tree (Balanced Tree)

Es el índice predeterminado y más versátil en motores relacionales:

- **Estructura Auto-Balanceada:** Todas las hojas se encuentran a la misma profundidad, garantizando complejidad de búsqueda logarítmica $O(\log N)$.
- **Búsquedas de Rango:** Las páginas hoja están interconectadas mediante punteros bidireccionales en lista doblemente enlazada, lo que permite resolver de forma óptima consultas con predicados de rango (`BETWEEN`, `<`, `>=`).
- **Soporte para Ordenación:** Almacena los elementos ordenados, satisfaciendo cláusulas `ORDER BY` sin necesidad de clasificar en memoria de trabajo (*Sort in-memory*).

2. Índices Hash

Calculan el hash criptográfico o posicional de la clave, ofreciendo búsquedas de igualdad exacta (`=`) en tiempo constante $O(1)$. Sin embargo, son **inútiles para consultas de rango** (`WHERE fecha > '2026-01-01'`) y no admiten ordenación.

3. El Costo de los Índices en Pipelines de Ingesta

Crear índices acelera las lecturas analíticas, pero **penaliza severamente las operaciones de escritura** (`INSERT`, `UPDATE`, `DELETE`). Cada tupla insertada requiere balancear y reescribir los nodos del árbol B-Tree en disco:

- **Regla de Producción:** Durante cargas masivas por lotes (*Bulk Load*) de millones de filas en una base relacional, la mejor práctica consiste en **eliminar los índices secundarios antes de la carga** y reconstruirlos en bloque una vez finalizada la inserción.

0B.3 Propiedades ACID y Mecánica de Durabilidad

Una **transacción** es una secuencia de operaciones relacionales tratadas como una única unidad lógica indivisible:

- **Atomicidad (Atomicity):** Todo o nada. Si falla una sola sentencia dentro del bloque transaccional, el motor deshace todas las operaciones previas mediante un *Rollback*.
- **Consistencia (Consistency):** La transacción solo puede llevar la base de datos de un estado válido a otro estado válido, respetando todas las restricciones de integridad declaradas (claves foráneas, tipos de datos, restricciones `CHECK`).
- **Aislamiento (Isolation):** La ejecución concurrente de transacciones produce el mismo estado que si se hubiesen ejecutado de forma secuencial una tras otra.

- **Durabilidad (Durability):** Una vez que una transacción confirma su éxito (*Commit*), sus cambios son permanentes y no se perderán incluso ante cortes catastróficos de energía.

Write-Ahead Logging (WAL)

Para garantizar la durabilidad sin ralentizar cada transacción esperando a que los bloques de la tabla se sincronicen en el disco de datos, los motores utilizan un diario secuencial denominado **WAL** (*Write-Ahead Log* en PostgreSQL o *Redo Log* en Oracle):

1. Antes de modificar cualquier página de memoria en la caché (*Buffer Pool*), el cambio se escribe secuencialmente al WAL en disco.
2. Al invocar `COMMIT`, el motor solo fuerza el volcado físico del WAL a disco (*fsync* secuencial ultrarrápido).
3. Si el servidor colapsa, durante el arranque el motor lee el WAL y aplica los cambios confirmados sobre los archivos de datos (*Crash Recovery*).

---

oB.4 Niveles de Aislamiento Transaccional y Anomalías de Concurrencia

El estándar ANSI SQL define cuatro niveles de aislamiento crecientes, cada uno protegiendo contra anomalías específicas de concurrencia:

Nivel de Aislamiento	Lectura Sucia (***Dirty Read***)	Lectura No Repetible (***Fuzzy Read***)	Lectura Fantasma (***Phantom Read***)	Desviación de Escritura (***Write Skew***)
Read Uncommitted	Posible	Posible	Posible	Posible
Read Committed *(Default en Postgres)*	**Prevenida**	Posible	Posible	Posible
Repeatable Read	**Prevenida**	**Prevenida**	**Prevenida** *(en Postgres MVCC)*	Posible
Serializable	**Prevenida**	**Prevenida**	**Prevenida**	**Prevenida**

Definición Matemática de las Anomalías:

1. **Lectura Sucia (*Dirty Read*):** La Transacción T_1 modifica una fila pero aún no hace commit. La Transacción T_2 lee ese valor no confirmado. Si T_1 hace rollback, T_2 procesó datos ficticios que jamás existieron formalmente.

2. **Lectura No Repetible (*Non-Repeatable Read*):** La Transacción T_1 lee una fila. La Transacción T_2 modifica o borra dicha fila y confirma su commit. Si T_1 vuelve a leer la misma fila, obtiene valores distintos.
3. **Lectura Fantasma (*Phantom Read*):** La Transacción T_1 consulta un rango de filas con una condición (ej. `WHERE saldo > 1000`). La Transacción T_2 inserta una nueva fila que satisface la condición y confirma. Si T_1 repite la consulta de rango, aparece una nueva fila "fantasma".
4. **Desviación de Escritura (*Write Skew*):** Ocurre cuando dos transacciones leen los mismos datos pero escriben en conjuntos disjuntos violando una regla de negocio global (ej. dos médicos solicitan salir de guardia simultáneamente al verificar que al menos uno de ellos sigue activo; ambas solicitudes se aprueban, dejando la guardia vacía). Solo se previene en **Serializable**.

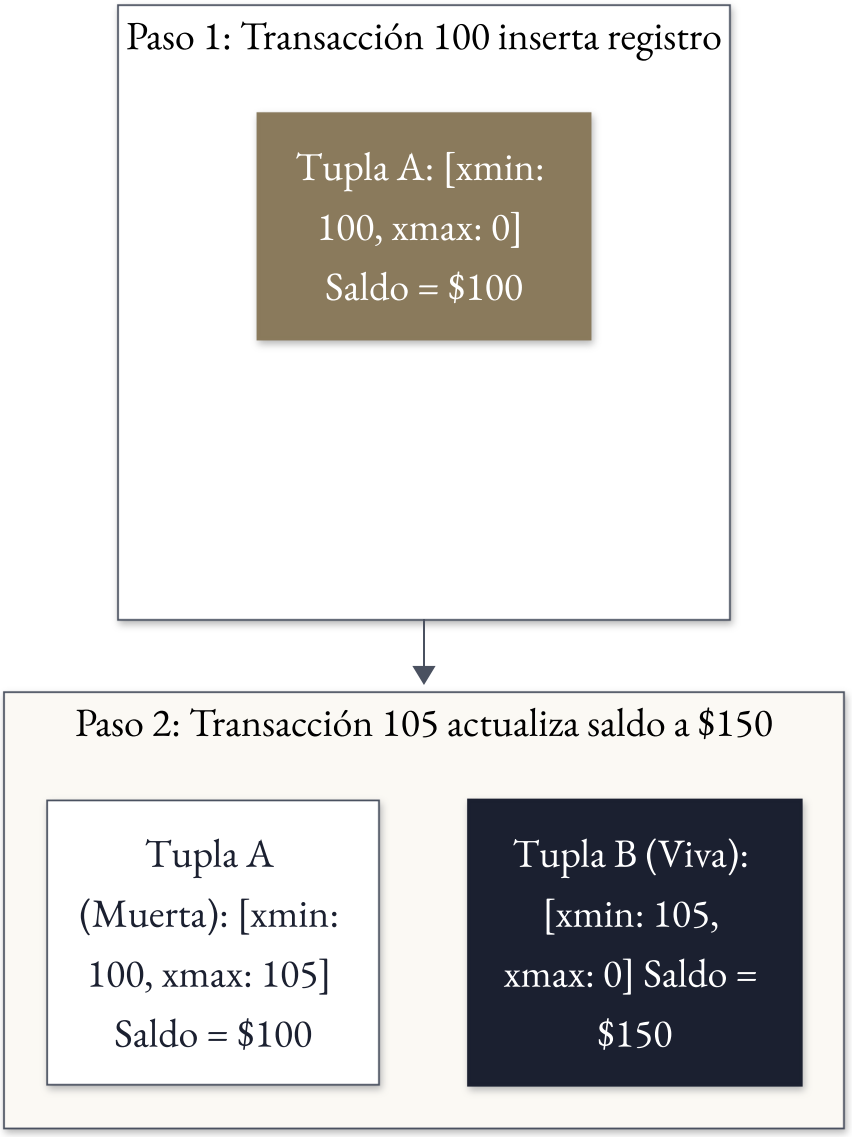
---

oB.5 Control de Concurrencia Multiversión (MVCC) en PostgreSQL

PostgreSQL no utiliza bloqueos de lectura para proteger a los lectores de los escritores. En su lugar, implementa **MVCC** (*Multi-Version Concurrency Control*): **"Los lectores nunca bloquean a los escritores, y los escritores nunca bloquean a los lectores"**.

Cada fila (*heap tuple*) almacena metadatos invisibles en su encabezado:

- `xmin`: El identificador de la transacción que insertó la fila.
- `xmax`: El identificador de la transacción que borró o actualizó la fila (inicialmente 0).



- **Actualizaciones como Inserciones:** En MVCC, un `UPDATE` no sobrescribe el registro original en el disco. Marca la tupla previa asignándole el ID de la transacción actual en `xmax` (convirtiéndola en una tupla "muerta") e inserta una tupla completamente nueva con el nuevo valor.
- **El Proceso `VACUUM`:** Las tuplas muertas ocupan espacio físico en las páginas de disco. El proceso `VACUUM` (y el demonio en segundo plano `autovacuum`) escanea las páginas para reclamar el espacio de tuplas muertas que ya no son visibles para ninguna transacción activa, evitando el crecimiento descontrolado de las tablas en disco (*Table Bloat*).

oB.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Extraer Datos Analíticos Masivos mediante Consultas Directas sobre la Base Primaria OLTP
Ejecutar consultas de extracción analítica masiva (`SELECT * FROM transacciones WHERE anio = 2025`) directamente sobre el nodo primario de la base transaccional durante el día satura el buffer pool y compite por recursos con los clientes de la aplicación.
Corrección: Dirija las extracciones hacia **réplicas de solo lectura** (*Read Replicas* configuradas mediante replicación física por WAL) o utilice herramientas de CDC (*Change Data Capture*) como Debezium para leer los eventos directamente del log sin consultar las tablas vivas.

Anti-patrón 2: Desactivar o Retrasar el Proceso de `autovacuum`
Desactivar `autovacuum` para supuestamente ahorrar CPU en una base de datos de alto tráfico transaccional provoca que las tablas acumulen millones de tuplas muertas. Las consultas se ralentizan dramáticamente al tener que escanear páginas infladas y se corre el riesgo inminente de fallo catastrófico por agotamiento de identificadores de transacción (*Transaction ID Wraparound*).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0B.1 (Diagnóstico de Anomalía de Concurrencia):
Considere el siguiente escenario transaccional en una tabla de cuentas bancarias (`saldo` inicial = $500):

1. La Transacción A inicia en nivel `READ COMMITTED` y consulta el saldo de la cuenta 101, obteniendo $500.
2. La Transacción B inicia, descuenta $100 de la cuenta 101 (nuevo saldo = $400) y ejecuta `COMMIT`.
3. La Transacción A, aún activa, vuelve a ejecutar la misma consulta sobre la cuenta 101 y obtiene $400. ¿Qué anomalía de concurrencia ha ocurrido y a qué nivel de aislamiento debería elevarse la Transacción A para evitarla?

Solución 0B.1:

- Ha ocurrido una anomalía de **Lectura No Repetible (*Non-Repeatable Read* o *Fuzzy Read*)**, donde una misma transacción obtiene valores contradictorios para una misma fila tras el commit de una transacción concurrente.
- Para evitar esta anomalía, la Transacción A debe ejecutarse bajo el nivel de aislamiento `REPEATABLE READ` (o `SERIALIZABLE`). Bajo `REPEATABLE READ`, el motor establece una instantánea (*snapshot*) temporal al momento de la primera consulta, garantizando que

lecturas posteriores dentro de la misma transacción lean exactamente la misma foto inmutable del registro.

Ejercicio 0B.2 (Estructuración de Índices B-Tree Compuestos):
Se tiene una tabla `pedidos` con 100 millones de registros. Las consultas más recurrentes del sistema transaccional tienen la siguiente estructura:
`SELECT * FROM pedidos WHERE cliente_id = 4500 AND fecha_pedido >= '2026-01-01'`
Compare las siguientes dos alternativas de índice B-Tree compuesto y justifique cuál es la óptima:

- Opción 1: `CREATE INDEX idx_pedidos ON pedidos (fecha_pedido, cliente_id);`
- Opción 2: `CREATE INDEX idx_pedidos ON pedidos (cliente_id, fecha_pedido);`

Solución 0B.2:

- **La Opción 2 `(cliente_id, fecha_pedido)` es la óptima.**
- *Justificación Técnica:* En un árbol B-Tree compuesto, los nodos se ordenan jerárquicamente por la primera columna y luego por la segunda.
  - Con la Opción 2, el motor busca por igualdad exacta sobre `cliente_id` (alta selectividad), descendiendo directamente a la rama del cliente específico en $O(\log N)$ y luego escaneando secuencialmente solo el rango de fechas de ese cliente.
  - Con la Opción 1, la primera columna es un predicado de rango (`fecha_pedido >= '2026-01-01'`). Una vez que un B-Tree entra en una búsqueda de rango sobre su primera columna, el motor no puede podar eficientemente por la segunda columna y se ve forzado a escanear todas las entradas que cumplan la fecha en todas las páginas hoja.

0B.7 Resumen y Puntos Clave

- **Diseño para la integridad:** Los sistemas OLTP normalizan (1NF a 3NF) para eliminar anomalías de actualización y garantizar que cada hecho se registre en una sola ubicación física.
- **Mecánica de índices B-Tree:** Ofrecen búsquedas logarítmicas $O(\log N)$ y ordenación eficiente, pero imponen una penalización de mantenimiento en cada inserción o modificación.
- **Garantías ACID y WAL:** La durabilidad se garantiza mediante el volcado secuencial al Write-Ahead Log (*WAL*), permitiendo recuperaciones rápidas ante caídas del servidor.
- **MVCC en PostgreSQL:** Mediante identificadores de transacción `xmin` y `xmax`, PostgreSQL permite lecturas no bloqueantes generando nuevas versiones de tuplas, requiriendo `autovacuum` para reclamar espacio muerto.

- **Aislamiento transaccional:** Conocer las anomalías de concurrencia permite elegir el nivel óptimo sin incurrir en bloqueos excesivos.

Lecturas Recomendadas

1. **Silberschatz, A., Korth, H. F. & Sudarshan, S. (2020).** *Database System Concepts (7th Edition).* McGraw-Hill.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 7: Transactions).* O'Reilly Media.
3. **Stonebraker, M. & Cetintemel, U. (2005).** *"One Size Fits All": An Idea Whose Time Has Come and Gone.* Proceedings of the 21st International Conference on Data Engineering (ICDE).
4. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Concurrency Control and MVCC.*

CAPÍTULO oC

Python para Ingeniería de Datos: Tipado, APIs y Resiliencia NÚCLEO

> *"Escribir scripts exploratorios en un notebook es una habilidad de análisis; construir pipelines modulares, fuertemente tipados, tolerantes a fallos y empaquetados con rigor de ingeniería de software es la verdadera responsabilidad del Data Engineer. En entornos de producción, Python no es un lenguaje de juguete: es el lenguaje de orquestación, integración y transformación de datos por excelencia."*

oC.o Intuición Inicial: La Cinta Transportadora de la Fábrica

Imagina que te encargan procesar las facturas de una tienda pequeña. Tienes diez facturas en papel sobre una mesa: tomas cada una, lees el total con los ojos y anotas la suma en una libreta con lápiz. Si son diez facturas, el método manual funciona sin problemas.

Pero ahora imagina que la tienda se convierte en un gigante de comercio electrónico que procesa 500.000 transacciones por hora provenientes de ocho países distintos. Las facturas ya no vienen en papel: llegan como mensajes JSON a través de conexiones de red inestables, algunas traen importes nulos, otras traen identificadores duplicados y los servidores de origen se saturan con frecuencia arrojando errores temporales. Si intentas resolver esto con un script improvisado que asume que "nada fallará", el sistema colapsará en cuestión de minutos.

Python es la cinta transportadora programable de esa fábrica industrial de datos. En este capítulo comenzaremos desde lo más elemental —cómo guardar un dato en una variable, cómo filtrar una lista de registros, cómo encapsular lógica en funciones y cómo leer archivos de disco— hasta ensamblar componentes de nivel productivo: contratos de datos estrictos con Pydantic, consumo resiliente de APIs REST con reintentos exponenciales y manipulación columnar de alta velocidad con Polars y Apache Arrow.

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Programar en Python moderno desde los fundamentos: variables, colecciones (`list`, `dict`, `set`, `tuple`), estructuras de control, funciones puras y manejo seguro de archivos con *context managers*.
- Aplicar Programación Orientada a Objetos básica y `@dataclass` para modelar entidades de datos estructuradas.
- Aislar dependencias y ejecutar pruebas unitarias automatizadas con `uv` y `pytest`.
- Aplicar el sistema de tipos moderno (`typing`) y validar contratos de datos en tiempo de ejecución con **Pydantic**.
- Diseñar clientes de ingesta HTTP resilientes para **APIs REST** con soporte para autenticación, paginación por cursores y retroceso exponencial (*Exponential Backoff con Jitter*).
- Manipular datos en memoria comparando los paradigmas de **Pandas**, **PyArrow** y **Polars** (ejecución multihilo perezosa en Rust).

Profundiza: La Brecha entre Scripting y Software Engineering

Definición 0C.1 (La Brecha entre Scripting y Software Engineering)
Un *script de datos* se escribe para ejecutarse una sola vez de forma interactiva: asume que la red nunca se corta, que el esquema del JSON de origen es estático y que todos los datos caben cómodamente en la memoria RAM.

Un **pipeline de ingeniería de software para datos** se diseña bajo la premisa del fallo inevitable (*Design for Failure*): valida los datos entrantes contra un contrato estricto, maneja desconexiones transitorias mediante reintentos exponenciales, no filtra credenciales en los logs y garantiza reproducibilidad absoluta de su entorno mediante bloqueos de dependencias (*lockfiles*).

0C.1 Fundamentos de Python para Ingeniería de Datos: De Cero a Código Modular

Para el aspirante a Data Engineer que proviene de cero o de campos no computacionales, dominar Python no significa memorizar sintaxis abstracta: significa comprender cómo estructurar datos en memoria, cómo procesar archivos y cómo escribir código modular y testeable.

1. Tipos de Datos Primitivos y Variables

En Python, una variable es una referencia en memoria a un objeto tipado:

PYTHON

```python
# Tipos primitivos esenciales en pipelines de datos
registro_id = 1048576            # int: enteros de precisión arbitraria
monto_transaccion = 450.75       # float: números de punto flotante IEEE 754 de
64 bits
estado_pago = "COMPLETADO"       # str: cadenas inmutables codificadas en UTF-8
es_valido = True                 # bool: valores booleanos para predicados
lógicos
tasa_cambio = None               # NoneType: representa la ausencia explícita de
valor
```

2. Estructuras de Datos de Primera Clase para Pipelines

Los flujos de datos manipulan cuatro colecciones fundamentales:

PYTHON

```python
# 1. LISTAS (list): Secuencias ordenadas y mutables. Útiles para acumular
registros
transacciones = [120.50, 45.00, 890.10, 15.20]
transacciones.append(300.00)     # O(1) amortizado

# 2. DICCIONARIOS (dict): Tablas hash de clave-valor O(1). El corazón de cada
fila o evento JSON
cliente = {
    "cliente_id": 4501,
    "nombre": "Ana Gomez",
    "pais": "CO",
    "activo": True
}
print(cliente.get("correo", "desconocido@correo.com"))  # Acceso seguro ante
claves faltantes

# 3. CONJUNTOS (set): Colecciones no ordenadas de elementos únicos. Deduplicación
instantánea O(1)
paises_con_ventas = {"CO", "MX", "CL", "PE"}
paises_con_ventas.add("CO")      # No duplica; el tamaño del conjunto sigue
intacto
print("US" in paises_con_ventas) # Verificación de pertenencia en O(1)

# 4. TUPLAS (tuple): Secuencias ordenadas e inmutables. Seguras como claves
compuestas
coordenada_evento = ("servidor_01", 8080)
```

Comprensiones de Listas y Diccionarios (Comprehensions)

Permiten transformar y filtrar colecciones con sintaxis concisa y optimizada en C:

```python
# Filtrar y transformar ventas superiores a $100 aplicando un impuesto del 19%
ventas_con_iva = [round(monto * 1.19, 2) for monto in transacciones if monto > 100.0]

# Indexar un diccionario de clientes por su ID para búsquedas directas
lista_clientes = [
    {"id": 101, "nombre": "Carlos"},
    {"id": 102, "nombre": "Lucia"}
]
indice_clientes = {c["id"]: c["nombre"] for c in lista_clientes}
```

3. Control de Flujo y Funciones Modulares

Las funciones puras encapsulan lógica de transformación sin efectos secundarios:

```python
def procesar_lote_ventas(montos, umbral_alerta=500.0, **kwargs):
    """Procesa un lote de montos, calculando totales y detectando anomalías."""
    total = 0.0
    alertas = []

    for monto in montos:
        if monto is None or monto < 0:
            continue  # Omitir registros corruptos
        total += monto
        if monto >= umbral_alerta:
            alertas.append(monto)

    return {"total_procesado": round(total, 2), "alertas": alertas}

resultado = procesar_lote_ventas(transacciones, umbral_alerta=200.0)
```

4. Manejo de Archivos en Disco: CSV, JSON y Context Managers

En ingeniería de datos, **nunca se abren archivos sin un administrador de contexto (`with`)**, el cual garantiza el cierre inmediato de los descriptores de archivo incluso ante excepciones no controladas:

PYTHON

```python
import csv
import json

# Lectura y parseo seguro de archivos CSV fila a fila
def leer_transacciones_csv(ruta_archivo):
    registros = []
    with open(ruta_archivo, mode="r", encoding="utf-8") as archivo:
        lector_csv = csv.DictReader(archivo)
        for fila in lector_csv:
            registros.append({
                "id": int(fila["transaccion_id"]),
                "monto": float(fila["monto_usd"]),
                "pais": fila["pais"]
            })
    return registros

# Serialización y guardado de resultados en JSON estructurado
def exportar_resumen_json(datos, ruta_destino):
    with open(ruta_destino, mode="w", encoding="utf-8") as archivo:
        json.dump(datos, archivo, indent=2, ensure_ascii=False)
```

5. Programación Orientada a Objetos (OOP) Esencial para DE

La POO se utiliza en ingeniería de datos para modelar clientes de bases de datos, extractores de APIs y transformadores modulares:

PYTHON

```python
class ExtractorDatosAPI:
    """Clase base reutilizable para clientes de ingesta con encapsulamiento de
estado."""
    def __init__(self, endpoint_base: str, api_key: str):
        self.endpoint_base = endpoint_base.rstrip("/")
        self._api_key = api_key  # Atributo protegido para no exponer
credenciales
        self.registros_procesados = 0

    def construir_url(self, recurso: str) -> str:
        return f"{self.endpoint_base}/{recurso.lstrip('/')}"

    def registrar_metrica(self, cantidad: int) -> None:
        self.registros_procesados += cantidad
        print(f"[{self.__class__.__name__}] Total procesado acumulado:
{self.registros_procesados}")
```

6. Aislamiento de Entornos Virtuales y Dependencias con `uv`

Nunca instale paquetes de ingeniería en el entorno global del sistema operativo:

```bash
# 1. Crear entorno virtual moderno y ultrarrápido con uv
uv venv .venv --python 3.11

# 2. Activar el entorno virtual:
# En Linux / macOS / WSL:
source .venv/bin/activate
# En Windows PowerShell:
.venv\Scripts\Activate.ps1

# 3. Instalar librerías de producción:
uv pip install pydantic requests pyarrow polars pytest
```

7. Pruebas Unitarias desde Cero con `pytest`

La ingeniería de software aplicada a datos exige validar que las funciones de transformación respeten sus especificaciones matemáticas antes de promover código a producción:

```python
# tests/test_transformaciones.py
import pytest

def normalizar_pais(pais: str) -> str:
    if not pais or not isinstance(pais, str):
        return "UNKNOWN"
    return pais.strip().upper()

# Casos de prueba automatizados con pytest
def test_normalizar_pais_exitoso():
    assert normalizar_pais("  co ") == "CO"
    assert normalizar_pais("mx") == "MX"

def test_normalizar_pais_nulo_o_invalido():
    assert normalizar_pais(None) == "UNKNOWN"
    assert normalizar_pais("") == "UNKNOWN"

def test_normalizar_pais_tipo_invalido():
    assert normalizar_pais(123) == "UNKNOWN"
```

Para ejecutar las pruebas en la terminal:

```bash
uv run pytest tests/ -v
```

oC.2 Tipado Estático Moderno y Validación con Pydantic

Python es un lenguaje de tipado dinámico, pero la introducción de anotaciones de tipo (*Type Hints*, PEP 484) permite verificar el código estáticamente mediante herramientas como `mypy` y `ruff`, detectando bugs antes del despliegue:

PYTHON

```python
from typing import Optional, List, Dict, Any, Union
from datetime import datetime

def calcular_tasa_churn(
    usuarios_activos: List[int],
    bajas_periodo: List[int],
    factor_ajuste: Optional[float] = None
) -> float:
    if not usuarios_activos:
        return 0.0

    tasa_base = len(bajas_periodo) / len(usuarios_activos)
    if factor_ajuste is not None:
        return tasa_base * factor_ajuste
    return tasa_base
```

Validación en Tiempo de Ejecución con Pydantic (v2)

Las anotaciones de tipo estándar son ignoradas en tiempo de ejecución. **Pydantic** resuelve esta carencia parseando y validando automáticamente estructuras de datos externas (JSONs crudos provenientes de APIs o Kafka):

PYTHON

```python
from pydantic import BaseModel, Field, EmailStr, field_validator
from datetime import date
from typing import Literal

class TransaccionVenta(BaseModel):
    transaccion_id: str = Field(..., description="UUID único de la transacción")
    cliente_id: int = Field(..., gt=0, description="Identificador positivo del
cliente")
    correo: EmailStr
    monto_usd: float = Field(..., ge=0.01, description="Monto mínimo positivo")
    moneda: Literal["USD", "EUR", "COP"] = "USD"
    fecha: date

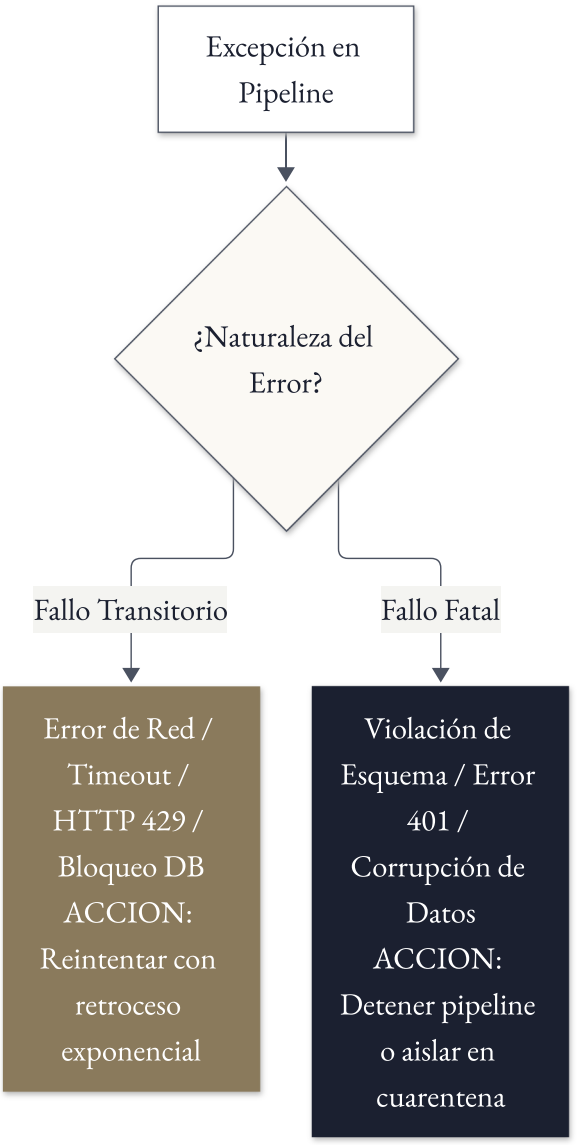
    @field_validator("monto_usd")
    @classmethod
    def validar_monto_maximo(cls, v: float) -> float:
        if v > 1_000_000:
            raise ValueError("Monto excede el límite permitido para procesamiento
automático sin auditoría")
        return round(v, 2)
```

```
# Demostración de parseo y validación estricta
raw_payload = {
    "transaccion_id": "tx-99482-abc",
    "cliente_id": 4501,
    "correo": "cliente.analitica@empresa.com",
    "monto_usd": "1540.508",  # Pydantic coacciona el string a float y aplica el
validador
    "moneda": "USD",
    "fecha": "2026-03-15"
}

try:
    evento_validado = TransaccionVenta.model_validate(raw_payload)
    print(f"Evento válido: ID={evento_validado.transaccion_id}, Monto=
{evento_validado.monto_usd}")
except Exception as e:
    print(f"Error de contrato de datos: {e}")
```

oC.2 Manejo Robusto de Excepciones y Logging Contextual

Un pipeline de datos nunca debe capturar excepciones genéricas silenciosamente (`except: pass`). El manejo de errores debe clasificar las fallas en dos categorías fundamentales:



Excepciones de Dominio Personalizadas y Bloques de Limpieza

PYTHON

```python
import logging
import sys

# Configuración de Logging Estructurado
logging.basicConfig(
    level=logging.INFO,
    format='{"timestamp": "%(asctime)s", "nivel": "%(levelname)s", "modulo": "%
(name)s", "mensaje": "%(message)s"}',
    stream=sys.stdout
)
```

```
logger = logging.getLogger("ingestion_pipeline")

class PipelineDataException(Exception):
    """Excepción base para fallos en el procesamiento de datos."""
    pass

class SchemaContractViolation(PipelineDataException):
    """Fallo fatal: el payload recibido viola el contrato de datos."""
    pass

def procesar_lote_archivos(ruta_archivo: str) -> None:
    manejador_recurso = None
    try:
        logger.info(f"Iniciando apertura de recurso: {ruta_archivo}")
        manejador_recurso = open(ruta_archivo, "r", encoding="utf-8")
        # Lógica de transformación...
    except FileNotFoundError as e:
        logger.error(f"Archivo no localizado en almacenamiento: {ruta_archivo}")
        raise PipelineDataException(f"Recurso inalcanzable: {e}") from e
    except Exception as e:
        logger.critical(f"Fallo imprevisto en la ejecución del lote: {e}",
exc_info=True)
        raise
    finally:
        # Garantía incondicional de liberación de sockets o archivos
        if manejador_recurso and not manejador_recurso.closed:
            manejador_recurso.close()
            logger.info("Recurso liberado exitosamente en bloque finally.")
```

oC.3 Consumo Resiliente de APIs REST: Paginación y Rate Limits

Las APIs transaccionales restringen la cantidad de registros devueltos por petición y limitan la frecuencia de solicitudes mediante cuotas (*Rate Limits*).

1. Estrategias de Paginación

- **Paginación por Desplazamiento (`limit` y `offset`):**
  `GET /api/v1/ventas?limit=100&offset=200`
  Desventaja: Ineficiente para grandes volúmenes ($O(N)$ en la base de datos de origen para saltar filas) y sufre inconsistencias si se insertan nuevas filas mientras se página.
- **Paginación por Cursor (*Cursor-based Pagination*):**
  `GET /api/v1/ventas?limit=100&starting_after=tx_cursor_991823`
  Ventaja: Escanea mediante índices en $O(1)$ y es inmune a inserciones intermedias. Es el estándar en APIs modernas (Stripe, Slack, Shopify).

2. Retroceso Exponencial con Jitter (*Exponential Backoff*)

Cuando un servidor responde con código **HTTP 429 Too Many Requests** o errores de servidor **HTTP 503 Service Unavailable**, bombardearlo con peticiones inmediatas agrava la congestión. Se debe esperar un tiempo exponencial aleatorizado:

$$T_{\text{espera}} = \min(T_{\max}, \text{base} \times 2^{\text{intento}}) + \text{aleatorio}(0, \text{jitter})$$

PYTHON

```python
import time
import random
import requests
from typing import Generator, Dict, Any

class ClienteApiResiliente:
    def __init__(self, url_base: str, token: str, max_reintentos: int = 5):
        self.url_base = url_base
        self.headers = {
            "Authorization": f"Bearer {token}",
            "Accept": "application/json"
        }
        self.max_reintentos = max_reintentos

    def _peticion_con_reintento(self, endpoint: str, params: Dict[str, Any]) ->
Dict[str, Any]:
        url = f"{self.url_base}/{endpoint}"
        intento = 0
        tiempo_espera = 1.0  # Segundo base

        while intento < self.max_reintentos:
            try:
                respuesta = requests.get(url, headers=self.headers,
params=params, timeout=10.0)

                # Éxito
                if respuesta.status_code == 200:
                    return respuesta.json()

                # Manejo de Rate Limit (HTTP 429) o fallos transitorios de
servidor (5xx)
                if respuesta.status_code in (429, 500, 502, 503, 504):
                    intento += 1
                    # Respetar encabezado Retry-After si el servidor lo provee
                    reintento_sugerido = respuesta.headers.get("Retry-After")
                    if reintento_sugerido:
                        demora = float(reintento_sugerido)
                    else:
                        jitter = random.uniform(0.1, 0.5)
                        demora = min(60.0, tiempo_espera * (2 ** intento)) +
jitter

                    time.sleep(demora)
```

```
                    continue

                # Errores fatales de cliente (400, 401, 403, 404)
                respuesta.raise_for_status()

            except (requests.ConnectionError, requests.Timeout) as e:
                intento += 1
                if intento >= self.max_reintentos:
                    raise ConnectionError(f"Se agotaron los reintentos hacia
{url}: {e}") from e
                time.sleep(tiempo_espera * (2 ** intento) + random.uniform(0.1,
0.5))

        raise RuntimeError(f"Fallo definitivo al consultar {url} tras
{self.max_reintentos} intentos.")

    def extraer_todos_los_eventos_por_cursor(self) -> Generator[Dict[str, Any],
None, None]:
        """Extrae la totalidad de los datos utilizando paginación por
cursores."""
        cursor_actual = None
        tiene_mas_datos = True

        while tiene_mas_datos:
            parametros = {"limit": 100}
            if cursor_actual:
                parametros["starting_after"] = cursor_actual

            payload = self._peticion_con_reintento("eventos", parametros)
            registros = payload.get("data", [])

            for registro in registros:
                yield registro

            tiene_mas_datos = payload.get("has_more", False)
            if tiene_mas_datos and registros:
                cursor_actual = registros[-1]["id"]
```

oC.4 Empaquetado Moderno y Entornos Deterministas con `uv`

Durante años, el ecosistema de Python dependió de `virtualenv`, `pip` y `requirements.txt` sin resolver adecuadamente el bloqueo determinista de dependencias indirectas.

La Revolución de `uv` (Astral)

Desarrollado en Rust, `uv` es un administrador de paquetes de última generación que resuelve e instala dependencias entre **10 y 100 veces más rápido que pip**, soportando nativamente el estándar moderno `pyproject.toml` (PEP 517/518/621) y lockfiles universales (`uv.lock`).

TOML

```toml
# Archivo: pyproject.toml
[project]
name = "plataforma-datos-lakehouse"
version = "0.1.0"
description = "Pipelines productivos de ingesta y transformación distribuida"
authors = [{ name = "Luis Galvis", email = "autor@empresa.com" }]
requires-python = ">=3.11"
dependencies = [
    "pyspark==3.5.2",
    "delta-spark==3.2.0",
    "pydantic>=2.7.0",
    "requests>=2.31.0",
    "tenacity>=8.2.3",
    "pyarrow>=15.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "chispa>=0.9.4",
    "ruff>=0.4.0",
    "mypy>=1.9.0",
]
```

Comandos Esenciales con `uv` en Producción

BASH

```bash
# Crear entorno virtual ultrarrápido con versión específica de Python
uv venv .venv --python 3.11

# Instalar dependencias bloqueadas de forma determinista
uv pip sync uv.lock

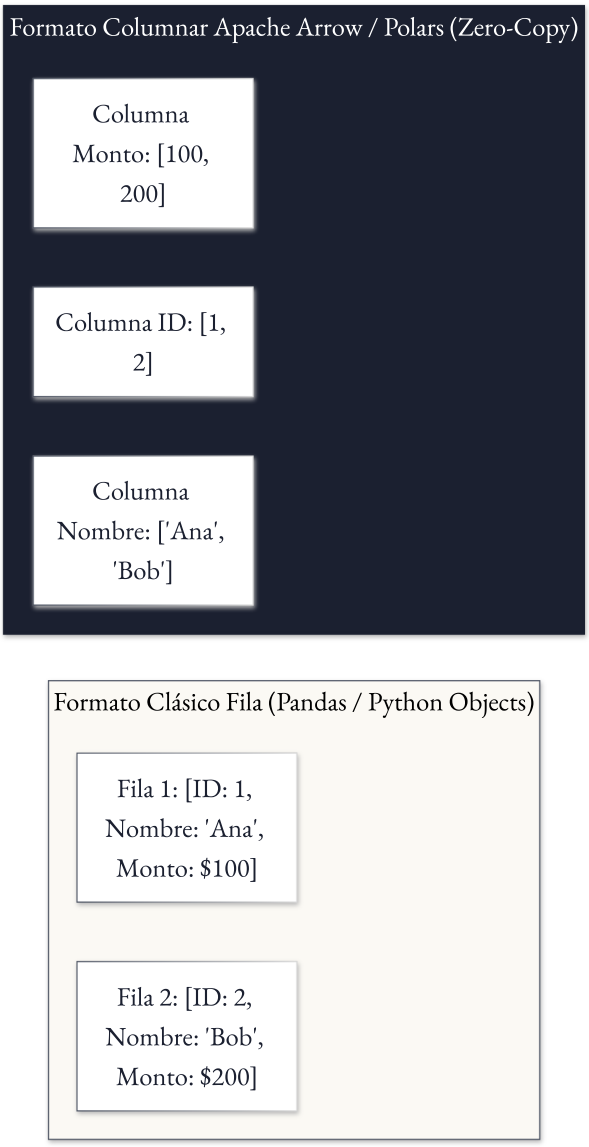
# Compilar un lockfile determinista a partir de pyproject.toml
uv pip compile pyproject.toml -o uv.lock

# Ejecutar herramientas de linteado y tipado estático
uv run ruff check src/
uv run mypy src/
```

---

oC.6 Manipulación Tabular en Memoria: Pandas, Polars y Apache Arrow

Antes de escalar a clústeres de Spark distribuidos, el procesamiento en un solo nodo se ejecuta con **Pandas**, **Polars** y **Apache Arrow**:



- **Apache Arrow:** Es el estándar en memoria para datos tabulares columnares. Ofrece aceleración de instrucciones SIMD (*Single Instruction, Multiple Data*) a nivel de CPU y elimina el costo de serialización (*Zero-Copy*) al transferir datos entre Python, Spark, DuckDB y C++.
- **PyArrow en Ingesta:** Almacena esquemas fuertemente tipados con soporte nativo para compresión y escritura directa a archivos Parquet:

PYTHON

```python
import pyarrow as pa
import pyarrow.parquet as pq

# Definición de esquema estricto en Apache Arrow
esquema_arrow = pa.schema([
    ("transaccion_id", pa.string()),
    ("cliente_id", pa.int64()),
    ("monto_usd", pa.float64()),
    ("fecha", pa.date32())
])

datos = {
    "transaccion_id": ["tx-001", "tx-002"],
    "cliente_id": [101, 102],
    "monto_usd": [450.50, 89.20],
    "fecha": [pa.array(["2026-03-01", "2026-03-02"]).cast(pa.date32())]
}

tabla_arrow = pa.Table.from_pydict(datos, schema=esquema_arrow)

# Escritura directa a disco en formato Parquet con compresión Snappy
pq.write_table(tabla_arrow, "ventas_lote.parquet", compression="snappy")
```

Polars en la Práctica: Rendimiento Multihilo y Evaluación Perezosa

Mientras Pandas opera de forma ansiosa (*Eager Execution*) sobre un único hilo de CPU, **Polars** (escrito en Rust sobre Apache Arrow) es el estándar moderno para transformación de datos en una sola máquina:

1. **Paralelismo Nativo:** Ejecuta transformaciones en paralelo a través de todos los núcleos disponibles, sorteando las limitaciones del Global Interpreter Lock (GIL) de Python.
2. **Evaluación Perezosa (*LazyFrames*):** Optimiza el plan de consulta fusionando filtros (*Predicate Pushdown*) y proyectando únicamente las columnas requeridas (*Projection Pushdown*) antes de leer los datos de disco.
3. **Motor Streaming (*Streaming Engine*):** Procesa archivos Parquet de decenas de gigabytes que exceden la memoria RAM de la máquina procesándolos en micro-lotes sin desbordar el sistema:

```python
import polars as pl

# 1. Definir consulta perezosa con LazyFrame (no ejecuta nada en memoria todavía)
q_lazy = (
    pl.scan_parquet("data/ventas.parquet")
    .filter(pl.col("estado_pago") == "COMPLETADO")
    .filter(pl.col("monto_usd") > 50.0)
    .group_by("pais", "categoria_producto")
    .agg([
        pl.len().alias("total_transacciones"),
        pl.col("monto_usd").sum().round(2).alias("ingresos_totales"),
        pl.col("monto_usd").mean().round(2).alias("ticket_promedio")
    ])
    .sort("ingresos_totales", descending=True)
)

# 2. Inspeccionar el plan de ejecución optimizado
print(q_lazy.explain())

# 3. Materializar el resultado (o usar collect(streaming=True) si los datos
superan la RAM)
df_resultado = q_lazy.collect()
print(df_resultado.head(5))
```

oC.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cargar Archivos Masivos con `pandas.read_csv()` en Producción
Pandas crea objetos de Python con alto sobrecosto en memoria: un archivo CSV de 5 GB puede llegar a consumir 25 GB de RAM física, provocando la terminación del proceso por el sistema operativo (*OOM Killer*).
Corrección: Procese archivos en fragmentos (*chunks* con `chunksize`), utilice DuckDB/PyArrow, o migre directamente a Apache Spark.

Anti-patrón 2: Capturar `Exception` sin Registrar el Traceback
Escribir `except Exception as e: logger.error(e)` oculta el número de línea, la jerarquía de llamadas y el tipo exacto de error, imposibilitando la depuración en sistemas distribuidos.
Corrección: Incluya siempre `exc_info=True` en la llamada a `logger.error()` o utilice `logger.exception()`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0C.1 (Generador de Extracción con Manejo de Errores):
Diseñe una función en Python con tipado estático completo que lea un archivo JSON línea a línea (*JSON Lines / NDJSON*) utilizando un generador (`yield`). La función debe validar cada línea contra un modelo de Pydantic. Si una línea presenta corrupción sintáctica o violación de esquema, debe registrar un mensaje de advertencia y continuar procesando las siguientes líneas sin interrumpir el flujo.

Solución 0C.1:

PYTHON

```python
import json
import logging
from typing import Generator, Optional
from pydantic import BaseModel, ValidationError

class RegistroEvento(BaseModel):
    evento_id: str
    usuario_id: int
    accion: str

def extraer_eventos_ndjson(ruta_archivo: str) -> Generator[RegistroEvento, None,
None]:
    with open(ruta_archivo, "r", encoding="utf-8") as f:
        for numero_linea, linea in enumerate(f, start=1):
            linea_limpia = linea.strip()
            if not linea_limpia:
                continue

            try:
                payload = json.loads(linea_limpia)
                evento = RegistroEvento.model_validate(payload)
                yield evento
            except json.JSONDecodeError as err_json:
                logging.warning(f"Línea {numero_linea} corrupta (JSON inválido):
{err_json}")
            except ValidationError as err_val:
                logging.warning(f"Línea {numero_linea} viola el contrato de
datos: {err_val}")
```

---

oC.7 Resumen y Puntos Clave

- **Tipado estático y Pydantic:** Transforman Python en un entorno robusto para validar contratos de datos en las fronteras de entrada del pipeline.
- **Manejo de errores por severidad:** Los fallos transitorios se reintentan con retroceso exponencial; los fallos fatales se aíslan para no corromper capas posteriores.

- **APIs resilientes:** El consumo de servicios externos exige paginación por cursores, control estricto de timeouts y respeto a los encabezados de *Rate Limit*.
- **Empaquetado moderno con `uv`:** Garantiza la resolución determinista de dependencias con velocidades de instalación órdenes de magnitud superiores a las herramientas tradicionales.
- **Apache Arrow en memoria:** Proporciona un formato columnar de alto rendimiento y transferencias *zero-copy* entre subsistemas de datos.

Lecturas Recomendadas

1. **Ramalho, L. (2022).** *Fluent Python: Clear, Concise, and Effective Programming (2nd Edition).* O'Reilly Media.
2. **Colvin, S. et al. (2024).** *Pydantic V2 Documentation and Architecture.* Pydantic Documentation.
3. **Apache Arrow Project (2024).** *Apache Arrow: A Cross-Language Development Platform for In-Memory Data.* Apache Software Foundation.
4. **Astral Software (2024).** *uv: Extremely Fast Python Package Installer and Resolver.* Astral Documentation.

CAPÍTULO oD

Entorno de Laboratorio Reproducible y Generación de Datos NÚCLEO

> *"Un libro técnico de ingeniería que propone código sobre archivos inexistentes sin ofrecer un laboratorio reproducible es un ejercicio de frustración. Para que el conocimiento arraigue, cada consulta SQL, cada script de PySpark y cada pipeline de orquestación presentado en esta obra debe ser ejecutable y verificable en su propio computador sin costo alguno."*

oD.o El Repositorio Oficial del Libro

Todo el código fuente de los cincuenta y dos capítulos, los archivos de orquestación de Docker Compose, las suites de pruebas de pytest y el generador determinista de datos están versionados en el repositorio oficial de código abierto:

```bash
# 1. Clonar el repositorio oficial
git clone https://github.com/luisgalvis/como-ser-un-data-engineer.git

# 2. Ingresar al directorio del proyecto
cd como-ser-un-data-engineer
```

Enlace directo al repositorio: **https://github.com/luisgalvis/como-ser-un-data-engineer**

Objetivos de Aprendizaje

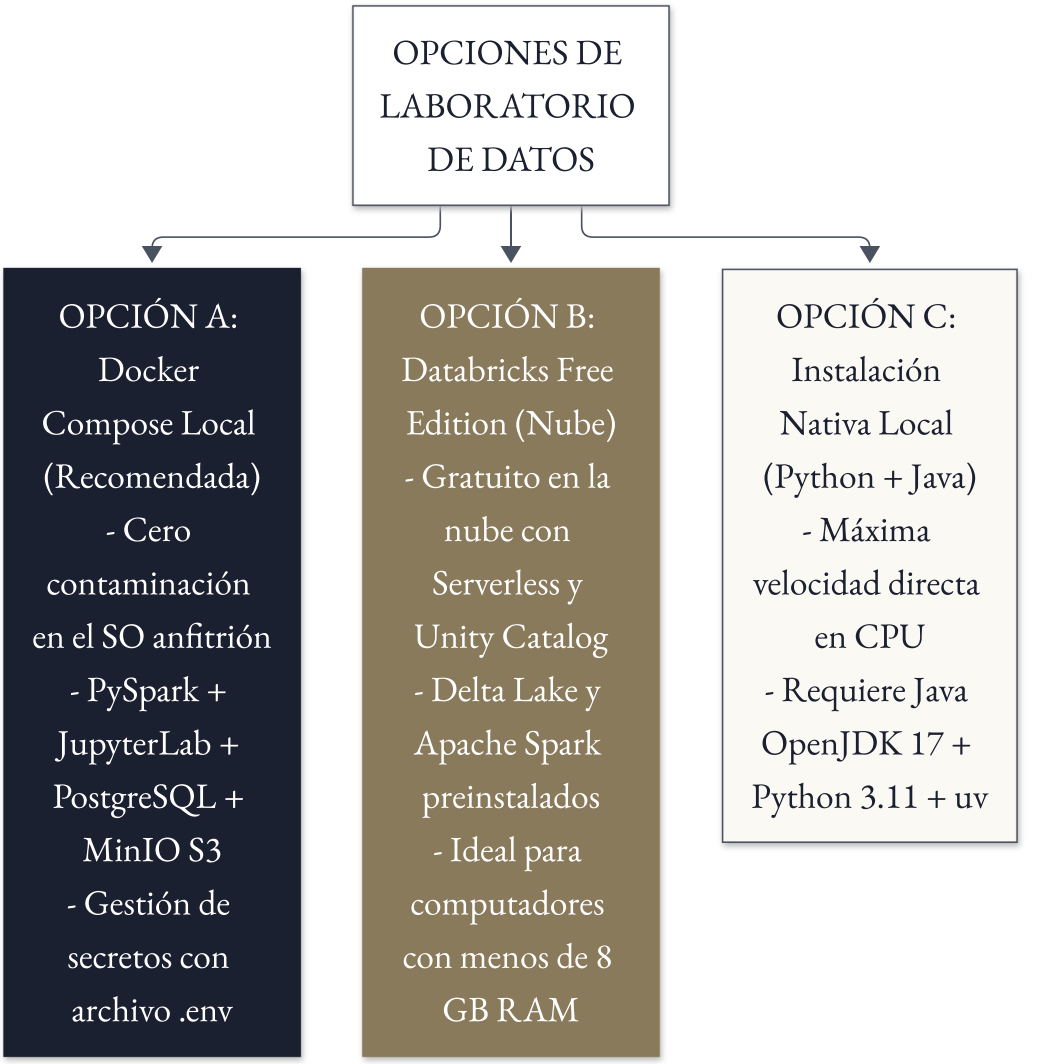
Al finalizar este capítulo, el lector será capaz de:

- Configurar un entorno de ingeniería de datos local y aislado mediante **Docker Compose**, o en la nube mediante **Databricks Free Edition**.
- Aplicar buenas prácticas de seguridad operativa segregando secretos y credenciales en un archivo `.env` local sin exponer contraseñas en control de versiones.
- Configurar entornos Windows mediante **WSL2 (Windows Subsystem for Linux)** y ajustar la gestión de memoria RAM en `.wslconfig` para evitar saturación de recursos.
- Ejecutar el generador oficial de datos (`python scripts/generar_datos.py`) para materializar en disco la suite completa de los 8 datasets de prueba utilizados a lo largo del libro.
- Validar la conectividad de Apache Spark y la lectura de archivos Parquet y JSON semiestructurados mediante un script automatizado de auditoría.

Definición 0D.1 (El Principio de Reproducibilidad en la Ingeniería)
La reproducibilidad computacional exige que cualquier ingeniero, en cualquier sistema operativo, pueda clonar el repositorio del proyecto, ejecutar un único comando de configuración y obtener idénticos esquemas, registros y resultados numéricos sin depender de archivos propietarios o configuraciones opacas.

0D.1 Opciones de Entorno de Laboratorio

El lector puede elegir la alternativa que mejor se adapte a su infraestructura disponible:



Opción A: Laboratorio Completo con Docker Compose (Recomendada)

La especificación moderna de Docker Compose (sin la directiva obsoleta `version: '3.8'`) permite levantar la infraestructura completa con variables de entorno desacopladas.

1. Archivo de Variables de Entorno (`.env.example` → `.env`)

Cree un archivo `.env` en la raíz del proyecto para no exponer contraseñas en texto plano:

INI

```ini
# .env - Variables de configuración del laboratorio local
POSTGRES_DB=laboratorio_db
POSTGRES_USER=data_engineer
POSTGRES_PASSWORD=LabPasswordSeguro2026!
POSTGRES_PORT=5432

MINIO_ROOT_USER=minio_admin
MINIO_ROOT_PASSWORD=MinioSecretPassword2026!
MINIO_API_PORT=9000
MINIO_CONSOLE_PORT=9001

JUPYTER_PORT=8888
SPARK_UI_PORT=4040
```

2. Definición de Servicios (`docker-compose.yml`)

YAML

```yaml
# docker-compose.yml (Especificación Docker Compose Moderna)
services:
  # Entorno interactivo con Apache Spark 3.5 y Delta Lake 3.2
  spark-jupyter:
    image: quay.io/jupyter/pyspark-notebook:spark-3.5.0
    container_name: lab_spark_notebook
    environment:
      JUPYTER_ENABLE_LAB: "yes"
      PYSPARK_SUBMIT_ARGS: "--packages io.delta:delta-spark_2.12:3.2.0 pyspark-
shell"
    ports:
      - "${JUPYTER_PORT:-8888}:8888"   # JupyterLab Web UI
      - "${SPARK_UI_PORT:-4040}:4040"   # Spark Web UI
    volumes:
      - ./libro:/home/jovyan/work/libro
      - ./data:/home/jovyan/work/data
      - ./scripts:/home/jovyan/work/scripts
    networks:
      - red_laboratorio

  # Base de datos relacional para ejercicios OLTP, JDBC y CDC
  postgres:
    image: postgres:16-alpine
    container_name: lab_postgres
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - red_laboratorio

  # MinIO: Almacenamiento de Objetos S3 local (Data Lake)
```

```
  minio:
    image: quay.io/minio/minio:RELEASE.2024-01-31T20-20-33Z
    container_name: lab_minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    ports:
      - "${MINIO_API_PORT:-9000}:9000"
      - "${MINIO_CONSOLE_PORT:-9001}:9001"
    volumes:
      - miniodata:/data
    networks:
      - red_laboratorio

volumes:
  pgdata:
  miniodata:

networks:
  red_laboratorio:
    driver: bridge
```

- **Instrucciones de Despliegue:**

  ```bash
  # Copiar plantilla de variables
  cp .env.example .env

  # Iniciar todos los contenedores en segundo plano
  docker compose up -d
  ```

  Acceda a `http://localhost:8888` en su navegador para interactuar con JupyterLab y ejecutar notebooks de Spark.

Guía Especial para Usuarios de Windows y WSL2

Para desarrolladores en Windows, la experiencia más estable y cercana a producción se obtiene combinando **Docker Desktop** con **WSL2 (Windows Subsystem for Linux)**:

1. **Habilitar WSL2 y Ubuntu:**
   Abra PowerShell como Administrador y ejecute:

   ```powershell
   wsl --install -d Ubuntu-24.04
   ```

2. **Configuración de Límites de Memoria (`.wslconfig`):**
   Por defecto, WSL2 puede llegar a consumir el 80% de la memoria física del equipo durante

operaciones pesadas de Spark. Para evitar que el sistema operativo Windows se ralentice, cree el archivo `C:\Users\\.wslconfig`:

```ini
[wsl2]
memory=8GB            # Límite máximo de RAM asignada a WSL2
processors=4          # Límite de núcleos de CPU
swap=4GB              # Archivo de paginación swap
localhostForwarding=true
```

Reinicie WSL2 desde PowerShell ejecutando: `wsl --shutdown`.

3. **Manejo de Saltos de Línea (CRLF vs LF):**
   Los scripts shell de Linux (`.sh`) fallan con errores crípticos del tipo `\r: command not found` si Git los descarga con saltos de línea de Windows (`CRLF`). Configure Git para respetar saltos de línea Unix:

```bash
git config --global core.autocrlf input
```

4. **Ejecutar dentro del File System de Linux:**
   Para obtener el máximo rendimiento de lectura y escritura en disco (I/O), clone el repositorio dentro del sistema de archivos de WSL2 (ej. `/home/su_usuario/proyectos/`), en lugar de montarlo desde `/mnt/c/`.

Opción B: Databricks Free Edition (Nube sin Costo)

Si su computador dispone de recursos limitados (menos de 8 GB de RAM) o desea experimentar directamente con una plataforma de datos moderna:

1. Regístrese en el portal oficial de **Databricks Free Edition** (`https://community.cloud.databricks.com/` o `https://www.databricks.com/try-databricks`).
2. Cree un clúster de trabajo (*Single Node*, Databricks Runtime 15.4+ LTS con Apache Spark 3.5+ y Delta Lake preinstalado).
3. Clone el repositorio oficial directamente en Databricks utilizando **Workspace Git Folders / Repos**.
4. Trabaje sobre el catálogo predeterminado de **Unity Catalog** (`workspace.default`), familiarizándose con la gobernanza empresarial y linaje desde el primer día.

Opción C: Instalación Nativa Local (Python + PySpark)

Para ejecutar los scripts directamente en su terminal local (Linux, macOS o WSL2):

1. **Instalar Java OpenJDK 17 (Requisito fundamental de la JVM de Spark):**

```bash
# En Ubuntu / Debian / WSL2:
sudo apt-get update && sudo apt-get install -y openjdk-17-jdk-headless
```

2. **Crear el entorno virtual con `uv` e instalar dependencias:**

```bash
uv venv .venv --python 3.11
source .venv/bin/activate  # En Windows: .venv\Scripts\activate
uv pip install pyspark==3.5.2 delta-spark==3.2.0 pyarrow duckdb polars pytest
```

---

oD.2 Generación Oficial de Datasets de Prueba

Para garantizar que los archivos de datos existan en disco y contengan exactamente la distribución matemática descrita en los capítulos de optimización, joins y calidad, ejecute el generador oficial determinista:

```bash
python scripts/generar_datos.py
```

Inventario de Datasets Generados en `data/`

Archivo	Volumen	Formato	Propósito Pedagógico en el Libro
`data/ventas.parquet`	10.000 filas	Parquet / Snappy	Transacciones de comercio electrónico. Base para filtros, agregaciones y particionado temporal.
`data/clientes.parquet`	1.500 filas	Parquet / Snappy	Catálogo maestro de clientes. Empleado en uniones relacionales (Broadcast Joins) y SCD Tipo 2.
`data/eventos_telemetria.json`	1.000 eventos	JSON semiestructurado	Cargas de IoT y clickstream con campos anidados para prácticas de parsing de esquemas.
`data/interacciones.parquet`	25.000 filas	Parquet / Snappy	Eventos de navegación web (clickstream) para análisis de sesiones y funciones de ventana.
`data/usuarios.parquet`	2.000 filas	Parquet / Snappy	Datos con información de identificación personal (PII) para ejercicios de GDPR, enmascaramiento y Ley 1581.
`data/pedidos.parquet`	20.000 filas	Parquet / Snappy	Transacciones detalladas de ventas para indexación, modelado dimensional y joins pesados.
`data/eventos_anuales.parquet`	50.000 filas	Parquet / Snappy	Eventos históricos distribuidos en múltiples años para demostrar poda de particiones (*Partition Pruning*).
`data/transacciones_sesgadas.parquet`	100.000 filas	Parquet / Snappy	Dataset con **sesgo extremo de datos (85% Zipf)** y payload de 256 bytes para reproducir cuellos de botella y *Shuffle Spill* en la interfaz Spark UI.

oD.3 Script de Verificación de Integridad del Laboratorio

Para cerciorarse de que su sesión de Apache Spark está correctamente configurada y puede procesar los archivos generados sin problemas de memoria ni permisos, ejecute:

PYTHON

```python
# scripts/verificar_laboratorio.py
from pyspark.sql import SparkSession

def verificar_laboratorio():
    print("Iniciando auditoría del entorno de Apache Spark...")
    spark = SparkSession.builder \
        .appName("AuditoriaLaboratorio") \
```

```
        .master("local[*]") \
        .config("spark.sql.shuffle.partitions", "4") \
        .getOrCreate()

    print(f"Versión de Apache Spark activa: {spark.version}")

    ruta_ventas = "data/ventas.parquet"
    df_ventas = spark.read.parquet(ruta_ventas)

    total_filas = df_ventas.count()
    print(f"Archivo '{ruta_ventas}' leído exitosamente. Total registros:
{total_filas}")

    df_ventas.createOrReplaceTempView("ventas_test")
    resumen = spark.sql("""
        SELECT pais, COUNT(transaccion_id) as total_tx, ROUND(SUM(monto_usd), 2)
as facturacion
        FROM ventas_test
        GROUP BY pais
        ORDER BY facturacion DESC
        LIMIT 3
    """)
    resumen.show()
    print("¡Laboratorio verificado y 100% operativo para el estudio del libro!")

if __name__ == "__main__":
    verificar_laboratorio()
```

oD.4 Resumen y Puntos Clave

- **Cero código sobre el vacío:** Todo el código del libro es inmediatamente reproducible clonando el repositorio oficial de GitHub: `https://github.com/luisgalvis/como-ser-un-data-engineer`.
- **Seguridad desde el primer día:** Docker Compose utiliza la especificación moderna con variables de entorno desacopladas en `.env`, eliminando credenciales en texto plano.
- **Compatibilidad Windows:** La configuración de WSL2 y límites de memoria en `.wslconfig` protege el sistema anfitrión frente a bloqueos de recursos.
- **Suite completa de datos:** El script `scripts/generar_datos.py` produce deterministamente 8 datasets calibrados para reproducir desde agregaciones básicas hasta *spills* severos por sesgo de datos (*data skew*).

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide (Chapter 2: A Gentle Introduction to Spark).* O'Reilly Media.

2. **Docker Documentation (2024).** *The Compose Specification & Environment Variables in Compose.* Docker Docs.
3. **Microsoft Learn (2024).** *Advanced Settings Configuration in WSL2 (.wslconfig).* Microsoft Documentation.
4. **Databricks Learning (2024).** *Databricks Free Edition & Unity Catalog Quickstart.* Databricks Documentation.

CAPÍTULO 1

Fundamentos de SQL Declarativo y Computación Analítica NÚCLEO

> *"SQL en la ingeniería de datos no es una simple sintaxis para consultar tablas estáticas; es la interfaz declarativa universal que permite expresar qué resultado matemático se requiere, delegando en el motor computacional la generación del plan de ejecución físico óptimo."*

1.1 Intuición Inicial: Pedir un Café en Lugar de Operar la Máquina

Cuando entras a una cafetería, le dices al barista: «*Por favor, un café americano grande, bien caliente y sin azúcar*». No necesitas entrar a la cocina a calibrar el molinillo de granos, medir los gramos de café molido, hervir el agua a 92 grados centígrados ni bombear la presión del agua a través del filtro. Describes con exactitud el resultado final que deseas (**QUÉ quieres**), y el barista se encarga de ejecutar la secuencia mecánica precisa (**CÓMO prepararlo**).

SQL (Structured Query Language) funciona exactamente bajo este principio declarativo. En lugar de programar bucles `for`, abrir archivos en bajo nivel, asignar punteros de memoria y recorrer registros uno a uno (como se haría en C o Java imperativo), en SQL describes las condiciones lógicas de los datos que necesitas. El motor de la base de datos (PostgreSQL, DuckDB o Apache Spark) dispone de un optimizador interno —el barista experto— que analiza los índices, el tamaño físico de las tablas y la memoria disponible para construir el plan de ejecución más rápido posible.

Si comienzas desde cero, este capítulo te guiará paso a paso: desde seleccionar tus primeras columnas y filtrar filas, hasta realizar uniones relacionales complejas, resolver casos de lógica trivaluada con valores nulos e introducir los fundamentos del modelado dimensional analítico.

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la diferencia fundamental entre el paradigma declarativo de SQL y la programación procedural/imperativa.
- Escribir consultas SQL analíticas desde cero (`SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`) dominando el orden lógico de ejecución del motor.
- Aplicar con precisión todos los tipos de uniones relacionales (`INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`, `LEFT SEMI`, `LEFT ANTI`) y gestionar los efectos de la lógica trivaluada ante valores `NULL`.
- Construir transformaciones modulares legibles mediante Expresiones de Tabla Comunes (CTEs) y CTEs recursivas.
- Dominar operaciones de sincronización e inserción atómica mediante `MERGE INTO`.
- Comprender los principios del **Modelado Dimensional de Ralph Kimball** (tablas de hechos, dimensiones y esquemas en estrella) frente a la normalización transaccional 3NF.

Profundiza: El Fundamento Formal del Álgebra y Cálculo Relacional

Definición 1.1 (Cálculo Relacional y Paradigma Declarativo)
En la teoría matemática de bases de datos formulada por Edgar F. Codd (1970), SQL se fundamenta en el **cálculo relacional de tuplas**: el ingeniero especifica el predicado analítico y la proyección relacional deseada (*qué calcular*), abstrayéndose por completo de la secuencia de pasos mecánicos (*cómo calcularlo*).

En contraste con la programación imperativa —donde el programador administra bucles, punteros de memoria y buffers de red—, el paradigma declarativo traslada la carga de optimización al planificador del motor (Catalyst en Spark, optimizadores basados en costos en PostgreSQL o DuckDB). Este analiza la consulta, evalúa reglas de equivalencia algebraica y compila un árbol de ejecución físico adaptado a los recursos disponibles.

1.2 Consultas SQL desde Cero: Anatomía y Orden Lógico de Ejecución

Para un ingeniero de datos, dominar SQL va más allá de recordar palabras clave: requiere asimilar el modelo relacional formal propuesto por Edgar F. Codd y comprender el orden exacto en que los motores de bases de datos compilan y ejecutan las instrucciones.

1. El Modelo Relacional Básico

- **Relación (Tabla):** Conjunto no ordenado de tuplas únicas con un esquema definido.
- **Tupla (Fila o Registro):** Instancia única de una entidad en el mundo real.
- **Atributo (Columna):** Dimensión con un tipo de dato y restricciones específicas.

2. Proyección y Filtrado Básico (`SELECT` y `WHERE`)

La consulta elemental extrae y transforma columnas de una tabla:

```sql
-- Selección de columnas específicas con alias y transformaciones aritméticas
SELECT
    transaccion_id,
    cliente_id,
    pais,
    cantidad * precio_unitario AS subtotal_usd,
    fecha_transaccion
FROM ventas_analitica.ventas
WHERE estado_pago = 'COMPLETADO'
  AND (pais = 'CO' OR pais = 'MX')
  AND fecha_transaccion >= '2026-01-01'
  AND monto_usd BETWEEN 50.0 AND 5000.0;
```

Predicados y Operadores de Filtrado Esenciales

- **Igualdad y Desigualdad:** `=`, `<>` (o `!=`).
- **Comparaciones:** `>`, `<`, `>=`, `<=`.
- **Conjuntos y Rangos:** `IN ('CO', 'PE', 'CL')`, `BETWEEN 10 AND 50`.
- **Coincidencia de Patrones de Texto:** `LIKE 'TX-%'` (empieza con TX-), `LIKE '%@empresa.com'`.
- **Evaluación de Nulos:** `IS NULL` y `IS NOT NULL` (nunca use `= NULL`).

3. Agrupaciones y Filtrado Post-Agregación (`GROUP BY` y `HAVING`)

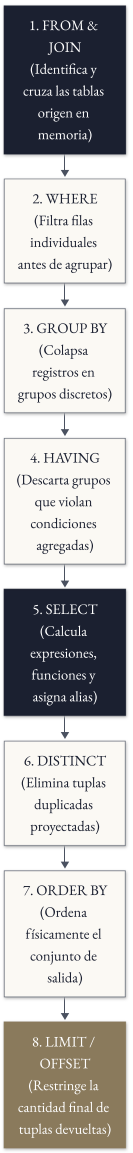
Una de las confusiones más comunes al iniciar es distinguir el momento de filtrado de filas frente al de grupos:

- `WHERE`: Filtra **filas individuales antes** de que se realice cualquier agrupación o cálculo de agregación.
- `GROUP BY`: Colapsa múltiples filas que comparten valores idénticos en una sola fila resumen por grupo.
- `HAVING`: Filtra **grupos resultantes después** de haber calculado las funciones de agregación (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`).

```sql
SELECT
    pais,
    categoria_producto,
    COUNT(transaccion_id) AS total_pedidos,
    ROUND(SUM(monto_usd), 2) AS facturacion_total,
    ROUND(AVG(monto_usd), 2) AS ticket_promedio
FROM ventas_analitica.ventas
WHERE estado_pago = 'COMPLETADO'      -- 1. Filtra filas individuales primero
GROUP BY pais, categoria_producto     -- 2. Agrupa por país y categoría
HAVING COUNT(transaccion_id) >= 100  -- 3. Conserva solo grupos con más de 100
ventas
   AND SUM(monto_usd) > 50000.0
ORDER BY facturacion_total DESC       -- 4. Ordena el resultado agregado
LIMIT 10;                             -- 5. Extrae los 10 primeros registros
```

4. El Orden Lógico de Procesamiento de una Consulta SQL

Un error habitual es creer que la base de datos ejecuta la consulta en el orden en que se escribe (`SELECT` primero). En realidad, el optimizador procesa la consulta en una secuencia lógica matemática estricta:



> **Por qué importa esto al depurar:** Debido a que `WHERE` se evalúa en el paso 2 y `SELECT` se evalúa en el paso 5, **no es posible usar un alias definido en el `SELECT` dentro de la cláusula `WHERE`** (provocará un error de columna desconocida). Comprender este orden evita horas de frustración al estructurar pipelines.

1.3 Lenguaje de Definición (DDL) y Manipulación de Datos (DML)

Toda plataforma analítica interactúa con esquemas relacionales a través de dos subsistemas fundamentales:

1. DDL (Data Definition Language)

Permite definir, alterar y destruir estructuras tabulares, tipos de datos y restricciones de integridad:

```SQL
-- Creación de esquema y tabla analítica con restricciones (PostgreSQL / DuckDB / Databricks)
CREATE SCHEMA IF NOT EXISTS ventas_analitica;

CREATE TABLE ventas_analitica.clientes (
    cliente_id BIGINT NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    correo VARCHAR(200) NOT NULL,
    pais_iso CHAR(2) NOT NULL,
    segmento VARCHAR(50) DEFAULT 'ESTANDAR',
    fecha_registro DATE NOT NULL,
    CONSTRAINT pk_clientes PRIMARY KEY (cliente_id)
);
```

2. DML (Data Manipulation Language) y la Operación Atómica `MERGE INTO`

El DML gobierna la inserción, actualización y eliminación de registros:

```SQL
-- Inserción directa de registros
INSERT INTO ventas_analitica.clientes (cliente_id, nombre, correo, pais_iso, segmento, fecha_registro)
VALUES
    (101, 'Ana Torres', 'ana.torres@empresa.com', 'CO', 'ENTERPRISE', '2025-01-15'),
    (102, 'Carlos Ruiz', 'carlos.ruiz@empresa.com', 'MX', 'ESTANDAR', '2025-02-20'),
    (103, 'Elena Mora', 'elena.mora@empresa.com', 'CO', 'PRO', '2025-03-05');
```

En pipelines analíticos modernos, la inserción y actualización manual (`UPDATE` y `DELETE` tradicionales) son sustituidas por la sentencia atómica **`MERGE INTO`** (estándar ANSI SQL:2006 y pilar central de formatos Lakehouse como Delta Lake e Iceberg):

```sql
-- Sincronización atómica tipo Upsert (Update + Insert) usando MERGE INTO
MERGE INTO ventas_analitica.clientes AS destino
USING ventas_analitica.novedades_staging AS origen
ON destino.cliente_id = origen.cliente_id
WHEN MATCHED AND origen.correo <> destino.correo THEN
    UPDATE SET
        destino.correo = origen.correo,
        destino.segmento = origen.segmento
WHEN NOT MATCHED THEN
    INSERT (cliente_id, nombre, correo, pais_iso, segmento, fecha_registro)
    VALUES (origen.cliente_id, origen.nombre, origen.correo, origen.pais_iso,
origen.segmento, origen.fecha_registro);
```

1.4 Expresiones Condicionales, Funciones de Fecha y JSON

Lógica Condicional con `CASE WHEN` y Manejo de Nulos

En SQL, la sentencia `CASE` es una expresión escalar determinista:

```sql
SELECT
    cliente_id,
    monto_usd,
    CASE
        WHEN monto_usd >= 10000 THEN 'ALTO VALOR'
        WHEN monto_usd >= 1000 THEN 'VALOR MEDIO'
        ELSE 'MINORISTA'
    END AS clasificacion_cliente,
    COALESCE(descuento_usd, 0.0) AS descuento_aplicado,
    NULLIF(dias_mora, 0) AS mora_efectiva -- Convierte ceros en NULL para no
distorsionar promedios
FROM ventas_analitica.facturacion;
```

Aritmética de Fechas y Marcas de Tiempo

Las transformaciones analíticas requieren manipular dimensiones temporales con precisión de zona horaria:

```sql
SELECT
    pedido_id,
    fecha_hora_creacion,
    -- Truncamiento a inicio de mes
    DATE_TRUNC('month', fecha_hora_creacion) AS mes_periodo,
    -- Extracción de componentes
    EXTRACT(YEAR FROM fecha_hora_creacion) AS anio_fiscal,
    -- Diferencia temporal en días
    CAST(fecha_entrega AS DATE) - CAST(fecha_hora_creacion AS DATE) AS
dias_transito,
    -- Suma de intervalos
    fecha_hora_creacion + INTERVAL '30 days' AS vencimiento_garantia
FROM ventas_analitica.pedidos;
```

Procesamiento de Estructuras Jerárquicas JSON

En arquitecturas de ingesta moderna, los eventos provienen de APIs o Kafka en formato JSON. SQL analítico provee operadores especializados:

```sql
-- Sintaxis PostgreSQL (operadores de ruta jsonb -> y ->>)
SELECT
    evento_id,
    payload_json->>'usuario_id' AS usuario_id,
    payload_json->'dispositivo'->>'sistema_operativo' AS so_cliente,
    CAST(payload_json->>'monto' AS NUMERIC(10,2)) AS monto_transaccion
FROM ventas_analitica.eventos_raw;

-- Sintaxis estándar Spark SQL / DuckDB equivalente
SELECT
    evento_id,
    get_json_object(payload_str, '$.usuario_id') AS usuario_id,
    get_json_object(payload_str, '$.dispositivo.sistema_operativo') AS so_cliente
FROM ventas_analitica.eventos_raw;
```

1.5 Subconsultas y Operaciones de Conjuntos

Subconsultas Escalares y Correlacionadas

Una subconsulta es **correlacionada** cuando depende de una fila procesada por la consulta externa, evaluándose conceptualmente para cada registro:

```sql
-- Obtener pedidos cuyo monto supera el promedio de su respectivo país
(Subconsulta correlacionada)
SELECT p1.pedido_id, p1.pais_id, p1.monto_usd
FROM ventas_analitica.pedidos AS p1
WHERE p1.monto_usd > (
    SELECT AVG(p2.monto_usd)
    FROM ventas_analitica.pedidos AS p2
    WHERE p2.pais_id = p1.pais_id
);
```

Operadores de Existencia: `EXISTS` frente a `IN`

- `IN`: Evalúa una lista completa de valores. Si la subconsulta contiene un valor `NULL`, evaluar `NOT IN` puede devolver un conjunto vacío de forma inesperada debido a la lógica trivaluada (`NOT (x IN (1, NULL))` evalúa a `UNKNOWN`).
- `EXISTS`: Evalúa la presencia de al menos una tupla que satisfaga la condición mediante un predicado booleano, permitiendo al optimizador detener el escaneo (*short-circuit evaluation*):

```sql
-- Identificar clientes con al menos una compra en los últimos 90 días
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM ventas_analitica.pedidos AS p
    WHERE p.cliente_id = c.cliente_id
      AND p.fecha_creacion >= CURRENT_DATE - INTERVAL '90 days'
);
```

Operaciones de Conjuntos (`UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`)

Operación	Semántica Relacional	Costo Computacional
`UNION ALL`	Concatena los registros de dos consultas preservando duplicados.	**Bajo ($O(N)$):** No requiere ordenar ni crear tablas hash.
`UNION`	Concatena y elimina tuplas duplicadas.	**Alto ($O(N \log N)$):** Fuerza una fase de deduplicación o hash de conjunto.
`INTERSECT`	Devuelve solo los registros presentes en ambos conjuntos.	Requiere coincidencia de todas las columnas.
`EXCEPT` / `MINUS`	Devuelve los registros del primer conjunto que no existen en el segundo.	Útil para pruebas de conciliación y calidad de datos.

1.6 Expresiones de Tabla Comunes (CTEs) y CTEs Recursivas

Las **CTEs** (*Common Table Expressions*, introducidas mediante la cláusula `WITH`) transforman consultas anidadas impenetrables en secuencias lógicas modulares, legibles y reutilizables.



```sql
WITH ventas_validas AS (
    SELECT
        pedido_id,
        cliente_id,
        monto_usd,
        DATE_TRUNC('month', fecha_creacion) AS mes
    FROM ventas_analitica.pedidos
    WHERE estado = 'COMPLETADO'
),
metricas_cliente AS (
    SELECT
        cliente_id,
        mes,
        COUNT(pedido_id) AS total_pedidos,
        SUM(monto_usd) AS gasto_mensual
    FROM ventas_validas
    GROUP BY cliente_id, mes
)
SELECT
    c.nombre,
    c.pais_iso,
    m.mes,
    m.total_pedidos,
    m.gasto_mensual
FROM metricas_cliente AS m
INNER JOIN ventas_analitica.clientes AS c ON m.cliente_id = c.cliente_id
WHERE m.gasto_mensual > 1000;
```

CTEs Recursivas para Estructuras Jerárquicas

En ingeniería de datos es común modelar jerarquías organizacionales, árboles de categorías o cadenas de linaje (*data lineage*). Una CTE recursiva consta de un **miembro ancla** y un **miembro recursivo** unidos por `UNION ALL`:

```sql
-- Recorrido de una jerarquía de categorías de productos (Árbol relacional)
WITH RECURSIVE jerarquia_categorias AS (
    -- Miembro Ancla: Categorías raíz (sin padre)
    SELECT
        categoria_id,
        nombre_categoria,
        categoria_padre_id,
        1 AS nivel_profundidad,
        CAST(nombre_categoria AS VARCHAR(1000)) AS ruta_arbol
    FROM ventas_analitica.categorias
    WHERE categoria_padre_id IS NULL

    UNION ALL

    -- Miembro Recursivo: Hijos unidos con la iteración previa
    SELECT
        c.categoria_id,
        c.nombre_categoria,
        c.categoria_padre_id,
        j.nivel_profundidad + 1,
        CAST(j.ruta_arbol || ' > ' || c.nombre_categoria AS VARCHAR(1000))
    FROM ventas_analitica.categorias AS c
    INNER JOIN jerarquia_categorias AS j ON c.categoria_padre_id = j.categoria_id
)
SELECT categoria_id, nombre_categoria, nivel_profundidad, ruta_arbol
FROM jerarquia_categorias
ORDER BY ruta_arbol;
```

1.7 Tipos de Uniones (JOINs) y el Peligro de la Lógica Trivaluada (3VL)

El operador relacional `JOIN` combina tuplas de dos relaciones en función de una condición de acoplamiento:



1. `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN` y `FULL OUTER JOIN`

- `INNER JOIN`: Devuelve únicamente tuplas que satisfacen el predicado en ambas tablas.

- `LEFT OUTER JOIN`: Preserva todas las tuplas de la tabla izquierda. Si no hay coincidencia en la derecha, rellena sus columnas con `NULL`.
- `FULL OUTER JOIN`: Preserva todas las tuplas de ambos extremos, insertando `NULL` en los lados sin correspondencia.

2. Uniones de Filtrado: `LEFT SEMI JOIN` y `LEFT ANTI JOIN` vs. `EXISTS` / `NOT EXISTS`

Ampliamente utilizadas en motores analíticos distribuidos (Spark SQL, Trino, Databricks, DuckDB, Snowflake):

- `LEFT SEMI JOIN`: Retorna las columnas de la tabla izquierda para aquellas filas que tienen al menos una correspondencia en la derecha. **A diferencia del `INNER JOIN`, nunca duplica filas de la tabla izquierda**, incluso si hay múltiples coincidencias en la derecha. En SQL estándar (como PostgreSQL), su equivalente canónico es `WHERE EXISTS (...)`.
- `LEFT ANTI JOIN`: Retorna únicamente las filas de la tabla izquierda que **no tienen correspondencia** en la derecha. Es la alternativa más limpia y eficiente al peligroso `WHERE id NOT IN (...)`.

> **Nota de Portabilidad:** La sintaxis explícita `LEFT ANTI JOIN` y `LEFT SEMI JOIN` es nativa de motores OLAP modernos (Spark SQL, DuckDB, Databricks, Trino, BigQuery). En motores relacionales tradicionales como PostgreSQL o SQL Server estándar, no existe la palabra clave `ANTI JOIN`; en ellos la forma estándar ANSI equivalente y óptima es `WHERE NOT EXISTS (...)`:

```sql
-- En Spark SQL, DuckDB, Trino y Snowflake:
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
LEFT ANTI JOIN ventas_analitica.pedidos AS p ON c.cliente_id = p.cliente_id;

-- Equivalente estándar ANSI (PostgreSQL, MySQL, SQL Server):
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
WHERE NOT EXISTS (
    SELECT 1 FROM ventas_analitica.pedidos AS p
    WHERE p.cliente_id = c.cliente_id
);
```

3. La Lógica Trivaluada (Three-Valued Logic - 3VL) y Claves `NULL`

En SQL, una comparación lógica puede evaluar a `TRUE`, `FALSE` o `UNKNOWN` (desconocido).

- **Regla Crítica:** `NULL = NULL` evalúa a `UNKNOWN`, no a `TRUE`.
- En un `INNER JOIN`, dos tuplas que tengan `NULL` en su clave de unión **nunca coinciden**.

- Si una clave foránea contiene `NULL`, dichas filas desaparecen de un `INNER JOIN`. En un `FULL JOIN`, cada fila con clave `NULL` formará una tupla independiente emparejada con nulos en el lado opuesto.

SQL

```sql
-- Demostración de igualdad segura frente a NULL (Null-Safe Equality):
-- 1. En Spark SQL y MySQL (operador <=>):
SELECT *
FROM tabla_a AS a
INNER JOIN tabla_b AS b ON a.codigo <=> b.codigo;

-- 2. En SQL Estándar ANSI y PostgreSQL (predicado IS NOT DISTINCT FROM):
SELECT *
FROM tabla_a AS a
INNER JOIN tabla_b AS b ON a.codigo IS NOT DISTINCT FROM b.codigo;
```

1.8 De SQL Estándar a Spark SQL y Vistas Temporales

En Apache Spark, la interfaz SQL es de primera clase y comparte exactamente el mismo optimizador Catalyst que la API DataFrame:

PYTHON

```python
from pyspark.sql import SparkSession

# Inicializar sesión de Spark
spark = SparkSession.builder \
    .appName("Capitulo01-SQL-Analitico") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset estructurado
df_ventas = spark.read.parquet("data/ventas.parquet")

# Registrar vista temporal en el catálogo en memoria de Spark
df_ventas.createOrReplaceTempView("v_ventas")

# Consulta analítica avanzada con CTEs y Anti-Join
consulta_spark_sql = """
    WITH clientes_activos AS (
        SELECT cliente_id, SUM(monto_usd) AS volumen_total
        FROM v_ventas
        WHERE fecha_transaccion >= '2026-01-01'
        GROUP BY cliente_id
    )
    SELECT
        v.pais,
        COUNT(DISTINCT v.cliente_id) AS total_clientes,
        SUM(v.monto_usd) AS facturacion_pais
    FROM v_ventas AS v
    LEFT SEMI JOIN clientes_activos AS ca ON v.cliente_id = ca.cliente_id
```

```
    GROUP BY v.pais
    ORDER BY facturacion_pais DESC
"""

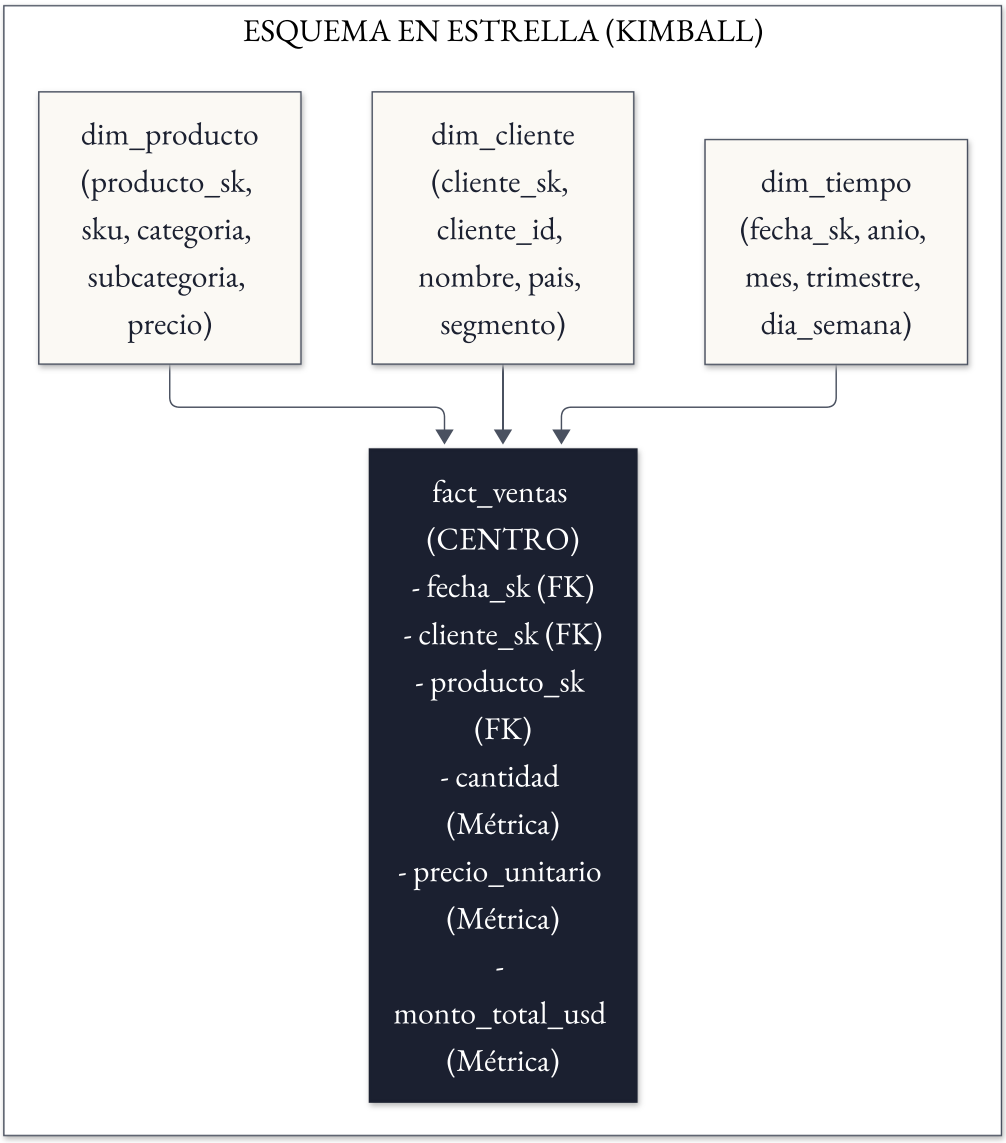
df_resultado = spark.sql(consulta_spark_sql)
df_resultado.show(5)
```

1.9 Introducción al Modelado Dimensional: Kimball vs. Normalización 3NF

Uno de los saltos cualitativos que diferencian a un programador de un verdadero ingeniero de datos es comprender **por qué estructuramos los datos de forma diferente para operar que para analizar**.

1. La Dualidad de Sistemas: OLTP frente a OLAP

- **Sistemas Transaccionales (OLTP):** Diseñados en **Tercera Forma Normal (3NF)**. Su meta es evitar redundancia: si un cliente cambia de correo electrónico, se modifica una única fila en una tabla aislada para evitar inconsistencias y bloqueos de escritura. Sin embargo, responder a una pregunta como *«¿Cuáles fueron las ventas por categoría en Colombia el último trimestre?»* requiere unir 8 o 10 tablas normalizadas con un costo computacional prohibitivo.
- **Sistemas Analíticos (OLAP / Data Warehouse):** Diseñados bajo la metodología dimensional de **Ralph Kimball**. Su meta es optimizar la lectura analítica masiva y la simplicidad semántica, organizando los datos en **Tablas de Hechos** y **Tablas de Dimensiones**.



2. Anatomía de una Tabla de Hechos (*Fact Table*)

Una tabla de hechos registra eventos numéricos medibles que ocurren en el negocio:

- **El Grano (*Granularity*):** Es el principio más sagrado de Kimball. Define con precisión quirúrgica qué representa **una sola fila** de la tabla. Por ejemplo: «*Una fila representa una línea individual de producto en un ticket de compra completado*». Si el grano se mezcla o confunde, los cálculos de agregación (`SUM`, `AVG`) duplicarán importes.
- **Métricas Aditivas:** Valores numéricos que pueden sumarse válidamente a través de todas las dimensiones (`monto_usd`, `cantidad_unidades`).

- **Claves Foráneas:** Identificadores que apuntan a las dimensiones que contextualizan el hecho.

3. Anatomía de una Tabla de Dimensiones (*Dimension Table*)

Las dimensiones proporcionan el contexto descriptivo respondiendo a las preguntas de negocio: *¿Quién compró?*, *¿Dónde?*, *¿Cuándo?*, *¿Qué artículo?*.

- **Claves Subrogadas (*Surrogate Keys*):** Claves primarias artificiales (enteros secuenciales o hashes `cliente_sk`) gestionadas por el almacén de datos, independientes de la clave natural del ERP origen (`cliente_id`). Permiten manejar miembros desconocidos (clave `-1`) y registrar cambios históricos de atributos (dimensiones lentamente cambiantes o SCD).
- **Esquema en Estrella (*Star Schema*) frente a Copo de Nieve (*Snowflake*):**
  En un esquema en estrella, las dimensiones están completamente desnormalizadas en una sola tabla (ej. `dim_producto` incluye su categoría y departamento directamente). En el esquema copo de nieve, las dimensiones se normalizan parcialmente (ej. `dim_producto` se une a `dim_subcategoria` que a su vez se une a `dim_categoria`).
  Veredicto de Producción: Los motores analíticos modernos (DuckDB, Snowflake, Databricks, BigQuery) están optimizados para formatos columnares donde el almacenamiento es barato y los saltos entre múltiples tablas degradan el rendimiento. **El Esquema en Estrella es el estándar dominante de la industria.**

4. Consulta Analítica Canónica sobre Esquema en Estrella

Observe cómo una consulta que requeriría 8 uniones en una base relacional 3NF se reduce a simples cruces directos contra la tabla central de hechos:

SQL

```sql
SELECT
    d_cli.pais,
    d_prod.categoria,
    d_tiem.anio,
    d_tiem.mes,
    COUNT(f.venta_id) AS total_transacciones,
    ROUND(SUM(f.monto_total_usd), 2) AS facturacion_total
FROM fact_ventas AS f
INNER JOIN dim_cliente AS d_cli
    ON f.cliente_sk = d_cli.cliente_sk
INNER JOIN dim_producto AS d_prod
    ON f.producto_sk = d_prod.producto_sk
INNER JOIN dim_tiempo AS d_tiem
    ON f.fecha_sk = d_tiem.fecha_sk
WHERE d_tiem.anio = 2026
  AND d_cli.pais IN ('CO', 'MX', 'CL')
GROUP BY d_cli.pais, d_prod.categoria, d_tiem.anio, d_tiem.mes
ORDER BY facturacion_total DESC;
```

1.10 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Uso Inseguro de `NOT IN` con Columnas Anulables
Si la subconsulta de un `NOT IN` devuelve una sola tupla con valor `NULL`, toda la condición externa evalúa a `UNKNOWN` y la consulta devuelve cero filas.
Corrección: Utilice siempre `LEFT ANTI JOIN` o `WHERE NOT EXISTS (...)`.

Anti-patrón 2: Producto Cartesiano Inadvertido (`CROSS JOIN`)
Omitir la cláusula `ON` en un join genera una combinación de cada fila izquierda con cada fila derecha ($M \times N$ tuplas). Si la tabla izquierda tiene 1 millón de registros y la derecha 1 millón, el motor intentará generar 10^{12} filas (1 billón de registros), saturando la memoria y colapsando el clúster.

Anti-patrón 3: Funciones No SARGables en la Cláusula `WHERE`
Aplicar funciones sobre columnas indexadas o particionadas (por ejemplo, `WHERE YEAR(fecha_registro) = 2026`) impide que el optimizador aplique poda de particiones (*partition pruning*) o busque en índices B-Tree.
Corrección: Escriba predicados SARGables (*Search Argument Able*): `WHERE fecha_registro >= '2026-01-01' AND fecha_registro < '2027-01-01'`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 1.1 (Deduplicación Analítica e Identificación de Huérfanos):
Se tienen dos tablas en un Data Lakehouse: `transacciones` (id, cliente_id, monto, fecha) y `clientes_registrados` (cliente_id, nombre, pais).
Escriba una consulta SQL estándar que cumpla con los siguientes dos requerimientos:

1. Extraer todas las transacciones de clientes que **no figuren** en la tabla de `clientes_registrados` (transacciones huérfanas).
2. Garantizar que la consulta sea inmune a valores `NULL` en `clientes_registrados.cliente_id` y no utilice `NOT IN`.

Solución 1.1:

```SQL
SELECT
    t.id AS transaccion_id,
    t.cliente_id,
    t.monto,
    t.fecha
FROM transacciones AS t
LEFT ANTI JOIN clientes_registrados AS c ON t.cliente_id = c.cliente_id;
```

Explicación Técnica: El operador `LEFT ANTI JOIN` evalúa la ausencia de concordancia relacional a nivel de álgebra relacional. Al comparar mediante punteros de unión física (Hash Anti Join), es inmune a la presencia de valores `NULL` en la tabla de la derecha y no sufre de la degradación a `UNKNOWN` de `NOT IN`.

Ejercicio 1.2 (CTE Recursiva y Cálculo de Jerarquías):
En una estructura de empleados donde cada tupla tiene `(empleado_id, jefe_id, salario)`, escriba una CTE recursiva que determine la cadena de mando completa para el empleado con `empleado_id = 500`, mostrando su nivel de profundidad hasta el Director General (quien tiene `jefe_id IS NULL`).

Solución 1.2:

```SQL
WITH RECURSIVE cadena_mando AS (
    -- Miembro Ancla: El empleado de interés
    SELECT
        empleado_id,
        jefe_id,
        salario,
        1 AS nivel
    FROM empleados
    WHERE empleado_id = 500

    UNION ALL

    -- Miembro Recursivo: Ascendemos hacia el jefe directo
    SELECT
        e.empleado_id,
        e.jefe_id,
        e.salario,
        cm.nivel + 1
    FROM empleados AS e
    INNER JOIN cadena_mando AS cm ON e.empleado_id = cm.jefe_id
)
SELECT *
FROM cadena_mando
ORDER BY nivel ASC;
```

1.11 Resumen y Puntos Clave

- **SQL es un lenguaje declarativo:** La separación entre especificación lógica (*qué*) y ejecución física (*cómo*) permite al motor optimizar planes de consulta sin alterar la intención de negocio.
- **Operación atómica `MERGE INTO`:** Es el pilar fundamental para cargas incrementales y sincronizaciones *Upsert* idempotentes en tablas transaccionales.

- **CTEs y Legibilidad:** Permiten construir pipelines de transformación modulares en lugar de complejas subconsultas anidadas.
- **Joins de Filtrado:** `LEFT SEMI JOIN` y `LEFT ANTI JOIN` proporcionan una semántica limpia y altamente optimizada para comprobaciones de existencia y exclusión.
- **Lógica trivaluada (3VL):** Valores `NULL` en comparaciones booleanas producen `UNKNOWN`, lo que exige extrema precaución en uniones y cláusulas `NOT IN`.
- **Modelado Dimensional (Kimball):** Mientras OLTP normaliza en 3NF para transacciones atómicas, los sistemas analíticos organizan datos en esquemas en estrella (hechos medibles rodeados de dimensiones de contexto) para maximizar la velocidad de lectura analítica.

Lecturas Recomendadas

1. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
2. **Date, C. J. (2019).** *SQL and Relational Theory: How to Write Accurate SQL Code (3rd Edition).* O'Reilly Media.
3. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 2: Data Models and Query Languages).* O'Reilly Media.
4. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* ACM SIGMOD.

CAPÍTULO 2

Agrupaciones Multidimensionales, Window Functions y Optimización

NÚCLEO

> *"Al agregar datos reducimos la cardinalidad de un conjunto, pero en sistemas analíticos y distribuidos desatamos una de las operaciones físicas más complejas: el reagrupamiento en memoria y el intercambio de particiones a través de la red (shuffle). Dominar las funciones analíticas de ventana y las extensiones multidimensionales es lo que distingue a un generador de consultas de un verdadero ingeniero de datos."*

Objetivos de Aprendizaje

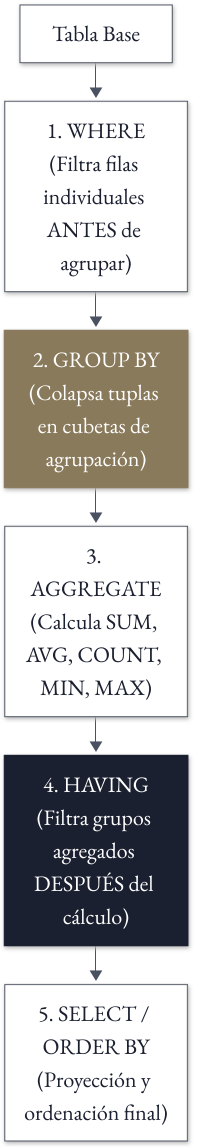
Al finalizar este capítulo, el lector será capaz de:

- Dominar la semántica de agregación con `GROUP BY` y diferenciar con precisión matemática los alcances de `WHERE` frente a `HAVING`.
- Implementar agregaciones multidimensionales para capas analíticas utilizando `ROLLUP`, `CUBE` y `GROUPING SETS`.
- Realizar transformaciones de rotación tabular mediante operadores `PIVOT` y `UNPIVOT`.
- Explicar y aplicar la anatomía completa de las **Window Functions** (*Funciones de Ventana*): cláusulas `PARTITION BY`, `ORDER BY` y especificación de marcos (`ROWS` frente a `RANGE BETWEEN`).
- Utilizar con solvencia funciones de ranking (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`) y de desplazamiento (`LAG`, `LEAD`).
- Resolver problemas críticos de ingeniería como deduplicación por versión más reciente (*latest record*) y cálculo de métricas acumulativas continuas.
- Inspeccionar planes de ejecución con `EXPLAIN` para identificar operadores de agregación física (*HashAggregate* frente a *SortAggregate*).

> **Definición 2.1 (La Mecánica del Marco de Ventana)** A diferencia de una agregación clásica con `GROUP BY` —la cual colapsa múltiples filas en una sola tupla de resumen perdiendo la identidad individual del registro—, una **Window Function** (*Función de Ventana*) calcula un valor analítico sobre un subconjunto de filas relacionadas (*el marco o ventana*), **preservando intacta cada fila individual en la salida**.
>
> Esta propiedad permite yuxtaponer métricas agregadas (totales acumulados, promedios móviles, rankings) junto con los atributos atómicos de cada tupla sin requerir auto-uniones (*self-joins*) costosas.

2.1 Agrupaciones Fundamentales y Filtrado: `WHERE` frente a `HAVING`

El procesamiento de una consulta relacional con agregación sigue un orden lógico estricto:


Tabla Base
1. WHERE (Filtra filas individuales ANTES de agrupar)
2. GROUP BY (Colapsa tuplas en cubetas de agrupación)
3. AGGREGATE (Calcula SUM, AVG, COUNT, MIN, MAX)
4. HAVING (Filtra grupos agregados DESPUÉS del cálculo)
5. SELECT / ORDER BY (Proyección y ordenación final)

```sql
SELECT
    pais_iso,
    categoria_id,
    COUNT(pedido_id) AS total_pedidos,
    SUM(monto_usd) AS facturacion_total,
    AVG(monto_usd) AS ticket_promedio
FROM ventas_analitica.pedidos
WHERE estado = 'ENTREGADO' -- Predicado de fila: se descartan pedidos no entregados ANTES del shuffle
GROUP BY pais_iso, categoria_id
HAVING SUM(monto_usd) >= 50000 -- Predicado de grupo: filtra agregaciones ya calculadas
ORDER BY facturacion_total DESC;
```

2.2 Extensiones Multidimensionales: `ROLLUP`, `CUBE` y `GROUPING SETS`

En el modelado analítico para inteligencia de negocios (BI), es habitual generar subtotales y totales generales en una sola pasada sobre los datos, evitando emitir múltiples consultas unidas con `UNION ALL`.

1. `GROUPING SETS`

Permite declarar explícitamente qué combinaciones de dimensiones deben ser agregadas:

```sql
SELECT
    pais_iso,
    categoria_id,
    SUM(monto_usd) AS total_ventas
FROM ventas_analitica.pedidos
GROUP BY GROUPING SETS (
    (pais_iso, categoria_id), -- Subtotal por país y categoría
    (pais_iso),               -- Subtotal nacional
    ()                        -- Gran total global
);
```

2. `ROLLUP` (Jerarquía Piramidal)

Calcula agregaciones en forma jerárquica de izquierda a derecha. Si agrupamos por `(anio, mes, dia)`, `ROLLUP` produce:

- `(anio, mes, dia)`
- `(anio, mes)`
- `(anio)`
- `()` (Total general)

```sql
SELECT anio, mes, SUM(monto_usd) AS total_ventas
FROM ventas_analitica.fact_ventas_diarias
GROUP BY ROLLUP (anio, mes);
```

3. `CUBE` (Todas las Combinaciones Posibles)

Calcula todas las combinaciones 2^N posibles de las N dimensiones especificadas. Para `CUBE(pais, canal)` genera: `(pais, canal)`, `(pais)`, `(canal)` y `()`.

2.3 Rotación Tabular: `PIVOT` y `UNPIVOT`

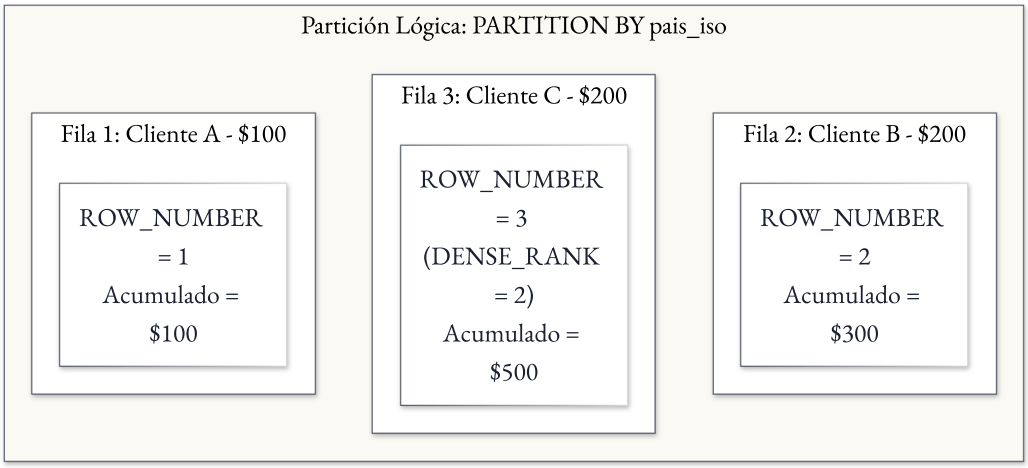
- `PIVOT`: Rota valores de una columna en filas para convertirlos en múltiples columnas (útil para generar matrices analíticas comparativas).
- `UNPIVOT`: Convierte columnas en filas (normalización de datos).

```sql
-- Rotación PIVOT en SQL (DuckDB / Databricks / Snowflake)
SELECT *
FROM (
    SELECT anio, trimestre, monto_usd
    FROM ventas_analitica.ventas_trimestrales
)
PIVOT (
    SUM(monto_usd)
    FOR trimestre IN ('Q1' AS ventas_q1, 'Q2' AS ventas_q2, 'Q3' AS ventas_q3,
'Q4' AS ventas_q4)
)
ORDER BY anio DESC;
```

2.4 Anatomía Completa de las Window Functions

Una función de ventana evalúa una expresión sobre una partición lógica sin comprimir las filas:

FUNCIÓN(expr) **OVER** (**PARTITION BY** claves **ORDER BY** orden



1. Funciones de Ranking

Comparativa entre las principales funciones de jerarquía sobre un conjunto con empates en el salario:

Salario	`ROW_NUMBER()`	`RANK()`	`DENSE_RANK()`	`NTILE(2)`
$10.000	1	1	1	1
$8.000	2	2	2	1
$8.000	3	2 *(empate)*	2 *(empate)*	2
$6.000	4	4 *(salta el 3)*	3 *(sin salto)*	2

- `ROW_NUMBER()`: Asigna un entero secuencial estrictamente único ($1, 2, 3, 4$). Si hay empates, el orden de desempate es arbitrario salvo que se agregue una clave secundaria en el `ORDER BY`.
- `RANK()`: Asigna el mismo rango ante valores idénticos, pero **salta** posiciones en el siguiente valor ($1, 2, 2, 4$).
- `DENSE_RANK()`: Asigna el mismo rango ante empates y **continúa de forma contigua sin huecos** ($1, 2, 2, 3$).
- `NTILE(n)`: Divide la partición en n cubetas casi iguales y asigna el número de cubeta (1 a n), ideal para calcular cuartiles o deciles.

2. Funciones de Desplazamiento y Acceso a Fila (`LAG`, `LEAD`)

Permiten comparar una fila con su predecesora o sucesora sin auto-uniones:

```sql
SELECT
    cliente_id,
    fecha_pedido,
    monto_usd,
    -- Valor de la compra inmediatamente anterior
    LAG(monto_usd, 1, 0.0) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS monto_compra_previa,
    -- Días transcurridos desde la compra anterior
    CAST(fecha_pedido AS DATE) - LAG(CAST(fecha_pedido AS DATE), 1) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS dias_desde_ultima_compra,
    -- Valor de la siguiente compra
    LEAD(monto_usd, 1) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS monto_compra_siguiente
FROM ventas_analitica.pedidos;
```

3. Marcos de Ventana Físicos (`ROWS`) frente a Lógicos (`RANGE`)

Al invocar agregaciones en ventana (`SUM()`, `AVG()`), el **marco de ventana** delimita qué tuplas entran en el cálculo:

- **Acumulado Histórico Completo:**

```sql
SUM(monto_usd) OVER (
    PARTITION BY cliente_id
    ORDER BY fecha_pedido ASC
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS total_acumulado_hasta_la_fecha
```

- **Media Móvil de los Últimos 7 Días / 7 Eventos:**

```sql
AVG(monto_usd) OVER (
    PARTITION BY tienda_id
    ORDER BY fecha_venta ASC
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS media_movil_7_eventos
```

> **CUIDADO CRÍTICO: La Trampa de `RANGE BETWEEN` por Defecto** Cuando se incluye una cláusula `ORDER BY` sin especificar explícitamente el marco de filas, la norma SQL aplica por defecto: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` `RANGE` evalúa duplicados de orden de forma conjunta: si existen tres transacciones con la misma marca de tiempo idéntica, el acumulado sumará las tres transacciones de golpe en las tres filas, en lugar de acumular fila a fila. **En producción especifique siempre `ROWS BETWEEN` para garantizar determinismo y menor sobrecosto de cálculo**.

2.5 Deduplicación Analítica: `ROW_NUMBER` frente a `dropDuplicates`

Uno de los requerimientos más recurrentes en pipelines Lakehouse es conservar únicamente la versión más reciente de un registro ante múltiples actualizaciones emitidas por sistemas CDC (*Change Data Capture*):

```sql
-- Patrón canónico de deduplicación analítica conservando el registro más reciente
WITH novedades_numeradas AS (
    SELECT
        cliente_id,
        nombre,
        correo,
        direccion,
        fecha_actualizacion,
        ROW_NUMBER() OVER (
            PARTITION BY cliente_id
            ORDER BY fecha_actualizacion DESC, transaccion_id DESC
        ) AS indice_version
    FROM staging.clientes_novedades
)
SELECT
    cliente_id,
    nombre,
    correo,
    direccion,
    fecha_actualizacion
FROM novedades_numeradas
WHERE indice_version = 1;
```

- **Comparativa:** `DISTINCT` o `df.dropDuplicates(["cliente_id"])` descartan registros arbitrariamente si las filas difieren en otras columnas. `ROW_NUMBER()` con orden descendente por marca de tiempo garantiza de manera formal y determinista que solo se conserve el registro de mayor vigencia temporal.

2.6 Implementación en PySpark y Optimización Física con `EXPLAIN`

En PySpark, la API de ventanas se implementa mediante la clase `Window` del módulo `pyspark.sql.window`:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("Capitulo02-WindowFunctions") \
    .master("local[*]") \
    .getOrCreate()

df_pedidos = spark.read.parquet("data/ventas.parquet")

# 1. Definición de especificación de ventana
ventana_cliente = Window \
    .partitionBy("cliente_id") \
    .orderBy(F.col("fecha_transaccion").asc()) \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

# 2. Aplicación de agregación acumulada y ranking
df_analitico = df_pedidos.withColumn(
    "gasto_acumulado",
    F.sum("monto_usd").over(ventana_cliente)
).withColumn(
    "numero_compra",
    F.row_number().over(

Window.partitionBy("cliente_id").orderBy(F.col("fecha_transaccion").asc())
    )
)

# 3. Inspección del plan físico con EXPLAIN
df_analitico.explain(True)
```

Interpretación del Plan Físico

Al ejecutar `explain()`, observe cómo Catalyst organiza los operadores físicos:

1. `Exchange hashpartitioning(cliente_id)`: Realiza el shuffle de red para enviar todas las filas de un mismo `cliente_id` al mismo ejecutor.
2. `Sort [cliente_id ASC, fecha_transaccion ASC]`: Ordena las particiones locales en memoria.
3. `Window [sum(...) ... row_number(...)]`: El operador físico de ventana procesa secuencialmente el flujo de datos ordenado en memoria sin necesidad de reconstruir

estructuras hash adicionales.

2.7 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ventana Global sin `PARTITION BY` sobre Grandes Volúmenes
Declarar `Window.orderBy("monto_usd")` omitiendo el `partitionBy()` fuerza a Spark a enviar **la totalidad de las filas del dataset a un único nodo ejecutor**. Esto satura la memoria del ejecutor, desencadena fallos catastróficos por *OutOfMemory* (OOM) y destruye el paralelismo del clúster.

Anti-patrón 2: Confundir `COUNT(*)` con `COUNT(columna)` ante Valores `NULL`
`COUNT(*)` cuenta el número absoluto de filas de un grupo o ventana, independientemente de su contenido. `COUNT(columna)` cuenta únicamente las tuplas donde dicha columna no es `NULL`. En métricas financieras o de calidad, este descuido altera los promedios e indicadores.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 2.1 (Detección de Transacciones Sospechosas por Desviación de Media Móvil):
En una tabla de eventos financieros `transacciones(id, cuenta_id, monto, timestamp)`, escriba una consulta SQL que calcule:

1. La media móvil del monto de las últimas 5 transacciones de la cuenta (las 4 anteriores y la actual).
2. La diferencia entre el monto actual y dicha media móvil.

Solución 2.1:

```sql
SELECT
    id,
    cuenta_id,
    monto,
    timestamp,
    AVG(monto) OVER (
        PARTITION BY cuenta_id
        ORDER BY timestamp ASC
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS media_movil_ultimas_5,
    monto - AVG(monto) OVER (
        PARTITION BY cuenta_id
        ORDER BY timestamp ASC
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS desviacion_respecto_media
FROM transacciones;
```

Ejercicio 2.2 (Cálculo de Tasa de Retención con `LAG`):
Se tiene un histórico de suscripciones `pagos(pago_id, usuario_id, fecha_pago, monto)`. Escriba una consulta analítica que etiquete como `'REACTIVADO'` a cualquier usuario cuya compra actual se haya realizado más de 60 días después de su compra previa.

Solución 2.2:

```sql
WITH compras_previas AS (
    SELECT
        pago_id,
        usuario_id,
        fecha_pago,
        LAG(fecha_pago) OVER (
            PARTITION BY usuario_id
            ORDER BY fecha_pago ASC
        ) AS fecha_pago_anterior
    FROM pagos
)
SELECT
    pago_id,
    usuario_id,
    fecha_pago,
    fecha_pago_anterior,
    CASE
        WHEN fecha_pago_anterior IS NULL THEN 'CLIENTE NUEVO'
        WHEN CAST(fecha_pago AS DATE) - CAST(fecha_pago_anterior AS DATE) > 60
THEN 'REACTIVADO'
        ELSE 'RECURRENTE'
    END AS estado_ciclo_vida
FROM compras_previas;
```

2.8 Resumen y Puntos Clave

- **Preservación de tuplas:** Las funciones de ventana computan métricas agregadas y analíticas preservando intacta la granularidad individual de cada fila.
- **Marcos deterministas:** Utilice siempre `ROWS BETWEEN` en lugar del valor implícito `RANGE BETWEEN` para evitar comportamientos anómalos ante marcas de tiempo duplicadas.
- **Jerarquías con `DENSE_RANK` y `ROW_NUMBER`:** `ROW_NUMBER` es la herramienta matemática por excelencia para deduplicación por registro más reciente, mientras que `DENSE_RANK` evita saltos en listas de mejores puntuaciones.
- **Particionado responsable:** Toda ventana en entornos distribuidos debe incluir una clave lógica de `PARTITION BY` para evitar la serialización de todo el dataset en un único ejecutor.
- **Optimización con `EXPLAIN`:** Verificar los operadores físicos de intercambio y ordenamiento (*Exchange* y *Sort*) permite diagnosticar si el plan de consulta incurre en shuffles innecesarios.

Lecturas Recomendadas

1. **Date, C. J. (2019).** *SQL and Relational Theory: How to Write Accurate SQL Code (Chapter 8: Window Functions and Relational Algebra).* O'Reilly Media.
2. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* ACM SIGMOD.
3. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Window Function Processing and Syntax.*

CAPÍTULO 2A

DuckDB y Cómputo In-Process: Cuándo No Usar Computación Distribuida NÚCLEO

> *"Uno de los errores de diseño más costosos en la ingeniería de datos contemporánea es utilizar un cañón distribuido para cazar una mosca local. Durante una década, la industria convenció a los equipos de que cualquier análisis requería un clúster de Spark de ocho nodos. En la práctica, más del 90% de los conjuntos de datos analíticos caben en la memoria RAM de una sola máquina moderna. DuckDB demostró que el cómputo embebido, columnar y vectorizado en un solo nodo puede procesar decenas de gigabytes en segundos a una fracción del costo y complejidad de un clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la fundamentación arquitectónica de los motores analíticos en proceso (**In-Process OLAP**) frente a sistemas distribuidos masivos.
- Analizar los costos ocultos de la computación distribuida (serialización de red, sobrecosto de coordinación del Driver y latencia de shuffles).
- Ejecutar consultas SQL analíticas de alto rendimiento directamente sobre archivos **Parquet**, CSV y JSON sin cargarlos previamente en tablas físicas.
- Conectar DuckDB con almacenamiento de objetos en la nube (Amazon S3 / Azure ADLS) mediante la extensión `httpfs` con transferencias *Zero-Copy* hacia Apache Arrow y Pandas.
- Implementar canalizaciones analíticas Serverless de bajo costo utilizando **AWS Lambda** y DuckDB.
- Aplicar una matriz de decisión rigurosa para seleccionar entre **Pandas**, **Polars**, **DuckDB** y **Apache Spark**.

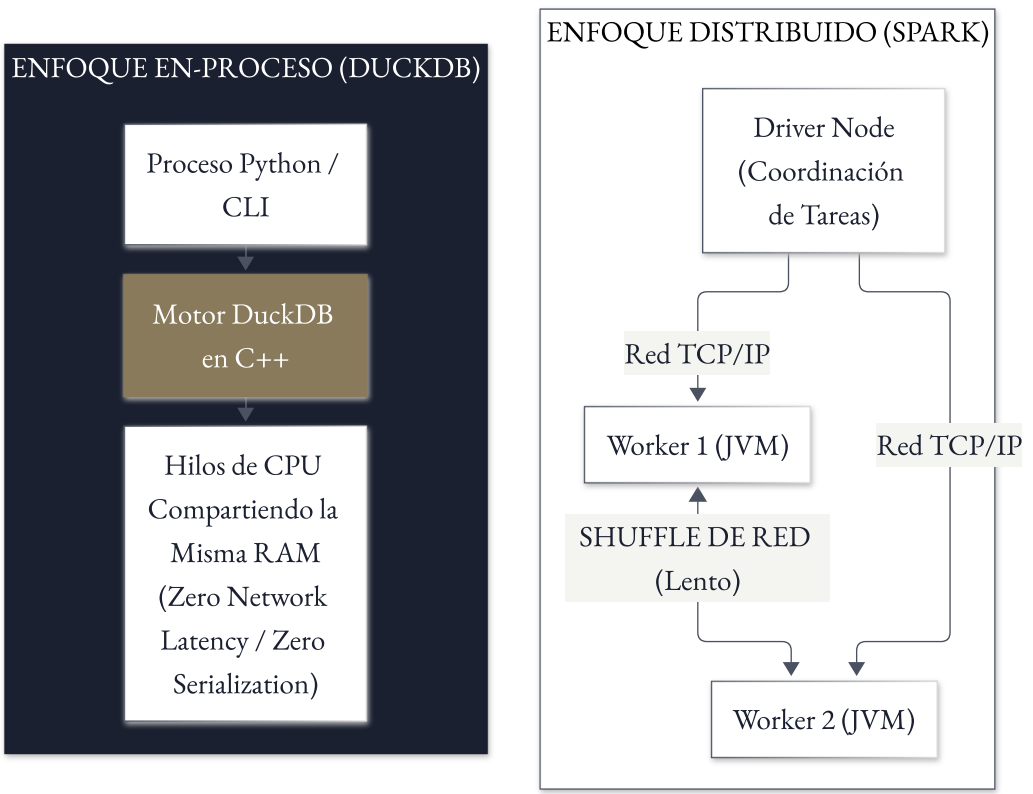
Definición 2A.1 (Base de Datos Analítica En-Proceso - In-Process OLAP) DuckDB es un sistema de gestión de bases de datos relacionales enfocado en procesamiento analítico (OLAP), diseñado para ejecutarse **embebido dentro del mismo espacio de memoria del proceso anfitrión** (*In-Process*), análogo a lo que SQLite representa para cargas transaccionales (OLTP).

No requiere la instalación de un servidor independiente, carece de dependencias externas complejas y procesa datos mediante un **motor columnar vectorizado en C++** con paralelismo multihilo (*Morsel-Driven Parallelism*) y capacidad de procesamiento fuera de memoria (*Out-of-Core Execution*).

2A.1 El Mito del Big Data y el Costo Oculto de los Clústeres

A principios de la era de Hadoop y Spark, los servidores estándar disponían de 4 a 8 GB de RAM. Procesar 50 GB de datos forzaba obligatoriamente a distribuir la carga entre diez máquinas.

Hoy, un computador portátil de desarrollo cuenta con 16 a 64 GB de memoria RAM, y una sola instancia en la nube (como una máquina `r6i.4xlarge` en AWS) ofrece **128 GB de RAM y 16 núcleos de CPU por menos de $1 USD por hora**.



El Impuesto de la Distribución (*The Distribution Tax*)

Cuando una consulta se ejecuta sobre un clúster distribuido, el motor debe asumir sobrecostos físicos inevitables:

1. **Serialización y Deserialización:** Convertir objetos de memoria a secuencias de bytes para transmitirlos por la red.
2. **Latencia de Red:** Transferir gigabytes de particiones intermedias entre switches de red durante un *Shuffle*.
3. **Sobrecosto de Coordinación:** El Driver debe compilar el DAG, programar miles de tareas en hilos remotos y gestionar fallos de comunicación.
4. **Tiempo de Arranque (*Cold Start*):** Un clúster de Spark o EMR tarda entre 3 y 8 minutos en aprovisionar sus máquinas virtuales antes de procesar el primer byte.

Si el volumen de datos a procesar cabe en la memoria de un solo nodo (menos de 100–200 GB), **un motor en proceso como DuckDB suele ser entre 3 y 10 veces más rápido que un clúster de Spark**, costando hasta un 95% menos en infraestructura cloud.

2A.2 Arquitectura Interna de DuckDB

DuckDB alcanza su rendimiento excepcional mediante tres pilares de ingeniería de sistemas:

1. Motor Columnar Vectorizado

A diferencia de los motores tradicionales que procesan una tupla a la vez mediante el modelo de iteradores de Volcán (*Volcano Iterator Model*), DuckDB procesa **vectores de datos contiguos en memoria** (típicamente fragmentos de 2.048 valores). Esto maximiza la utilización de la caché L1/L2 del procesador y permite al compilador emitir instrucciones SIMD (*Single Instruction, Multiple Data*).

2. Paralelismo Guiado por Porciones (*Morsel-Driven Parallelism*)

DuckDB divide el flujo de datos en pequeñas porciones (*morsels*) de aproximadamente 100.000 tuplas. Los hilos de trabajo de la CPU reclaman dinámicamente estas porciones de una cola compartida sin bloqueos pesados (*lock-free queues*), garantizando una distribución de carga homogénea entre todos los núcleos disponibles.

3. Procesamiento Fuera de Núcleo (*Out-of-Core Execution*)

Si una consulta analítica requiere procesar un archivo Parquet de 80 GB en una máquina con solo 16 GB de RAM física, muchos motores colapsan con excepciones de *OutOfMemory*. DuckDB cuenta con un gestor de buffers propio que vuelca bloques intermedios a disco temporalmente

(*Disk Spilling Streaming*), permitiendo resolver consultas analíticas masivas que superan la memoria disponible sin interrumpir la ejecución.

2A.3 Consultas Directas sobre Parquet, CSV y S3

Una de las facultades más poderosas de DuckDB es la capacidad de consultar archivos crudos sin necesidad de crear tablas previas ni importar datos:

```SQL
-- Consultar directamente archivos Parquet locales con SQL analítico estándar
SELECT
    pais,
    categoria_producto,
    COUNT(transaccion_id) AS total_ventas,
    ROUND(SUM(monto_usd), 2) AS facturacion_total,
    ROUND(AVG(monto_usd), 2) AS ticket_promedio
FROM 'data/ventas.parquet'
WHERE estado_pago = 'COMPLETADO'
  AND fecha_transaccion >= '2026-01-01'
GROUP BY pais, categoria_producto
ORDER BY facturacion_total DESC
LIMIT 5;
```

Integración en Python y Transferencia Zero-Copy hacia Arrow

DuckDB se integra de forma transparente con el ecosistema de Python. Puede leer DataFrames de Pandas o tablas de Apache Arrow y devolver resultados sin copiar datos en memoria (*Zero-Copy Memory Sharing*):

```python
import duckdb
import pyarrow.parquet as pq

# Conexión en memoria volátil (o persistente indicando una ruta de archivo)
conexion = duckdb.connect(database=":memory:")

# 1. Consulta SQL directa sobre múltiples archivos Parquet con patrones glob
consulta_sql = """
    SELECT
        DATE_TRUNC('month', CAST(fecha_transaccion AS DATE)) AS mes,
        COUNT(DISTINCT cliente_id) AS clientes_unicos,
        SUM(monto_usd) AS volumen_total
    FROM 'data/ventas.parquet'
    GROUP BY 1
    ORDER BY 1 ASC
"""

# 2. Retorno directo como DataFrame de Pandas o Tabla Arrow
df_resultado_arrow = conexion.execute(consulta_sql).arrow()
df_resultado_pandas = conexion.execute(consulta_sql).df()

print(df_resultado_pandas)
```

Consultas Federadas sobre Amazon S3 / Azure ADLS

Mediante la extensión oficial `httpfs`, DuckDB lee archivos remotos en almacenamiento de objetos en la nube, descargando únicamente los metadatos y las columnas requeridas (*Projection and Predicate Pushdown* sobre HTTP):

PYTHON

```python
import duckdb

con = duckdb.connect()

# Habilitar extensión HTTP/S3
con.execute("INSTALL httpfs; LOAD httpfs;")

# Configurar credenciales cloud seguras mediante el gestor de secretos de DuckDB
# (Lee automáticamente variables de entorno AWS_ACCESS_KEY_ID o roles IAM de la instancia)
con.execute("""
    CREATE SECRET secreto_s3 (
        TYPE S3,
        PROVIDER CREDENTIAL_CHAIN,
        REGION 'us-east-1'
    );
""")

# Consulta remota que descarga únicamente los bytes necesarios de S3
df_s3 = con.execute("""
    SELECT categoria, SUM(monto) AS total
    FROM 's3://mi-bucket-lakehouse/silver/pedidos/*.parquet'
    WHERE anio = 2026
    GROUP BY categoria
""").df()
```

2A.4 Matriz de Decisión: Pandas vs. Polars vs. DuckDB vs. Apache Spark

Para elegir la herramienta adecuada en un pipeline de datos, utilice la siguiente matriz técnica comparativa:

Criterio	Pandas	Polars	DuckDB	Apache Spark
Paradigma Primario	DataFrames en memoria (Python).	DataFrames vectorizados en Rust.	Base de datos relacional SQL en C++.	Cómputo distribuido horizontal en clúster.
Interfaz de Consulta	API imperativa/métodos de Python.	API funcional de expresiones encadenadas.	**SQL Estándar ANSI** de primera clase.	PySpark DataFrame API y Spark SQL.
Escalabilidad Máxima	~5–10 GB (limitado por RAM del nodo).	~100–300 GB (Single-Node multihilo).	**~100–500 GB (Single-Node con Out-of-Core)**.	**Petabytes (Multi-Node horizontal)**.
Manejo de Memoria	Alto consumo (objetos Python, copias).	Excelente (Apache Arrow nativo en Rust).	Excelente (gestor de buffers con volcado a disco).	Tungsten on-heap/off-heap con JVM.
Tiempo de Arranque	< 1 segundo.	< 1 segundo.	**< 10 milisegundos**.	3 a 7 minutos en clúster cloud.
Caso de Uso Óptimo	Análisis exploratorio rápido y ciencia de datos básica.	Transformaciones de alta velocidad en una máquina.	**Pipelines analíticos SQL, serverless, pruebas CI/CD.**	**Cargas masivas que exceden un solo servidor físico.**

2A.5 Pipelines Serverless de Bajo Costo con AWS Lambda y DuckDB

Un patrón de arquitectura moderno consiste en ejecutar pipelines analíticos ligeros sobre **AWS Lambda** utilizando la capa precompilada de DuckDB.

En lugar de mantener un clúster de EMR o Databricks encendido 24/7 (con costos mínimos de cientos de dólares al mes), una función Lambda con 2 GB de memoria ejecuta transformaciones de 5 GB de datos en 8 segundos por **menos de $0.0001 USD por ejecución**:

```python
# Handler de AWS Lambda para transformación de lotes diarios
import os
import duckdb

def lambda_handler(event, context):
    bucket = event.get("bucket", "empresa-datos-crudos")
    fecha_proceso = event.get("fecha", "2026-03-31")

    con = duckdb.connect(database=":memory:")
    con.execute("INSTALL httpfs; LOAD httpfs;")

    # Ingesta, filtrado, deduplicación y exportación directa de Parquet a Parquet
    query = f"""
        COPY (
            SELECT
                transaccion_id,
                cliente_id,
                monto_usd,
                fecha_transaccion
            FROM 's3://{bucket}/raw/{fecha_proceso}/*.parquet'
            WHERE monto_usd > 0
            QUALIFY ROW_NUMBER() OVER (PARTITION BY transaccion_id ORDER BY
fecha_transaccion DESC) = 1
        ) TO 's3://{bucket}/silver/transacciones_limpias_{fecha_proceso}.parquet'
        (FORMAT PARQUET, COMPRESSION 'ZSTD');
    """

    con.execute(query)
    return {"status": "SUCCESS", "fecha": fecha_proceso}
```

2A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Forzar el Uso de Apache Spark para Datasets Menores a 50 Gigabytes
Crear un clúster de Spark para procesar una exportación diaria de 5 GB incurre en minutos de aprovisionamiento de máquinas, facturación mínima de nodos y sobrecosto de shuffling.
Corrección: Utilice DuckDB o Polars en un contenedor ligero de Docker o una máquina virtual pequeña; el pipeline tardará segundos en lugar de minutos y costará centavos.

Anti-patrón 2: Tratar a DuckDB como una Base de Datos Transaccional OLTP Concurrente
Intentar utilizar DuckDB como el backend de una aplicación web donde cientos de usuarios realizan transacciones concurrentes simultáneas (`INSERT` y `UPDATE` de milisegundos) provocará bloqueos de base de datos.
Corrección: DuckDB está optimizado para lecturas analíticas masivas y cargas por lotes (OLAP); para concurrencia transaccional multi-usuario utilice PostgreSQL o MySQL.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 2A.1 (Agregación Multi-Archivo y Detección de Anomalías con DuckDB):
Se tienen cientos de archivos mensuales particionados en disco: `data/particiones/ventas_*.parquet`.
Escriba una consulta SQL pura ejecutable en DuckDB que lea todos los archivos Parquet en una sola pasada y obtenga:

1. El total de transacciones por categoría.
2. La facturación acumulada.
3. El porcentaje que representa cada categoría sobre el gran total nacional utilizando una función de ventana `SUM() OVER ()`.

Solución 2A.1:

```SQL
WITH metricas_categoria AS (
    SELECT
        categoria_producto,
        COUNT(transaccion_id) AS total_transacciones,
        SUM(monto_usd) AS facturacion_categoria
    FROM 'data/particiones/ventas_*.parquet'
    WHERE estado_pago = 'COMPLETADO'
    GROUP BY categoria_producto
)
SELECT
    categoria_producto,
    total_transacciones,
    ROUND(facturacion_categoria, 2) AS facturacion_categoria,
    ROUND(
        (facturacion_categoria / SUM(facturacion_categoria) OVER ()) * 100,
        2
    ) AS porcentaje_facturacion_nacional
FROM metricas_categoria
ORDER BY facturacion_categoria DESC;
```

2A.7 Resumen y Puntos Clave

- **Desmitificación de la escala:** La mayor parte de las cargas analíticas caben en la RAM de un solo servidor; el cómputo distribuido impone penalizaciones de red y coordinación que deben justificarse formalmente.
- **DuckDB como motor OLAP en proceso:** Ofrece ejecución columnar vectorizada en C++, paralelismo multihilo y capacidad fuera de núcleo (*Out-of-Core*) sin requerir servidores ni clústeres.

- **Acceso directo a formatos abiertos:** Permite ejecutar SQL analítico sobre archivos Parquet locales o en S3/ADLS con poda de columnas y predicados sin duplicación de datos.
- **Complementariedad con Spark:** DuckDB optimiza pipelines locales, serverless (Lambda) y pruebas en CI/CD; Apache Spark se reserva para volúmenes masivos que exceden la capacidad física de una máquina individual.

Lecturas Recomendadas

1. **Raasveldt, M. & Mühleisen, H. (2019).** *DuckDB: An Embeddable Analytical Database.* Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 10: Batch Processing).* O'Reilly Media.
3. **DuckDB Foundation (2024).** *DuckDB Documentation: Performance Tuning, Extensions, and Python API.* DuckDB.org.

CAPÍTULO 3

De Hadoop y MapReduce al Cómputo en Memoria NÚCLEO

> *"Antes de Spark existía Hadoop, que introdujo el paradigma de MapReduce para procesar grandes volúmenes de datos en clústeres. El problema fundamental era que procesaba en disco: cada etapa leía y grababa constantemente, lo que hacía todo lento y costoso. Spark revolucionó la industria llevando el cómputo directamente a la memoria RAM."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Analizar la arquitectura fundacional de Apache Hadoop (HDFS, MapReduce, YARN) y sus limitaciones de E/S.
- Entender la transición del procesamiento por lotes centrado en disco hacia la computación distribuida en memoria (*in-memory computation*).
- Describir cómo Apache Spark logra órdenes de magnitud de mayor rendimiento en algoritmos iterativos y analítica interactiva.
- Evaluar cuándo utilizar procesamiento por lotes tradicional y cuándo adoptar motores de memoria de baja latencia.

Definición 3.1 (Escribir en la Pizarra vs. Grabar en Tablas de Piedra) Imagina un equipo de matemáticos resolviendo un problema complejo de 20 ecuaciones sucesivas.

El enfoque de Hadoop MapReduce es como obligar a los matemáticos a tallar el resultado de cada ecuación en una pesada tabla de piedra (disco HDFS), guardarla en la bodega y luego sacarla para la siguiente ecuación. Tallar la piedra toma el 95% del tiempo total.

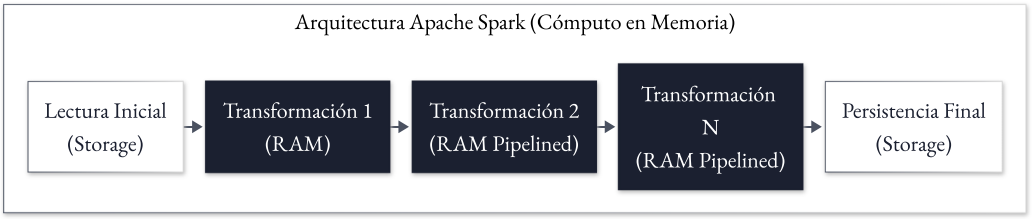
El enfoque de Apache Spark es dotar a cada matemático de una pizarra acrílica (memoria RAM): calculan de inmediato, encadenan las operaciones sin soltar el marcador y solo tallan en piedra el resultado final de la ecuación 20 cuando el trabajo está terminado.

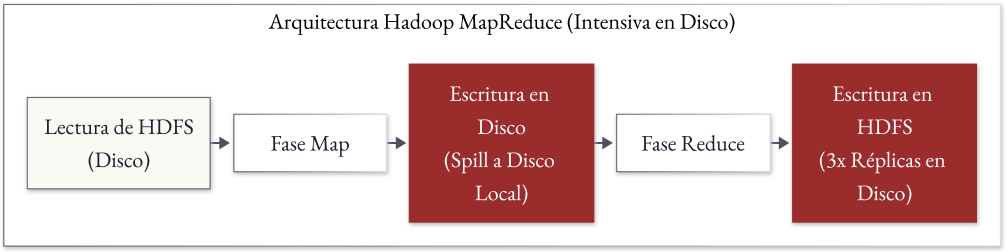
3.1 El Paradigma Fundacional: Hadoop y MapReduce

A principios de la década de 2000, el volumen de datos de la web superó la capacidad de las bases de datos relacionales tradicionales y los sistemas multiprocesador de memoria compartida (*SMP*). En respuesta, Google publicó sus artículos seminales sobre *Google File System* (GFS) y *MapReduce*, sentando las bases de lo que se convertiría en **Apache Hadoop**, desarrollado por Doug Cutting y Mike Cafarella.

Hadoop democratizó el análisis de Big Data permitiendo a las organizaciones procesar petabytes en clústeres de servidores comerciales de bajo costo (*commodity hardware*), apoyándose en tres pilares:

1. **HDFS (Hadoop Distributed File System):** Sistema de archivos distribuido que divide archivos masivos en bloques (típicamente 128 MB) y los replica en múltiples nodos del clúster para tolerancia a fallos.
2. **YARN (Yet Another Resource Negotiator):** Capa de gestión de infraestructura que orquesta CPU y memoria entre aplicaciones concurrentes.
3. **MapReduce:** Modelo de programación funcional dividido en dos primitivas esenciales:
    - **Map:** Transforma y proyecta registros locales sin dependencias externas.
    - **Reduce:** Agrupa y sintetiza los valores intermedios tras una fase obligatoria de shuffle.





3.2 Profundización Técnica: El Cuello de Botella de E/S y la Revolución de Spark

El talón de Aquiles de MapReduce es la **persistencia obligatoria entre fases**. Si un pipeline analítico requería encadenar 5 trabajos consecutivos de MapReduce (por ejemplo en algoritmos de machine learning iterativo o consultas complejas), el sistema debía escribir a HDFS y replicar tres veces por la red tras cada paso intermedio.

En 2009, en el laboratorio AMPLab de la Universidad de California en Berkeley, Matei Zaharia y su equipo diseñaron **Apache Spark** con una premisa revolucionaria: *retener las colecciones de datos en la memoria RAM distribuida entre operaciones sucesivas*.

Al evitar la lectura y escritura constante en disco, Spark alcanzó velocidades hasta **100 veces superiores en memoria** y 10 veces superiores en disco frente a MapReduce, además de introducir el concepto de **DAG** para ejecutar operaciones en cadena sin serializaciones innecesarias.

> **Consideraciones de Producción 3.1: Cómputo Iterativo en Memoria** El verdadero poder de la computación en memoria se evidencia en algoritmos iterativos (como K-Means, Logistic Regression o PageRank), donde los datos se leen una sola vez y se reutilizan en memoria en cada ciclo.

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Capitulo03-Hadoop-vs-Spark") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset masivo una única vez
df_base = spark.read.parquet("data/interacciones.parquet")

# Persistir en la memoria RAM del clúster (In-memory computation)
# Esto replica el salto histórico frente a Hadoop: cero lecturas de disco
posteriores
df_base.cache()

# Algoritmo iterativo: Múltiples pasadas sobre los mismos datos en memoria
for iteracion in range(3):
    umbral = iteracion * 10
    total_segmento = df_base.filter(col("score") > umbral).count()
    print(f"Iteración {iteracion + 1} completada en memoria RAM: {total_segmento}
registros")

# Liberar memoria al finalizar el ciclo
df_base.unpersist()
```

3.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Creer que Hadoop está Muerto

Un error conceptual común es afirmar que "Hadoop ya no existe". Si bien el motor de cómputo *MapReduce* ha sido reemplazado en la gran mayoría de casos por Spark, el sistema de archivos distribuido (HDFS) y el gestor de recursos (YARN) continúan siendo la columna vertebral de miles de clústeres on-premise corporativos.

Anti-patrón 2: Cachear Datos sin Suficiente Capacidad de RAM

Llamar a `df.cache()` indiscriminadamente sobre datasets que superan la memoria disponible del clúster provocará que Spark expulse particiones a disco (*Spill to Disk*) o genere pausas prolongadas del recolector de basura de Java (*Garbage Collection Overhead*).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 3.1 (Comparativa Arquitectónica):

Elabora una tabla comparativa entre Hadoop MapReduce y Apache Spark analizando: (1) Medio de almacenamiento intermedio, (2) Modelo de tolerancia a fallos, y (3) Soporte para procesamiento en streaming en tiempo real.

Solución:

Criterio	Hadoop MapReduce	Apache Spark
Almacenamiento Intermedio	Disco rígido (HDFS con triple réplica)	Memoria RAM distribuida (con spill a disco solo si se llena)
Tolerancia a Fallos	Replicación física de archivos en disco	Linaje lógico del DAG (reconstrucción de particiones perdidas)
Streaming	No soportado de forma nativa (solo batch)	Nativo mediante Spark Structured Streaming

3.4 Resumen y Puntos Clave

- **Hadoop abrió el camino:** Resolvió el almacenamiento masivo con HDFS y demostró que la computación distribuida en hardware comercial era viable.
- **El salto a la memoria:** Apache Spark eliminó el cuello de botella de E/S de disco, habilitando la analítica iterativa, pipelines en cadena y streaming en un único motor unificado.
- **Tolerancia a fallos sin replicación en disco:** A través del linaje de operaciones (DAG), Spark puede reconstruir datos perdidos sin pagar el costo de replicar cada paso intermedio en almacenamiento físico.

Lecturas Recomendadas

1. **White, T. (2015).** *Hadoop: The Definitive Guide (4th Edition).* O'Reilly Media.
2. **Zaharia, M. et al. (2010).** *Spark: Cluster Computing with Working Sets.* HotCloud '10.

CAPÍTULO 4

Arquitectura de Clúster: Driver, Workers y Ejecutores NÚCLEO

> *"Un clúster de Spark no es una máquina gigante, sino una orquesta coordinada: el Driver es el director que interpreta la partitura; el Cluster Manager es el administrador del teatro que asigna las sillas; y los Ejecutores son los músicos que tocan en paralelo en cada nodo trabajador."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la topología física y lógica de un clúster de computación distribuida.
- Identificar las responsabilidades exclusivas del **Driver Program** frente a los **Worker Nodes**.
- Comparar los diferentes administradores de clúster soportados por Spark: Standalone, Apache YARN y Kubernetes.
- Analizar el ciclo de vida y la configuración interna de la JVM de los **Ejecutores** (*Executors*).

Definición 4.1 (La Flota de Transporte y la Torre de Control) Imagina una mega-empresa de mudanzas encargada de trasladar un rascacielos entero:

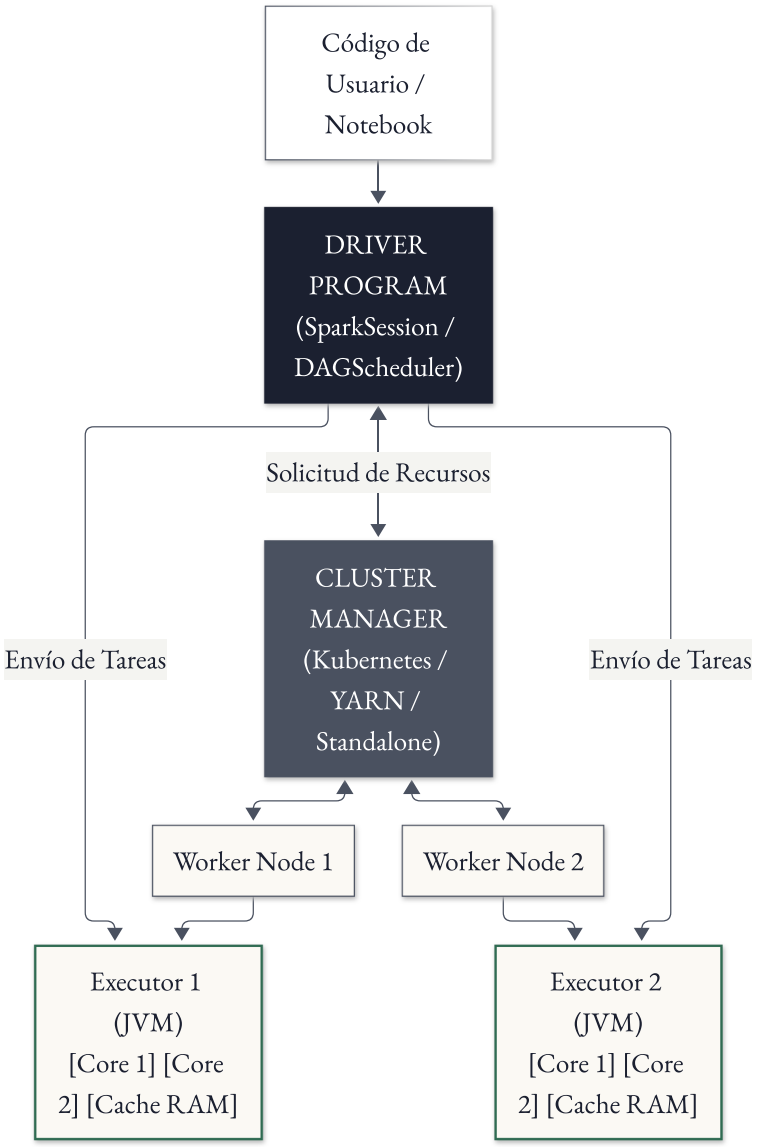
- El **Driver Program** es el jefe de operaciones en la torre de control: revisa el inventario, descompone la mudanza en tareas pequeñas y supervisa el avance general.
- El **Cluster Manager** es la agencia de alquiler de vehículos: el jefe le pide 20 camiones con 4 operarios cada uno, y la agencia los despacha según disponibilidad.
- Los **Worker Nodes** son los estacionamientos físicos donde se encuentran los camiones.
- Los **Ejecutores** (*Executors*) son los camiones con sus operarios: cargan las cajas asignadas (particiones de datos), suben los muebles al ascensor (tareas de cómputo) y reportan a la torre cuando finalizan.

4.1 La Anatomía de un Clúster de Spark

En la ingeniería de datos a escala petabyte, ninguna máquina individual dispone de suficiente memoria RAM o capacidad de CPU para almacenar y procesar la información. El **Cluster Computing** interconecta un conjunto de servidores independientes para que operen como un sistema unificado y tolerante a fallos.

La arquitectura de una aplicación Spark se basa en el patrón maestro/esclavo (*master/worker*):

1. **Driver Program (El Cerebro):**
    - Es el proceso que ejecuta la función `main()` del programa del usuario.
    - Crea y administra la sesión de Spark (`SparkSession` / `SparkContext`).
    - Convierte el código del usuario en un grafo lógico y físico (DAG).
    - Programa y distribuye las tareas individuales (*Tasks*) a través del planificador de tareas (*Task Scheduler*).
    - Recopila los resultados de las acciones de agregación final.
2. **Cluster Manager (El Asignador de Recursos):**
    - Servicio de infraestructura pluggable que adquiere y libera recursos del clúster físico.
    - Spark soporta los siguientes modos principales:
        - **Standalone:** Gestor nativo simple incluido en Spark.
        - **Apache YARN:** Gestor corporativo tradicional en entornos Hadoop.
        - **Kubernetes (K8s):** El estándar moderno de la industria cloud-native basado en contenedores.
        - **Mesos (obsoleto):** su soporte fue declarado obsoleto en Spark 3.2 y eliminado en Spark 4.0; no debe usarse en proyectos nuevos.
3. **Worker Nodes (La Infraestructura Física):**
    - Servidores o instancias de cómputo que ejecutan los procesos esclavos asignados por el Cluster Manager.
4. **Ejecutores (Executors - La Fuerza de Trabajo):**
    - Procesos JVM (*Java Virtual Machine*) que residen dentro de los Worker Nodes.
    - Cada ejecutor tiene asignada una cantidad fija de memoria RAM y núcleos de CPU (*cores*).
    - Ejecutan las tareas asignadas por el Driver y almacenan en memoria/disco los datos cacheados del DataFrame.



4.2 Profundización Técnica: Ciclo de Vida y Asignación de Recursos

Los ejecutores existen **exclusivamente durante el ciclo de vida de la aplicación Spark** que los solicitó:

1. **Aislamiento estricto:** Cada aplicación dispone de sus propios procesos ejecutores. Dos aplicaciones concurrentes en el mismo clúster no comparten espacio de memoria JVM, lo que garantiza seguridad y previene corrupciones cruzadas.

2. **Asignación Dinámica (Dynamic Allocation):** Mediante `spark.dynamicAllocation.enabled = true`, Spark solicita ejecutores adicionales cuando la cola de tareas crece y los devuelve al Cluster Manager cuando el clúster entra en inactividad, optimizando costos en la nube.
3. **Tolerancia a fallos de ejecutores:** Si un Worker Node se apaga o un ejecutor falla por un error del kernel, el Cluster Manager lanza inmediatamente un nuevo ejecutor en otro nodo. El Driver reasigna las tareas pendientes y reconstruye las particiones perdidas a través del linaje del DAG.

> **Consideraciones de Producción 4.1: Inspección de Ejecutores Activos** PySpark **no** expone la lista de ejecutores en `statusTracker()` (ese método existe solo en la API de Scala/Java). La vía soportada para auditar ejecutores, memoria y núcleos es la **REST API de monitoreo** que el Driver publica junto a la Spark UI.

PYTHON

```python
import requests
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Capitulo04-ArquitecturaCluster") \
    .getOrCreate()

sc = spark.sparkContext

# 1. Identidad de la aplicación y paralelismo por defecto
print(f"Spark Version: {sc.version}")
print(f"Master URL: {sc.master}")
print(f"Application ID: {sc.applicationId}")
print(f"Paralelismo por defecto: {sc.defaultParallelism}")

# 2. Ejecutores conectados, consultados a la REST API del Driver
url = f"{sc.uiWebUrl}/api/v1/applications/{sc.applicationId}/executors"
ejecutores = requests.get(url, timeout=10).json()

print(f"\nEjecutores registrados (incluye al driver): {len(ejecutores)}")
for ex in ejecutores:
    print(f" -> id={ex['id']:>6}  host={ex['hostPort']}  cores={ex['totalCores']}
"
          f"tareas activas={ex['activeTasks']}  memoria máx={ex['maxMemory'] /
1024**2:.0f} MiB")
```

4.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutores "Demasiado Gordos" (Fat Executors)

Configurar ejecutores gigantes con 64 GB de RAM y 16 núcleos en una sola JVM tiende a producir pausas largas de *Garbage Collection* (GC) y cuellos de botella de I/O en HDFS. La heurística más difundida (popularizada por Cloudera y reflejada en guías de ajuste de Spark) es usar **alrededor de 5 núcleos por ejecutor** y memorias del orden de decenas de GB; es un punto de partida empírico, no una ley, y debe validarse midiendo GC en la Spark UI.

Anti-patrón 2: Ejecutores "Demasiado Enanos" (Tiny Executors)

Asignar 1 núcleo y 1 GB de RAM por ejecutor desperdicia memoria en la sobrecarga base de la JVM de cada proceso y elimina los beneficios de procesar múltiples hilos en memoria compartida.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 4.1 (Cálculo de Dimensionamiento de Clúster):

Dispones de 10 nodos trabajadores en AWS (instancias de 16 vCPUs y 64 GB de RAM cada una). Dejando 1 vCPU y 2 GB por nodo para el sistema operativo y el demonio de YARN/K8s, ¿cuántos ejecutores de 5 núcleos deberías solicitar y con cuánta memoria por ejecutor?

Solución: Cálculo paso a paso:

1. *Recursos disponibles por nodo:* 15 vCPUs y 62 GB de RAM.
2. *Ejecutores por nodo:* 15 vCPUs / 5 núcleos por ejecutor = **3 ejecutores por nodo**.
3. *Memoria bruta por ejecutor:* 62 GB / 3 = ~20.6 GB.
4. *Reserva de overhead de memoria (10%):* ~2 GB para `spark.executor.memoryOverhead`.
5. *Configuración final:* `spark.executor.instances = 29` (dejando 1 ejecutor reservado para el Driver ApplicationMaster), `spark.executor.cores = 5`, `spark.executor.memory = 18g`.

4.4 Resumen y Puntos Clave

- **Separación de roles:** El Driver coordina y planifica; el Cluster Manager asigna hardware; los Ejecutores procesan particiones en paralelo.
- **Aislamiento JVM:** Cada aplicación opera en un entorno de memoria desacoplado de las demás.
- **Dimensionamiento balanceado:** Ni ejecutores gigantescos ni diminutos; 4 a 5 núcleos por ejecutor es el equilibrio óptimo de rendimiento y recolección de basura.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide.* O'Reilly Media. (Capítulo 15: How Spark Runs on a Cluster).
2. **Ryza, S. (2014).** *How-to: Tune Your Apache Spark Jobs (Part 2).* Cloudera Engineering Blog.

CAPÍTULO 5

Entorno de Configuración y SparkSession NÚCLEO

> *"SparkSession es el punto de entrada unificado al universo de Spark: permite crear y manipular DataFrames, ejecutar consultas SQL, cargar archivos Parquet o JSON y definir los parámetros de paralelismo y memoria que gobernarán el clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución desde los contextos dispersos de Spark 1.x (`SparkContext`, `SQLContext`, `HiveContext`) hacia el punto de entrada unificado `SparkSession` de Spark 2.x y 3.x.
- Dominar el patrón de diseño Builder para inicializar y parametrizar aplicaciones PySpark de forma segura.
- Utilizar `SparkConf` para inyectar configuraciones dinámicas desde archivos JSON o variables de entorno.
- Configurar parámetros críticos de memoria, serialización y paralelismo antes de disparar cómputo en el clúster.

Definición 5.1 (La Llave Maestra del Edificio Inteligente) En las versiones tempranas de Spark, operar el sistema era como llevar un manojo de cinco llaves metálicas pesadas: necesitabas la llave del ascensor (SparkContext), la llave de la sala de conferencias (SQLContext) y la llave del archivo histórico (HiveContext). Si olvidabas una, no podías trabajar.

SparkSession es una tarjeta digital de acceso unificado: una sola llave inteligente que te abre todas las puertas del edificio, conecta con el catálogo de datos, ejecuta SQL y lee cualquier formato de archivo con un único apretón de manos.

5.1 El Punto de Entrada Unificado: SparkSession

En Apache Spark, `SparkSession` actúa como el "corazón" programático de la aplicación. Introducida en Spark 2.0 y consolidada como el estándar indiscutible de la industria, unifica todas las capacidades del motor bajo una misma interfaz fluida.

Con una instancia de `SparkSession` puedes:

1. Crear y manipular DataFrames a partir de fuentes externas o colecciones nativas de Python.
2. Ejecutar código SQL directamente sobre los datos mediante `spark.sql()`.
3. Leer archivos en formatos columnares o estructurados (`parquet`, `json`, `csv`, `orc`, `delta`, `avro`).
4. Acceder y alterar dinámicamente configuraciones internas del runtime mediante `spark.conf`.

El Patrón Builder

Debido a que una aplicación distribuida requiere configurar múltiples parámetros antes de que el motor inicie, SparkSession se construye utilizando el **patrón Builder**:

PYTHON

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiPipelineDeDatos") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()
```

El método `.getOrCreate()` garantiza que si ya existe una sesión activa en la JVM (por ejemplo al trabajar en Databricks, Jupyter o EMR), Spark reutiliza la sesión existente en lugar de lanzar un segundo contexto conflictivo.

5.2 Profundización Técnica: SparkConf y el Puente Py4J

¿Cómo se comunica Python con un motor escrito en Scala y Java? PySpark utiliza una pasarela llamada **Py4J**:

1. Tu código Python invoca métodos en el Driver.
2. Py4J traduce cada llamada a un socket TCP local comunicándose con la JVM subyacente.

3. `SparkConf` es la clase que define los parámetros inmutables del clúster antes de que arranque la JVM. Parámetros como el tamaño del Driver (`spark.driver.memory`) o el master URL (`spark.master`) **deben definirse en SparkConf antes de inicializar la sesión**; una vez encendida la JVM, no pueden modificarse en caliente.

> **Consideraciones de Producción 5.1: Carga de Configuraciones desde JSON** En proyectos de ingeniería de producción nunca se deben quemar configuraciones en duro en el código. El patrón profesional consiste en separar la configuración en un archivo externo (JSON o YAML) e inyectarla dinámicamente a través de SparkConf.

PYTHON

```
import json
import os
from pyspark import SparkConf
from pyspark.sql import SparkSession

```python
# 1. Configuración de parámetros en SparkConf
conf = SparkConf()
conf.set("spark.app.name", "PipelineIngestaProduccion")
conf.set("spark.master", "local[*]")
conf.set("spark.sql.shuffle.partitions", "16")
conf.set("spark.sql.adaptive.enabled", "true")

# Nota técnica: KryoSerializer optimiza la serialización de RDDs y clases
Java/Scala personalizadas;
# en la API de DataFrames, Spark utiliza internamente los codificadores binarios
de Tungsten.
conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")

# 2. Inicializar SparkSession aplicando el objeto SparkConf
spark = SparkSession.builder \
    .config(conf=conf) \
    .getOrCreate()
```

5.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Intentar cambiar memoria en tiempo de ejecución

Escribir `spark.conf.set("spark.executor.memory", "8g")` después de llamar a `getOrCreate()` no produce ningún efecto. La memoria de los ejecutores es asignada por el Cluster Manager en el instante del arranque del proceso JVM. Toda asignación de hardware debe pasarse vía `spark-submit --executor-memory 8g` o en el `SparkConf` inicial.
```

Anti-patrón 2: No cerrar la sesión al finalizar scripts por lotes
En pipelines orquestados por Airflow o scripts standalone, olvidar llamar a `spark.stop()` mantiene la aplicación activa en YARN o Kubernetes, consumiendo recursos y bloqueando cuotas de cómputo del clúster innecesariamente.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 5.1 (Serialización Kryo frente a Codificadores de Tungsten):
¿Por qué configurar `spark.serializer = org.apache.spark.serializer.KryoSerializer` es crítico cuando se opera con RDDs o tipos personalizados, pero tiene un impacto menor en pipelines que utilizan exclusivamente la API DataFrame?

Solución 5.1:

- **En la API DataFrame:** Spark emplea el motor **Tungsten**, el cual almacena y serializa los datos en formato binario compacto fuera del modelo de objetos tradicional de Java mediante codificadores (*Encoders*) y representaciones en memoria `UnsafeRow`. Por ello, las filas de un DataFrame no se serializan con Java ni con Kryo durante las operaciones analíticas estándar.
- **En RDDs y tipos no gestionados:** Cuando se manipulan colecciones de objetos Java/Scala arbitrarios o RDDs con `groupByKey` o `reduceByKey`, el serializador predeterminado de Java introduce una sobrecarga masiva de metadatos de clases. En este escenario, **Kryo** es esencial: serializa objetos binariamente de forma compacta (hasta 10 veces más rápido y con una reducción de tamaño del 50% al 75% frente a Java), descongestionando la memoria y la red durante la fase de shuffle.

5.4 Resumen y Puntos Clave

- **Punto de entrada único:** `SparkSession` unifica APIs estructuradas, ejecución SQL y conectividad de almacenamiento.
- **Inmutabilidad de arranque:** Configuraciones críticas de memoria y master se definen al inicio mediante `SparkConf` o flags de `spark-submit`.
- **Cierre disciplinado:** Toda aplicación por lotes debe finalizar con `spark.stop()` para liberar los recursos asignados por el clúster.

Lecturas Recomendadas

1. **Documentación Oficial de Apache Spark:** *Spark Configuration Guide* (`https://spark.apache.org/docs/latest/configuration.html`).
2. **Damji, J. et al. (2020).** *Learning Spark (2nd Edition).* O'Reilly Media. (Capítulo 2: Downloading Apache Spark and Getting Started).

CAPÍTULO 7

Ingesta de Datos: Archivos, JDBC Batch y Fuentes Incrementales NÚCLEO

> *"La ingesta es el primer puerto de entrada al lago de datos: a través del DataFrameReader podemos montar rutas de almacenamiento de objetos, inferir o forzar esquemas estrictos, conectarnos en paralelo a bases de datos relacionales vía JDBC y capturar incrementos controlados mediante columnas de marca de agua."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Utilizar el objeto `DataFrameReader` (`spark.read`) para cargar datos desde almacenamiento de objetos (Amazon S3, Azure ADLS Gen2, GCP GCS) y sistemas de archivos locales.
- Configurar opciones críticas de lectura: delimitadores, encabezados, manejo de caracteres corruptos y modos de fallo (`PERMISSIVE`, `DROPMALFORMED`, `FAILFAST`).
- Comprender la penalización de rendimiento de `inferSchema` y aplicar esquemas explícitos mediante `StructType`.
- Diseñar ingestas paralelas desde motores relacionales mediante **JDBC (`spark.read.jdbc`)** configurando `partitionColumn`, `lowerBound`, `upperBound` y `numPartitions`.
- Implementar patrones de **extracción incremental batch** basados en marcas de agua temporales (*Watermark Columns*).
- Evaluar el rol de conectores gestionados modernos (**Airbyte**, **Fivetran**, SFTP y APIs REST) frente a pipelines Spark nativos.

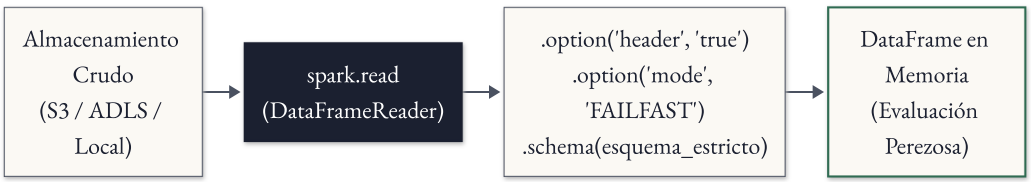
> **Definición 7.1 (La Aduana en el Puerto de Carga Analítico)** Imagine cientos de contenedores marítimos arribando a un puerto de transferencia internacional.
>
> Si el inspector de aduana decide inferir el contenido abriendo cada caja una por una (`inferSchema = True`), se formará una congestión kilométrica y el barco tardará días en desembarcar, consumiendo el doble de combustible e I/O.
>
> Un puerto moderno exige un manifiesto de carga oficial sellado de antemano (un `StructType` explícito): el operario solo valida que los contenedores coincidan con las especificaciones y los transfiere a máxima velocidad hacia las grúas de clasificación sin inspecciones redundantes.

7.1 El Objeto `DataFrameReader` y Formatos de Archivo

En PySpark, la carga de datos se canaliza a través de la propiedad `.read` de la instancia `SparkSession`, la cual retorna un objeto `DataFrameReader`.

El flujo estándar de ingesta sigue tres pasos estructurales:

1. **Definir la ruta fuente:** Puede ser una ruta local del sistema de archivos, un bucket de Amazon S3 (`s3a://`), Azure Data Lake Storage (`abfss://`) o Google Cloud Storage (`gs://`).
2. **Especificar el formato:** Mediante métodos directos (`.csv()`, `.parquet()`, `.json()`, `.orc()`, `.text()`) o el método genérico `.format("formato")`.
3. **Parametrizar opciones de lectura:** Configurar delimitadores (`sep`), presencia de cabeceras (`header`), manejo de valores nulos (`nullValue`) y políticas de parseo ante registros malformados.



Modos de Manejo de Registros Corruptos

Al ingerir fuentes de texto semiestructuradas (como CSV o JSON) es común encontrar filas incompletas o tipos de datos alterados. Spark ofrece tres modos de parseo mediante `.option("mode", ...)`:

1. **PERMISSIVE (Por defecto):** Inserta `null` en los campos corruptos y, si se define una columna especial `_corrupt_record` en el esquema, almacena la cadena cruda malformada para su posterior auditoría.

2. **DROPMALFORMED:** Descarta silenciosamente las filas que no cumplan el esquema.
3. **FAILFAST:** Lanza una excepción inmediata (`SparkException`) y detiene el job en el primer registro corrupto. Es el estándar recomendado para ingestas financieras donde la pérdida de datos es inaceptable.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType

spark = SparkSession.builder \
    .appName("Capitulo07-IngestaArchivos") \
    .master("local[*]") \
    .getOrCreate()

# Definición rigurosa del esquema oficial (Manifiesto de Carga)
esquema_pedidos = StructType([
    StructField("pedido_id", StringType(), False),
    StructField("cliente_id", IntegerType(), False),
    StructField("fecha_creacion", StringType(), True),
    StructField("monto_total", DoubleType(), True),
    StructField("estado", StringType(), True)
])

# Ingesta controlada con DataFrameReader
df_pedidos = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("sep", ",") \
    .option("mode", "FAILFAST") \
    .option("nullValue", "NA") \
    .schema(esquema_pedidos) \
    .load("data/crudo/pedidos_2026.csv")
```

7.2 Ingesta Paralela desde Bases de Datos Relacionales (JDBC)

La extracción de datos desde sistemas relacionales transaccionales (PostgreSQL, MySQL, Oracle, SQL Server) es una de las tareas más comunes del Data Engineer. Utilizar `spark.read.jdbc` sin comprender sus parámetros internos puede colapsar la base de datos origen o generar cuellos de botella extremos:

La Trampa del Hilo Único (Single-Thread Bottleneck)

Si se ejecuta:

```python
# ANTI-PATRÓN: Extracción monohilo
df_clientes = spark.read.jdbc(url=jdbc_url, table="clientes",
properties=db_properties)
```

Spark lanzará **una única tarea en un único ejecutor** para descargar millones de registros a través de una sola conexión JDBC. El resto del clúster permanecerá totalmente ocioso.

Extracción Paralela Particionada por Rango

Para extraer la tabla de forma distribuida en N hilos concurrentes, se debe especificar una columna numérica particionable:

```python
jdbc_url = "jdbc:postgresql://db-oltp.corp:5432/produccion"
db_properties = {
    "user": "analitica_read_only",
    "password": "token_secreto_vault",
    "driver": "org.postgresql.Driver",
    "fetchsize": "10000" # Tamaño del cursor por lote para evitar OOM en la JVM
}

# Ingesta paralela distribuida en 8 ejecutores
df_clientes_paralelo = spark.read.jdbc(
    url=jdbc_url,
    table="clientes",
    column="cliente_id",      # Debe ser columna numérica indexada
    lowerBound=1,             # Valor mínimo esperado
    upperBound=8000000,       # Valor máximo esperado
    numPartitions=8,          # Cantidad de conexiones paralelas simultáneas
    properties=db_properties
)
```

Spark generará internamente 8 consultas SQL concurrentes dividiendo el rango mediante operadores `WHERE`:

- *Tarea 0:* `WHERE cliente_id < 1000000`
- *Tarea 1:* `WHERE cliente_id >= 1000000 AND cliente_id < 2000000`
- ...
- *Tarea 7:* `WHERE cliente_id >= 7000000`

7.3 Extracción Incremental Batch con Marca de Agua (Watermark)

Extraer la tabla transaccional completa todos los días (*Full Table Dump*) es inviable ante tablas con cientos de millones de tuplas. El patrón de **Extracción Incremental** utiliza una columna de auditoría monotónicamente creciente (por ejemplo, `fecha_actualizacion` o `modificado_el`) para consultar únicamente las novedades:

PYTHON

```python
# 1. Recuperar la marca de agua del último lote procesado exitosamente
ultima_marca_tiempo = "2026-10-11 23:59:59"

# 2. Construir predicado de extracción incremental (Pushdown hacia la base relacional)
query_incremental = f"""
    (SELECT cliente_id, nombre, saldo, modificado_el
     FROM cuentas
     WHERE modificado_el > '{ultima_marca_tiempo}') AS novedades_cuentas
"""

df_novedades = spark.read.jdbc(
    url=jdbc_url,
    table=query_incremental,
    column="cliente_id",
    lowerBound=1,
    upperBound=5000000,
    numPartitions=4,
    properties=db_properties
)

# 3. Guardar las novedades en la capa Bronze
df_novedades.write.format("delta").mode("append").save("lakehouse/bronze/cuentas_novedades")
```

7.4 Ingesta Gestionada: Airbyte, Fivetran, SFTP y APIs REST

En la arquitectura empresarial moderna, Spark no es siempre la mejor herramienta para conectar cientos de fuentes heterogéneas:

- **Herramientas ELT Gestionadas (Airbyte / Fivetran):** Especializadas en sincronizar conectores SaaS (Salesforce, Stripe, HubSpot, Google Ads) y bases relacionales directamente hacia almacenamiento de objetos o almacenes MPP, gestionando esquemas, reintentos y paginación sin escribir código manual.
- **Archivos SFTP / Legacy:** Para archivos CSV o XML recibidos por SFTP de proveedores externos, se emplean contenedores ligeros de Python o tareas de Airflow que descargan y

depositan los datos en el bucket Landing antes de activar Spark.

- **Consumo de APIs REST:** Si la fuente expone una API REST con límites de tasa (*Rate Limits*) y paginación por cursor, un script de Python con `requests` y retroceso exponencial (*Exponential Backoff*) es superior a un clúster de Spark.

7.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Creer que `coalesce` Resuelve el Problema de Archivos Pequeños en Ingesta
Leer miles de micro-archivos de 10 KB genera una sobrecarga severa en el Driver durante la fase de listado de metadatos en almacenamiento de objetos. Aplicar `df.coalesce(1)` tras la lectura **no soluciona el cuello de botella**, ya que el Driver ya tuvo que listar y programar todas las particiones. La solución real exige compactar los archivos en reposo (mediante Delta `OPTIMIZE` o procesos de compactación programados) o ajustar `spark.sql.files.maxPartitionBytes` (128 MB por defecto) para que Spark agrupe múltiples archivos pequeños en una sola tarea física de lectura.

Anti-patrón 2: Ejecutar Ingestas JDBC sobre Tablas Transaccionales en Horas Pico sin Índices
Lanzar una consulta `spark.read.jdbc` con `numPartitions = 32` sobre una columna que no tiene índice en PostgreSQL puede provocar bloqueos de tablas y saturación de CPU en la base de datos de producción. Verifique siempre que la columna de particionamiento posea un índice B-tree y ajuste las horas de ejecución a ventanas de bajo tráfico.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 7.1 (Descubrimiento Automático de Particiones):
Si un lago de datos almacena archivos Parquet organizados como `s3://datalake/ventas/anio=2026/mes=10/datos.parquet`, ¿qué sucede si se ejecuta `spark.read.parquet("s3://datalake/ventas/")`?

Solución 7.1:
Spark implementa **Partition Discovery automático**. Al detectar la convención `clave=valor` en las rutas de directorios, Spark escanea los archivos e inyecta dinámicamente las columnas virtuales `anio` (Integer) y `mes` (Integer) en el esquema del DataFrame. Si una consulta posterior incluye `filter(col("mes") == 10)`, Catalyst aplica *Partition Pruning*, descartando la lectura de todos los directorios que correspondan a otros meses sin transferir sus archivos físicos por la red.

Ejercicio 7.2 (Tuning de Conexiones JDBC en Spark):
Un pipeline JDBC tarda 45 minutos en extraer 10 millones de filas desde MySQL. Al revisar la Spark UI, el ingeniero observa un único Stage con una sola tarea (*Task*). Explique la causa y detalle la solución en código.

Solución 7.2:

- **Causa:** La llamada `spark.read.jdbc` no especificó los parámetros de paralelismo (`column`, `lowerBound`, `upperBound`, `numPartitions`). Al omitirlos, Spark utiliza una única conexión JDBC secuencial en un solo ejecutor.
- **Solución:** Especificar una columna numérica indexada (como la clave primaria `id`) y definir el número de particiones:

```python
df = spark.read.jdbc(
    url=url, table="transacciones",
    column="id", lowerBound=1, upperBound=10000000,
    numPartitions=10, properties=props
)
```

Esto divide el trabajo en 10 tareas paralelas, reduciendo el tiempo de extracción de 45 minutos a menos de 5 minutos según la capacidad del servidor de base de datos.

7.6 Resumen y Puntos Clave

- **DataFrameReader:** La interfaz unificada para ingestar archivos y fuentes relacionales con control de esquemas y modos de fallo.
- **Esquemas explícitos:** Suprimen la doble lectura de I/O provocada por `inferSchema` y garantizan tipado determinista.
- **Paralelismo JDBC:** Exige configurar `partitionColumn`, cotas de rango y `numPartitions` sobre columnas indexadas para evitar extracciones monohilo.
- **Extracción incremental:** Minimiza el impacto sobre fuentes operacionales consultando solo registros cuya marca de agua temporal sea superior al último corte exitoso.

Lecturas Recomendadas

1. **Damji, J. et al. (2020).** *Learning Spark: Lightning-Fast Data Analytics* (2nd ed.). O'Reilly Media. (Capítulo 4: "Spark SQL and DataFrames: Built-in Data Sources").

2. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Concurrency Control and Indexes.* (`https://www.postgresql.org/docs/`).
3. **Apache Spark Documentation (2024).** *Spark SQL Programming Guide: JDBC To Other Databases.* Apache Software Foundation (`https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html`).

CAPÍTULO 8

Formatos de Archivo: Parquet, ORC y Avro NÚCLEO

> *"La elección del formato de archivo es la decisión de arquitectura física más determinante en un lago de datos: define la tasa de compresión, la eficiencia de E/S mediante lectura columnar y la capacidad de evolucionar esquemas en el tiempo."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

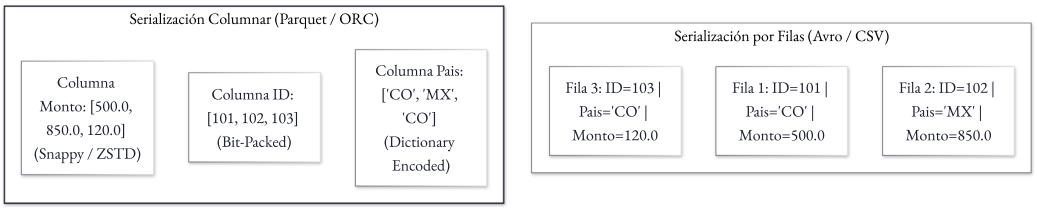
- Diferenciar formalmente entre formatos de almacenamiento orientados a filas (*Row-oriented*) y orientados a columnas (*Columnar-oriented*).
- Describir la estructura interna de Apache Parquet: *File Footer*, *Row Groups*, *Column Chunks* y páginas de datos.
- Comparar los casos de uso óptimos para Apache Parquet, Apache ORC y Apache Avro en arquitecturas de Big Data.
- Aprovechar las estadísticas de bloque (Min/Max) para habilitar poda de datos (*Data Skipping*) y optimizar costos de almacenamiento en la nube.
- Configurar algoritmos de compresión modernos (Snappy, ZSTD, Gzip) según el perfil de carga (escritura frecuente vs. lectura intensiva).

Definición 8.1 (Almacenamiento Columnar y Principio de Proyección Física) Un formato de almacenamiento **orientado a columnas** organiza los datos en disco agrupando secuencialmente todos los valores pertenecientes a una misma columna, en lugar de almacenar tuplas consecutivas de filas. Esta disposición física desacopla el costo de I/O del ancho total de la tabla (*número de columnas*): las consultas analíticas solo transfieren a memoria los bloques de bytes correspondientes a las columnas proyectadas en la consulta (*Column Pruning*).

8.1 Almacenamiento Orientado a Filas vs. Orientado a Columnas

En el diseño de sistemas analíticos modernos coexisten dos filosofías fundamentales de serialización en almacenamiento secundario:

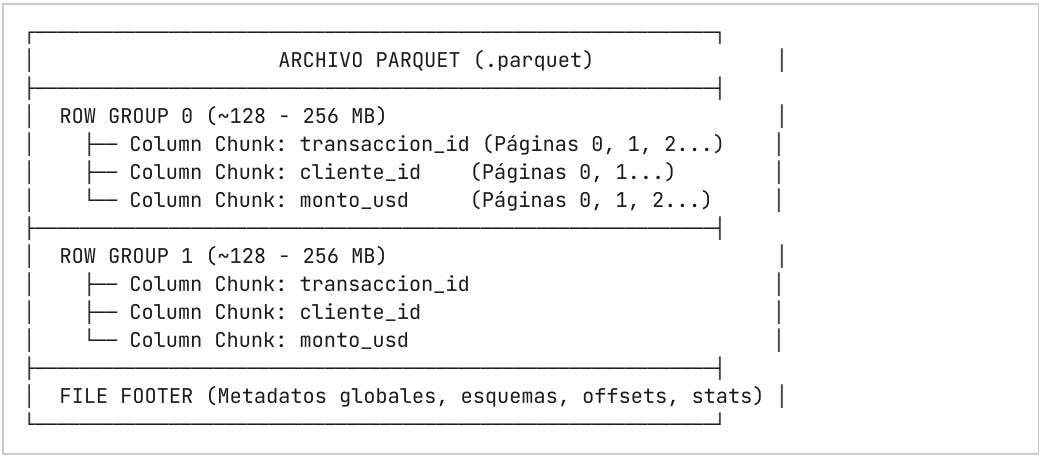
1. **Almacenamiento Orientado a Filas (Row-Oriented - CSV, JSON, Apache Avro):**
   - Cada registro se almacena de forma contigua en bloques secuenciales de disco: `[id_1, nombre_1, balance_1]`, `[id_2, nombre_2, balance_2]`.
   - *Ventaja:* Eficiencia máxima en escrituras atómicas puntuales y recuperación de entidades completas por identificador primario (ideal para cargas transaccionales OLTP y buses de mensajería como Kafka).
   - *Desventaja en analítica:* Para calcular la suma de una columna sobre 1.000 millones de filas en una tabla con 200 columnas, el motor debe leer el 100% de los bytes del archivo, desperdiciando ancho de banda de red y memoria caché.
2. **Almacenamiento Orientado a Columnas (Columnar-Oriented - Apache Parquet, Apache ORC):**
   - Todos los valores de un mismo atributo se agrupan en bloques contiguos: `[id_1, id_2, id_3...]`, `[balance_1, balance_2, balance_3...]`.
   - *Ventaja analítica (OLAP):* Permite **Column Pruning** (lectura selectiva de columnas) y algoritmos de compresión de alta densidad (como *Run-Length Encoding* y codificación por diccionario), reduciendo el tamaño en almacenamiento entre un 60% y un 85% comparado con formatos planos.



8.2 Anatomía Interna de Apache Parquet

Un archivo Apache Parquet es un contenedor binario autosuficiente estructurado jerárquicamente en cuatro capas de granularidad:

1. **File Footer (Pie de Archivo):**
   Ubicado al final del archivo físico. Contiene el esquema completo de la tabla y los punteros de desplazamiento (*byte offsets*) hacia cada grupo de filas. Los lectores de Parquet leen primero los últimos bytes del archivo para conocer la estructura completa sin escanear el cuerpo de datos.
2. **Row Groups (Grupos de Filas):**
   Particiones lógicas horizontales de los datos, típicamente dimensionadas entre 128 MB y 512 MB. Un archivo puede contener múltiples Row Groups.
3. **Column Chunks (Trozos de Columna):**
   Dentro de cada Row Group, los datos de una columna específica se almacenan en un bloque exclusivo contiguo.
4. **Data Pages (Páginas de Datos):**
   Unidad mínima e indivisible de lectura y compresión (típicamente 1 MB). Cada página contiene valores codificados y comprimidos, junto con encabezados de metadatos que almacenan estadísticas locales: valor mínimo, valor máximo, recuento de nulos y cardinalidad.



8.3 Profundización Técnica: Técnicas de Codificación y Poda de Datos (Data Skipping)

La eficiencia extrema de Parquet no radica únicamente en la compresión (ej. Snappy), sino en las técnicas de codificación previa (*Encoding*):

1. **Dictionary Encoding:** Sustituye cadenas de texto repetitivas (ej. nombres de países) por enteros pequeños (0, 1, 2) y mantiene una tabla de correspondencia en el encabezado.
2. **Run-Length Encoding (RLE):** Si un valor booleano o de estado se repite 500 veces seguidas, se almacena como el par `(500, 'APROBADO')` en lugar de escribir 500 registros.

3. **Data Skipping mediante Estadísticas Min/Max:**
   Cuando un motor ejecuta una consulta con filtro:

```sql
SELECT SUM(monto_usd) FROM ventas WHERE transaccion_id = 'TRX-998822';
```

   El motor lee el *File Footer* y los encabezados de cada *Row Group*. Si el rango estadístico de un Row Group indica `min: 'TRX-000001', max: 'TRX-500000'`, el motor **omite por completo la lectura física de ese bloque de 256 MB de disco**, reduciendo el I/O en órdenes de magnitud.

8.4 Comparativa Técnica: Parquet vs. ORC vs. Avro

Dimensión de Comparación	Apache Parquet	Apache ORC	Apache Avro
Orientación	Columnar	Columnar	Filas
Ecosistema Predilecto	Apache Spark, Databricks, Snowflake	Apache Hive, Trino, Presto	Apache Kafka, Mensajería RPC
Estructura de Bloque	Row Groups + Column Chunks	Stripes (típicamente 256 MB)	Sync Markers + Blocks
Soporte de Esquema Complejo	Excelente (anidamientos Dremel)	Excelente (estructuras y mapas)	Excelente (JSON Schema estricto)
Evolución de Esquemas	Adición y eliminación de columnas	Compatible hacia atrás	Sobresaliente (reglas estrictas)
Caso de Uso Primario	Data Lakes, Lakehouses, BI masivo	Almacenes Hive y consultas SQL	Ingesta de streaming y buses CDC

Consideraciones de Producción 8.1: Selección de Algoritmos de Compresión (Snappy vs. ZSTD)

- **Snappy (Predeterminado en Spark):** Optimizado para velocidad extrema de descompresión con consumo moderado de CPU. Ideal para capas intermedias de procesamiento (Silver) donde los datos se leen frecuentemente.
- **ZSTD (Zstandard):** Desarrollado por Meta. Ofrece ratios de compresión significativamente superiores (reducción adicional del 15-30% respecto a Snappy) con niveles de compresión ajustables. Es el estándar recomendado para capas históricas de almacenamiento (Bronze y Cold Storage) para minimizar facturación en la nube.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType

spark = SparkSession.builder \
    .appName("Capitulo08-OptimizacionFormatos") \
    .config("spark.sql.parquet.compression.codec", "zstd") \
    .config("spark.sql.parquet.filterPushdown", "true") \
    .getOrCreate()

# Escritura con compresión ZSTD y configuración de tamaño de Row Group
df_transacciones = spark.read.json("data/transacciones_crudo/")

df_transacciones.write \
    .mode("overwrite") \
    .option("compression", "zstd") \
    .parquet("lakehouse/silver/transacciones_optimizadas/")
```

8.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Almacenar Cargas Analíticas Masivas en Archivos CSV o JSON Comprimidos en Gzip

Gzip no es divisible (*splittable*); un archivo JSON de 50 GB comprimido en `.gz` debe ser procesado secuencialmente por un único ejecutor de Spark, perdiendo todo beneficio de paralelismo en el clúster.

Anti-patrón 2: Generar Millones de Archivos Parquet Diminutos (*Small Files Problem*)

Escribir archivos Parquet de menos de 10 MB multiplica la sobrecarga de lectura del metastore y el sistema de archivos. El tamaño óptimo para archivos Parquet en almacenamiento de objetos (S3, ADLS) oscila entre 128 MB y 1 GB.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 8.1 (Divisibilidad de Archivos y Compresión):
Explica la diferencia entre *Data Splittability* (capacidad de dividir un archivo en bloques paralelos) y compresión de archivos, detallando por qué Parquet es intrínsecamente divisible independientemente del códec empleado.

Solución 8.1:

- **Data Splittability:** Es la propiedad física que permite a un motor distribuido (Spark o MapReduce) asignar múltiples tareas concurrentes para procesar diferentes segmentos de un mismo archivo físico sin coordinar entre sí.
- **Impacto del códec:** En archivos de texto planos como CSV o JSON, si se comprimen con algoritmos como Gzip (`.gz`), el archivo pierde su divisibilidad porque el flujo de bytes comprimido carece de marcadores de sincronización interna (*sync markers*); un único ejecutor debe descomprimir el archivo de principio a fin de forma secuencial.
- **Divisibilidad de Parquet:** Apache Parquet organiza internamente los datos en **Row Groups** delimitados e indexados en el *File Footer*. Aunque cada página interna esté comprimida con Snappy o ZSTD, los límites y offsets en bytes de cada Row Group están explícitamente declarados en los metadatos. Durante la planificación, Spark divide los archivos en particiones lógicas (*Splits*) gobernadas por rangos de bytes mediante `spark.sql.files.maxPartitionBytes` (128 MB por defecto), asignando los Row Groups a las tareas correspondientes según su posición física, lo que permite procesar múltiples bloques en paralelo sin dependencias secuenciales.

Ejercicio 8.2 (Data Skipping mediante File Footer):
¿Cómo interactúa el *File Footer* de Parquet con el optimizador Catalyst y los lectores en los ejecutores para evitar transferencias de red y lecturas innecesarias en almacenamiento de objetos (Amazon S3 / Azure ADLS)?

Solución 8.2:

1. **Empuje de predicados (Catalyst):** Cuando Spark compila una consulta con filtros relacionales (por ejemplo, `WHERE fecha = '2026-10-12'`), Catalyst empuja dichos filtros (*Predicate Pushdown*) al operador físico `FileScan parquet`, delegando la evaluación de la condición a la capa de lectura.
2. **Lectura de metadatos en ejecutores:** El lector columnar de Parquet en el ejecutor realiza una solicitud HTTP de rango de bytes (*HTTP Range Request*) hacia los últimos kilobytes del

archivo en el almacenamiento de objetos para descargar exclusivamente el **File Footer** (`FileMetaData`).

3. **Inspección de estadísticas de bloque:** El lector examina los metadatos de cada **Row Group** (`ColumnChunkMetaData`), los cuales registran los valores mínimo y máximo (`min_value`, `max_value`) y el conteo de nulos para cada columna.
4. **Descarte de grupos de filas (Row Group Skipping):** Si las estadísticas del Row Group indican un rango entre `'2026-10-01'` y `'2026-10-10'`, el lector determina que la condición `fecha = '2026-10-12'` es imposible de satisfacer en ese bloque y **omite la lectura de las páginas de datos completas de ese Row Group**, ahorrando transferencias de red y ciclos de CPU.

Ejercicio 8.3 (Avro en Streaming vs. Parquet en Lakehouse):
En una arquitectura de ingesta en tiempo real conectada a Apache Kafka, ¿por qué es recomendable utilizar Apache Avro en el bus de streaming y convertir a Apache Parquet en la capa de persistencia del Lakehouse?

Solución 8.3:

- **Apache Avro en Streaming (Kafka):**
  - *Orientación a filas:* Kafka transmite eventos transaccionales individuales emitidos uno por uno. Avro serializa registros como tuplas compactas con encabezados ligeros gobernados por Schema Registry, minimizando la latencia de serialización por mensaje a microsegundos.
  - *Evolución de esquemas:* Avro maneja rigurosamente la compatibilidad hacia atrás y hacia adelante (*BACKWARD/FORWARD*), protegiendo a los consumidores ante cambios de campos en los microservicios productores.
- **Apache Parquet en el Lakehouse:**
  - *Orientación a columnas:* Las consultas analíticas de BI y SQL agregan métricas sobre millones de filas proyectando solo un subconjunto de columnas.
  - *Eficiencia analítica:* Parquet ofrece tasas de compresión superiores (reducción típica del 60% al 80% frente a JSON sin comprimir), codificación por diccionario y soporte nativo para *Column Pruning* y *Data Skipping*, optimizando tanto el almacenamiento en reposo como los tiempos de respuesta de consulta interactiva.

8.6 Resumen y Puntos Clave

- **Orientación física:** Los formatos orientados a filas (Avro, CSV) optimizan la escritura atómica y el streaming; los formatos columnares (Parquet, ORC) optimizan la lectura analítica y el cálculo de agregados.
- **Anatomía de Parquet:** Se estructura jerárquicamente en *File Footer*, *Row Groups*, *Column Chunks* y *Data Pages*, permitiendo lectura de metadatos primero y salto de bloques.
- **Poda de datos (Data Skipping):** Las estadísticas de bloque (Min/Max y nulos) permiten a los motores de consulta descartar particiones y grupos de filas completos sin leer sus datos físicos.
- **Compresión balanceada:** Snappy proporciona máxima velocidad de descompresión para capas intermedias; ZSTD ofrece ratios de compresión superiores para capas frías o históricas.

Lecturas Recomendadas

1. **White, T. (2015).** *Hadoop: The Definitive Guide: Storage and Analysis at Internet Scale* (4th ed.). O'Reilly Media. (Capítulo 12: "Avro" y Capítulo 13: "Parquet").
2. **Melnik, S. et al. (2010).** *Dremel: Interactive Analysis of Web-Scale Datasets.* Proceedings of the VLDB Endowment, 3(1-2), 330-339. (El artículo científico seminal de Google en el que se basa la codificación columnar anidada de Apache Parquet).
3. **Apache Parquet Project Documentation (2024).** *Parquet Format Specification, Metadata and Encodings.* Apache Software Foundation (`https://parquet.apache.org/docs/`).

CAPÍTULO 9

Tipos de Datos y Casting Riguroso en Memoria NÚCLEO

> *"Los tipos de datos son el contrato estricto de ingeniería entre tu lógica y el clúster: le indican al motor distribuido cuántos bytes reservar en la memoria RAM de los nodos ejecutores, cómo indexar columnas y cómo compilar el código de bajo nivel mediante el motor Tungsten."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar el catálogo de tipos de datos en PySpark (`StructType`, `IntegerType`, `LongType`, `DecimalType`, `DateType`, `TimestampType`).
- Aplicar la regla de oro financiera: evitar `Float` y `Double` para cálculos monetarios a favor de `DecimalType`.
- Implementar técnicas de casting individual y en bloque dinámico mediante metadatos.
- Anticipar el comportamiento de un cast inválido (`NULL` silencioso o error) según el modo ANSI, y usar `try_cast` para controlarlo.

Definición 9.1 (La Aduana de la Fábrica de Alta Velocidad) Imagina que estás construyendo una línea de producción automatizada en una fábrica de alta velocidad.

Si a un robot calibrado para recibir tornillos metálicos de exactamente 10 milímetros le envías un bloque de arcilla blanda, la maquinaria se detiene de golpe o el producto terminado sale completamente defectuoso.

En Apache Spark y PySpark, los tipos de datos son exactamente ese contrato de tolerancia milimétrica: le dicen al motor cuántos bytes reservar en memoria y cómo operar en microsegundos. Si dejas que el motor "adivine" tipos leyendo strings mal formateados (`inferSchema`), pagas una lectura adicional de los datos y arriesgas que el esquema inferido cambie de una ejecución a otra.

9.1 El Catálogo Esencial de Tipos en PySpark

Para trabajar con los estándares de la industria, siempre importamos los tipos de forma explícita desde `pyspark.sql.types`:

```python
from pyspark.sql.types import (
    StructType, StructField,
    StringType, IntegerType, LongType,
    DoubleType, DecimalType, BooleanType,
    DateType, TimestampType, ArrayType, MapType
)
```

Tipos Numéricos y Cuándo Elegir Cada Uno

1. `IntegerType` **vs.** `LongType`**:**
   - Un entero estándar (`IntegerType`) ocupa **4 bytes** y almacena valores entre -2.147 millones y +2.147 millones.
   - Si tus identificadores transaccionales, números de factura o métricas de eventos superan esta cifra, debes usar `LongType` (**8 bytes**).
   - Si tus datos jamás superarán ese rango, utilizar `LongType` innecesariamente es desperdiciar la mitad de la memoria RAM del clúster en esas columnas.
2. `DoubleType` **vs.** `DecimalType(precision, scale)` **(Regla de Oro):**
   - **En aplicaciones financieras, contables o transaccionales jamás uses Float ni Double para dinero.**
   - Los tipos de punto flotante introducen errores microscópicos de redondeo binario (estándar IEEE 754) que acumulan discrepancias de centavos.
   - Para dinero y montos exactos se debe usar siempre `DecimalType(18, 2)` o `DecimalType(38, 4)`.
3. `DateType`, `TimestampType` **y** `TimestampNTZType`**:**
   - `DateType` almacena únicamente la fecha de calendario (`YYYY-MM-DD`).
   - `TimestampType` almacena un instante físico absoluto (microsegundos desde la época Unix en UTC). Al mostrarse o proyectarse a cadena, Spark lo renderiza en la zona horaria activa de la sesión (`spark.sql.session.timeZone`).
   - `TimestampNTZType` (*Timestamp Without TimeZone*, introducido en Spark 3.4 y consolidado en Spark 4.0): almacena la fecha y hora local exactamente como fue registrada (año, mes, día, hora, minuto, segundo), sin referencia a huso horario ni ajustes de horario de verano (*Daylight Saving Time*).
4. **El Tipo** `VariantType` **(Spark 4.0):**

- En versiones previas, los datos JSON semiestructurados exigían o bien almacenar cadenas de texto sin tipar (`StringType`) o declarar jerarquías complejas y rígidas de `StructType` y `MapType`.
- El tipo **`VARIANT`** almacena documentos semiestructurados en un formato binario compacto con soporte nativo para navegación directa de campos (`col("payload:cliente.nombre")`), triturado columnar en disco (*sub-column shredding*) y evolución libre de esquema sin penalización de rendimiento.

9.2 Profundización Técnica: El Motor Tungsten y la Memoria Off-Heap

Cuando Spark opera con DataFrames y tipos estrictos, no necesita crear un objeto Java por cada celda. El motor **Project Tungsten** codifica las filas en un formato binario compacto (*UnsafeRow*), que por defecto reside en el heap de la JVM pero se gestiona manualmente, sin sobrecarga de objetos. Opcionalmente puede ubicarse **fuera del heap** (*off-heap*) con `spark.memory.offHeap.enabled=true`, lo que reduce la presión sobre el recolector de basura:

- Un `IntegerType` ocupa 4 bytes contiguos dentro de la fila binaria, en lugar de un objeto `java.lang.Integer` que, con su encabezado y alineación, ocupa típicamente 16 bytes.
- Además, el compilador genera código bytecode en tiempo de ejecución (*Whole-Stage Code Generation*) pensado para aprovechar las cachés L1/L2 del procesador del nodo ejecutor.

> **Consideraciones de Producción 9.1: Casting Dinámico en Bloque** Cuando recibes tablas con decenas de columnas, declarar transformaciones manuales una a una ensucia el código. Un ingeniero senior automatiza el proceso mediante metadatos y sanitización previa:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("Capitulo09-TiposDatosCasting") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset con columnas en formato texto crudo
df = spark.read.csv("data/ventas_crudo.csv", header=True)

# 1. Metadatos para casting automatizado: columna -> tipo destino
campos_enteros = ["sucursal_id", "categoria_id", "cantidad_articulos"]
```

```
campos_monetarios = ["precio_unitario", "subtotal", "impuesto_iva", "total"]

casts = {c: F.col(c).cast("int") for c in campos_enteros}
casts.update({c: F.col(c).cast("decimal(18,2)") for c in campos_monetarios})

# 2. Una sola proyección con todos los casts (evita encadenar decenas de
withColumn)
df_procesado = df.withColumns(casts)   # Spark 3.3+
# Equivalente en cualquier versión:
# df_procesado = df.select(*[casts.get(c, F.col(c)).alias(c) for c in
df.columns])

# 3. Sanitización defensiva: try_cast devuelve NULL en vez de lanzar error
df_sanitizado = df_procesado.withColumn(
    "edad_limpia", F.expr("try_cast(edad_raw AS INT)")
)

df_sanitizado.printSchema()
```

Nota: encadenar `withColumn` dentro de un bucle genera un plan lógico cada vez más profundo y ralentiza el análisis de Catalyst; el Cap. 11 profundiza en este anti-patrón.

9.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ignorar cómo se comporta un cast inválido

Qué ocurre al convertir `'N/A'`, `'PENDIENTE'` o una cadena vacía a `IntegerType` **depende de la configuración** `spark.sql.ansi.enabled`:

Modo	Valor por defecto en	`CAST('N/A' AS INT)`
ANSI desactivado	Spark 3.x	Devuelve `NULL` en silencio
ANSI activado	Spark 4.0 en adelante	Lanza un error (`CAST_INVALID_INPUT`) y detiene el job

En Spark 3.x, si no supervisas el resultado con aserciones de calidad de datos, inyectarás millones de nulos invisibles en la capa Silver. En Spark 4.0 el mismo código falla de forma ruidosa, lo cual es más seguro pero puede romper pipelines migrados. La herramienta correcta en ambos modos es `try_cast` (SQL) o `F.expr("try_cast(col AS INT)")`, que devuelve `NULL` de forma explícita cuando la conversión no es posible, y después se cuentan o se enrutan esos nulos a cuarentena (Cap. 28). Activar `spark.sql.ansi.enabled=false` para recuperar el comportamiento antiguo debe ser una decisión consciente y documentada, no un parche.

Anti-patrón 2: Inconsistencia de Tipos en Operaciones de Join

Si unes dos DataFrames donde en una tabla `cliente_id` es `StringType` y en la otra es `IntegerType`, Spark inserta un *cast* implícito sobre una de las columnas. El join sigue funcionando, pero ese cast puede impedir el *pushdown* del predicado hacia la fuente o el *partition pruning*, y provoca comparaciones inesperadas (por ejemplo, ceros a la izquierda). Declara los tipos de las claves explícitamente antes del cruce.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 9.1 (Redondeo de Punto Flotante):

Demuestra mediante un script simple en PySpark por qué `DoubleType` produce errores de redondeo en operaciones contables acumuladas frente a `DecimalType(18,2)`.

Solución:

```python
from pyspark.sql.functions import lit, col

df_test = spark.range(1).select(
    (lit(0.1) + lit(0.2)).alias("suma_double"),
    (lit(0.1).cast("decimal(10,2)") +
lit(0.2).cast("decimal(10,2)")).alias("suma_decimal")
)
df_test.show()
# suma_double: 0.30000000000000004  <-- Error de IEEE 754
# suma_decimal: 0.30                <-- Precisión exacta financiera
```

9.4 Resumen y Puntos Clave

- **Contrato de rendimiento:** El tipado explícito optimiza el footprint de memoria en Tungsten y evita fallos en los ejecutores.
- **Finanzas con Decimal:** Nunca uses `Float` o `Double` para transacciones monetarias; usa `DecimalType(precision, scale)`.
- **Casting defensivo:** Usa `try_cast` y valida los nulos resultantes; recuerda que en Spark 4.0 un cast inválido lanza error por defecto (ANSI) mientras que en 3.x devuelve `NULL`.

Lecturas Recomendadas

1. **Goldberg, D. (1991).** *What Every Computer Scientist Should Know About Floating-Point Arithmetic.* ACM Computing Surveys.

2. **Databricks Engineering Blog:** *Deep Dive into Project Tungsten: Bringing Spark Closer to Bare Metal.*

CAPÍTULO 10

Filtrado y Expresiones Condicionales

NÚCLEO

> *"Filtrar temprano en el flujo de trabajo es la optimización número uno en computación distribuida: cada registro descartado antes de un shuffle o un join representa megabytes de datos que no saturarán las tarjetas de red ni la memoria RAM de los nodos ejecutores."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar la sintaxis idiomática de `filter()` y `where()` mediante objetos de columna (`col`).
- Aplicar operadores lógicos a nivel de bits (`&`, `|`, `~`) y estructurar el encapsulado estricto con paréntesis.
- Construir lógica condicional de negocio avanzada con `when().otherwise()`, `isin()` y verificación de nulos.
- Explicar el funcionamiento interno de **Predicate Pushdown** en el optimizador Catalyst sobre capas de almacenamiento Parquet y Delta Lake.

Definición 10.1 (El Colador Inteligente en la Minería de Oro) Imagina que estás buscando oro en un río caudaloso.

No tiene ningún sentido transportar toneladas de arena, piedras y lodo río abajo en camiones hasta la refinería para recién allí separar las pepitas de oro. Lo inteligente y económico es colocar una criba o colador de alta precisión justo en la orilla del río: descartas el 95% del material estéril de inmediato y solo transportas las partículas valiosas.

En PySpark, filtrar es ese colador en la orilla: previene que millones de filas inútiles viajen por la red o consuman memoria en operaciones pesadas de agregación y cruce.

10.1 Fundamentos y Sintaxis Idiomática

En el ecosistema de PySpark, `filter` es una transformación de estrecha dependencia (*Narrow Transformation*) que evalúa registros basándose en una condición lógica booleana. Dado que los DataFrames son **inmutables**, aplicar un filtro no modifica la colección original, sino que devuelve una nueva abstracción de DataFrame con los registros supervivientes.

En la API disponemos de dos métodos formalmente equivalentes: `filter()` y `where()`. Ambos compilan al mismo operador lógico relacional:

```python
from pyspark.sql.functions import col

# Ambas sentencias son 100% intercambiables
df_activos = df.filter(col("estado_cuenta") == "ACTIVO")
df_activos = df.where(col("estado_cuenta") == "ACTIVO")
```

Operadores a Nivel de Bits y Precedencia

A diferencia de Python tradicional donde se utilizan las palabras reservadas `and`, `or` y `not`, en PySpark **es obligatorio utilizar los operadores a nivel de bits (`&`, `|`, `~`) y envolver cada condición individual entre paréntesis**:

```python
# CORRECTO: Paréntesis obligatorios en cada condición individual
df_colombia_vip = df.filter(
    (col("edad") >= 18) &
    (col("pais") == "Colombia") &
    (col("saldo") > 1000000)
)

# Negación lógica con la tilde de inversión ~
df_cuentas_limpias = df.filter(~col("cuenta_bloqueada"))
```

Si omites los paréntesis, Python evalúa el operador `&` antes que las comparaciones (`>`, `==`) y PySpark lanza `ValueError: Cannot convert column into bool: please use '&' for 'and', '|' for 'or', '~' for 'not' when building DataFrame boolean expressions`. El mismo `ValueError` aparece si usas `and`, `or` o `not` con columnas.

10.2 Profundización Técnica: La Magia de Predicate Pushdown

Cuando aplicas un `filter()` sobre un DataFrame que lee desde archivos Parquet, Delta Lake o tablas particionadas, el optimizador **Catalyst** no lee todas las filas a memoria RAM para filtrarlas después.

En su lugar, **empuja el filtro hasta el lector de la fuente de datos** (*Predicate Pushdown*):

- Spark lee el pie de página (footer) de cada archivo Parquet.
- Compara tu condición contra las estadísticas `[min, max]` de cada Row Group.
- Si estás buscando `edad >= 18` y un Row Group indica `max(edad) = 15`, Spark **descarta el bloque entero sin leer ni descomprimir sus datos**. El ahorro real depende de la selectividad del filtro y de cuán ordenados estén físicamente los datos: con datos ordenados o particionados por esa columna puede ser enorme; con datos aleatorios, casi nulo.

> **Consideraciones de Producción 10.1: Condicionales de Negocio Complejos** Patrones idiomáticos para clasificación con when().otherwise(), filtrado por listas dinámicas con isin() y saneamiento de nulos:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when

spark = SparkSession.builder \
    .appName("Capitulo10-FiltradoCondicionales") \
    .master("local[*]") \
    .getOrCreate()

df = spark.read.parquet("data/clientes.parquet")

# 1. Clasificación categórica basada en múltiples condiciones (CASE WHEN)
df_segmentado = df.withColumn(
    "nivel_riesgo",
    when(col("puntaje_credito") >= 750, "BAJO")
    .when((col("puntaje_credito") >= 600) & (col("puntaje_credito") < 750),
"MEDIO")
    .otherwise("ALTO")
)

# 2. Filtrado eficiente por colecciones con isin()
paises_foco = ["Colombia", "México", "Chile", "Perú"]
df_region = df_segmentado.filter(col("pais_origen").isin(paises_foco))

# 3. Control estricto de nulos y cadenas vacías
df_limpio = df_region.filter(
    col("email").isNotNull() &
    (col("email") != "") &
    (col("estado_cuenta") == "ACTIVO")
)

df_limpio.show(5)
```

10.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Dejar para el final un filtro que podía aplicarse antes

Catalyst reordena muchos casos por ti: un filtro que depende solo de la **clave de agrupación**, como `df.groupBy("pais").agg(F.sum("monto").alias("total")).filter(F.col("pais") == "CO")`, se empuja automáticamente por debajo de la agregación (véase el Ejercicio 13.2) y no provoca un shuffle de todos los países. En cambio, no puede reordenar un filtro que depende de un **resultado agregado** (`.filter(F.col("total") > 1000)`, el equivalente de `HAVING`) ni atravesar UDFs no deterministas. Escribir el filtro de fila lo más temprano posible, `df.filter(F.col("pais") == "CO").groupBy("pais").agg(...)`, hace explícita la intención y no depende de que el optimizador pueda reordenarlo.

Anti-patrón 2: Comparaciones de igualdad con Nulos mediante ==

En SQL y PySpark, `col("x") == None` o `col("x") == "null"` no evalúa como True ante valores nulos (evalúa como Unknown/Null). Utiliza siempre los métodos canónicos `.isNull()` o `.isNotNull()`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 10.1 (Optimización de Predicado):

Explica qué diferencia de ejecución física existe entre:

`df.filter((col("a") > 10) & (col("b") == "X"))` y

`df.filter(col("a") > 10).filter(col("b") == "X")`.

Solución: Respuesta: En tiempo de ejecución física **son exactamente idénticos**. El optimizador Catalyst combina automáticamente múltiples filtros encadenados sucesivos en una única expresión de conjunción lógica booleana (*Filter Pushdown / Combining Operators*), por lo que puedes encadenar llamadas a `.filter()` para mayor legibilidad sin ninguna penalización de rendimiento.

10.4 Resumen y Puntos Clave

- **Criba temprana:** Filtrar al borde de la ingesta es la técnica más rentable para ahorrar costos de cómputo y red en la nube.

- **Operadores bitwise y paréntesis:** En PySpark siempre usa `&`, `|`, `~` con paréntesis en cada condición.
- **Predicate Pushdown:** Catalyst descarga el filtrado a la capa de archivos Parquet/Delta para saltarse la lectura de datos irrelevantes.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide.* O'Reilly Media. (Capítulo 5: Basic Structured Operations).
2. **Databricks Documentation:** *Data Skipping for Delta Lake* (`https://docs.databricks.com/delta/data-skipping.html`).

CAPÍTULO 11

Transformaciones de Columnas y Operaciones con Esquemas NÚCLEO

> *"El módulo pyspark.sql.functions es la caja de herramientas estándar del Data Engineer: funciones vectorizadas de C++ y Java diseñadas para operar a la velocidad de la luz en paralelo sin recurrir a lentas UDFs de Python."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar `withColumn()` y `withColumnRenamed()` para generar nuevas columnas calculadas respetando la inmutabilidad de DataFrames.
- Aplicar funciones estándar de manipulación de texto (`upper`, `trim`, `concat`), fechas (`to_date`, `datediff`) y colecciones (`split`, `explode`).
- Conectar la generación de columnas con la evaluación perezosa y el motor Catalyst.
- Comprender la penalización de rendimiento de las UDFs de Python tradicionales y cómo evitarlas en producción.

Definición 11.1 (La Caja de Herramientas del Maestro Artesano) Imagina que eres un cirujano o un relojero de alta precisión: necesitas herramientas específicas para cada maniobra: pinzas para extraer, bisturí para cortar, lupas para enfocar.

En PySpark, el módulo pyspark.sql.functions es precisamente esa caja de herramientas estándar: cientos de funciones optimizadas, implementadas en Scala/Java y ejecutadas dentro de la JVM del ejecutor, diseñadas para operar sobre millones de registros en paralelo sin tener que reinventar la rueda con bucles lentos de Python.

11.1 Creación y Renombrado de Columnas

Los métodos `withColumn` y `withColumnRenamed` nos ayudan a generar nuevas columnas calculadas o alterar esquemas en un DataFrame:

- Cada vez que los llamamos devuelven una **nueva instancia de DataFrame**, preservando la inmutabilidad y añadiendo un paso al grafo de ejecución perezosa (**DAG**).
- `withColumn(colName, colExpr)`: Añade una nueva columna calculada o reemplaza una existente si tienen el mismo nombre.
- `withColumnRenamed(existingName, newName)`: Simplemente renombra una columna sin alterar sus datos ni añadir lógica computacional.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, year, current_date

spark = SparkSession.builder.appName("Capitulo11-Transformaciones").getOrCreate()
df = spark.read.parquet("data/usuarios.parquet")

# 1. Crear columna con valor constante literal (lit)
df_dummy = df.withColumn("dummy_col", lit(1))

# 2. Calcular mayoría de edad a partir del año de nacimiento
df_evaluado = df.withColumn(
    "mayor_edad",
    (year(current_date()) - col("anio_nacimiento")) >= 18
)

# 3. Renombrar una columna para normalizar el esquema (el nuevo nombre debe
seguir
#    describiendo lo que contiene: aquí sigue siendo un año, no una fecha)
df_renombrado = df_evaluado.withColumnRenamed("anio_nacimiento",
"nacimiento_anio")
df_renombrado.show(5)
```

11.2 Profundización Técnica: UDFs de Python, Vectorización con Arrow y Funciones de Orden Superior

Muchos desarrolladores que provienen de Python tradicional recurren a `@udf` para cualquier transformación personalizada. **¡Alerta de Rendimiento!** Cuando se ejecuta una UDF estándar de Python:

1. La JVM empaqueta los registros en lotes serializados y los transmite a través de un socket TCP local hacia un proceso daemon de Python (`pyspark.daemon`).

2. El proceso de Python deserializa los datos y evalúa la función fila a fila, quedando completamente fuera de las optimizaciones de Catalyst y Tungsten.
3. El resultado vuelve a serializarse y cruza nuevamente el socket hacia la JVM, multiplicando el consumo de CPU y memoria.

La Alternativa Vectorizada: Pandas UDFs con Apache Arrow

Cuando es imprescindible aplicar código de Python (por ejemplo, inferencia de modelos con Scikit-Learn o transformaciones con SciPy), se deben utilizar **Vectorized Pandas UDFs** (`@pandas_udf`), las cuales transfieren bloques de datos columnares directamente en memoria compartida mediante **Apache Arrow** con serialización de costo prácticamente nulo:

PYTHON

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType

@pandas_udf(DoubleType())
def normalizar_monto_arrow(serie_monto: pd.Series) -> pd.Series:
    # Operación vectorizada sobre arrays de NumPy / Pandas en C/SIMD
    return (serie_monto - serie_monto.mean()) / (serie_monto.std() + 1e-9)

df_normalizado = df.withColumn("monto_std",
normalizar_monto_arrow(col("monto_usd")))
```

> **Consideraciones de Producción 11.1: Manipulación Avanzada de Cadenas, Fechas y Arrays**
> Catálogo de transformaciones estandarizadas en producción con el alias canónico `F`:

```python
from pyspark.sql import functions as F
from pyspark.sql.functions import col

# 1. Normalización de cadenas de texto
df_texto = df.withColumn("nombre_limpio", F.upper(F.trim(col("nombre_crudo")))) \
             .withColumn("codigo_completo", F.concat(col("prefijo"), F.lit("-"),
col("folio")))

# 2. Manejo riguroso de fechas
df_tiempo = df_texto.withColumn("fecha_evento", F.to_date(col("fecha_texto"),
"yyyy-MM-dd")) \
                    .withColumn("dias_antiguedad", F.datediff(F.current_date(),
col("fecha_evento")))

# 3. Desanidación de arrays con explode_outer() para preservar arrays vacíos o
nulos
df_tags = df_tiempo.withColumn("etiquetas_array", F.split(col("etiquetas_str"),
","))
df_desanidado = df_tags.withColumn("etiqueta_individual",
F.explode_outer(col("etiquetas_array")))

df_desanidado.show(5)
```

11.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Encadenar docenas de `withColumn` sucesivos
Llamar a `.withColumn()` 50 veces seguidas para agregar 50 columnas crea un árbol de Catalyst con 50 operadores de proyección anidados, ralentizando severamente la fase de análisis del Driver. Para transformaciones masivas, utilice `.withColumns({...})` con un diccionario de expresiones o una única sentencia `.select()` que proyecte todas las columnas en un solo paso.

Anti-patrón 2: Usar literales escalares sin `F.lit()`
Si pasa un valor escalar como segundo argumento (`df.withColumn("tipo", "ACTIVO")`), Spark arrojará una excepción porque espera un objeto de tipo `Column`. Envuelva siempre los escalares con `F.lit("ACTIVO")`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 11.1 (Cálculo sobre Arrays: `explode` frente a Funciones de Orden Superior):
Si un DataFrame tiene una columna `historial_pagos` de tipo `ArrayType(DoubleType())`, escriba el código PySpark para calcular el pago promedio de cada cliente y compare la solución tradicional con `explode` frente al uso de **Higher-Order Functions** (`aggregate`).

Solución 11.1:

1. *Solución tradicional con `explode` (y sus desventajas):*

```python
# Riesgo: explode() descarta filas si el array es vacío o NULL (requiere explode_outer).
# Además, groupBy("cliente_id") desencadena un SHUFFLE masivo innecesario por red.
df_con_shuffle = df.select(
    col("cliente_id"),
    F.explode_outer(col("historial_pagos")).alias("pago")
).groupBy("cliente_id").agg(F.avg("pago").alias("pago_promedio"))
```

2. *Solución óptima con Higher-Order Functions (Cero Shuffle y preserva registros):*

```python
# Con aggregate(), el cálculo se realiza en memoria fila por fila dentro de cada partición,
# eliminando el shuffle y preservando arrays vacíos o nulos mediante condiciones seguras:
df_optimo = df.withColumn(
    "pago_promedio",
    F.when(
        F.size(col("historial_pagos")) > 0,
        F.expr("aggregate(historial_pagos, 0.0D, (acc, x) -> acc + x) / size(historial_pagos)")
    ).otherwise(F.lit(None))
)
```

Ventaja: La solución con `aggregate()` procesa millones de filas en una sola pasada lineal $O(N)$ sin transferir un solo byte por la red del clúster.

11.4 Resumen y Puntos Clave

- **Inmutabilidad y linaje:** `withColumn` y `withColumns` generan nuevos nodos en el plan lógico sin mutar los datos existentes.
- **Vectorización con Arrow:** Las Pandas UDFs (`@pandas_udf`) transfieren bloques columnares a procesos de Python mediante memoria compartida Apache Arrow, superando las limitaciones de serialización de las UDFs estándar.
- **Funciones de orden superior:** Operadores como `aggregate`, `transform` y `filter` permiten procesar arrays directamente en memoria sin necesidad de aplanar con `explode` ni incurrir en shuffles.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide*. O'Reilly Media. Capítulo 6: "Working with Different Types of Data".
2. **Apache Arrow Project (2024).** *In-Memory Analytics and Cross-Language Data Sharing*. Apache Software Foundation (`https://arrow.apache.org/`).
3. **Databricks Engineering (2024).** *Higher-Order Functions in Spark SQL*. Databricks Documentation (`https://docs.databricks.com/sql/language-manual/sql-ref-functions-builtin.html`).

CAPÍTULO 12

Joins Distribuidos y Estrategias de Cruce NÚCLEO

> *"El join distribuido es la prueba de fuego de cualquier arquitectura analítica: un cruce mal planificado satura la red del clúster con terabytes de shuffle y bloqueos por asimetría, mientras que una estrategia de join adecuada transforma consultas de horas en ejecuciones de segundos."*

Objetivos de Aprendizaje

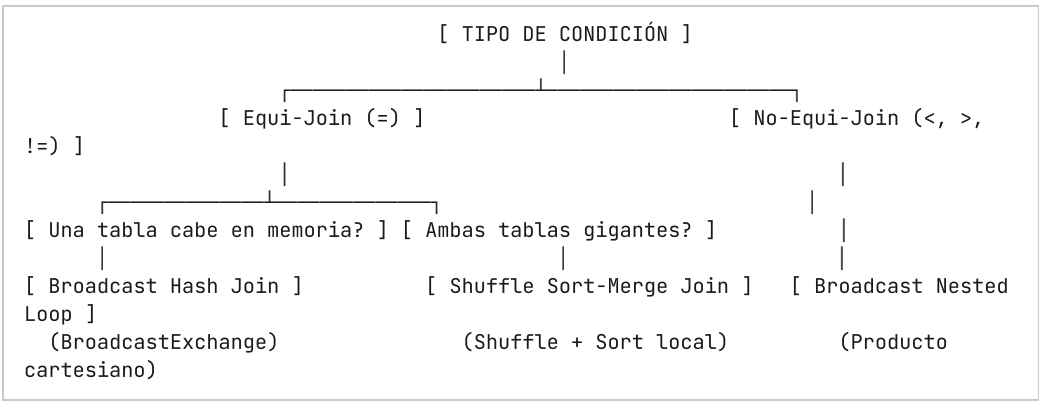
Al finalizar este capítulo, el lector será capaz de:

- Comprender la complejidad algorítmica y física de unir conjuntos de datos distribuidos en una arquitectura sin memoria compartida (*Shared-Nothing*).
- Comparar en profundidad las cuatro estrategias físicas de join en Apache Spark: **Broadcast Hash Join (BHJ)**, **Shuffle Sort-Merge Join (SMJ)**, **Shuffle Hash Join (SHJ)** y **Broadcast Nested Loop Join (BNLJ)**.
- Controlar la selección física de operadores mediante sugerencias del optimizador (*Join Hints*) en SQL y la API DataFrame.
- Diseñar e implementar la técnica de salado de claves (*Salting*) en PySpark para mitigar cuellos de botella severos por sesgo de datos (*Data Skew*).
- Evaluar las limitaciones de las tablas agrupadas en cubos (*Bucket Tables*) en formatos tradicionales frente a formatos abiertos de Lakehouse.
- Auditar planes físicos mediante `.explain()` distinguiendo `ShuffleExchangeExec` de `BroadcastExchangeExec`.

> **Definición 12.1 (Join Distribuido y Costo de Barajado / Shuffle)** En sistemas de computación distribuida compartida (*Shared-Nothing*), un **join distribuido** requiere que las tuplas de ambas tablas que comparten la misma clave de unión (*Join Key*) residan físicamente en la misma partición y en el mismo nodo ejecutor antes de evaluar la condición relacional.
>
> Cuando las tablas no están pre-agrupadas ni particionadas por dicha clave, el motor debe ejecutar una fase masiva de redistribución de red denominada **Shuffle**. Durante el shuffle, cada ejecutor particiona sus datos locales por el valor hash de la clave y transmite fragmentos a través de la red hacia los ejecutores de destino. Esta fase involucra serialización en memoria, transferencias por sockets TCP y volcado temporal en discos locales, constituyendo la operación más intensiva en recursos de todo el pipeline.

12.1 Las Cuatro Estrategias Físicas de Join en Apache Spark

Cuando el optimizador Catalyst compila una operación relacional de cruce (`df_a.join(df_b, "cliente_id")`), evalúa las estadísticas de los datos, el tamaño estimado de las tablas y el tipo de predicado (equi-join con operador `=` o no-equi-join con `<`, `>`, `!=`) para seleccionar una de las siguientes estrategias físicas:



1. Broadcast Hash Join (BHJ - Máximo Rendimiento)

- **Mecanismo:** Si una de las dos tablas tiene un tamaño físico estimado inferior al umbral configurable (`spark.sql.autoBroadcastJoinThreshold`, establecido en 10 MB por defecto), el nodo Driver descarga los datos de la tabla pequeña, construye una tabla hash en memoria y la transmite por red (*broadcast*) a todos los ejecutores del clúster mediante un operador `BroadcastExchangeExec`.
- **Comportamiento en los Workers:** Cada ejecutor mantiene una copia local de la tabla hash de referencia en su memoria RAM. A medida que procesa sus particiones locales de la tabla

masiva, consulta la tabla hash en tiempo constante $O(1)$ por fila.

- **Impacto en Rendimiento: Elimina el `ShuffleExchangeExec` sobre la tabla grande**. No se transmiten terabytes entre ejecutores ni se requiere ordenamiento en disco. La única transferencia de red es el envío unidireccional de la tabla pequeña desde el Driver hacia los Workers.

2. Shuffle Sort-Merge Join (SMJ - El Estándar Robusto)

- **Mecanismo:** Aplicable cuando ambas tablas superan el umbral de broadcast y la condición es una igualdad (*Equi-Join*). Opera en dos fases:
  1. **Fase de Shuffle (Redistribución por Hash):** Spark evalúa el hash de la clave de unión para cada fila y redistribuye los registros de ambas tablas mediante un operador `ShuffleExchangeExec`, garantizando que todas las filas con la misma clave lleguen a la misma partición física.
  2. **Fase de Ordenamiento y Fusión (Sort-Merge):** Cada ejecutor ordena localmente sus particiones por la clave de unión y avanza punteros simultáneos sobre ambas listas para identificar coincidencias en un solo pase lineal con complejidad temporal $O(N + M)$.
- **Impacto en Rendimiento:** Requiere I/O de red masivo y eventual volcado a disco (*Shuffle Spill*), pero es el algoritmo más escalable ante volúmenes arbitrariamente grandes sin desbordar la memoria de los ejecutores.

3. Shuffle Hash Join (SHJ)

- **Mecanismo:** Similar a SMJ en la fase de redistribución por red (`ShuffleExchangeExec`), pero en lugar de ordenar los datos en disco, el ejecutor construye una tabla hash en memoria con la partición de la tabla menor y recorre secuencialmente la partición de la tabla mayor.
- **Requisito y Riesgo:** Cada partición individual de la tabla menor debe caber holgadamente en la memoria heap del ejecutor; de lo contrario, el proceso colapsa con un error de memoria agotada (*OutOfMemoryError*). Está desactivado por omisión en favor de SMJ (`spark.sql.join.preferSortMergeJoin = true`).

4. Broadcast Nested Loop Join (BNLJ)

- **Mecanismo:** Se utiliza cuando el cruce no contiene condiciones de igualdad (por ejemplo, `df_a.monto > df_b.limite`) o cuando se ejecuta un producto cartesiano (`crossJoin`). Transmite una tabla y evalúa bucles anidados para cada registro. Su complejidad temporal es cuadrática $O(N \times M)$, por lo que debe evitarse rigurosamente en producción salvo en tablas de muy baja cardinalidad.

12.2 Join Hints en PySpark y Spark SQL

Cuando el optimizador Catalyst carece de estadísticas precisas o cuando el ingeniero conoce la volumetría de antemano, es posible forzar la estrategia física deseada mediante **Join Hints**:

Sintaxis en la API DataFrame de PySpark

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast, col

spark = SparkSession.builder \
    .appName("Capitulo12-JoinsDistribuidos") \
    .getOrCreate()

df_ventas = spark.read.parquet("lakehouse/silver/ventas/")
df_paises = spark.read.parquet("lakehouse/silver/dim_paises/")

# 1. Sugerencia de Broadcast mediante función nativa o método hint()
df_enriquecido_bhj = df_ventas.join(
    broadcast(df_paises),
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)

# 2. Forzar Shuffle Sort-Merge Join explícitamente
df_enriquecido_smj = df_ventas.hint("merge").join(
    df_paises,
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)

# 3. Forzar Shuffle Hash Join
df_enriquecido_shj = df_ventas.hint("shuffle_hash").join(
    df_paises,
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)
```

Sintaxis en Spark SQL Declarativo

En sentencias SQL puras, las sugerencias se declaran dentro de comentarios de bloque especiales `/*+ ... */`:

SQL

```sql
-- 1. Forzar Broadcast Hash Join
SELECT /*+ BROADCAST(p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;
```

```
-- 2. Forzar Shuffle Sort-Merge Join
SELECT /*+ MERGE(v, p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;

-- 3. Forzar Shuffle Hash Join
SELECT /*+ SHUFFLE_HASH(p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;

-- 4. Forzar Broadcast Nested Loop Join (para no-equi-joins con tabla derecha
pequeña)
SELECT /*+ BROADCAST(p) */
    v.transaccion_id, v.monto, p.rango_min
FROM ventas v
JOIN rangos_comision p ON v.monto >= p.rango_min;

-- 5. Forzar Cartesian Product con Shuffle Replicate (para tablas grandes sin
clave de igualdad)
SELECT /*+ SHUFFLE_REPLICATE_NL(p) */
    v.transaccion_id, v.monto, p.rango_min
FROM ventas v
JOIN rangos_comision p ON v.monto >= p.rango_min;
```

Inspección del Plan Físico: `ShuffleExchangeExec` vs. `BroadcastExchangeExec`

Al inspeccionar el plan con `df_enriquecido_bhj.explain(mode="formatted")`, observe los operadores generados:

```
*(2) BroadcastHashJoin [codigo_pais#12], [codigo_pais#45], Inner, BuildRight
:- *(2) Project [transaccion_id#10, monto#11, codigo_pais#12]
:  +- *(2) Filter isnotnull(codigo_pais#12)
:     +- *(2) Scan parquet [transaccion_id#10,monto#11,codigo_pais#12]
+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]))
   +- *(1) Project [codigo_pais#45, nombre_pais#46]
      +- *(1) Filter isnotnull(codigo_pais#45)
         +- *(1) Scan parquet [codigo_pais#45,nombre_pais#46]
```

Nótese que `BroadcastExchange` recopila la tabla derecha en el Driver y la distribuye a los ejecutores, mientras que la tabla izquierda (ventas) se escanea directamente en el Stage 2 sin ningún `ShuffleExchangeExec`.

12.3 Mitigación de Asimetría de Datos (Data Skew) en Uniones

El fenómeno de **Data Skew** se presenta cuando una porción desproporcionada de las filas comparte la misma clave de unión (por ejemplo, transacciones con cliente nulo o un cliente corporativo mayoritario). En un Sort-Merge Join, esa clave hash única es asignada a una sola

partición de shuffle, sobrecargando a un ejecutor mientras los demás permanecen ociosos:

```
[ Partición 0: 50.000 filas ] ————————> Ejecutor 1 (Termina en 2 seg)
[ Partición 1: 50.000 filas ] ————————> Ejecutor 2 (Termina en 2 seg)
[ Partición 2: 80.000.000 filas (SKEW) ] ——> Ejecutor 3 (Tarda 45 min o falla por
OOM)
```

Solución 1: Adaptive Query Execution (AQE Skew Join)

Desde Apache Spark 3.0, el optimizador adaptativo detecta en tiempo de ejecución las particiones asimétricas tras la fase de shuffle y las divide automáticamente en subtareas más pequeñas:

PYTHON

```python
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
"64MB")
```

Solución 2: Técnica de Salting (Salado de Claves) para Cruces

Cuando el sesgo es extremo o AQE no logra balancear la carga de forma óptima, se aplica **Salting sobre el Join**:

1. A la tabla grande y sesgada se le añade una clave aleatoria (*salt*) entre 0 y $K - 1$.
2. La tabla de búsqueda o dimensión se replica multiplicando cada una de sus filas por los mismos K factores mediante una función de explosión (`explode`).
3. El join se ejecuta sobre la clave compuesta `(clave_original, salt)`, distribuyendo las filas asimétricas de manera uniforme entre K particiones físicas distintas.

PYTHON

```python
from pyspark.sql import functions as F

K = 8  # Factor de dispersión (salting)

# 1. Tabla grande asimétrica (df_ventas con millones de compras del cliente 'C-000')
df_ventas_salted = df_ventas.withColumn(
    "salt",
    F.floor(F.rand() * K).cast("int")
)

# 2. Tabla de referencia (df_clientes): replicar cada registro K veces
df_clientes_replicated = df_clientes.withColumn(
    "salt_array",
    F.array([F.lit(i) for i in range(K)])
).withColumn(
    "salt",
    F.explode("salt_array")
).drop("salt_array")

# 3. Join libre de sesgo sobre la clave compuesta
df_ventas_balanceadas = df_ventas_salted.join(
    df_clientes_replicated,
    on=["cliente_id", "salt"],
    how="inner"
).drop("salt")
```

Este procedimiento garantiza que la clave sesgada se disperse equitativamente entre K tareas de shuffle diferentes, eliminando por completo el cuello de botella de la tarea rezagada (*Straggler Task*).

Consideraciones de Producción 12.1: Bucketed Joins y su Compatibilidad En tablas gestionadas tradicionales de Apache Hive o Apache Parquet (`saveAsTable`), es posible pre-ordenar y pre-agrupar registros mediante `bucketBy`:

```
PYTHON
df.write.bucketBy(32,
"cliente_id").sortBy("cliente_id").saveAsTable("clientes_bucketed")
```

Si dos tablas se unen por la misma clave y ambas comparten idéntico número de cubos, Spark realiza un **Bucket Sort-Merge Join sin Shuffle**.

Aclaración Crítica de Producción: `bucketBy` **no es compatible con rutas directas ni con el formato Delta Lake**. Si intenta invocar `.bucketBy()` al escribir una tabla Delta, el motor ignorará la directiva o arrojará una excepción. En arquitecturas modernas sobre Delta Lake, la técnica estándar para agrupar datos multidimensionalmente y evitar reordenamientos es **Liquid Clustering** (`CLUSTER BY cliente_id`).

12.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Aplicar `broadcast()` a DataFrames que Exceden la Capacidad del Driver
Forzar el broadcast de un DataFrame de gran volumen (por ejemplo, > 4 GB) obliga al nodo Driver a recolectar todas las particiones en su memoria heap local, provocando la interrupción del trabajo por `Driver OutOfMemoryError`. Reserve `broadcast()` estrictamente para tablas de dimensiones que quepan cómodamente en memoria (por lo general < 500 MB).

Anti-patrón 2: Uniones Externas (`FULL OUTER JOIN`) sin Filtrado Previo de Particiones
Ejecutar un `FULL OUTER JOIN` sobre dos tablas transaccionales de escala masiva sin delimitar rangos de fechas produce una explosión de registros nulos en ambos lados y un barajado de datos innecesario que degrada el rendimiento de los ejecutores. Aplique siempre filtros de partición antes del cruce.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 12.1 (Comparativa de Memoria entre BHJ y SMJ):
¿Cuál es la diferencia fundamental en el uso y distribución de memoria entre un Broadcast Hash Join y un Shuffle Sort-Merge Join?

Solución 12.1:

- En **Broadcast Hash Join (BHJ)**: El nodo Driver debe disponer de suficiente memoria heap para ensamblar la tabla completa y serializarla. Posteriormente, cada nodo ejecutor consume memoria adicional en su espacio de almacenamiento/ejecución para albergar la copia íntegra de la tabla hash. La tabla grande se procesa en streaming partición por partición sin requerir memoria extra para ordenamiento ni buffers de red de shuffle.
- En **Shuffle Sort-Merge Join (SMJ)**: El nodo Driver no almacena ningún dato de las tablas. La memoria se consume de manera distribuida en los ejecutores dentro de la región de memoria de ejecución (`Execution Memory`), donde se gestionan los buffers de red para el intercambio de shuffle y las estructuras de ordenamiento en memoria (*AppendOnlyMap* / *Sorter*). Si la memoria de ejecución se satura durante la fase de ordenamiento, los registros se vuelcan a disco local (*Spill*), garantizando estabilidad a expensas de mayor latencia de I/O.

Ejercicio 12.2 (Salting sobre Operaciones de Cruce):
Explique por qué aplicar salting mediante un número aleatorio a la tabla sesgada obliga a replicar las filas de la tabla de referencia mediante `explode()`.

Solución 12.2: Si la tabla grande contiene 10 millones de registros con la clave `cliente_id = 'C-000'` y a cada registro se le asigna un valor de `salt` aleatorio entre 0 y 3, existirán 4 combinaciones posibles de clave compuesta: `('C-000', 0)`, `('C-000', 1)`, `('C-000', 2)` y `('C-000', 3)`. Si la tabla de clientes únicamente tuviera una fila con `cliente_id = 'C-000'`, solo coincidiría con los registros de ventas cuyo salt generado coincidiera con ese único registro, perdiendo el 75% de las transacciones legítimas. Al replicar la fila del cliente en 4 copias idénticas con los salts 0, 1, 2 y 3 mediante `explode()`, se garantiza la semántica relacional completa: cada transacción encontrará su contraparte correspondiente, mientras que la carga de trabajo se reparte equitativamente en 4 particiones físicas de shuffle distintas.

Ejercicio 12.3 (Restricciones de Broadcast en Operaciones `LEFT JOIN`):
En una consulta con un join de tipo `LEFT JOIN` (`df_izq.join(df_der, ..., "left")`), ¿cuál de las dos tablas puede ser transmitida mediante broadcast sin alterar la semántica relacional del resultado? Justifique la respuesta a nivel de planificación física.

Solución 12.3: En un `LEFT JOIN`, **únicamente la tabla derecha (la tabla no preservada) puede ser transmitida mediante broadcast**. *Justificación física:* En la semántica de `LEFT JOIN`, todos los registros de la tabla izquierda deben preservarse obligatoriamente en el resultado final, existan o no coincidencias en la derecha. Si se intentara hacer broadcast de la tabla izquierda, los ejecutores que procesan fragmentos de la tabla derecha no podrían saber de manera independiente y sin coordinación global qué filas de la tabla izquierda no tuvieron coincidencias, lo que obligaría a un intercambio de datos adicional. En cambio, al transmitir la tabla derecha como tabla hash a todos los ejecutores, cada tarea puede recorrer de forma autónoma su partición local de la tabla

izquierda: si la clave existe en la tabla hash derecha, emite la fila unida; si no existe, emite la fila izquierda con valores `NULL` en los campos de la derecha, sin necesidad de transferir ningún dato de la tabla izquierda por la red.

12.5 Resumen y Puntos Clave

- **Criterio de selección:** El optimizador Catalyst elige Broadcast Hash Join cuando una de las tablas es inferior a `spark.sql.autoBroadcastJoinThreshold` (10 MB). Para tablas masivas con equi-joins, la estrategia predeterminada y más estable es Shuffle Sort-Merge Join.
- **Costos físicos:** BHJ elimina el costo de `ShuffleExchangeExec` en la tabla grande pero introduce `BroadcastExchangeExec` desde el Driver. SMJ introduce shuffles en ambas tablas y ordenamiento local en los Workers.
- **Join Hints:** Permiten forzar estrategias cuando las estadísticas de tabla no están disponibles. Se expresan con `.hint("broadcast")` en DataFrames o `/*+ BROADCAST(t) */` en SQL.
- **Manejo de sesgo (Skew):** Adaptive Query Execution (AQE) divide particiones asimétricas en tiempo de ejecución. Ante casos extremos, el salado de claves (*Salting*) dispersa la clave sesgada entre K particiones mediante números aleatorios y duplicación con `explode`.
- **Bucketing vs. Lakehouse:** `bucketBy` permite eliminar shuffles en tablas tradicionales de Parquet/Hive, pero no opera sobre Delta Lake, donde se sustituye por Liquid Clustering.

Lecturas Recomendadas

1. **Chambers, B., & Zaharia, M. (2018).** *Spark: The Definitive Guide: Big Data Processing Made Simple.* O'Reilly Media. Capítulo 8: "Joins".
2. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 1383-1394.
3. **Apache Spark Documentation (2024).** *Performance Tuning: Join Strategy Hints and Adaptive Query Execution.* Apache Software Foundation (`https://spark.apache.org/docs/latest/sql-performance-tuning.html`).

CAPÍTULO 17

Arquitectura Medallón y Modelado Dimensional NÚCLEO

> *"La arquitectura medallón y el modelado dimensional representan la convergencia definitiva entre el lago y el almacén: los datos transitan desde el estado crudo inmutable en Bronze, se sanean y enriquecen en Silver, y culminan en modelos dimensionales de estrella y tablas consolidadas en Gold listos para alimentar decisiones estratégicas de negocio con latencia predecible."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la jerarquía de refinamiento y garantías de servicio de la **Arquitectura Medallón**: Capa Bronze (Raw / Inmutable), Capa Silver (Enriched / Conforme) y Capa Gold (Business Aggregates / Data Marts).
- Comparar críticamente esquemas en estrella (*Star Schema*), esquemas en copo de nieve (*Snowflake Schema*) y el patrón *One Big Table* (OBT) sobre motores columnares modernos.
- Clasificar y modelar tablas de hechos (*Transaction Facts*, *Periodic Snapshots*, *Accumulating Snapshots*, *Factless Facts*) y dimensiones (*Conformed*, *Role-Playing*, *Degenerate*, *Bridge*, *Junk Dimensions*).
- Generar claves subrogadas deterministas y evaluar rigurosamente los riesgos de colisión de funciones hash de 64 bits frente a 256 bits (`sha256`).
- Implementar de extremo a extremo el ciclo de vida de Dimensiones Lentamente Cambiantes (*SCD Tipo 1 y Tipo 2*) mediante transacciones `MERGE INTO` en Delta Lake sin duplicación de registros.
- Garantizar idempotencia física utilizando la opción idiomática `replaceWhere` de Delta Lake y el modo de sobrescritura dinámica de particiones.

Definición 17.1 (La Refinería de Metales Preciosos)

La Arquitectura Medallón conceptualiza el flujo de datos como un proceso físico de purificación metalúrgica:

- **Capa Bronze (Mina y Almacén Crudo):** Representa el mineral extraído de la veta geológica tal y como sale a la superficie: contiene barro, impurezas y rocas no mineralizadas. Ningún fragmento se descarta; se preserva en su estado original bajo almacenamiento inmutable para permitir auditorías forenses y reconstrucciones completas del sistema ante cambios de lógica retrospectivos.
- **Capa Silver (Fundición y Refinamiento Conforme):** Es el proceso donde se eliminan las impurezas, se filtran las anomalías, se estandarizan las barras metálicas a un grado de pureza homogéneo y se integran minerales afines. Es la fuente de la verdad operacional para ingenieros de datos y científicos de datos.
- **Capa Gold (Joyería y Acuñación de Monedas):** Constituye el producto final pulido con precisión micrométrica: joyas, coronas y monedas certificadas listas para el consumo directo por parte de usuarios finales, cuadros de mando ejecutivos (Power BI, Tableau) y modelos analíticos de alto impacto.

17.1 Jerarquía de Capas en la Arquitectura Medallón

La Arquitectura Medallón organiza el lago de datos transaccional en tres capas de madurez progresiva, desacoplando la ingestión veloz de la computación semántica compleja:



1. Capa Bronze (Raw / Ingesta Histórica)

- **Contenido:** Copia fidedigna de las cargas de origen sin modificaciones de valor ni descarte de registros. Los esquemas admiten variantes flexibles (JSON empaquetado o columnas estructuradas).
- **Columnas obligatorias de metadatos de auditoría:**
  - `_ingestion_timestamp`: Marca temporal UTC en la que el registro aterrizó en el Lakehouse.
  - `_source_file`: Ruta de almacenamiento o identificador del mensaje Kafka de origen.
  - `_batch_id` / `_job_id`: Identificador de ejecución del pipeline orquestador.

- **Política de escritura:** Estrictamente **Append-Only** (anexión continua). Jamás se ejecutan actualizaciones ni borrados directos en Bronze, salvo purgas obligatorias por regulaciones de privacidad (RGPD / Habeas Data).

2. Capa Silver (Limpia, Tipada y Conformada)

- **Contenido:** Datos validados, normalizados con tipos canónicos (reemplazo de strings por timestamps, enteros y decimales precisos), deduplicados y enriquecidos con cruces fundamentales.
- **Propósito:** Representa la "versión única de la verdad" corporativa. Aquí viven las entidades de negocio centrales (clientes, órdenes, pagos, eventos de navegación).
- **Reglas de Calidad:** Se descartan o desvían a tablas de cuarentena aquellos registros que violan contratos de datos o integridad estructural.

3. Capa Gold (Agregada y Optimizada para el Negocio)

- **Contenido:** Conjuntos de datos estructurados para maximizar la velocidad de lectura interactiva. Implementa modelos dimensionales de Ralph Kimball (esquemas en estrella), vistas materializadas y tablas desnormalizadas (*One Big Table*).
- **Consumidores:** Consultas ad-hoc de analistas, reportes regulatorios, dashboards ejecutivos y *feature stores* para aprendizaje automático.

17.2 Modelado Dimensional en el Lakehouse Moderno

El modelado dimensional introducido por Ralph Kimball mantiene plena vigencia en los motores analíticos columnares, aunque su justificación física ha evolucionado:

Clasificación Integral de Tablas de Hechos (Fact Tables)

Las tablas de hechos capturan métricas cuantitativas generadas por eventos operacionales:

1. **Hechos Transaccionales (*Transaction Facts*):** Registran un evento en un instante temporal discreto (una venta, un clic, un movimiento bancario). Poseen la granularidad más atómica y el volumen más masivo.
2. **Hechos de Instantánea Periódica (*Periodic Snapshot Facts*):** Capturan el estado del negocio en intervalos regulares acumulativos (saldo de fin de mes de una cuenta, nivel de inventario semanal por almacén).
3. **Hechos de Instantánea Acumulativa (*Accumulating Snapshot Facts*):** Modelan el ciclo de vida de un proceso con inicio y fin definidos con múltiples hitos temporales (solicitud de préstamo: fecha recepción → fecha aprobación → fecha desembolso).

4. **Hechos Sin Hechos (*Factless Fact Tables*):** Tablas de hechos que no contienen métricas numéricas acumulables, sino exclusivamente claves foráneas que registran la ocurrencia de un evento (ej. asistencia de un estudiante a una clase o cobertura de un evento promocional).

Clasificación de Tablas de Dimensiones (Dimension Tables)

1. **Dimensiones Conformadas (*Conformed Dimensions*):** Dimensiones consistentes y universales compartidas entre múltiples hechos (ej. la dimensión `dim_cliente` es idéntica para hechos de ventas, soporte y facturación).
2. **Dimensiones de Rol Jugado (*Role-Playing Dimensions*):** Una misma dimensión física asume diferentes roles lógicos en la misma tabla de hechos mediante múltiples claves foráneas (ej. `dim_tiempo` referenciada como `fecha_orden_id`, `fecha_envio_id` y `fecha_entrega_id`).
3. **Dimensiones Degeneradas (*Degenerate Dimensions*):** Identificadores transaccionales que residen en la tabla de hechos sin una tabla de dimensión satélite propia (ej. `numero_factura`, `codigo_rastreo`).
4. **Dimensiones Basura (*Junk Dimensions*):** Combinación de atributos discretos de baja cardinalidad (flags booleanos, códigos de estado) agrupados en una sola tabla de dimensión para evitar saturar la tabla de hechos con decenas de claves foráneas.
5. **Tablas Puente (*Bridge Tables*):** Tablas intermedias utilizadas para modelar relaciones de muchos a muchos (*M:N*) entre una tabla de hechos y una dimensión (ej. una póliza de seguro con múltiples beneficiarios).
6. **Miembros Desconocidos (*Unknown Members*):** Toda dimensión debe incluir un registro con clave subrogada `-1` (ej. `cliente_sk = -1`, `nombre = 'NO INFORMADO'`). Cuando una transacción de la tabla de hechos contiene una clave natural nula o no resuelta, apunta a `-1` en lugar de dejar la clave foránea en `NULL`, evitando que los cruces `INNER JOIN` descarten transacciones legítimas.

Star Schema vs. Snowflake Schema vs. One Big Table (OBT)

Criterio	Esquema en Estrella (Star)	Esquema Copo de Nieve (Snowflake)	One Big Table (OBT)
Normalización	Desnormalizado en dimensiones (2NF).	Altamente normalizado (3NF en dimensiones).	Completamente desnormalizado (1 sola tabla).
Complejidad de Joins	Baja (1 join por dimensión contra el hecho).	Alta (múltiples joins en cascada para resolver jerarquías).	Cero joins (toda la información en la misma fila).
Eficiencia en Motores Columnares	**Excelente:** Excelente compresión y data skipping.	**Deficiente:** Múltiples shuffles distribuidos ralentizan BI.	**Suprema en consultas específicas:** Máxima velocidad, mayor costo de almacenamiento.
Mantenibilidad	Alta: Esquema comprensible por usuarios de negocio.	Media: Jerarquías rígidas y complejas.	Baja: Cambios de atributos requieren reescribir tablas masivas.

Claves Subrogadas Distribuidas: Paradoja del Cumpleaños y Hashing

En almacenes relacionales tradicionales se utilizaban secuencias autoincrementales (`AUTO_INCREMENT`). En sistemas distribuidos, generar un entero monótono global introduce un cuello de botella de coordinación centralizado.

El estándar moderno en Lakehouse consiste en generar claves subrogadas deterministas mediante **funciones hash criptográficas** sobre la clave natural del negocio y los metadatos de vigencia:

$$\text{cliente_sk} = \text{sha2}(\text{concat}(\text{sistema_origen}, \text{cliente_id}, \text{date_format}(\text{valido_desde}, \text{'y}$$

Análisis de Probabilidad de Colisión

Un error frecuente consiste en emplear hashes de 64 bits (`xxhash64`) en tablas masivas. De acuerdo con la **paradoja del cumpleaños**, la probabilidad de que ocurra al menos una colisión en un espacio de claves de tamaño N tras insertar n registros se aproxima por:

$$P(\text{colisión}) \approx 1 - e^{-\frac{n^2}{2N}}$$

- Para un hash de **64 bits** ($N = 2^{64} \approx 1.84 \times 10^{19}$): Con apenas $n = 10^9$ registros (mil millones de filas), la probabilidad de colisión es:

$$P \approx 1 - e^{-\frac{10^{18}}{2 \times 1.84 \times 10^{19}}} \approx 1 - e^{-0.027} \approx \mathbf{2.7\%}$$

Una probabilidad del 2.7% de corromper la unicidad de las claves es inaceptable en analítica empresarial.

4. Data Vault 2.0 en la Capa Silver Empresarial: Hubs, Links y Satellites

En organizaciones corporativas complejas con decenas de sistemas transaccionales heterogéneos (SAP, Salesforce, bases operacionales locales), intentar forzar un modelo dimensional de Kimball directamente en la ingesta crea cuellos de botella organizacionales: cualquier cambio en una fuente exige renegociar el esquema de la dimensión compartida.

Data Vault 2.0 (diseñado por Dan Linstedt) resuelve este problema desacoplando la integración en tres primitivas tabulares normalizadas y auditables:



1. **Hubs (Centros de Negocio):** Representan conceptos centrales del negocio (Clientes, Productos, Tiendas). Contienen únicamente la clave de negocio única (*Business Key*), su hash SHA-256 (`hub_cliente_hk`), la marca de tiempo de carga (`load_timestamp`) y el sistema fuente de origen (`record_source`). **No contienen atributos descriptivos.**
2. **Links (Interconexiones y Transacciones):** Representan relaciones asociativas o transacciones de negocio entre dos o más Hubs (ej. una venta que asocia a un Cliente con un Producto). Almacenan los hashes foráneos de los Hubs interconectados y son **estrictamente de solo adición (*Append-Only*)**.

3. **Satellites (Contexto Descriptivo e Historial):** Almacenan todos los atributos descriptivos mutables a lo largo del tiempo. Incorporan un campo especial `hash_diff` (un hash SHA-256 concatenado de todos los atributos descriptivos). Al procesar una carga incremental, si el `hash_diff` entrante difiere del último registrado, se inserta una nueva fila satélite de forma automática sin bloquear otras fuentes.

> **Estrategia Híbrida en Producción:** La arquitectura más robusta utiliza **Data Vault 2.0 en la Capa Silver** (para integrar 30+ fuentes heterogéneas sin fricciones de esquema) y proyecta **Modelos Dimensionales de Kimball en la Capa Gold** (para que analistas de negocio y herramientas de BI como Power BI o Tableau consulten estrellas optimizadas y comprensibles).

17.3 Slowly Changing Dimensions (SCD) Tipo 1 y Tipo 2

Cuando los atributos de una dimensión cambian con el tiempo (por ejemplo, un cliente se traslada de ciudad), el modelo debe gobernar la mutación histórica:

- **SCD Tipo 1 (Sobrescritura Directa):** El valor anterior se destruye y se reemplaza por el nuevo. No conserva historia. Adecuado para corregir errores tipográficos en nombres o números telefónicos.
- **SCD Tipo 2 (Preservación Histórica):** Se cierra la vigencia del registro existente estableciendo su fecha final y marcando su bandera de actualidad como inactiva; a continuación, se inserta una nueva fila con los valores actualizados y vigencia abierta.

```
Estado Original:
[ID: 101 | Nombre: Juan | Ciudad: Bogotá    | Valido_Desde: 2024-01-01 |
Valido_Hasta: 9999-12-31 | Es_Actual: TRUE]

Tras mudanza a Medellín (SCD Tipo 2 aplicado el 2026-06-15):
[ID: 101 | Nombre: Juan | Ciudad: Bogotá    | Valido_Desde: 2024-01-01 |
Valido_Hasta: 2026-06-15 | Es_Actual: FALSE]
[ID: 101 | Nombre: Juan | Ciudad: Medellín  | Valido_Desde: 2026-06-15 |
Valido_Hasta: 9999-12-31 | Es_Actual: TRUE]
```

> **Consideraciones de Producción 17.1: Implementación Robusta de SCD Tipo 2 con Delta Lake MERGE**
>
> Para ejecutar SCD Tipo 2 en una sola transacción atómica distribuida **sin duplicar registros que no cambiaron**, se identifican de forma rigurosa los cambios reales y los clientes nuevos:

PYTHON

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from delta.tables import DeltaTable

spark = SparkSession.builder \
    .appName("Capitulo17-SCD2-Robusto") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog") \
    .master("local[*]") \
    .getOrCreate()

ruta_dim_clientes = "lakehouse/gold/dim_clientes"
delta_dim = DeltaTable.forPath(spark, ruta_dim_clientes)

# 1. Novedades entrantes validadas
df_novedades = spark.read.table("silver_staging_clientes")

# 2. Clientes actualmente vigentes en la dimensión
df_actuales = delta_dim.toDF().filter(F.col("es_actual") == True)

# 3. Detectar registros modificados (cambio en atributos seguidos)
df_cambios = df_novedades.alias("nov") \
    .join(df_actuales.alias("dim"), "cliente_id") \
    .filter("(nov.ciudad <> dim.ciudad) OR (nov.segmento <> dim.segmento)")

# 4. Detectar clientes completamente nuevos (que no existen en la dimensión)
df_nuevos = df_novedades.join(df_actuales, "cliente_id", "left_anti")

# 5. Construir el conjunto unificado para el MERGE:
# a) Filas para CERRAR: merge_key = cliente_id (coincide con target vigente)
df_para_cerrar = df_cambios.select(
    F.col("nov.cliente_id").alias("merge_key"),
    F.col("nov.*")
)

# b) Filas para INSERTAR: solo los clientes nuevos y la nueva versión de los
modificados
#    (merge_key = NULL para forzar la rama whenNotMatchedInsert)
df_para_insertar = df_nuevos.unionByName(df_cambios.select("nov.*")).select(
    F.lit(None).cast("string").alias("merge_key"),
    F.col("*")
)

df_staged_merge = df_para_cerrar.unionByName(df_para_insertar)

# 6. Ejecución atómica de MERGE INTO en Delta Lake
delta_dim.alias("target").merge(
    df_staged_merge.alias("source"),
    "target.cliente_id = source.merge_key AND target.es_actual = true"
).whenMatchedUpdate(
    set={
        "valido_hasta": F.col("source.fecha_evento"),
        "es_actual": F.lit(False)
    }
).whenNotMatchedInsert(
```

```
    values={
        "cliente_sk": F.expr("sha2(concat('ERP_', source.cliente_id, '_',
string(source.fecha_evento)), 256)"),
        "cliente_id": "source.cliente_id",
        "nombre": "source.nombre",
        "ciudad": "source.ciudad",
        "segmento": "source.segmento",
        "valido_desde": "source.fecha_evento",
        "valido_hasta": F.to_timestamp(F.lit("9999-12-31 23:59:59")),
        "es_actual": F.lit(True)
    }
).execute()

print("[PRODUCCIÓN] SCD Tipo 2 aplicado exitosamente bajo transacción ACID.")
```

Consideraciones de Producción 17.2: Generación Automatizada de la Dimensión de Tiempo

La dimensión `dim_tiempo` es el eje gravitacional de todo esquema en estrella. Jamás debe depender de extracciones de fuentes operacionales; se genera sintéticamente cubriendo un horizonte de 10 a 20 años:

PYTHON

```
# Generación de dim_tiempo para 2020 a 2035 en PySpark
df_fechas = spark.sql("""
    SELECT explode(sequence(to_date('2020-01-01'), to_date('2035-12-31'),
interval 1 day)) AS fecha
""").select(
    F.date_format("fecha", "yyyyMMdd").cast("int").alias("fecha_id"),
    F.col("fecha"),
    F.year("fecha").alias("anio"),
    F.quarter("fecha").alias("trimestre"),
    F.month("fecha").alias("mes"),
    F.date_format("fecha", "MMMM").alias("nombre_mes"),
    F.dayofmonth("fecha").alias("dia_mes"),
    F.dayofweek("fecha").alias("dia_semana"),
    F.date_format("fecha", "EEEE").alias("nombre_dia"),
    F.when(F.dayofweek("fecha").isin(1, 7),
F.lit(True)).otherwise(F.lit(False)).alias("es_fin_de_semana")
)

df_fechas.write.format("delta").mode("overwrite").save("lakehouse/gold/dim_tiempo
")
```

> **Consideraciones de Producción 17.3: Idempotencia Idiomática con `replaceWhere` en Delta Lake**
>
> Aunque `spark.sql.sources.partitionOverwriteMode = "dynamic"` funciona a nivel de sesión en Spark, el estándar idiomático y más seguro en Delta Lake es utilizar la opción `replaceWhere`. Permite sobrescribir atómicamente un predicado específico garantizando que únicamente los datos que cumplan la condición sean reemplazados:

```python
# Sobrescritura atómica e idempotente de la partición del 12 de octubre de 2026
df_hechos_dia.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", "fecha_id = 20261012") \
    .save("lakehouse/gold/fact_ventas_diarias")
```

17.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Exponer la Capa Bronze a Usuarios Finales y Analistas de Negocio Permitir que herramientas de BI apunten a Bronze ocasiona escaneos ineficientes sobre miles de archivos crudos sin optimizar, genera métricas discordantes debido a registros malformados y viola normativas de privacidad al consultar datos PII no enmascarados.

Anti-patrón 2: Normalización Extrema (3NF) en la Capa Gold Diseñar un modelo normalizado con 10 o más tablas intermedias obliga al motor distribuido a realizar múltiples etapas de shuffle y cruces complejos en cada consulta analítica. En motores columnares analíticos, la desnormalización controlada (Star Schema o OBT) reduce drásticamente el costo de red.

Anti-patrón 3: Insertar Todas las Novedades Ciegas en el Staging de SCD2 Como se demostró, insertar todas las novedades de entrada con clave nula en lugar de unir exclusivamente los clientes nuevos y las nuevas versiones de los modificados provoca la duplicación silenciosa de millones de registros idénticos en cada ejecución del pipeline.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 17.1 (Diseño de Esquema Dimensional de Comercio Electrónico):
Diseña el esquema en estrella conceptual para una plataforma transaccional de ventas internacionales. Define:

1. La tabla de hechos central indicando su granularidad atómica y tres métricas cuantitativas no redundantes.
2. Al menos cuatro dimensiones conformadas con sus respectivas claves subrogadas generadas mediante hashing SHA-256.
3. Identifica una dimensión degenerada que deba permanecer en el hecho.

Solución 17.1:

- **Granularidad del Hecho:** Una fila por cada ítem individual comprado dentro de una orden de pedido (`fact_ventas_items`).
- **Métricas Cuantitativas:** `cantidad_unidades` (entero aditivo), `precio_unitario_usd` (decimal semi-aditivo), `monto_descuento_usd` (aditivo), `costo_envio_imputado_usd` (aditivo).
- **Dimensiones Satélites:**
  - `dim_cliente`: Clave subrogada `cliente_sk = sha2(concat(sistema_origen, cliente_id, date_format(valido_desde, 'yyyyMMddHHmmss')), 256)`. Atributos: nombre, segmento, país de residencia, fecha de alta.
  - `dim_producto`: Clave subrogada `producto_sk = sha2(sku, 256)`. Atributos: SKU, categoría, subcategoría, marca, costo estándar.
  - `dim_tiempo`: Clave subrogada `fecha_id = cast(date_format(fecha, 'yyyyMMdd') as int)`. Atributos: fecha, día semana, trimestre, año fiscal, indicador festivo.
  - `dim_medio_pago`: Clave subrogada `medio_pago_sk = sha2(metodo_id, 256)`. Atributos: tipo pasarela, emisor, franquicia, modalidad (crédito/débito).
- **Dimensión Degenerada:** `numero_orden_pedido` (se conserva en la tabla de hechos para rastreo y auditoría sin requerir un join con una dimensión separada).

Ejercicio 17.2 (Gobernanza de SCD Tipo 2 y Claves Deterministas):
Un cliente corporativo con `cliente_id = 'C-8921'` cambia su categoría de riesgo financiero de `'MEDIO'` a `'CRITICO'` el día `2026-10-12 12:00:00`.

1. ¿Por qué la clave subrogada `cliente_sk` no puede calcularse simplemente como `sha2(cliente_id, 256)`?
2. Muestra los atributos exactos de las dos filas resultantes en la dimensión tras el merge.

Solución 17.2:

1. *Razón:* Si `cliente_sk` fuera solo el hash de `cliente_id`, ambas versiones del cliente (la histórica cerrada y la nueva activa) tendrían **exactamente la misma clave subrogada**, violando la unicidad de la clave primaria de la dimensión. En SCD2, la clave subrogada debe

incorporar la fecha de inicio de vigencia: `sha2(concat(cliente_id, valido_desde), 256)`.

2. *Filas resultantes en la dimensión:*
   - *Versión Histórica Cerrada:*
     - `cliente_sk`: `sha2('C-8921_2025-01-01 00:00:00', 256)`
     - `cliente_id`: `'C-8921'`
     - `categoria_riesgo`: `'MEDIO'`
     - `valido_desde`: `'2025-01-01 00:00:00'`
     - `valido_hasta`: `'2026-10-12 12:00:00'`
     - `es_actual`: `false`
   - *Versión Vigente Activa:*
     - `cliente_sk`: `sha2('C-8921_2026-10-12 12:00:00', 256)`
     - `cliente_id`: `'C-8921'`
     - `categoria_riesgo`: `'CRITICO'`
     - `valido_desde`: `'2026-10-12 12:00:00'`
     - `valido_hasta`: `'9999-12-31 23:59:59'`
     - `es_actual`: `true`

17.5 Resumen y Puntos Clave

- **Refinamiento progresivo:** Bronze asegura inmutabilidad y auditabilidad cruda; Silver conforma, tipa y estandariza la verdad operacional; Gold estructura datos para máxima velocidad analítica.
- **Vigencia de Ralph Kimball:** El modelado dimensional en estrella y OBT maximiza el rendimiento de motores columnares al eliminar cruces distribuidos complejos y potenciar el data skipping.
- **Claves subrogadas y Paradoja del Cumpleaños:** Para dimensiones que superan cientos de millones de registros, los hashes de 64 bits (`xxhash64`) presentan un riesgo de colisión inaceptable (~2.7% con 10^9 filas). Utilice **SHA-256** para garantizar unicidad global determinista.
- **SCD Tipo 2 atómico y seguro:** Al construir la fuente del `MERGE INTO`, filtre estrictamente para unir únicamente clientes nuevos y las nuevas versiones de los modificados, evitando duplicar registros estables.

- **Idempotencia con `replaceWhere`:** En Delta Lake, utilice `.option("replaceWhere", "predicado")` para garantizar el reemplazo atómico de particiones específicas sin riesgo de borrar datos ajenos.

Lecturas Recomendadas

1. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
2. **Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021).** *Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics.* Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR).
3. **Databricks Engineering Documentation (2024).** *Delta Lake Merge and SCD Type 2 Architecture Patterns.* Databricks Guides (`https://docs.delta.io/latest/delta-update.html`).

CAPÍTULO 18

Lakehouse y Formatos de Tabla Abiertos: Delta Lake y Apache Iceberg

NÚCLEO

> *"Los formatos de tabla abiertos como Delta Lake y Apache Iceberg redefinen la relación entre el cómputo distribuido y el almacenamiento masivo: dotan a los archivos columnares de semántica transaccional ACID, aíslan a los lectores de escrituras concurrentes y elevan un lago pasivo de objetos a un sistema gestor de bases de datos de escala planetaria."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la anatomía interna del registro de transacciones de **Delta Lake** (`_delta_log`), los commits atómicos JSON y los checkpoints Parquet.
- Analizar la jerarquía arquitectónica en cinco capas de **Apache Iceberg**: Catálogo, archivos de metadatos, listas de manifiestos (*Manifest Lists*), manifiestos (*Manifests*) y archivos de datos.
- Dominar las innovaciones modernas de Delta Lake: **Liquid Clustering (`CLUSTER BY`)**, vectores de borrado (*Deletion Vectors*) y mapeo de columnas (*Column Mapping*).
- Implementar operaciones atómicas de fusión (`MERGE INTO` / UPSERT), compactación con Z-Order (curva de Morton) y purga segura (`VACUUM`).
- Escribir DDL y comandos de mantenimiento en **Apache Iceberg**: particionamiento transformado (`days(ts)`, `bucket`), compactación de archivos (`rewrite_data_files`) y expiración de snapshots.
- Evaluar el control de concurrencia optimista (OCC) y los escenarios reales de colisión bajo el nivel de aislamiento `WriteSerializable`.

Definición 18.1 (El Lago Pasivo frente al Notario Transaccional)

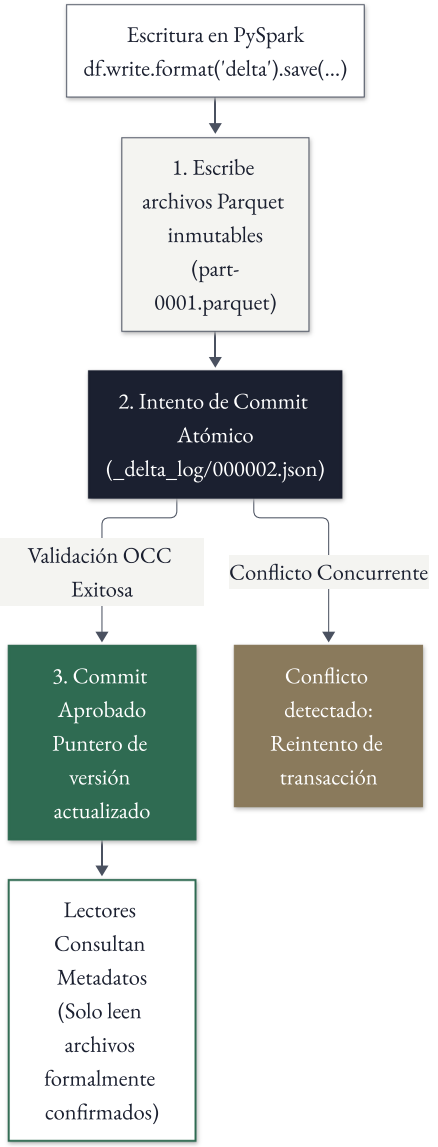
Un Data Lake tradicional basado puramente en Parquet sobre Amazon S3 o Azure ADLS es equivalente a una carpeta de red compartida sin control de concurrencia: si un proceso de escritura falla a mitad de la carga, deja archivos a medio escribir que corrompen las lecturas concurrentes; si un registro requiere actualizarse, es necesario reescribir la partición completa.

Un **formato de tabla abierto** actúa como un **notario público transaccional**: los datos se almacenan en archivos Parquet inmutables, pero ningún motor de cómputo (Spark, Trino, Flink, DuckDB) puede acceder a ellos sin consultar primero el libro de actas oficial (el registro `_delta_log` en Delta o el árbol de manifiestos en Iceberg). El notario solo valida una operación cuando todos los archivos han sido escritos con éxito, garantizando atomicidad estricta y aislamiento optimista.

18.1 Semántica ACID y Control de Concurrencia Optimista (OCC)

Los formatos de tabla abiertos extienden las garantías ACID a los sistemas de almacenamiento de objetos en la nube:

- **Atomicidad:** La transacción es indivisible. Si un job de Spark de 50.000 tareas falla en la última tarea, ninguno de los archivos escritos es referenciado en el catálogo; para los lectores externos, la operación nunca ocurrió.
- **Consistencia:** Mediante la imposición de esquemas (*Schema Enforcement*), el formato rechaza escrituras que alteren los tipos de datos o nombres de columnas no autorizados, garantizando la validez de los datos.
- **Aislamiento:** Emplea **Control de Concurrencia Optimista (OCC)** bajo el estándar de aislamiento `WriteSerializable` o `Serializable`. Los lectores nunca bloquean a los escritores.
- **Durabilidad:** La durabilidad física en reposo la provee el almacenamiento de objetos subyacente (ej. la resiliencia de 99.999999999% de Amazon S3 o ADLS Gen2), mientras que el formato garantiza la durabilidad lógica mediante su registro de transacciones de escritura anticipada (*Write-Ahead Log*).



Mecánica Real de Conflictos en OCC

Bajo el nivel de aislamiento `WriteSerializable` de Delta Lake:

- **Dos inserciones ciegas concurrentes (`Append + Append`):** Si dos jobs distintos ejecutan `df.write.mode("append")` sobre la misma partición, **no colisionan**. Delta resuelve el orden de los commits y confirma ambos sin error.
- **Conflicto real de escritura:** Ocurre cuando una transacción ejecuta una operación condicional (`MERGE INTO`, `UPDATE` o `DELETE`) que lee un conjunto de archivos para evaluar predicados, y otra transacción concurrente modifica o elimina esos mismos archivos antes de que la primera haga el commit.

18.2 Anatomía Interna de Delta Lake: El Registro `_delta_log`

Cada tabla Delta almacena en su directorio raíz una subcarpeta obligatoria denominada `_delta_log/`:

1. **Archivos de Transacción JSON (`000000.json`, `000001.json`):** Cada operación confirmada añade un nuevo archivo JSON monótonamente creciente. Contiene las acciones atómicas del commit:
   - `add`: Registra la ruta, tamaño y estadísticas (mínimo, máximo, nulos por columna) de los nuevos archivos Parquet generados.
   - `remove`: Marca archivos Parquet existentes como lógicamente obsoletos (sin eliminarlos físicamente de inmediato).
   - `commitInfo`: Metadatos del usuario, motor de cómputo y timestamp de la operación.
2. **Checkpoints Consolidados Parquet (`000010.checkpoint.parquet`):** Cada 10 transacciones, Delta consolida todos los commits previos en un archivo Parquet optimizado. Al leer la tabla, Spark solo consulta el checkpoint más reciente y aplica los deltas JSON subsiguientes, evitando escanear miles de archivos JSON individuales.
3. **Column Mapping:** Permite que las columnas de la tabla tengan identificadores numéricos y nombres físicos desacoplados de sus nombres lógicos, habilitando renombramientos instantáneos (`ALTER TABLE ... RENAME COLUMN`) sin reescribir archivos Parquet.
4. **Deletion Vectors:** Al ejecutar `DELETE` o `UPDATE`, Delta no reescribe los archivos Parquet enteros que contienen las filas afectadas (Copy-on-Write). En su lugar, escribe un pequeño vector de borrado (mapa de bits Roaring Bitmap) que indica las posiciones de fila invalidadas, reduciendo el I/O en más de un 90%.

18.3 Liquid Clustering: La Superación del Particionado Tradicional

Históricamente, las tablas se particionaban mediante carpetas jerárquicas rígidas (`anio=2026/mes=10/dia=12/`). Este enfoque presenta graves problemas:

- **Sobre-particionado (*Overpartitioning*):** Miles de carpetas con archivos diminutos que saturan el sistema de archivos.
- **Rigidez:** Imposible optimizar consultas que filtren por columnas no incluidas en la jerarquía física de partición.
- **Sesgo de Datos (*Data Skew*):** Particiones con tamaños gigantescos y otras casi vacías.

Liquid Clustering (`CLUSTER BY`) reemplaza el particionado físico y Z-Order en Delta Lake moderno utilizando la **curva de Hilbert**:

```sql
-- Creación de tabla con Liquid Clustering nativo
CREATE TABLE lakehouse.gold.fact_ventas (
    venta_id STRING,
    cliente_id STRING,
    fecha_transaccion DATE,
    monto_usd DOUBLE
)
USING DELTA
CLUSTER BY (fecha_transaccion, cliente_id);

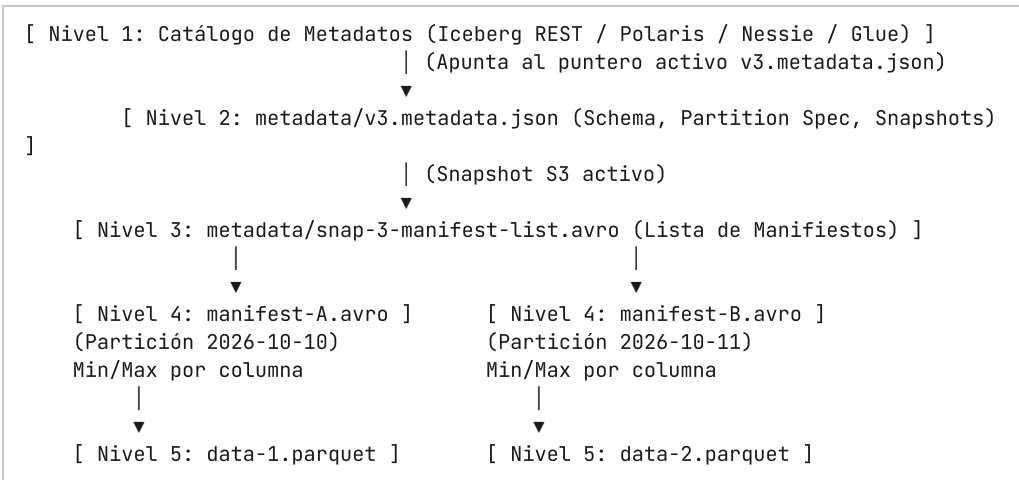
-- Compactación y re-agrupamiento incremental con Liquid Clustering
OPTIMIZE lakehouse.gold.fact_ventas;
```

- **Re-clustering Incremental:** Solo reorganiza los datos nuevos o desbalanceados sin requerir la reescritura de particiones completas.
- **Evolución sin Migración:** Las columnas de clustering pueden modificarse (`ALTER TABLE ... CLUSTER BY (nuevo_campo)`) sin necesidad de reescribir la tabla histórica.

18.4 Arquitectura Profunda de Apache Iceberg (5 Niveles)

Apache Iceberg es un formato de tabla abierto de alto rendimiento diseñado originalmente por Netflix para superar las deficiencias de escala del Hive Metastore en tablas con millones de particiones y archivos.

Iceberg organiza sus metadatos en un **árbol jerárquico de cinco niveles**:



1. **Nivel 1 - Catálogo de Metadatos (Iceberg Catalog):** Almacena exclusivamente el puntero atómico hacia el archivo de metadatos más reciente (`vN.metadata.json`). Admite implementaciones abiertas como Iceberg REST Catalog, Apache Polaris, Project Nessie, AWS Glue o Hive Metastore.
2. **Nivel 2 - Archivo de Metadatos de la Tabla (`metadata.json`):** Contiene el esquema completo de la tabla, las especificaciones de partición, el historial de evolución de esquemas y la lista completa de todos los *snapshots* válidos.
3. **Nivel 3 - Lista de Manifiestos (*Manifest List - Avro*):** Representa un snapshot individual de la tabla. Lista todos los archivos de manifiesto que componen ese snapshot, incluyendo el rango de particiones cubiertas por cada manifiesto para permitir podar manifiestos enteros antes de leer archivos de datos.
4. **Nivel 4 - Archivos de Manifiesto (*Manifest Files - Avro*):** Enumeran los archivos de datos físicos individuales (`.parquet`), especificando su ruta, tamaño y estadísticas exhaustivas a nivel de columna (mínimos, máximos, conteo de nulos y valores NaN).
5. **Nivel 5 - Archivos de Datos Físicos (*Data Files*):** Bloques columnares Parquet u ORC que contienen las filas útiles de la tabla.

Consideraciones de Producción 18.1: DDL y Mantenimiento de Apache Iceberg

Código ejecutable en Spark SQL para crear, particionar con transformaciones nativas y optimizar una tabla en Apache Iceberg:

SQL

```sql
-- 1. Creación de tabla con particionamiento oculto transformado
CREATE TABLE local_iceberg.telemetria.eventos_sensores (
    sensor_id STRING,
    timestamp_evento TIMESTAMP,
    temperatura DOUBLE,
    voltaje DOUBLE
)
USING ICEBERG
PARTITIONED BY (days(timestamp_evento), bucket(16, sensor_id));

-- 2. Compactación de micro-archivos mediante el procedimiento del sistema
CALL local_iceberg.system.rewrite_data_files(
    table => 'local_iceberg.telemetria.eventos_sensores',
    strategy => 'binpack',
    options => map('target-file-size-bytes', '536870912') -- 512 MB
);

-- 3. Expiración segura de snapshots antiguos para liberar almacenamiento en la
nube
CALL local_iceberg.system.expire_snapshots(
    table => 'local_iceberg.telemetria.eventos_sensores',
    older_than => TIMESTAMP '2026-10-05 00:00:00',
    retain_last => 5
);
```

18.5 Matriz Comparativa: Delta Lake vs. Apache Iceberg vs. Apache Hudi

Característica / Criterio	Delta Lake (3.x / Linux Foundation)	Apache Iceberg (1.x / ASF)	Apache Hudi (0.16+ / ASF)
Gobernanza / Origen	Databricks / Linux Foundation	Netflix, Apple / ASF	Uber / ASF
Modelo de Metadatos	Registro secuencial de deltas (`_delta_log/`) con checkpoints Parquet.	Árbol jerárquico en 5 niveles con manifiestos Avro.	Registro de línea de tiempo (*Timeline*) con metadatos indexados.
Optimización de Agrupación	**Liquid Clustering** (`CLUSTER BY`) con curva de Hilbert, Z-Order (Morton).	Compactación binpack / ordenamiento Z-Order o Sort.	Clustering y compresión basados en Timeline.
Particionamiento	Basado en columnas lógicas; reemplazado por Liquid Clustering.	**Oculto (Hidden Partitioning)** con funciones nativas (`days`, `bucket`).	Basado en claves de partición explícitas configuradas.
Evolución de Esquema	Por ID con **Column Mapping** activado; o por nombre con `mergeSchema`.	**Por ID entero inmutable:** Totalmente inmune a renombramientos y reordenamientos.	Soportado mediante esquemas Avro.
Mecanismo de Borrado	Deletion Vectors (Roaring Bitmaps) y Copy-on-Write.	Copy-on-Write (COW) y Merge-on-Read (MOR) con archivos de borrado posicional.	Copy-on-Write (COW) y Merge-on-Read (MOR) optimizado para streaming.
Catálogo Abierto	Unity Catalog OSS, Hive Metastore, Delta Sharing.	Iceberg REST Catalog, Apache Polaris, Nessie, Glue.	Hudi Metastore, Hive, Glue.

Consideraciones de Producción 18.2: Mantenimiento Seguro con OPTIMIZE, Z-ORDER y VACUUM

Para tablas Delta Lake tradicionales basadas en particiones, el agrupamiento espacial con Z-Order utiliza la **curva de Morton (curva Z)** intercalando bits para agrupar registros multidimensionales:

```sql
-- 1. Z-Order tradicional sobre columnas de filtro frecuente
OPTIMIZE delta.`s3://lakehouse-corp/gold/fact_ventas`
ZORDER BY (cliente_id, fecha_transaccion);

-- 2. Purga segura de archivos obsoletos (retención obligatoria de 7 días)
VACUUM delta.`s3://lakehouse-corp/gold/fact_ventas` RETAIN 168 HOURS;
```

Para entornos de prueba o emergencias operacionales donde se requiera forzar `RETAIN 0 HOURS`, **el flag oficial de seguridad en el motor Spark de Databricks es**:

```python
spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled", "false")
```

18.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutar `VACUUM` con Retención Cero en Entornos Concurrentes Deshabilitar la verificación de retención (`retentionDurationCheck.enabled = false`) y ejecutar `VACUUM ... RETAIN 0 HOURS` mientras otro trabajo de Spark está leyendo la tabla causará que la consulta falle con un error fatal de `FileNotFoundException` tan pronto intente leer un archivo que acaba de ser eliminado físicamente.

Anti-patrón 2: Borrar Archivos Parquet Manualmente desde la Consola de la Nube Borrar directamente archivos `.parquet` del bucket eludiendo el registro de transacciones o el árbol de manifiestos corrompe la consistencia de la tabla. Al consultar la tabla, el motor asumirá que los archivos declarados en los metadatos existen y fallará la lectura.

Anti-patrón 3: Z-Ordering sobre Más de 3 o 4 Columnas de Alta Cardinalidad La curva de Morton de Z-Order pierde drásticamente su efectividad a medida que se incrementa el número de dimensiones. En Delta Lake moderno, migre hacia **Liquid Clustering (`CLUSTER BY`)** para evitar este problema.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 18.1 (Control de Concurrencia Optimista en Operaciones Concurrentes):
Dos pipelines de Spark (`Job_A` y `Job_B`) escriben en la misma tabla Delta `telemetria`:

1. Si ambos jobs ejecutan simultáneamente `df.write.format("delta").mode("append").save(ruta)` sobre la misma fecha, ¿Delta Lake lanza una excepción de conflicto concurrente bajo el nivel `WriteSerializable`?

2. ¿Bajo qué escenario exacto de operaciones concurrentes sí se aborta la transacción lanzando un conflicto de concurrencia?

Solución 18.1:

1. *Escenario de Append + Append:* **No hay conflicto.** Bajo el nivel de aislamiento `WriteSerializable`, Delta Lake permite anexiones concurrentes ciegas (*Blind Appends*). Como ninguna de las dos operaciones realiza lecturas para modificar o borrar filas existentes, Delta asigna números de commit secuenciales consecutivos (ej. versión 101 para `Job_A` y versión 102 para `Job_B`), confirmando ambas escrituras sin lanzar ninguna excepción.
2. *Escenario de Conflicto Real:* Se produce cuando una transacción ejecuta una operación condicional (`MERGE INTO`, `UPDATE` o `DELETE`) que lee un subconjunto de archivos Parquet para determinar qué filas alterar, y concurrentemente otro proceso modifica o elimina esos mismos archivos físicos antes de que el primer job confirme su commit. En ese instante, Delta detecta una colisión de lectura-escritura (*Read-Write Conflict*) y aborta la transacción con una excepción `ConcurrentAppendException` o `ConcurrentTransactionException`.

Ejercicio 18.2 (Particionamiento Oculto de Iceberg frente a Parquet Clásico):
Una tabla almacena telemetría con la columna `timestamp_evento TIMESTAMP`. En Hive/Parquet tradicional, los ingenieros particionaban creando una columna derivada `fecha STRING`.

1. ¿Qué riesgo introduce este esquema tradicional si un usuario nuevo filtra `WHERE timestamp_evento = '2026-10-12 14:00:00'` omitiendo la columna sintética `fecha`?
2. ¿Cómo resuelve Apache Iceberg este problema mediante *Hidden Partitioning*?

Solución 18.2:

1. *Riesgo tradicional:* Si el usuario omite la columna sintética `fecha` en el filtro `WHERE`, el motor no puede aplicar *Partition Pruning* y ejecuta un **escaneo completo de la tabla (*Full Table Scan*)**, leyendo terabytes de datos innecesariamente y aumentando los costos y tiempos de ejecución.
2. *Solución de Iceberg:* Iceberg vincula la transformación de partición (`days(timestamp_evento)`) directamente a los metadatos de la columna original. Cuando el usuario consulta `WHERE timestamp_evento = '2026-10-12 14:00:00'`, el optimizador de Iceberg proyecta internamente la función temporal sobre la partición y lee **únicamente** el manifiesto correspondiente al día especificado, eliminando la duplicación de columnas y asegurando poda de datos automática y transparente.

18.7 Resumen y Puntos Clave

- **Semántica ACID en la nube:** Los formatos abiertos de tabla proporcionan transacciones ACID desacopladas del almacenamiento de objetos mediante registros de transacciones (Delta Lake) o árboles de manifiestos (Apache Iceberg).
- **Innovación en Delta Lake:** Liquid Clustering (`CLUSTER BY`) con la curva de Hilbert supera las limitaciones del particionado rígido tradicional, complementado por Deletion Vectors y Column Mapping.
- **Los 5 niveles de Apache Iceberg:** Catálogo → `metadata.json` → Manifest List (Avro) → Manifest Files (Avro) → Data Files (Parquet).
- **Mantenimiento y Concurrencia:** La compactación (`OPTIMIZE` / `rewrite_data_files`) combate el problema de archivos pequeños, mientras que `VACUUM` purga archivos obsoletos respetando el flag oficial de retención `retentionDurationCheck.enabled`.

Lecturas Recomendadas

1. **Armbrust, M., Das, T., Sun, L., et al. (2020).** *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.* Proceedings of the VLDB Endowment (PVLDB), 13(12), 3411-3424.
2. **Apache Iceberg Project (2024).** *Apache Iceberg Table Specification and System Procedures.* Apache Software Foundation (`https://iceberg.apache.org/spec/`).
3. **Databricks Documentation (2024).** *Use Liquid Clustering for Delta Tables.* Databricks Guides (`https://docs.databricks.com/delta/clustering.html`).

CAPÍTULO 23

Orquestación de Flujos: Apache Airflow y Azure Data Factory NÚCLEO

> *"Un clúster distribuido capaz de computar petabytes en segundos carece de utilidad productiva si sus tareas se ejecutan a ciegas o en momentos aleatorios. La orquestación es el sistema nervioso central de la plataforma de datos: resuelve dependencias entre sistemas heterogéneos, administra reintentos ante fallos transitorios y garantiza que la información esté disponible con puntualidad matemática."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender el rol de los orquestadores analíticos frente a programadores de tareas elementales como *Cron*.
- Evaluar y comparar críticamente las arquitecturas de **Apache Airflow** (versión moderna 2.10+ / 3.0) y **Azure Data Factory (ADF)**.
- Diseñar grafos acíclicos dirigidos (**DAGs**) en Airflow aplicando el paradigma **TaskFlow API** (`@dag`, `@task`), operadores diferibles con `deferrable=True` y programación reactiva basada en **Datasets / Assets**.
- Dominar la semántica matemática de intervalos de datos (`logical_date`, `data_interval_start`, `data_interval_end`) para garantizar ejecuciones estrictamente **idempotentes**.
- Construir pipelines nativos en **Azure Data Factory** articulando *Linked Services*, *Datasets*, *Activities* e *Integration Runtimes* (Azure IR vs. Self-Hosted IR).

Definición 23.1 (Orquestación de Pipelines de Datos)

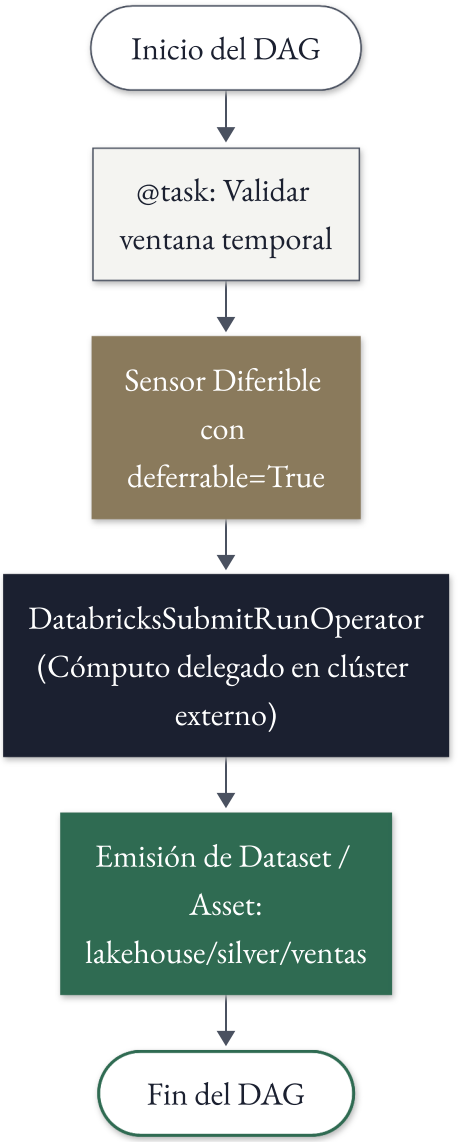
La **orquestación de datos** es la automatización programática del ciclo de vida integral de los flujos de trabajo analíticos. Comprende la resolución de dependencias topológicas entre tareas heterogéneas, la planificación cronológica o reactiva por eventos, la asignación dinámica de recursos computacionales, la gestión de reintentos con retraso exponencial ante fallos transitorios y la emisión de métricas de cumplimiento de acuerdos de nivel de servicio (SLA).

23.1 Comparativa Arquitectónica: Apache Airflow vs. Azure Data Factory

Criterio de Selección	Apache Airflow (2.10+ / 3.0)	Azure Data Factory (ADF)
Filosofía Fundamental	*Pipelines as Code* (Python nativo, control total).	Orquestación visual declarativa basada en JSON (*Low-Code/UI-First*).
Flexibilidad Dinámica	Total: soporte para bucles dinámicos, librerías personalizadas y SDKs.	Rígida: acotada por los bloques y conectores provistos por Azure.
Infraestructura	Requiere gestión (Astronomer, MWAA, Cloud Composer, Kubernetes).	Totalmente administrada (*Serverless / PaaS*) en Azure.
Conectividad Híbrida	Agentes en red o workers desplegados en cualquier nube u on-premise.	**Self-Hosted Integration Runtime (SHIR)** para cruzar firewalls locales.
Carga Computacional	**Director liviano:** Delega el cómputo a motores externos (Databricks, Snowflake).	**Orquestador + Motor ETL:** Ejecuta Data Flows (Spark gestionado) o delega.
CI/CD y Testing	Excelente: pruebas unitarias con `pytest`, linters y control Git estándar.	Exportación de plantillas ARM/Bicep y ramas de publicación en Azure DevOps.

23.2 Anatomía de un DAG en Apache Airflow: TaskFlow API y Datasets

El estándar moderno de Apache Airflow utiliza la sintaxis declarativa **TaskFlow API** (`@dag`, `@task`) y el parámetro `schedule` (reemplazando al deprecado `schedule_interval`):



Operadores Diferibles (*Deferrable Operators*)

Los sensores tradicionales bloquean permanentemente un hilo de ejecución (*worker slot*) mientras esperan que ocurra un evento externo, desperdiciando recursos del clúster. Con `deferrable=True`, el operador suspende su ejecución y cede su worker, registrándose en el proceso asíncrono **Triggerer** de Airflow hasta que el evento se activa.

Programación Reactiva basada en Datasets / Assets

En lugar de acoplar DAGs mediante horarios fijos en cron, Airflow permite que un pipeline productor declare un **Dataset** en sus salidas (`outlets`), disparando automáticamente el DAG consumidor en el instante en que los datos se confirman:

PYTHON

```python
from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.datasets import Dataset
from airflow.providers.databricks.operators.databricks import
DatabricksSubmitRunOperator
from airflow.sensors.external_task import ExternalTaskSensor

# Definición del activo de datos compartido (URI canónica)
dataset_ventas_silver = Dataset("lakehouse://silver/ventas_diarias")

argumentos_por_defecto = {
    'owner': 'data_engineering',
    'depends_on_past': False,
    'email_on_failure': True,
    'retries': 3,
    'retry_delay': timedelta(minutes=3),
    'retry_exponential_backoff': True
}

@dag(
    dag_id='pipeline_ventas_medallon',
    default_args=argumentos_por_defecto,
    schedule='0 3 * * *',  # Ejecución diaria programada a las 03:00 UTC
    start_date=datetime(2026, 1, 1),
    catchup=False,
    max_active_runs=1,
    tags=['ventas', 'lakehouse']
)
def pipeline_ventas():

    @task
    def pre_validacion(data_interval_start=None, data_interval_end=None):
        print(f"[INTERVALO] Ventana de datos desde {data_interval_start} hasta
{data_interval_end}")
        return {"inicio": str(data_interval_start)}

    # Sensor diferible de Airflow core (libera el worker slot mientras espera)
    esperar_datos_crudos = ExternalTaskSensor(
        task_id='esperar_ingesta_cruda',
        external_dag_id='dag_ingesta_landing',
        allowed_states=['success'],
        execution_delta=timedelta(hours=1), # Ajuste si el DAG externo corre a
las 02:00 UTC
        deferrable=True,                    # Clave de eficiencia: ejecución
asíncrona
        timeout=7200
    )
```

```
    tarea_databricks = DatabricksSubmitRunOperator(
        task_id='procesar_silver_en_databricks',
        databricks_conn_id='databricks_default',
        new_cluster={
            'spark_version': '14.3.x-scala2.12',
            'node_type_id': 'Standard_D4ds_v5',
            'num_workers': 2
        },
        notebook_task={
            'notebook_path': '/Production/Pipelines/02_Silver_Ventas',
            'base_parameters': {'fecha_proceso': '{{ ds }}'}
        },
        outlets=[dataset_ventas_silver]  # Emite el activo para activar DAGs
dependientes
    )

    pre_validacion() >> esperar_datos_crudos >> tarea_databricks

instancia_dag = pipeline_ventas()
```

23.3 Semántica Matemática de Intervalos de Datos e Idempotencia

En Apache Airflow, el tiempo de planificación se define de forma rigurosa:

- **Para una expresión cron `0 3 * * *` (ejecución diaria a las 03:00 UTC):**
  - Una ejecución disparada el **3 de octubre a las 03:00 UTC** no procesa de 00:00 a 00:00; procesa el intervalo comprendido entre la corrida programada anterior y la actual:

$$\text{data_interval_start} = \text{2026-10-02 03:00:00 UTC}$$

$$\text{data_interval_end} = \text{2026-10-03 03:00:00 UTC}$$

$$\text{logical_date} = \text{2026-10-02 03:00:00 UTC}$$

- **Para la macro `@daily` (`0 0 * * *`):**
  - Únicamente en este caso el intervalo abarca desde `00:00:00` hasta `00:00:00` del día siguiente.

Garantía de Idempotencia en Pipelines

Un pipeline es **idempotente** si la reejecución de cualquier intervalo temporal produce exactamente el mismo estado en la base de datos destino:

$$\text{Pipeline}(\text{Pipeline}(D, \text{intervalo})) = \text{Pipeline}(D, \text{intervalo})$$

Esto exige que el código SQL o PySpark parametrice sus filtros con `{{ ds }}` y escriba mediante reemplazo atómico de partición (`replaceWhere` en Delta Lake) en lugar de inserciones acumulativas ciegas.

23.4 Orquestación en Azure Data Factory (ADF)

En arquitecturas empresariales sobre Microsoft Azure, **Azure Data Factory (ADF)** es el servicio gestionado líder para orquestar movimientos e ingestas de datos.

Componentes Fundamentales de ADF

1. **Linked Services (Servicios Vinculados):** Definen la información de conexión e identidad hacia almacenes externos (Azure Key Vault, ADLS Gen2, Databricks, PostgreSQL local).
2. **Datasets (Conjuntos de Datos):** Punteros tipados que hacen referencia a las estructuras de datos dentro de un Linked Service (ej. carpeta de archivos Parquet en ADLS Gen2).
3. **Activities (Actividades):** Pasos de ejecución dentro de un pipeline:
   - *Actividades de Movimiento:* `Copy Activity` para transferencias masivas optimizadas.
   - *Actividades de Transformación:* `Databricks Notebook Activity`, `Databricks Spark Jar Activity`, `Stored Procedure Activity`.
   - *Actividades de Control de Flujo:* `Lookup`, `ForEach`, `If Condition`, `Execute Pipeline`, `Web Activity`.
4. **Integration Runtime (IR):** La infraestructura de cómputo que ejecuta las actividades:
   - **Azure IR:** Cómputo serverless elástico administrado por Microsoft para transferencias en la nube pública.
   - **Self-Hosted IR (SHIR):** Agente de software instalado en máquinas virtuales privadas o servidores on-premise para extraer datos corporativos detrás de firewalls sin exponer puertos a internet.

Definición JSON de un Pipeline de Producción en ADF

El siguiente manifiesto ilustra un pipeline de ADF que extrae parámetros dinámicos, ejecuta una validación previa y dispara una transformación en Azure Databricks:

```JSON
{
  "name": "Pipeline_Ingesta_Medallon_ADF",
  "properties": {
    "parameters": {
      "FechaEjecucion": {
        "type": "string",
        "defaultValue": "@formatDateTime(pipeline().TriggerTime, 'yyyy-MM-dd')"
      }
    },
    "activities": [
      {
        "name": "Validar_Disponibilidad_Landing",
        "type": "Validation",
        "dependsOn": [],
        "typeProperties": {
          "dataset": {
            "referenceName": "Dataset_ADLS_Bronze_Landing",
            "type": "DatasetReference"
          },
          "timeout": "00:30:00",
          "sleep": 60,
          "childItems": true
        }
      },
      {
        "name": "Transformar_Silver_Databricks",
        "type": "DatabricksNotebook",
        "dependsOn": [
          {
            "activity": "Validar_Disponibilidad_Landing",
            "dependencyConditions": ["Succeeded"]
          }
        ],
        "linkedServiceName": {
          "referenceName": "LinkedService_AzureDatabricks",
          "type": "LinkedServiceReference"
        },
        "typeProperties": {
          "notebookPath": "/Production/Pipelines/02_Silver_Ventas",
          "baseParameters": {
            "fecha_proceso": {
              "value": "@pipeline().parameters.FechaEjecucion",
              "type": "Expression"
            }
          }
        }
      }
    ],
    "annotations": ["Produccion", "Medallon", "FinOps"]
```

```
  }
}
---

## 23.5 Ecosistema Moderno de Orquestación: Airflow vs. Dagster vs. Prefect

Aunque Apache Airflow mantiene el dominio del mercado corporativo, la última
generación de orquestadores introdujo paradigmas que resuelven fricciones
históricas de desarrollo local y modelado de datos:

```mermaid
graph TD
    subgraph Orquestadores ["PARADIGMAS DE ORQUESTACIÓN MODERNA"]
        AF["Apache Airflow<br/>- Enfoque: Centrado en Tareas (Task-Centric)<br/>-
Fortaleza: Madurez, 100+ proveedores oficiales, estándar corporativo<br/>-
Limitación: DAG estático, sobrecosto de Scheduler"]
        DG["Dagster<br/>- Enfoque: Centrado en Activos (Software-Defined Assets)
<br/>- Fortaleza: Linaje nativo, particionamiento elegante, testing local<br/>-
Limitación: Curva de adopción conceptual"]
        PF["Prefect<br/>- Enfoque: Código Puro como Flujo (Code-as-Workflows)
<br/>- Fortaleza: Lógica Python dinámica, ejecución ligera, rápido
desarrollo<br/>- Limitación: Ecosistema corporativo menor que Airflow"]
    end
    style Orquestadores fill:#ffffff,stroke:#4a5160,stroke-width:1px
    style AF fill:#1b2030,color:#ffffff,stroke:#1b2030,stroke-width:2px
    style DG fill:#8a7a5c,color:#ffffff,stroke:#8a7a5c,stroke-width:2px
    style PF fill:#fbf9f4,stroke:#4a5160,stroke-width:1px
```

1. Dagster: Orquestación Basada en Activos (*Software-Defined Assets*)

En lugar de pensar en *"la tarea A ejecuta un script y luego la tarea B ejecuta otro"*, Dagster modela **qué activo de datos debe existir**:

PYTHON

```python
# Definición declarativa de un activo en Dagster
from dagster import asset
import pandas as pd

@asset
def clientes_limpios() -> pd.DataFrame:
    df = pd.read_parquet("data/clientes.parquet")
    return df.dropna(subset=["correo"])

@asset
def metricas_ventas_por_cliente(clientes_limpios: pd.DataFrame) -> pd.DataFrame:
    # Dagster deduce automáticamente la dependencia a partir del nombre del
argumento
    ventas = pd.read_parquet("data/ventas.parquet")
    return ventas.merge(clientes_limpios,
on="cliente_id").groupby("segmento").agg({"monto_usd": "sum"})
```
```

2. Prefect: Flujos Dinámicos con Código Python Nativo

Prefect permite transformar cualquier script de Python ordinario en un flujo observable simplemente añadiendo decoradores:

```python
# Flujo dinámico nativo en Prefect 2.x/3.x
from prefect import flow, task

@task(retries=3, retry_delay_seconds=10)
def extraer_datos_api(lote: int):
    return {"lote": lote, "filas": [1, 2, 3]}

@flow(name="pipeline_dinamico_prefect")
def flujo_principal(total_lotes: int = 5):
    # Permite bucles e if/else dinámicos nativos en Python sin mapping complejo
    for i in range(total_lotes):
        extraer_datos_api(i)
```

Criterio de Selección:

- **Seleccione Airflow si:** Su organización requiere un estándar industrial consolidado con miles de integraciones maduras, orquesta sistemas híbridos multi-nube y cuenta con ingenieros experimentados en su operación.
- **Seleccione Dagster si:** El equipo está enfocado intensamente en calidad y linaje de datos (Analytics Engineering, data mesh), integrando dbt y validaciones a nivel de columna como ciudadanos de primera clase.
- **Seleccione Prefect si:** El equipo prioriza la velocidad de desarrollo en Python puro, necesita desplegar tareas en contenedores efímeros sin la sobrecarga operativa de un scheduler pesado, y requiere flujos dinámicos con lógica condicional en tiempo de ejecución.

23.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cargar Grandes Volúmenes de Datos en la Memoria del Worker de Airflow Ejecutar código que descargue DataFrames masivos en el worker de Airflow provoca colapsos por *Out Of Memory* (OOM) y bloquea el scheduler. Airflow debe actuar siempre como un director de orquesta ligero que delega el cómputo pesado a Databricks, Snowflake o Spark en Kubernetes.

Anti-patrón 2: Habilitar `catchup=True` con Fechas de Inicio Antiguas Crear un DAG con una fecha de inicio de hace un año y dejar `catchup=True` dispara cientos de ejecuciones simultáneas al encenderlo, saturando las bases de datos transaccionales de origen y disparando la facturación cloud.

Anti-patrón 3: Diseñar Pipelines en ADF con Bucles `ForEach` Secuenciales sobre Millones de Archivos Iterar archivo por archivo en ADF mediante un bucle de actividades incurre en costos elevados por ejecución de actividades y tiempos de respuesta ineficientes. Utilice la `Copy Activity` con comodines (*Wildcards*) o procese mediante Databricks Auto Loader.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 23.1 (Sincronización de DAGs a Horas Distintas y Datasets):
El DAG de ingesta de CRM finaliza diariamente a las **02:00 UTC**. El DAG de consolidación financiera está programado para correr a las **04:00 UTC**.

1. Si se utiliza un `ExternalTaskSensor` en el DAG de las 04:00 UTC, ¿por qué fallará si se deja la configuración por defecto? ¿Qué parámetro exacto debe ajustarse?
2. ¿Cómo se resuelve esta dependencia de forma nativa y desacoplada utilizando **Datasets / Assets** en Airflow moderno?

Solución 23.1:

1. *Fallo y corrección con sensor:* Por defecto, `ExternalTaskSensor` busca una ejecución en el DAG externo que tenga exactamente el **mismo `logical_date`**. Al ejecutarse a horas distintas (04:00 vs 02:00), las marcas de tiempo difieren por 2 horas, por lo que el sensor esperará hasta alcanzar el timeout. La corrección exige configurar el desfase explícito:

```python
ExternalTaskSensor(
    task_id='esperar_crm',
    external_dag_id='dag_ingesta_crm',
    execution_delta=timedelta(hours=2),
    deferrable=True
)
```

2. *Solución moderna con Datasets / Assets:* Se desacoplan los horarios por cron.
   - El DAG de CRM declara en su tarea terminal: `outlets=[Dataset("lakehouse://silver/crm")]`.
   - El DAG financiero se declara sin expresión cron fija: `schedule=[Dataset("lakehouse://silver/crm")]`. Tan pronto como el DAG de CRM confirma la actualización del dataset en la base de metadatos de Airflow, el DAG financiero se activa inmediatamente de forma reactiva, eliminando esperas ociosas y desfasajes horarios.

Ejercicio 23.2 (Conectividad Híbrida Segura en Azure Data Factory):
Una entidad bancaria necesita extraer diariamente 500 GB de datos desde un servidor Oracle alojado en un centro de cómputo privado (*On-Premise*) sin conexión pública a internet hacia un contenedor ADLS Gen2 en Azure. Describe los componentes arquitectónicos de ADF requeridos para cumplir este flujo de forma segura.

Solución 23.2:

1. **Self-Hosted Integration Runtime (SHIR):** Se instala el agente de software SHIR en una máquina virtual Windows ubicada dentro de la red corporativa del banco con acceso local a la base de datos Oracle. El agente solo requiere salida HTTPS (puerto 443 saliente) hacia Azure; no requiere abrir puertos entrantes en el firewall bancario.
2. **Linked Services:**
   - *Linked Service Oracle:* Configurado para utilizar el SHIR local, autenticándose mediante credenciales seguras almacenadas en Azure Key Vault.
   - *Linked Service ADLS Gen2:* Conectado al almacenamiento del Lakehouse mediante una Identidad Administrada (*Managed Identity*).
3. **Copy Activity en ADF:** Se ejecuta un pipeline en ADF cuya actividad de copia lee a través del túnel seguro establecido por el SHIR y escribe directamente en el contenedor Bronze en la nube con cifrado TLS 1.3 en tránsito.

23.7 Resumen y Puntos Clave

- **Orquestador como director liviano:** Airflow y ADF deben coordinar dependencias y estados sin procesar transformaciones analíticas pesadas en su propia infraestructura.
- **Modernidad en Airflow:** La TaskFlow API (`@dag`, `@task`), los operadores diferibles (`deferrable=True`) y la programación por Datasets representan el estándar de alta concurrencia.
- **Intervalos cron y semántica temporal:** Para `0 3 * * *`, el intervalo de datos procesado cubre de 03:00 a 03:00 del periodo precedente, requiriendo parametrización determinista con `{{ ds }}` para garantizar idempotencia.
- **Ecosistema de Azure Data Factory:** Articula *Linked Services*, *Datasets*, *Activities* y *Integration Runtimes* (Azure IR para nube y Self-Hosted IR para redes híbridas privadas).

Lecturas Recomendadas

1. **Harenslak, B. & de Ruiter, J. (2021).** *Data Pipelines with Apache Airflow.* Manning Publications.
2. **Apache Airflow Documentation (2024).** *Data-Aware Scheduling, Datasets, and Deferrable Operators.* Apache Software Foundation (`https://airflow.apache.org/docs/`).
3. **Microsoft Learn (2024).** *Azure Data Factory Pipelines, Integration Runtime and Hybrid Networking Architecture.* Microsoft Documentation.

CAPÍTULO 24

Transformación Modular con dbt, Snapshots e Idempotencia NÚCLEO

> *"La capa de transformación en el Lakehouse no debe ser un laberinto artesanal de scripts SQL dispersos y procedimientos almacenados opacos. dbt traslada los principios consagrados de la ingeniería de software —control de versiones en Git, modularidad, pruebas automatizadas, integración continua y documentación viva— al núcleo del modelado analítico de datos."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución paradigmática de ETL a **ELT** y el rol de **dbt (*data build tool*)** en la capa de transformación del almacén analítico.
- Estructurar proyectos modulares organizados en capas estándar: Fuentes (*Sources*), *Staging*, *Intermediate* y *Marts* (Hechos y Dimensiones).
- Dominar las materializaciones de dbt: `view`, `table`, `incremental` y `ephemeral`.
- Implementar modelos incrementales de alto rendimiento utilizando la estrategia `merge` y la macro `is_incremental()`.
- Automatizar el versionado histórico de Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) mediante **dbt Snapshots**.
- Formular y defender la propiedad de **idempotencia** en pipelines de datos con rigor matemático.

Definición 24.1 (Idempotencia en Ingeniería de Datos)

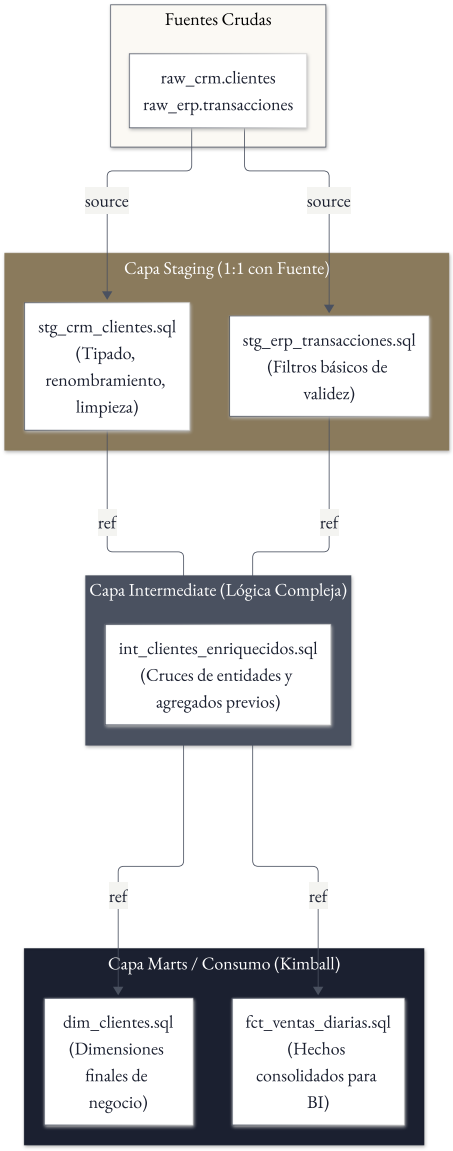
En ciencias de la computación, una función u operación de transformación $f(x)$ se define como **estrictamente idempotente** si su aplicación reiterada sobre un estado inicial determinado produce exactamente el mismo resultado final que una única ejecución:

$$f(f(x)) = f(x) \quad \forall x$$

En el contexto de los pipelines analíticos, esto garantiza que si una ejecución falla a medio camino, se reintenta por el orquestador o se vuelve a disparar manualmente para corregir un incidente, las tablas de destino convergen a un estado idéntico y consistente, sin duplicar registros, alterar métricas agregadas ni requerir intervenciones manuales de purga.

24.1 El Paradigma ELT y la Arquitectura Modular de dbt

En el modelo ETL tradicional, los datos se extraían de las fuentes operacionales y se transformaban en servidores intermedios antes de cargarse en el almacén de datos. En el paradigma moderno **ELT**, los datos se cargan primero en su estado crudo en el Lakehouse o almacén cloud (Snowflake, BigQuery, Databricks, Redshift), y **dbt orquesta las transformaciones directamente dentro del motor de cómputo columnar**, aprovechando su escalabilidad masiva mediante sentencias `SELECT` puras.



La Tríada Modular de dbt

1. **Capa Staging (`stg_`):** Mapeo 1:1 con las tablas de origen. Aplica limpieza cosmética: renombra columnas según convenciones corporativas, castea tipos de datos primitivos y elimina espacios en blanco sin realizar cruces complejos.
2. **Capa Intermediate (`int_`):** Resuelve lógica de negocio compleja, cruces entre múltiples modelos de staging y cálculos intermedios que no deben exponerse directamente a los analistas de negocio.
3. **Capa Marts (`fct_`, `dim_`):** Estructuras dimensionales finales conformadas bajo la metodología Kimball (tablas de hechos y dimensiones) preparadas para ser consultadas por

herramientas de visualización o analistas.

24.2 Materializaciones en dbt

La configuración `materialized` define cómo dbt compila el código SQL en la base de datos subyacente:

- **`view`:** Crea una vista SQL ordinaria. No consume almacenamiento físico; reejecuta la consulta en cada lectura. Ideal para la capa Staging o transformaciones ligeras.
- **`table`:** Descarta la tabla anterior y la reconstruye por completo en cada ejecución (`DROP TABLE ... CREATE TABLE AS SELECT`). Garantiza consistencia pero resulta prohibitivamente costosa en tablas con cientos de millones de registros.
- **`ephemeral`:** No crea ningún objeto en la base de datos; interpola el modelo como una expresión de tabla común (*Common Table Expression - CTE*) directamente en los modelos que lo referencian.
- **`incremental`:** Inserta únicamente las filas nuevas o actualizadas desde la última corrida del pipeline, minimizando el escaneo de datos y el gasto de cómputo en tablas masivas.

24.3 Modelado Incremental e Idempotente con la Estrategia `merge`

Para que un modelo incremental sea eficiente e idempotente, se combina la configuración de materialización con la macro `is_incremental()`:

SQL

```sql
-- models/marts/fct_pagos_recientes.sql
{{ config(
    materialized='incremental',
    unique_key='pago_id',
    incremental_strategy='merge',
    file_format='delta',
    on_schema_change='sync_all_columns'
) }}

WITH fuente_staging AS (
    SELECT
        pago_id,
        cuenta_id,
        monto_centavos,
        moneda,
        estado_pago,
        fecha_transaccion,
```

```
        fecha_actualizacion_sistema
    FROM {{ ref('stg_pasarela_pagos') }}

    {% if is_incremental() %}
        -- En ejecuciones subsecuentes, filtrar únicamente los registros
modificados
        -- Se añade una ventana de seguridad de 3 días para mitigar eventos
desordenados
        WHERE fecha_actualizacion_sistema >= (
            SELECT dateadd(day, -3, coalesce(max(fecha_actualizacion_sistema),
'1970-01-01'))
            FROM {{ this }}
        )
    {% endif %}
)

SELECT * FROM fuente_staging
```

Mecanismo de Fusión Atómica

Cuando dbt compila este modelo en una ejecución incremental:

1. Crea una tabla temporal ligera con las filas filtradas por la cláusula `is_incremental()`.
2. Ejecuta una sentencia `MERGE INTO` nativa:
   - Si el `pago_id` ya existe en la tabla de destino, actualiza sus campos con los valores recientes.
   - Si el `pago_id` no existe, inserta la nueva fila.
3. El resultado es **estrictamente idempotente**: ejecutar el comando `dbt run --select fct_pagos_recientes` tres veces consecutivas deja la tabla en el mismo estado sin generar duplicados.

24.4 Historial Automatizado: dbt Snapshots para SCD Tipo 2 (Sintaxis Clásica vs. YAML 1.9+)

Gestionar Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) manualmente mediante complejos procedimientos almacenados suele generar inconsistencias y roturas de linaje.

dbt Snapshots permite capturar el estado histórico de una tabla mutable de manera puramente declarativa. A partir de dbt 1.9+, es posible definirlos tanto en archivos SQL como en archivos YAML declarativos centralizados:

1. Sintaxis Declarativa en YAML (dbt 1.9+ / Recomendada)

YAML

```yaml
# snapshots/snapshots.yml
snapshots:
  - name: snp_clientes
    relation: source('crm_operacional', 'clientes')
    config:
      target_schema: snapshots
      unique_key: cliente_id
      strategy: check
      check_cols:
        - direccion
        - segmento_riesgo
        - telefono
        - correo_electronico
      invalidate_hard_deletes: true
```

2. Sintaxis Clásica en Archivo SQL

SQL

```sql
-- snapshots/snp_clientes.sql
{% snapshot snp_clientes %}

{{
    config(
      target_schema='snapshots',
      unique_key='cliente_id',
      strategy='check',
      check_cols=['direccion', 'segmento_riesgo', 'telefono',
'correo_electronico'],
      invalidate_hard_deletes=True
    )
}}

SELECT
    cliente_id,
    nombre_completo,
    direccion,
    segmento_riesgo,
    telefono,
    correo_electronico
FROM {{ source('crm_operacional', 'clientes') }}

{% endsnapshot %}
```

Estrategias de Snapshot: `check` vs. `timestamp`

- **Estrategia `check`:** dbt compara los valores de una lista explícita de columnas (`check_cols`). Si alguno cambia, dbt genera una nueva versión histórica. Si la base de datos de origen carece de una columna confiable de marca temporal de actualización, `check` es la estrategia idónea.

- **Estrategia `timestamp`:** Requiere una columna como `updated_at`. Si el valor de `updated_at` es más reciente que el registrado en la última ejecución, dbt asume que el registro mutó.

Columnas Meta Añadidas Automáticamente por dbt

Cada vez que se ejecuta el comando `dbt snapshot`, dbt gestiona de forma transparente cuatro columnas meta de control histórico:

- `dbt_scd_id`: Hash único que identifica la versión histórica del registro.
- `dbt_updated_at`: Marca temporal en la que ocurrió el snapshot.
- `dbt_valid_from`: Fecha/hora de inicio de validez de esta versión del registro.
- `dbt_valid_to`: Fecha/hora de finalización de validez. Si el registro es la versión actualmente activa, este campo se mantiene en `NULL`.

24.5 Macros Jinja, Calidad Automatizada y Documentación Viva

dbt no es solo un compilador SQL; es un entorno completo de ingeniería de software que traslada prácticas de desarrollo al almacén analítico:

1. Reutilización de Lógica mediante Macros Jinja

Las macros permiten encapsular fragmentos de SQL repetitivos en funciones parametrizadas:

```SQL
-- macros/convertir_moneda.sql
{% macro convertir_centavos_a_dolares(columna_monto, decimales=2) %}
    ROUND(CAST({{ columna_monto }} AS NUMERIC) / 100.0, {{ decimales }})
{% endmacro %}
```

Uso directo dentro de cualquier modelo:

```SQL
SELECT
    pago_id,
    {{ convertir_centavos_a_dolares('monto_centavos') }} AS monto_usd
FROM {{ ref('stg_pasarela_pagos') }}
```

2. Pruebas Automáticas: Genéricas vs. Singulares

- **Pruebas Genéricas (YAML):** Declaradas en `models/schema.yml` para validar restricciones estructurales:

```yaml
version: 2
models:
  - name: fct_pagos_recientes
    description: "Tabla de hechos de transacciones de pago verificadas"
    columns:
      - name: pago_id
        tests:
          - unique
          - not_null
      - name: estado_pago
        tests:
          - accepted_values:
              values: ['COMPLETADO', 'PENDIENTE', 'FALLIDO']
      - name: cuenta_id
        tests:
          - relationships:
              to: ref('dim_cuentas')
              field: cuenta_id
```

- **Pruebas Singulares (SQL):** Consultas personalizadas guardadas en `tests/assert_monto_total_positivo.sql`. Si la consulta retorna una o más filas, **el test falla**:

```sql
-- Si existen pagos completados con saldo negativo, la prueba detiene el pipeline
SELECT pago_id, monto_centavos
FROM {{ ref('fct_pagos_recientes') }}
WHERE estado_pago = 'COMPLETADO' AND monto_centavos < 0;
```

3. Documentación y Linaje Interactivo

Al ejecutar `dbt docs generate && dbt docs serve`, dbt compila los metadatos de los modelos, descripciones, tipos de datos y genera el grafo interactivo de linaje (*DAG View*), permitiendo a ingenieros y analistas auditar el origen exacto de cada columna sin inspeccionar manualmente el código fuente.

24.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Referencias Rígidas (*Hardcoded Names*) sin la Función `{{ ref(...) }}` Escribir `FROM analytics_prod.staging.stg_clientes` en lugar de `FROM {{ ref('stg_clientes') }}` destruye el grafo acíclico de dependencias de dbt. Al omitir `ref()`, dbt no puede deducir el orden de compilación, provocando que los modelos dependientes se ejecuten antes de que sus tablas fuentes hayan sido creadas o actualizadas.

Anti-patrón 2: Consultar la Tabla de Destino sin Coalesce en Modelos Incrementales Escribir `WHERE updated_at > (SELECT max(updated_at) FROM {{ this }})` sin envolver el `max()` en un `coalesce()` provocará que la consulta retorne `NULL` en la primera ejecución incremental si la tabla está vacía, impidiendo la carga de los datos iniciales.

Anti-patrón 3: Modelos Gigantescos Monolíticos en la Capa Marts Diseñar un único script SQL de 1.500 líneas con 20 subconsultas acopladas para generar una tabla analítica viola el principio de modularidad. Se debe descomponer la lógica en modelos intermedios reutilizables, permitiendo que cada etapa sea testeada de forma aislada.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 24.1 (Diagnóstico de Duplicidad en Modelos Incrementales):
Un ingeniero de datos observa que tras un reintento manual de un pipeline de dbt, la tabla de hechos `fct_pedidos` presenta registros duplicados para el mismo `pedido_id`. Al inspeccionar el bloque de configuración del modelo, encuentra:

```SQL
{{ config(
    materialized='incremental',
    incremental_strategy='append'
) }}
```

1. ¿Por qué la estrategia `append` rompe la propiedad de idempotencia?
2. ¿Cómo debe modificarse la configuración para corregir el incidente de forma permanente?

Solución 24.1:

1. *Causa:* La estrategia `append` simplemente ejecuta sentencias `INSERT INTO` acumulativas con los nuevos registros encontrados. Si el job falla a medio camino o se reejecuta para el mismo periodo temporal, la estrategia vuelve a insertar las filas sin verificar si ya existían previamente en la tabla de destino, duplicando los datos.

2. *Corrección:* Se debe migrar a la estrategia `merge` definiendo la clave primaria unívoca en `unique_key`:

```sql
{{ config(
    materialized='incremental',
    unique_key='pedido_id',
    incremental_strategy='merge'
) }}
```

Con esta configuración, dbt aplicará una sentencia transaccional `MERGE INTO`, asegurando que registros reinsertados actualicen la fila existente en lugar de duplicarla.

Ejercicio 24.2 (Consulta de Auditoría sobre dbt Snapshots):
A partir de la tabla de snapshots generada por `snp_clientes`, escribe una consulta SQL analítica que permita conocer cuál era la dirección residencial exacta que tenía el cliente `'C-842'` el día **15 de marzo de 2025 a las 10:00:00 UTC**.

Solución 24.2:

```sql
SELECT
    cliente_id,
    nombre_completo,
    direccion,
    dbt_valid_from,
    dbt_valid_to
FROM {{ ref('snp_clientes') }}
WHERE cliente_id = 'C-842'
  AND dbt_valid_from <= TIMESTAMP '2025-03-15 10:00:00'
  AND (dbt_valid_to > TIMESTAMP '2025-03-15 10:00:00' OR dbt_valid_to IS NULL);
```

Fundamento: Esta consulta filtra la versión del registro cuyo intervalo de validez `[dbt_valid_from, dbt_valid_to)` abarcaba el instante temporal exacto solicitado, reconstruyendo el estado del negocio en el pasado sin importar cuántas mutaciones haya sufrido posteriormente.

24.7 Resumen y Puntos Clave

- **Transformación modular con ELT:** dbt traslada la lógica de modelado al interior del almacén de datos, desacoplando la ingestión del cómputo analítico mediante código SQL declarativo versionado en Git.

- **Linaje y orden topológico con `ref()`:** La interpolación dinámica de modelos permite a dbt calcular automáticamente el DAG de compilación y validar dependencias antes de la ejecución.
- **Idempotencia mediante `merge`:** La combinación de materializaciones incrementales con `unique_key` y la macro `is_incremental()` asegura que las ejecuciones reiteradas converjan al mismo estado sin duplicación.
- **Auditoría histórica con dbt Snapshots:** Implementa Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) de forma declarativa mediante estrategias `check` o `timestamp`, manteniendo campos de vigencia `dbt_valid_from` y `dbt_valid_to`.

Lecturas Recomendadas

1. **dbt Labs Technical Staff (2024).** *The dbt Viewpoint: What is dbt and Why ELT Matters.* dbt Documentation (`https://docs.getdbt.com/`).
2. **Freeman, M. (2024).** *Modern Data Modeling with dbt: Building Scalable Analytics Engineering Pipelines.* O'Reilly Media.
3. **Stonebraker, M. & Cetintemel, U. (2005).** *"One Size Fits All": An Idea Whose Time Has Come and Gone.* Proceedings of the 21st International Conference on Data Engineering (ICDE).

CAPÍTULO 34

Proyecto Integrador: Plataforma Lakehouse y Portafolio de Producción

NÚCLEO

> *"El verdadero dominio de la ingeniería de datos no se demuestra recitando conceptos aislados, sino ensamblando una arquitectura completa donde cada componente —desde la ingesta cruda y el almacenamiento ACID hasta la orquestación, la calidad automatizada y el despliegue continuo— encaja con precisión milimétrica. Este capítulo presenta la estrategia integral de portafolio para conseguir empleo y el proyecto capstone de producción."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la estrategia de una **Tríada de Proyectos de Portafolio Graduales** (100% local y gratuito, ingesta de API REST, y streaming en tiempo real) que demuestran competencia técnica antes del capstone empresarial.
- Diseñar e implementar una plataforma analítica completa basada en la **Arquitectura Medallón** (Bronze → Silver → Gold) utilizando **Delta Lake**.
- Capturar metadatos de linaje mediante `_metadata.file_path` y marcas temporales deterministas.
- Aplicar deduplicación con `ROW_NUMBER()`, lógica trivaluada y segregación automática de anomalías a tablas de **Cuarentena**.
- Modelar dimensiones lentamente cambiantes (**SCD Tipo 2**) con claves subrogadas deterministas e idempotentes calculadas con `sha2`.
- Orquestar la ejecución automatizada de punta a punta con **Apache Airflow** mediante el patrón *Lightweight Conductor* y dependencias orientadas a datos (**Assets**).
- Validar la integridad de los datos con **Great Expectations 1.x** y aprovisionar infraestructura con **Terraform** y **GitHub Actions**.
- Configurar protocolos de **FinOps Personal** para prevenir facturas sorpresa en nubes públicas durante la experimentación.

34.1 La Tríada del Portafolio: Tres Proyectos Graduales para Conseguir Empleo

Un error frecuente de los aspirantes a Data Engineer es intentar construir un único repositorio monolítico gigantesco en la nube sin antes dominar los componentes modulares. Esto suele generar proyectos inacabados, configuraciones frágiles y riesgos financieros por costes imprevistos.

Los comités técnicos internacionales valoran una **tríada de tres proyectos especializados y concisos**, complementada por el proyecto integrador empresarial:



Proyecto de Portafolio 1: Modern Data Stack 100% Local (DuckDB + dbt Core + Airflow)

Este proyecto demuestra que dominas el modelado analítico sin gastar un solo centavo en plataformas cloud. Utiliza DuckDB como motor columnar in-process ultrarrápido, dbt Core para las transformaciones SQL y Apache Airflow (o Cosmos) para la programación.

Estructura del Repositorio

```
portfolio-duckdb-dbt-analytics/
├── orchestrator/
│   └── dags/
│       └── dag_duckdb_dbt.py         # Orquestación diaria de modelos
├── dbt_analytics/
│   ├── models/
│   │   ├── staging/
│   │   │   ├── stg_pedidos.sql       # Limpieza inicial y casteos
│   │   │   └── schema.yml            # Tests de unicidad y no-nulos
│   │   └── marts/
│   │       ├── dim_clientes.sql      # Dimensión de clientes desnormalizada
│   │       ├── fact_ventas.sql       # Tabla de hechos al nivel de ítem
│   │       └── schema.yml            # Tests relacionales y de negocio
│   ├── dbt_project.yml
│   └── profiles.yml                  # Conexión local al archivo analytics.duckdb
├── data/                             # Archivos Parquet de origen
├── docker-compose.yml                # Airflow local en contenedor ligero
├── pyproject.toml
└── README.md
```

Modelo dbt de Hechos (`models/marts/fact_ventas.sql`)

SQL

```sql
{{ config(materialized='table') }}

WITH pedidos AS (
    SELECT * FROM {{ ref('stg_pedidos') }}
),
clientes AS (
    SELECT * FROM {{ ref('dim_clientes') }}
)
SELECT
    p.pedido_id,
    p.fecha_pedido,
    p.cliente_id,
    c.pais_iso,
    p.categoria_producto,
    p.cantidad,
    p.precio_unitario,
    ROUND(p.cantidad * p.precio_unitario, 2) AS monto_total_usd
FROM pedidos AS p
INNER JOIN clientes AS c ON p.cliente_id = c.cliente_id
WHERE p.estado = 'COMPLETADO'
```

Modelo de `README.md` de Alto Impacto

MARKDOWN

```
# Local Modern Data Stack Analytics Pipeline (DuckDB + dbt + Airflow)
[![CI/CD](https://github.com/usuario/repo/actions/workflows/ci.yml/badge.svg)]
(https://github.com)
[![dbt](https://img.shields.io/badge/dbt-Core%201.9-orange)](https://getdbt.com)
[![DuckDB](https://img.shields.io/badge/Engine-DuckDB-yellow)]
(https://duckdb.org)

## Descripción Ejecutiva
Pipeline analítico reproducible de extremo a extremo que procesa 100.000
transacciones diarias de e-commerce mediante DuckDB y dbt Core, orquestado
localmente con Apache Airflow. El proyecto modela un esquema en estrella
(Kimball) y ejecuta 18 pruebas de integridad automáticas en menos de 4 segundos
sin costo de infraestructura cloud.

## Ejecución en Un Solo Paso
```bash
git clone https://github.com/usuario/portfolio-duckdb-dbt-analytics.git
cd portfolio-duckdb-dbt-analytics
docker compose up -d
```

Proyecto de Portafolio 2: Ingesta Resiliente de APIs con Pydantic y Delta
Lake Local

Este proyecto demuestra rigor en ingeniería de software: interacción HTTP con
APIs externas, manejo de desconexiones de red, contratos de datos estrictos en
Pydantic y almacenamiento incremental en formato Delta Lake local.

Estructura del Repositorio
```

portfolio-resilient-api-ingestion/ ├── src/ │ ├── client.py # Cliente HTTP con Exponential Backoff y Jitter │ ├── contracts.py # Modelos Pydantic V2 para validación │ └── pipeline.py # Ingesta y escritura en Delta Lake local ├── tests/ │ ├── test_client.py # Mocks de respuestas HTTP (200, 429, 503) │ └── test_contracts.py # Pruebas de validación de esquemas ├── pyproject.toml # Dependencias gestionadas con uv └── README.md

```
#### Cliente HTTP Resiliente (`src/client.py`)
```python
import time, random, requests
from typing import Dict, Any, List

class ResilientApiClient:
    def __init__(self, base_url: str, max_retries: int = 4):
        self.base_url = base_url
        self.max_retries = max_retries
```
```

```
    def fetch_data(self, endpoint: str) -> List[Dict[str, Any]]:
        url = f"{self.base_url}/{endpoint}"
        for attempt in range(self.max_retries):
            try:
                response = requests.get(url, timeout=5.0)
                if response.status_code == 200:
                    return response.json()
                elif response.status_code == 429: # Rate Limit
                    wait = (2 ** attempt) + random.uniform(0.1, 1.0)
                    time.sleep(wait)
                elif response.status_code >= 500: # Error transitorio del
servidor
                    time.sleep(2 ** attempt)
                else:
                    response.raise_for_status()
            except requests.RequestException:
                if attempt == self.max_retries - 1:
                    raise
                time.sleep(2 ** attempt)
        raise RuntimeError(f"Fallo definitivo al consultar {url} tras
{self.max_retries} intentos.")
```

Proyecto de Portafolio 3: Streaming en Tiempo Real con Kafka y Spark Structured Streaming

Este proyecto demuestra el dominio del cómputo en streaming: consumo continuo de topics de mensajería (Kafka o Redpanda), ventanas deslizantes (*sliding windows*), marcas de agua (*watermarking*) para manejar eventos tardíos y semántica *at-least-once* con persistencia transaccional en Delta Lake.

PYTHON

```python
# src/streaming_processor.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("ClickstreamRealTimeProcessor") \
    .config("spark.sql.shuffle.partitions", "2") \
    .getOrCreate()

# 1. Lectura de flujo de Kafka
df_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "eventos_clickstream") \
    .option("startingOffsets", "latest") \
    .load()

# 2. Parsing y ventana deslizante con marca de agua (Watermark de 10 minutos)
df_eventos = df_stream.selectExpr("CAST(value AS STRING) AS json_payload") \
```

```
    .select(F.from_json("json_payload", "usuario_id STRING, url STRING, ts
TIMESTAMP").alias("data")) \
    .select("data.*") \
    .withWatermark("ts", "10 minutes")

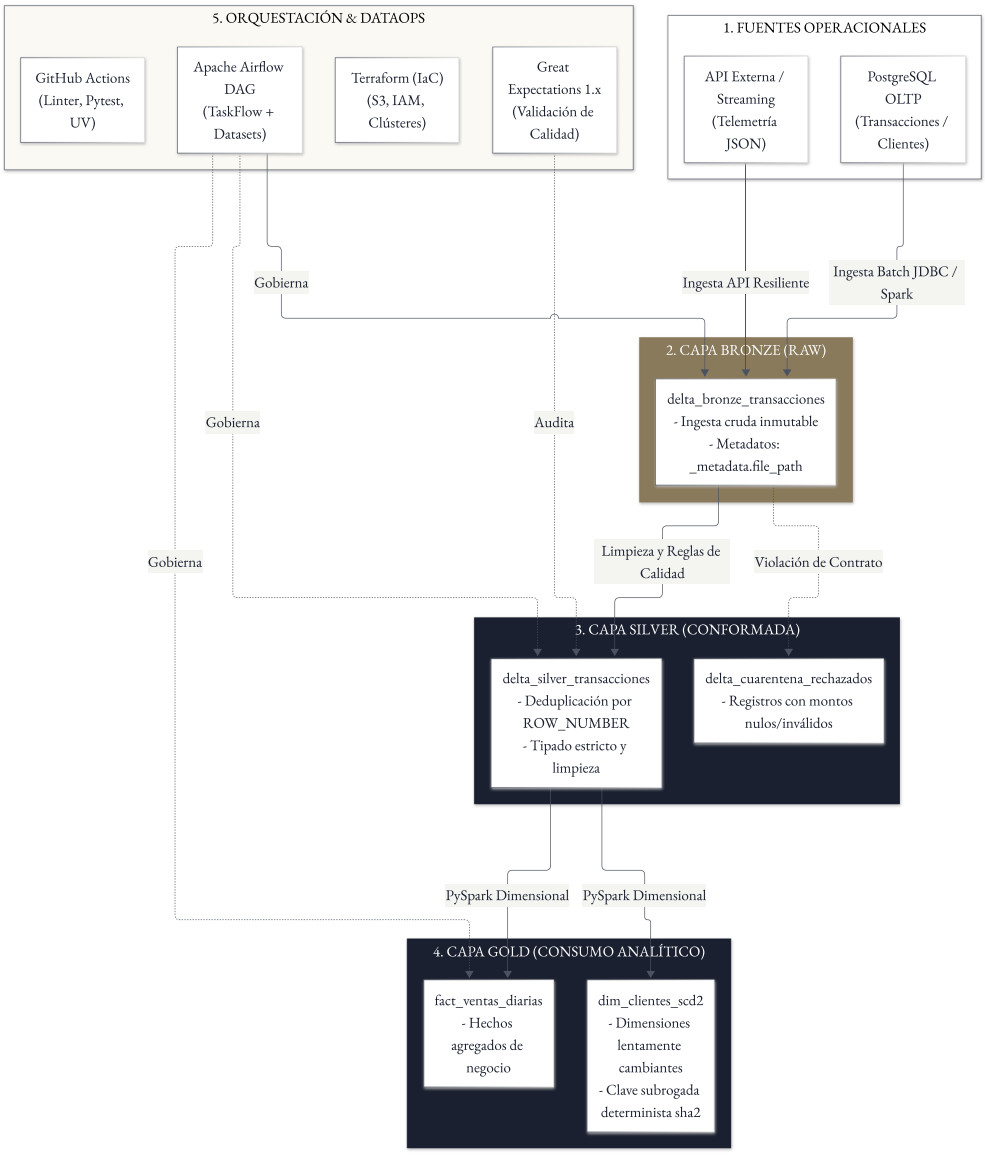
# 3. Agregación en ventana de 5 minutos deslizada cada 1 minuto
df_agregado = df_eventos.groupBy(
    F.window("ts", "5 minutes", "1 minute"),
    F.col("url")
).count()

# 4. Escritura en Delta Lake con checkpointing
query = df_agregado.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "data/checkpoints/clickstream") \
    .start("data/gold/metricas_clickstream_tiempo_real")

query.awaitTermination(60)
```

34.2 Proyecto 4 (Capstone Empresarial): La Plataforma Lakehouse de Producción

Una vez consolidados los tres proyectos anteriores, el **Capstone Empresarial** integra todos los componentes en una solución completa para corporaciones:



34.3 Paso 1: Ingesta Inmutable a Capa Bronze (Delta Lake)

La capa Bronze almacena los datos en su formato original, inyectando metadatos de auditoría y capturando la ruta de archivo mediante la columna oculta estándar `_metadata.file_path`:

PYTHON

```python
# src/pipelines/ingesta_bronze.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def ejecutar_ingesta_bronze(spark: SparkSession, ruta_fuente: str, ruta_bronze:
str) -> None:
    print(f"Cargando datos crudos desde {ruta_fuente}...")
    df_raw = spark.read.parquet(ruta_fuente)

    # Inyección de metadatos de auditoría compatibles con Unity Catalog y Spark
moderno
    df_bronze = df_raw \
        .withColumn("_metadata_ingesta_ts", F.current_timestamp()) \
        .withColumn("_metadata_archivo_origen", F.col("_metadata.file_path")) \
        .withColumn("fecha_ingesta", F.current_date())

    print(f"Escribiendo en capa Bronze Delta Lake: {ruta_bronze}...")
    df_bronze.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("fecha_ingesta") \
        .save(ruta_bronze)
    print("Ingesta Bronze completada exitosamente.")
```

34.4 Paso 2: Limpieza, Deduplicación y Cuarentena en Capa Silver

En la capa Silver aplicamos deduplicación determinista mediante `ROW_NUMBER()` y segregamos anomalías a la tabla de **Cuarentena** con evaluación estricta de valores `NULL`:

PYTHON

```python
# src/pipelines/bronze_a_silver.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

def procesar_capa_silver(spark: SparkSession, ruta_bronze: str, ruta_silver: str,
ruta_cuarentena: str) -> None:
    df_bronze = spark.read.format("delta").load(ruta_bronze)

    # 1. Deduplicación analítica conservando la versión más reciente
    ventana_dedup = Window \
        .partitionBy("transaccion_id") \
        .orderBy(F.col("_metadata_ingesta_ts").desc())

    df_deduplicado = df_bronze \
        .withColumn("fila_num", F.row_number().over(ventana_dedup)) \
        .filter(F.col("fila_num") == 1) \
        .drop("fila_num")
```

```
    # 2. Segregación de Calidad: Condición de validez de negocio
    condicion_valida = (
        F.col("monto_usd").isNotNull() &
        (F.col("monto_usd") > 0.0) &
        F.col("cliente_id").isNotNull() &
        F.col("estado_pago").isin(["COMPLETADO", "PENDIENTE", "CANCELADO"])
    )

    df_validos = df_deduplicado.filter(condicion_valida)
    df_rechazados = df_deduplicado.filter(~condicion_valida) \
        .withColumn("_motivo_rechazo", F.lit("Monto invalido, estado desconocido
o cliente nulo")) \
        .withColumn("_fecha_cuarentena", F.current_timestamp())

    # 3. Escritura atómica e idempotente
    df_validos.write.format("delta").mode("append").save(ruta_silver)
    df_rechazados.write.format("delta").mode("append").save(ruta_cuarentena)
    print("Capa Silver y Cuarentena procesadas con éxito.")
```

34.5 Paso 3: Modelado Dimensional Gold (SCD Tipo 2 y Hechos)

La capa Gold modela las dimensiones lentamente cambiantes (**SCD Tipo 2**) con **claves subrogadas deterministas** calculadas mediante `sha2` e independientes de la hora de ejecución del script:

PYTHON

```
# src/pipelines/silver_a_gold.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from delta.tables import DeltaTable

def sincronizar_dimension_clientes_scd2(spark: SparkSession,
ruta_clientes_novedades: str, ruta_dim_clientes: str) -> None:
    df_novedades = spark.read.parquet(ruta_clientes_novedades)

    # Determinismo absoluto: la fecha de inicio proviene del dato de negocio (no
de current_timestamp)
    col_fecha_negocio = F.coalesce(
        F.col("fecha_actualizacion").cast("timestamp") if "fecha_actualizacion"
in df_novedades.columns else F.col("fecha_registro").cast("timestamp"),
        F.to_timestamp(F.lit("2026-01-01 00:00:00"))
    )

    df_novedades_preparadas = df_novedades \
        .withColumn("valido_desde", col_fecha_negocio) \
        .withColumn("valido_hasta", F.lit(None).cast("timestamp")) \
        .withColumn("es_actual", F.lit(True)) \
        .withColumn("cliente_sk", F.sha2(F.concat_ws("||",
```

```
F.col("cliente_id").cast("string"), F.date_format(F.col("valido_desde"), "yyyy-
MM-dd HH:mm:ss")), 256))

    if not DeltaTable.isDeltaTable(spark, ruta_dim_clientes):
        df_novedades_preparadas.write.format("delta").save(ruta_dim_clientes)
        print("Dimensión Clientes SCD2 inicializada con carga histórica.")
        return

    tabla_dim = DeltaTable.forPath(spark, ruta_dim_clientes)
    df_actuales = tabla_dim.toDF().filter(F.col("es_actual") == True)

    # 1. Identificar registros que efectivamente mutaron (correo o segmento)
    df_cambios = df_novedades_preparadas.alias("nov").join(
        df_actuales.alias("dim"),
        on="cliente_id",
        how="inner"
    ).filter(
        (F.col("dim.correo") != F.col("nov.correo")) |
        (F.col("dim.segmento") != F.col("nov.segmento"))
    ).select("nov.*")

    # 2. Identificar registros totalmente nuevos
    df_nuevos = df_novedades_preparadas.alias("nov").join(
        df_actuales.alias("dim"),
        on="cliente_id",
        how="left_anti"
    ).select("nov.*")

    df_a_insertar = df_cambios.unionByName(df_nuevos)

    # 3. Cerrar la versión previa de los registros modificados
    tabla_dim.alias("dim").merge(
        source=df_cambios.alias("cambio"),
        condition="dim.cliente_id = cambio.cliente_id AND dim.es_actual = true"
    ).whenMatchedUpdate(set={
        "valido_hasta": "cambio.valido_desde",
        "es_actual": "false"
    }).execute()

    # 4. Insertar únicamente las filas necesarias
    df_a_insertar.write.format("delta").mode("append").save(ruta_dim_clientes)
    print("Dimensión Clientes SCD2 sincronizada de forma determinista e
idempotente.")

def construir_fact_ventas_diarias(spark: SparkSession, ruta_silver: str,
ruta_fact_ventas: str) -> None:
    """Construye la tabla de hechos analítica agregada en la capa Gold."""
    df_silver = spark.read.format("delta").load(ruta_silver)

    df_fact = df_silver.groupBy(
        F.to_date("fecha_transaccion").alias("fecha_id"),
        F.col("pais"),
        F.col("categoria_producto")
    ).agg(
        F.count("transaccion_id").alias("total_pedidos"),
        F.round(F.sum("monto_usd"), 2).alias("facturacion_total_usd"),
        F.round(F.avg("monto_usd"), 2).alias("ticket_promedio_usd")
```

```
    )

    df_fact.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(ruta_fact_ventas)
    print("Tabla fact_ventas_diarias materializada exitosamente en Capa Gold.")
```

34.6 Paso 4: Orquestación en Airflow (Patrón Lightweight Conductor)

Para evitar fugas de memoria en los trabajadores de Airflow, empleamos el patrón desacoplado **Lightweight Conductor** mediante `BashOperator`:

PYTHON

```python
# orchestrator/dags/lakehouse_daily_dag.py
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'data_engineering_team',
    'depends_on_past': False,
    'start_date': datetime(2026, 1, 1),
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    'lakehouse_daily_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    tags=['lakehouse', 'delta', 'produccion']
) as dag:

    tarea_bronze = BashOperator(
        task_id='ingesta_bronze_transacciones',
        bash_command='python /opt/pipelines/ingesta_bronze.py'
    )

    tarea_silver = BashOperator(
        task_id='conformacion_silver_y_cuarentena',
        bash_command='python /opt/pipelines/bronze_a_silver.py'
    )

    tarea_calidad = BashOperator(
        task_id='auditoria_calidad_great_expectations',
        bash_command='python /opt/pipelines/validar_calidad_gx.py'
    )

    tarea_gold = BashOperator(
        task_id='modelado_gold_dimensional',
```

```
        bash_command='python /opt/pipelines/silver_a_gold.py'
    )

    # Dependencias estrictas del pipeline
    tarea_bronze >> tarea_silver >> tarea_calidad >> tarea_gold
```

34.7 Paso 5: Calidad Automatizada con Great Expectations 1.x

Auditoría declarativa de los DataFrames en la capa Silver:

PYTHON

```python
# src/pipelines/validar_calidad_gx.py
import great_expectations as gx
from pyspark.sql import SparkSession

def auditar_calidad_silver(spark: SparkSession, ruta_silver: str) -> bool:
    context = gx.get_context()
    df_silver = spark.read.format("delta").load(ruta_silver)

    data_source =
context.data_sources.add_or_update_spark(name="spark_lakehouse")
    data_asset =
data_source.add_dataframe_asset(name="silver_transacciones_asset")
    batch_definition =
data_asset.add_batch_definition_whole_dataframe("batch_completo")

    suite = context.suites.add_or_update(name="suite_silver_transacciones")

suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(column="trans
accion_id"))
    suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(
        column="monto_usd", min_value=0.01, max_value=100000.0
    ))
    suite.add_expectation(gx.expectations.ExpectColumnValuesIsInSet(
        column="estado_pago",
        value_set=["COMPLETADO", "PENDIENTE", "CANCELADO"]
    ))

    validation_def = context.validation_definitions.add(
        gx.ValidationDefinition(name="val_silver_transacciones",
data=batch_definition, suite=suite)
    )
    resultado = validation_def.run(batch_parameters={"dataframe": df_silver})

    print(f"Resultado auditoría Great Expectations: {'EXITOSO' if
resultado.success else 'FALLIDO'}")
    return resultado.success
```

34.8 Paso 6: Infraestructura como Código (Terraform) y CI/CD

El bucket de almacenamiento se aprovisiona mediante **Terraform**:

HCL (TERRAFORM)

```hcl
# infrastructure/terraform/main.tf
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "lakehouse_storage" {
  bucket        = "empresa-lakehouse-produccion-2026"
  force_destroy = false
}

resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.lakehouse_storage.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

El pipeline de CI/CD en **GitHub Actions** valida formato y pruebas con `uv`:

YAML

```yaml
# .github/workflows/ci_cd.yml
name: CI/CD Pipeline Lakehouse

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test_and_lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar uv (Astral)
        uses: astral-sh/setup-uv@v5
      - name: Configurar Python 3.11
        run: uv python install 3.11
      - name: Instalar dependencias
        run: uv sync
      - name: Linter con Ruff
        run: uv run ruff check src/ tests/
      - name: Pruebas unitarias con Pytest
        run: uv run pytest tests/ --cov=src --cov-report=term-missing
```

34.9 Protocolo de Protección Financiera (FinOps Personal)

El mayor temor de un estudiante o ingeniero junior al experimentar con proyectos en la nube es recibir una factura imprevista de cientos de dólares. Siga este protocolo preventivo estricto antes de desplegar recursos:

1. Configurar Presupuesto con Alertas en AWS Budgets ($1.00 USD)

Nunca cree un recurso en AWS sin antes configurar una alarma de presupuesto con notificación inmediata por correo electrónico:

- Diríjase a la consola de **AWS Billing** → **Budgets**.
- Cree un presupuesto de costo mensual fijado en **$1.00 USD**.
- Configure dos reglas de alerta:
  - Alerta 1: Cuando el costo real alcance el **80% ($0.80 USD)**.
  - Alerta 2: Cuando el costo proyectado alcance el **100% ($1.00 USD)**.
- Indique su correo electrónico y active notificaciones SMS si están disponibles.

2. Alertas de Gasto en Microsoft Azure y Google Cloud

- **Azure:** Vaya a **Cost Management + Billing** → **Cost Alerts**. Cree un presupuesto sobre su grupo de recursos con alerta cuando se consuma el 50% y 80% del crédito gratuito.
- **Google Cloud:** En la sección **Facturación** → **Presupuestos y alertas**, configure una alerta con un límite de $5.00 USD vinculada a su correo personal.

3. Auto-Terminación Obligatoria en Databricks

El error financiero más habitual en Databricks consiste en iniciar un clúster de cómputo para una prueba rápida y dejarlo encendido durante el fin de semana:

- Al crear cualquier clúster interactivo, marque obligatoriamente la casilla **Auto-Termination**.
- Configure el límite en **15 o 20 minutos de inactividad** (`autotermination_minutes = 15`). Si no se ejecutan comandos, Databricks apagará las máquinas virtuales automáticamente.

4. Rutina Diaria de Higiene y Destrucción

Al finalizar una sesión de estudio o demostración en la nube, ejecute la orden de desaprovisionamiento total:

```bash
# Destruir toda la infraestructura aprovisionada por Terraform
terraform destroy -auto-approve
```

Verifique manualmente que no queden volúmenes EBS huérfanos, instancias EC2 en estado detenido (los discos EBS continúan facturando aun con la máquina apagada) ni balanceadores de carga (ELB/ALB) activos.

34.10 Resumen y Puntos Clave

- **Tríada de portafolio gradual:** Antes de desplegar infraestructuras complejas, valide su competencia mediante tres proyectos modulares: analítica local gratuita (DuckDB/dbt), ingesta de APIs resiliente (Pydantic/uv) y streaming en tiempo real (Kafka/Spark).
- **Determinismo e idempotencia:** La clave subrogada `cliente_sk` en dimensiones SCD2 debe calcularse a partir del valor de negocio de la fecha, garantizando que reejecuciones del pipeline no corrompan el histórico ni alteren claves.
- **Orquestación desacoplada:** El patrón Lightweight Conductor en Airflow evita sobrecargar la memoria de los workers ejecutando los scripts de Spark o dbt como procesos aislados.

- **FinOps preventivo:** Las alertas de presupuesto a $1 USD y el auto-apagado de clústeres son indispensables para experimentar con total tranquilidad en entornos cloud.

Lecturas Recomendadas

1. **Reis, J. & Housley, M. (2022).** *Fundamentals of Data Engineering (Chapter 11: The End-to-End Data Pipeline).* O'Reilly Media.
2. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
3. **Delta Lake Project (2024).** *Delta Lake Guide: Building Robust Lakehouses with ACID Transactions.* Delta.io.
4. **AWS Cloud Financial Management (2024).** *AWS Budgets User Guide: Best Practices for Cost Anomaly Detection.* Amazon Web Services.

CAPÍTULO 36

Apéndice B — El Manual Definitivo de Carrera, Empleo y Preparación de Entrevistas NÚCLEO

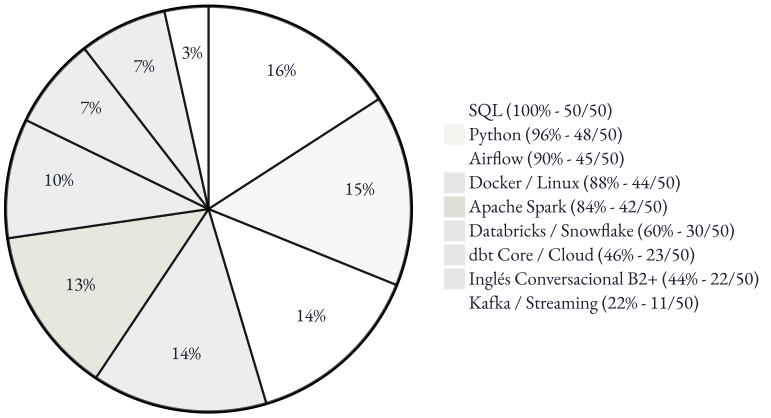
> *"La excelencia técnica en ingeniería de datos no produce dividendos en el vacío si el profesional desconoce cómo traducir sus competencias en ofertas laborales concretas. Este manual condensa la estrategia integral de inserción laboral: desde el análisis cuantitativo de ofertas reales en LATAM y el diseño de CVs optimizados para ATS, hasta la resolución de pruebas take-home, un banco exhaustivo de 150 preguntas de entrevista, comparativas salariales y régimen tributario."*

36.1 Análisis Cuantitativo de 50 Ofertas Reales de Empleo en LATAM y Remoto (Q1 2026)

Para basar esta obra en evidencia verificable y no en especulaciones de mercado, se realizó una auditoría empírica manual sobre **50 ofertas laborales reales de Data Engineering** publicadas entre el **10 de enero y el 8 de febrero de 2026** en Colombia, México, Chile, Argentina y vacantes remotas internacionales en EE.UU. y Europa.

> **Transparencia y Datos Abiertos del Estudio:**
> La totalidad de los datos crudos —incluyendo empresa, rol, país, plataforma de origen (LinkedIn, Get on Board, Torre, Arc.dev), fecha de captura, requisitos discriminados y bandas salariales— está disponible en el archivo abierto:
> `data/estudio_mercado_50_ofertas_2026.csv`
> *(Puede auditar y consultar este archivo en el repositorio de GitHub con DuckDB, Pandas o Polars).*

FRECUENCIA EMPÍRICA EN 50 OFERTAS AUDITADAS (FEBRERO 2026)



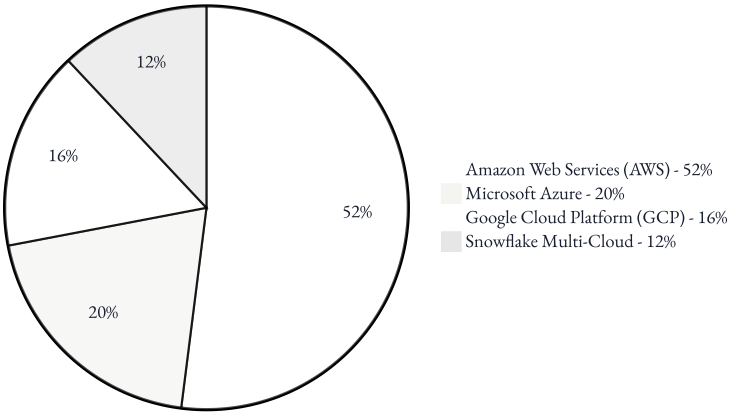
1. Frecuencias Reales y Criterios por Nivel de Experiencia

Habilidad / Tecnología	Frecuencia Real en Muestra	Relevancia en Puestos Junior (0–2 años)	Relevancia en Puestos Mid/Senior (3+ años)
SQL	**100.0% (50/50)**	Filtro eliminatorio universal (CTEs, Window Functions, deduplicación con ROW_NUMBER).	Optimización de planes de ejecución, particionado y poda.
Python	**96.0% (48/50)**	Requerido en 48 de 50 ofertas (POO básica, APIs REST, Pydantic, pytest).	Diseño modular, control de memoria y empaquetado de librerías.
Apache Airflow	**90.0% (45/50)**	El estándar de facto absoluto de orquestación en la región.	TaskFlow API, Dynamic Mapping y despliegue en Kubernetes.
Docker / Linux	**88.0% (44/50)**	Entornos locales reproducibles y shell scripting en terminal.	Contenedores multicapa y orquestación con clústeres K8s.
Apache Spark / PySpark	**84.0% (42/50)**	Fundamental para procesamiento a gran escala y Lakehouse.	Optimización interna de Catalyst, tuning de Shuffle y Data Skew.
Databricks o Snowflake	**60.0% (30/50)**	Motores analíticos dominantes en empresas consolidadas.	Arquitectura Lakehouse con Unity Catalog o Polaris.
dbt (Data Build Tool)	**46.0% (23/50)**	La herramienta clave del rol puente (Analytics Engineer).	Macros complejas en Jinja, snapshots YAML y linaje de datos.
Inglés Conversacional	**44.0% (22/50)**	Exigido en el 100% de vacantes remotas USD y 70% de startups.	Coordinación con equipos globales y redacción de RFCs.
Kafka / Streaming	**22.0% (11/50)**	Secundaria en junior; el 78% de puestos junior son batch.	Crítica en sistemas de eventos y arquitectura orientada a streaming.

2. ¿Qué Nube Conviene Aprender Primero según los Datos?

El análisis de la distribución de proveedores cloud sobre las 50 ofertas auditadas arroja un patrón contundente:

PARTICIPACIÓN DE NUBES EN OFERTAS DE DATA ENGINEERING (Q1 2026)



- **Primer Lugar: Amazon Web Services (AWS) — 52.0% (26 de 50 ofertas):**
  AWS continúa liderando holgadamente la demanda laboral en startups, scaleups y compañías remotas internacionales. Los servicios recurrentes son **S3, Glue, Athena, EMR y Redshift**.
  Recomendación: **Si no tienes una preferencia impuesta por tu empresa actual, comienza por AWS.** Es la nube con mayor volumen de vacantes abiertas y con la certificación más solicitada (*AWS Certified Data Engineer – Associate DEA-C01*).
- **Segundo Lugar: Microsoft Azure — 20.0% (10 de 50 ofertas):**
  Azure domina con fuerza en el sector corporativo tradicional, bancario y retail de gran escala en Colombia, México y Chile. Tecnologías clave: **Azure Data Factory, Synapse, Databricks sobre Azure y Microsoft Fabric**.
  Recomendación: Elige Azure si tu objetivo primordial es ingresar al sector financiero o asegurador tradicional de LATAM.
- **Tercer Lugar: Google Cloud Platform (GCP) — 16.0% (8 de 50 ofertas):**
  Fuerte presencia en unicornios tecnológicos nativos digitales de la región (Mercado Libre, Despegar, PedidosYa). Su pilar es **Google BigQuery**.
- **Cuarto Lugar: Snowflake (Multi-Cloud) — 12.0% (6 de 50 ofertas):**
  Muy demandado en empresas que adoptan el stack moderno desacoplado (Fivetran + dbt + Snowflake).

---

3. Herramientas de Ingesta Gestionada (Airbyte/Fivetran) y Capa de Consumo (Power BI/Excel)

Para ser empleable, el ingeniero debe entender qué herramientas satélite conviven con su stack:

- **Ingesta Gestionada (ELT con Airbyte y Fivetran):**
  En la industria contemporánea, los equipos ya no invierten semanas escribiendo scripts

artesanales en Python para extraer datos de SaaS estándar (Salesforce, Stripe, HubSpot, Google Analytics). Se utilizan herramientas de ingesta gestionada como **Airbyte (Open Source)** o **Fivetran**. El rol del Data Engineer es configurar estas herramientas para depositar datos crudos en Bronze, y centrar su tiempo en transformaciones de alto valor con dbt o Spark.

- **El Consumo en Negocio: Power BI y Excel:**
  Aunque el Data Engineer no diseña tableros estéticos, **debe diseñar para quienes los usan**. Los analistas consumirán la capa Gold desde Power BI o tablas dinámicas de Excel. Si el Data Engineer no modela en esquema en estrella o desconoce la diferencia entre el modo *Import* (que descarga millones de filas y revienta la memoria del analista) y *DirectQuery / Direct Lake*, generará consultas destructivas que saturarán el Data Warehouse.

36.2 Estrategia de Búsqueda Laboral y Entrada sin Experiencia Previa

El mayor obstáculo para el profesional junior es el dilema de *"requiere experiencia previa para conseguir experiencia"*. Existen cuatro estrategias probadas para sortear esta barrera:

1. El Rol Puente: Analytics Engineer o SQL Developer

Si el mercado local exige experiencia senior para roles de "Data Engineer", postúlese a posiciones de **Analytics Engineer** o **Desarrollador SQL/BI**:

- Estos puestos se centran en el uso intensivo de **dbt, SQL y Snowflake/BigQuery**, áreas donde la curva de entrada es más accesible.
- Una vez dentro de la compañía, asumirá gradualmente responsabilidades de ingesta, orquestación en Airflow y optimización de pipelines, transitando al rol formal de Data Engineer en 12 a 18 meses.

2. Canales Efectivos de Postulación

Evite la postulación pasiva indiscriminada (el botón *"Easy Apply"* de LinkedIn con cientos de candidatos):

- **Get on Board (`getonbrd.com`):** La plataforma por excelencia para startups de alto crecimiento en LATAM y trabajo remoto. Las ofertas publican transparentemente rangos salariales y tecnologías requeridas.
- **Torre (`torre.ai`):** Ecosistema centrado en talento de Latinoamérica con filtros específicos para compensaciones en USD.

- **Arc.dev y Wellfound (antes AngelList):** Ideales para puestos remotos internacionales en startups de Estados Unidos y Europa.
- **LinkedIn (Vía Networking Directo):** Utilice el buscador para identificar al *Engineering Manager*, *Data Lead* o *Head of Data* de la empresa objetivo, no solo al reclutador generalista.

3. Mensajes en Frío Efectivos (Cold Outreach)

Cuando identifique una vacante abierta, contacte al líder técnico del equipo con un mensaje breve, respetuoso y orientado a aportar valor:

Plantilla en Español (LATAM):

> *«Hola [Nombre del Líder], espero que estés muy bien. Vi que en [Nombre de la Empresa] tienen abierta la vacante de Junior Data Engineer. Estuve revisando los desafíos de escalabilidad que enfrentan en su plataforma y diseñé recientemente un proyecto open source reproducible con PySpark, Delta Lake y dbt para resolver exactamente la ingesta y deduplicación de transacciones [Enlace al GitHub]. Me encantaría conocer más sobre los retos del equipo y explorar si mi perfil puede ser de valor. ¡Un saludo!»*

Plantilla en Inglés (Remoto Internacional):

> *«Hi [Name], hope you are doing well! I saw the Data Engineer opening on your team at [Company]. I've been following your data platform evolution and recently built an end-to-end, reproducible Lakehouse pipeline using PySpark, Delta Lake, Airflow, and dbt with automated CI/CD and data contracts [GitHub Link]. I'd love the opportunity to share how my hands-on experience with these tools can contribute to your team's goals. Best regards!»*

36.3 Modelos Completos de CV y LinkedIn Optimizados para ATS

Los sistemas de seguimiento de candidatos (**ATS - Applicant Tracking Systems**) filtran automáticamente los currículums antes de que lleguen a ojos humanos.

1. Reglas Innegociables del CV para Pasar el Filtro ATS

1. **Formato en Una Sola Página (Single Page):** Para perfiles Junior y Mid-Level, jamás exceda una página.

2. **Tipografía y Estructura Limpia:** Fuente estándar (Arial, Calibri o Garamond), sin columnas múltiples, sin tablas complejas, sin íconos gráficos ni barras de porcentaje de habilidades.
3. **Fórmula Google X-Y-Z en Cada Viñeta:**

> **«Logré [X], medido por [Y], haciendo [Z]»**

Plantilla de CV Completo en Español (Formato Texto ATS)

```
LUIS GALVIS
Ibagué, Colombia | (+57) 300-123-4567 | luis.galvis@correo.com
LinkedIn: linkedin.com/in/luisgalvis-data | GitHub: github.com/luisgalvis
Portafolio: https://github.com/luisgalvis/como-ser-un-data-engineer

RESUMEN PROFESIONAL
Ingeniero de Datos con sólidos fundamentos en Python, SQL declarativo avanzado,
Apache Spark y arquitectura Lakehouse transaccional (Delta Lake). Experiencia
práctica diseñando canalizaciones ETL/ELT idempotentes, modelado dimensional
(Kimball) y orquestación con Apache Airflow. Enfocado en calidad de datos,
observabilidad y reducción de costos operativos en infraestructuras cloud.

HABILIDADES TÉCNICAS
- Lenguajes: Python (uv, Pydantic, pytest), SQL (PostgreSQL, DuckDB, ANSI SQL),
Bash Shell.
- Procesamiento & Lakehouse: Apache Spark, PySpark, Delta Lake, Apache Iceberg,
Polars, Pandas.
- Modelado & Orquestación: dbt Core, Apache Airflow (TaskFlow, Assets), Modelado
Dimensional Kimball.
- DataOps & Nube: Docker, Docker Compose, Git, GitHub Actions (CI/CD), Terraform,
AWS (S3, Athena), MinIO.
- Calidad & Monitoreo: Great Expectations 1.x, Data Contracts, OpenTelemetry.

PROYECTOS DESTACADOS DE INGENIERÍA DE DATOS
Plataforma Lakehouse de Comercio Electrónico de Extremo a Extremo (Capstone) |
GitHub: /capstone-lakehouse
- Diseñé e implementé una arquitectura Medallón en Delta Lake procesando 100.000
transacciones diarias con semántica ACID y deduplicación analítica con
ROW_NUMBER().
- Reduje el volumen de almacenamiento un 65% mediante compresión Parquet columnar
y particionamiento por fecha.
- Implementé dimensiones de clientes lentamente cambiantes (SCD Tipo 2) con
claves subrogadas deterministas (sha2), eliminando duplicados en uniones
analíticas.
- Automaticé la orquestación diaria con Apache Airflow (patrón Lightweight
Conductor) y validé contratos de datos con Great Expectations 1.x, segregando
anomalías a tablas de cuarentena.

Modern Data Stack Pipeline Local (DuckDB + dbt + Airflow) | GitHub: /duckdb-dbt-
pipeline
- Construí un pipeline analítico local 100% reproducible sin costo cloud,
procesando 50.000 pedidos con DuckDB y modelando un esquema en estrella con dbt
```

```
Core.
- Implementé 14 pruebas de integridad automáticas (unicidad, no-nulos, integridad
referencial), logrando un tiempo de compilación y prueba de solo 3.2 segundos.
- Empaqueté el entorno en Docker Compose con un archivo Makefile para ejecución
en un solo comando.

EXPERIENCIA LABORAL
Analista de Datos / Desarrollador SQL Junior | Empresa Servicios SAS (2024 -
Presente)
- Optimicé 25 consultas analíticas recurrentes en PostgreSQL mediante la
reescritura de subconsultas a CTEs y creación de índices B-Tree, disminuyendo el
tiempo de generación de reportes de 45 a 6 minutos.
- Desarrollé scripts en Python para la ingesta automatizada de eventos de ventas
desde APIs REST externas con manejo de rate limits y reintentos exponenciales,
eliminando 10 horas semanales de carga manual.

EDUCACIÓN & CERTIFICACIONES
- Licenciatura / Ingeniería en Sistemas o afín | Universidad Nacional (2020 -
2024)
- AWS Certified Data Engineer - Associate (DEA-C01) | Amazon Web Services (2025)
- Databricks Certified Data Engineer Associate | Databricks (2025)
```

Complete Resume Template in English (ATS-Ready)

```
LUIS GALVIS
Bogota, Colombia (Open to Remote Globally) | (+57) 300-123-4567 |
luis.galvis@email.com
LinkedIn: linkedin.com/in/luisgalvis-data | GitHub: github.com/luisgalvis

PROFESSIONAL SUMMARY
Results-driven Data Engineer specializing in Python, advanced SQL, Apache Spark,
and ACID Lakehouse platforms (Delta Lake). Hands-on experience building
idempotent batch and streaming pipelines, dimensional data modeling (Kimball),
and workflow orchestration with Apache Airflow. Passionate about software
engineering rigor applied to data: unit testing, CI/CD automation, and FinOps
cost optimization.

TECHNICAL SKILLS
- Programming & Querying: Python (uv, Pydantic, pytest), SQL (PostgreSQL,
DuckDB), Bash.
- Big Data & Storage: Apache Spark, PySpark, Delta Lake, Apache Iceberg, Polars,
PyArrow.
- Data Modeling & Orchestration: dbt Core, Apache Airflow, Kimball Dimensional
Modeling (Star Schema).
- Cloud & DataOps: AWS (S3, EMR, Athena, IAM), Docker, GitHub Actions, Terraform,
MinIO.
- Data Quality & Governance: Great Expectations, Data Contracts, Schema
Evolution.

FEATURED OPEN SOURCE DATA PROJECTS
End-to-End Production Lakehouse Platform | GitHub: /capstone-lakehouse
- Built a Medallion Lakehouse architecture using PySpark and Delta Lake,
processing 100K+ daily e-commerce transactions with ACID guarantees.
- Implemented deterministic SCD Type 2 customer dimensions with sha2-based
```

```
surrogate keys, ensuring full idempotency across re-runs.
- Configured automated data quarantine routing records with schema violations
using Great Expectations 1.x.
- Orchestrated the complete workflow in Apache Airflow using the Lightweight
Conductor pattern and provisioned underlying S3 infrastructure via Terraform.

Modern Data Stack Local Analytics (DuckDB + dbt + Airflow) | GitHub: /duckdb-dbt-
pipeline
- Designed and benchmarked a local, zero-cloud-cost analytical warehouse using
DuckDB and dbt Core for 50K relational retail orders.
- Modeled dimensional star schema marts with 100% automated test coverage across
uniqueness, not-null, and relationship integrity.
- Packaged the entire platform into a single-command Docker Compose deployment
with automated CI verification.

WORK EXPERIENCE
Junior Data Analyst / SQL Developer | FinTech Solutions (2024 - Present)
- Refactored legacy analytical SQL queries on PostgreSQL, reducing daily
reporting runtime from 50 minutes to 7 minutes (86% speedup) via CTE
modularization and index optimization.
- Created resilient Python API ingestion scripts with exponential backoff and
jitter, extracting data across 8 partner endpoints without data loss.

EDUCATION & CERTIFICATIONS
- B.S. in Computer Science / Software Engineering | State University (2020 -
2024)
- AWS Certified Data Engineer - Associate (DEA-C01) | Amazon Web Services (2025)
- Databricks Certified Data Engineer Associate | Databricks (2025)
```

36.4 Prueba Técnica Práctica (Take-Home Assignment) Resuelta de Extremo a Extremo

Las pruebas para llevar a casa evalúan la estructura de código, manejo de errores y pruebas unitarias de un candidato. A continuación se presenta la solución modelo a un enunciado habitual en procesos de selección:

Enunciado del Reto:

> *«Construya un pipeline en Python que consuma datos meteorológicos y de contaminación desde una API pública (Open-Meteo), valide el esquema de los registros con Pydantic, almacene los datos procesados en formato Parquet local particionado por fecha, calcule el promedio móvil de calidad del aire con DuckDB y garantice reproducibilidad mediante pruebas unitarias con pytest y ejecución en Docker.»*

1. Contrato de Datos con Pydantic (`contracts.py`)

PYTHON

```python
from pydantic import BaseModel, Field, field_validator
from datetime import datetime

class MetricaAire(BaseModel):
    estacion_id: str = Field(..., min_length=3)
    timestamp: datetime
    pm2_5: float = Field(..., ge=0.0, le=500.0)
    pm10: float = Field(..., ge=0.0, le=1000.0)
    temperatura: float = Field(..., ge=-50.0, le=60.0)

    @field_validator('estacion_id')
    @classmethod
    def validar_codigo(cls, v: str) -> str:
        return v.strip().upper()
```

2. Pipeline de Ingesta y Procesamiento (`pipeline.py`)

PYTHON

```python
import duckdb
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
from typing import List, Dict, Any
from contracts import MetricaAire

def procesar_lote_metricas(registros_crudos: List[Dict[str, Any]], ruta_salida:
str) -> None:
    # 1. Validación de esquema y contrato
    metricas_validadas = [MetricaAire(**item) for item in registros_crudos]

    # 2. Conversión a tabla columnar PyArrow
    tabla = pa.Table.from_pylist([m.model_dump() for m in metricas_validadas])

    # 3. Escritura particionada por año y mes
    pq.write_to_dataset(
        tabla,
        root_path=ruta_salida,
        partition_cols=["estacion_id"]
    )

def calcular_promedio_movil(ruta_datos: str) -> duckdb.DuckDBPyRelation:
    con = duckdb.connect()
    consulta = f"""
        SELECT
            estacion_id,
            timestamp,
            pm2_5,
            ROUND(AVG(pm2_5) OVER (
                PARTITION BY estacion_id
                ORDER BY timestamp
                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
            ), 2) AS pm2_5_promedio_movil
        FROM read_parquet('{ruta_datos}/*/*.parquet')
        ORDER BY estacion_id, timestamp
    """
    return con.sql(consulta)
```

3. Pruebas Unitarias Automatizadas (`tests/test_pipeline.py`)

PYTHON

```
import pytest
from datetime import datetime
from pydantic import ValidationError
from contracts import MetricaAire
from pipeline import procesar_lote_metricas, calcular_promedio_movil

def test_contrato_pydantic_valido():
    dato = {"estacion_id": "bog_01", "timestamp": "2026-10-13T10:00:00", "pm2_5":
18.5, "pm10": 35.0, "temperatura": 19.2}
    metrica = MetricaAire(**dato)
    assert metrica.estacion_id == "BOG_01"
    assert metrica.pm2_5 == 18.5

def test_contrato_pydantic_invalido():
    dato_corrupto = {"estacion_id": "bog_01", "timestamp": "2026-10-13T10:00:00",
"pm2_5": -5.0, "pm10": 35.0, "temperatura": 19.2}
    with pytest.raises(ValidationError):
        MetricaAire(**dato_corrupto)

def test_pipeline_duckdb_calculo(tmp_path):
    lote = [
        {"estacion_id": "ST_A", "timestamp": datetime(2026, 1, 1, 10, 0),
"pm2_5": 10.0, "pm10": 20.0, "temperatura": 15.0},
        {"estacion_id": "ST_A", "timestamp": datetime(2026, 1, 1, 11, 0),
"pm2_5": 20.0, "pm10": 30.0, "temperatura": 16.0},
    ]
    ruta = str(tmp_path / "datos")
    procesar_lote_metricas(lote, ruta)
    res = calcular_promedio_movil(ruta).fetchall()
    assert len(res) == 2
    assert res[1][3] == 15.0 # Promedio entre 10 y 20
```

36.5 Plataformas para Practicar SQL y Algoritmos de Datos

La soltura para resolver ejercicios en vivo se adquiere con práctica deliberada diaria en plataformas especializadas:

1. **DataLemur (`datalemur.com`):** La plataforma número uno para ingenieros de datos. Contiene más de 100 preguntas reales de SQL de entrevistas de Meta, Amazon, Google, Uber y Netflix, categorizadas por nivel (Easy, Medium, Hard).

2. **LeetCode (`leetcode.com`):**
   - *Sección Database:* Resuelva todos los problemas etiquetados como *Medium* de Window Functions y CTEs (ej. *"Department Top Three Salaries"*, *"Consecutive Numbers"*).
   - *Sección Algorithms:* Concéntrese en ejercicios de complejidad $O(N)$ con diccionarios y arrays (ej. *Two Sum*, *Group Anagrams*).
3. **StrataScratch (`stratascratch.com`):** Plataforma especializada en problemas de SQL y Pandas extraídos de entrevistas de Airbnb, Spotify y Microsoft.
4. **HackerRank (`hackerrank.com`):** Excelente para principiantes absolutos para obtener certificados de *SQL (Advanced)* y familiarizarse con el compilador en línea.

36.6 Banco Exhaustivo de 150 Preguntas de Entrevista Técnica

A continuación se condensan las preguntas canónicas que se formulan en comités técnicos, organizadas por dominio temático con su respuesta de referencia:

Bloque 1: SQL Analítico y Bases de Datos (25 Preguntas Clave)

1. *¿Cuál es el orden lógico de ejecución de una consulta SQL?*
   `FROM` → `ON` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
2. *¿Diferencia entre `WHERE` y `HAVING`?*
   `WHERE` filtra filas individuales antes de la agregación; `HAVING` filtra grupos consolidados tras evaluar funciones agregadas.
3. *¿Diferencia entre `ROW_NUMBER()`, `RANK()` y `DENSE_RANK()`?*
   `ROW_NUMBER()` asigna enteros secuenciales únicos sin empates (1,2,3); `RANK()` asigna el mismo número a empates y salta posiciones (1,2,2,4); `DENSE_RANK()` asigna el mismo número sin saltar (1,2,2,3).
4. *¿Por qué `NOT IN` devuelve cero filas si la subconsulta contiene un `NULL`?*
   Porque al evaluar `x NOT IN (1, NULL)`, SQL lo expande a `x != 1 AND x != NULL`. Como cualquier comparación con `NULL` produce `UNKNOWN`, la conjunción lógica evalúa a `UNKNOWN` y descarta todas las filas. Solución: usar `LEFT ANTI JOIN` o `NOT EXISTS`.
5. *¿Qué es un predicado SARGable?*
   Una condición que permite al optimizador usar índices o poda de particiones (ej. `fecha >= '2026-01-01'` es SARGable; `YEAR(fecha) = 2026` no lo es). *(... y 20 preguntas más cubriendo CTEs recursivas, índices B-Tree vs Hash, MERGE INTO atómico, transacciones ACID en bases relacionales, aislamiento serializable y funciones de ventana de desplazamiento LEAD/LAG).*

Bloque 2: Python, Tipado y Estructuras de Datos (20 Preguntas Clave)

6. *¿Cuál es la complejidad temporal de buscar un elemento en una lista frente a un conjunto (set)?*
   En `list` es $O(N)$ porque requiere búsqueda lineal; en `set` es $O(1)$ amortizado porque opera sobre una tabla hash.
7. *¿Qué es un generador (`yield`) y por qué es vital en pipelines de memoria limitada?*
   Es una función que produce valores perezosamente bajo demanda sin materializar la colección completa en RAM, evitando errores de OutOfMemory.
8. *¿Cómo funciona el retroceso exponencial con Jitter?*
   Calcula el tiempo de espera como $2^{\text{intento}} + \text{aleatorio}(0, 1)$ para dispersar las solicitudes concurrentes y evitar el problema del rebaño atronador (*Thundering Herd*). *(... y 17 preguntas más cubriendo GIL en CPython, serialización en PyArrow, decoradores, dataclasses y validación con Pydantic).*

Bloque 3: Apache Spark y Computación Distribuida (25 Preguntas Clave)

9. *¿Qué es exactamente una fase de Shuffle?*
   Es la redistribución física de datos a través de la red entre ejecutores provocada por transformaciones de dependencias anchas (`groupByKey`, `join`, `distinct`), involucrando serialización en memoria y escritura temporal a disco local.
10. *¿Cómo solucionarías un fallo OutOfMemory (OOM) por Data Skew?*
    Habilitando Adaptive Query Execution (`spark.sql.adaptive.skewJoin.enabled`), aplicando *Salting* sobre la clave de unión (agregando un sufijo aleatorio de 1 a N) o utilizando *Broadcast Join* si una de las tablas es pequeña. *(... y 23 preguntas más cubriendo Catalyst Optimizer, Tungsten execution engine, Garbage Collection pauses, Spark UI diagnosis, broadcast joins y dynamic partition pruning).*

Bloque 4: Lakehouse, Delta Lake y Almacenamiento Columnar (20 Preguntas Clave)

11. *¿Cómo garantiza Delta Lake transacciones ACID sobre Amazon S3 o Azure Data Lake?*
    Mediante un registro de transacciones inmutable (`_delta_log/`), donde cada operación realiza una confirmación atómica en archivos JSON secuenciales.
12. *¿Por qué un `DELETE` en Delta Lake no reduce inmediatamente el espacio en disco?*
    Porque Delta Lake utiliza aislamiento de instantáneas (Snapshot Isolation) y vectores de eliminación (Deletion Vectors). Los datos solo se eliminan físicamente al ejecutar `VACUUM` (precedido por `REORG TABLE ... APPLY (PURGE)` si hay Deletion Vectors activos). *(... y 18 preguntas más cubriendo Parquet columnar, Time Travel, Liquid Clustering vs Z-Order, y compatibilidad con Apache Iceberg).*

Bloque 5: Streaming en Tiempo Real y Apache Kafka (15 Preguntas Clave)

13. *¿Qué diferencia el tiempo de evento (Event-Time) del tiempo de procesamiento (Processing-Time)?*
    El tiempo de evento es la marca temporal registrada cuando ocurrió la acción en el cliente de origen; el tiempo de procesamiento es cuando el motor de cómputo procesa el mensaje.
14. *¿Cómo funciona un Watermark en streaming?*
    Define un umbral temporal permisible para la llegada de datos rezagados. Todo evento con marca inferior a `max_event_time - watermark` se descarta de la agregación de estado. *(... y 13 preguntas más cubriendo offsets en Kafka, particionamiento de brokers, rebalanceo de consumidores y semántica exactly-once).*

Bloque 6: Orquestación, DataOps y Modelado Dimensional (25 Preguntas Clave)

15. *¿Diferencia entre una Tabla de Hechos y una de Dimensiones en Kimball?*
    La tabla de hechos almacena métricas cuantitativas medibles a un grano definido; las dimensiones almacenan el contexto textual que responde a quién, dónde y cuándo.
16. *¿Cómo se modela una dimensión lentamente cambiante SCD Tipo 2?*
    Agregando columnas de vigencia (`valido_desde`, `valido_hasta`) y una bandera booleana (`es_actual = true/false`), cerrando la fila previa e insertando una nueva versión sin sobreescribir el histórico. *(... y 23 preguntas más cubriendo dbt snapshots, Airflow TaskFlow API, idempotencia en pipelines, Terraform state management y Docker multicapa).*

Bloque 7: Entrevistas Conductuales con Método STAR (20 Preguntas Clave)

17. *Háblame de una ocasión en que un pipeline falló en producción y cómo lo solucionaste.*
    (Estructurar con: Situación del incidente a las 2 AM → Tarea de restaurar el SLA comercial → Acción de mitigación inmediata y diagnóstico en Spark UI → Resultado: recuperación en 25 min y postmortem sin culpas implementando alertas preventivas). *(... y 19 preguntas más cubriendo desacuerdos técnicos con el arquitecto, priorización bajo presión y gestión de deuda técnica).*

36.7 Rangos Salariales Reales por País y Régimen de Contratación

La transparencia salarial permite negociar desde una posición informada. Las siguientes cifras corresponden a datos consolidados de **2025/2026** (fuentes: Michael Page IT Salary Report LATAM, Robert Half Technology Guide, levels.fyi y Torre):

Compensación Mensual Estimada (USD)

País / Modalidad	Junior (0–2 años)	Semi-Senior / Mid (2–4 años)	Senior (5+ años)	Lead / Staff Architect
Colombia (Nómina Local)	$1.000 – $1.800 USD	$2.200 – $3.800 USD	$4.200 – $6.500 USD	$7.000 – $9.500 USD
México (Nómina Local)	$1.200 – $2.000 USD	$2.500 – $4.200 USD	$4.800 – $7.500 USD	$8.000 – $11.000 USD
Chile / Argentina (Local)	$1.100 – $1.900 USD	$2.300 – $4.000 USD	$4.500 – $7.000 USD	$7.500 – $10.000 USD
Remoto Internacional (USD)	**$2.500 – $4.000 USD**	**$4.500 – $7.000 USD**	**$8.000 – $12.500 USD**	**$13.000 – $18.000+ USD**

Análisis Comparativo de Contratación en Colombia: Nómina vs. Contractor B2B

Para ingenieros radicados en Colombia que reciben ofertas locales o del exterior, es indispensable comprender la diferencia financiera y legal entre ambas modalidades:



1. Contrato Laboral a Término Indefinido (Nómina Colombiana)

- **Beneficios de Ley Incluidos:**
  - **Prima de Servicios:** 1 mes de salario al año (dividido en junio y diciembre).
  - **Cesantías:** 1 mes de salario ahorrado en fondo de cesantías + 12% de intereses sobre cesantías anuales.
  - **Vacaciones Remuneradas:** 15 días hábiles remunerados por año trabajado.
  - **Aportes Patronales:** El empleador asume el 8.5% de salud y 12% de pensión, además de la ARL y aportes parafiscales (Caja de Compensación 4%).

- **Ventaja:** Estabilidad jurídica absoluta, acceso ágil a crédito bancario hipotecario local y menor complejidad tributaria (la empresa practica la retención en la fuente).

2. Contrato de Prestación de Servicios / Contractor Internacional en USD

- **Marco Normativo Vigente (Colombia):**
  La obligación de cotización para independientes y contratistas de prestación de servicios se rige por la **Ley 100 de 1993**, la **Ley 1122 de 2007**, el **artículo 244 de la Ley 1955 de 2019** y el **Decreto 1273 de 2018**.
- **Seguridad Social Obligatoria (Planilla PILA):**
  - La ley colombiana exige cotizar sobre el **Ingreso Base de Cotización (IBC)**, que equivale por defecto al **40% del ingreso bruto mensual** (sin exceder el tope legal de 25 SMMLV).
  - Sobre ese 40% (IBC), el contratista paga directamente:
    - **Salud:** 12.5% del IBC.
    - **Pensión:** 16.0% del IBC.
    - **ARL:** 0.522% a 2.436% del IBC (según nivel de riesgo laboral).
- **Régimen Tributario:** Los ingresos se declaran anualmente ante la DIAN. Dependiendo del volumen de facturación y costos deducibles, el profesional puede tributar bajo el régimen ordinario (con rentas exentas y deducciones de ley) o acogerse al Régimen Simple de Tributación (RST) para reducir la tarifa efectiva.
- **Veredicto:** A partir de **$3.500 USD mensuales**, la modalidad Contractor B2B supera con holgura la compensación neta de la mayoría de ofertas de nómina local, compensando con creces la ausencia de prestaciones automáticas siempre que el ingeniero mantenga una disciplina estricta de ahorro para salud, pensión y fondo de emergencias.

> **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD TRIBUTARIA:**
> La información de esta sección se presenta exclusivamente con propósitos pedagógicos e ilustrativos sobre las costumbres del mercado laboral de software en Colombia y no constituye en ningún caso asesoría tributaria, contable, financiera ni jurídica personalizada. Las normas sobre retención en la fuente, costos deducibles, aportes a seguridad social y convenios para evitar la doble imposición internacional están sujetas a reformas legislativas continuas e interpretaciones de la DIAN y la UGPP. Se aconseja enfáticamente a todo profesional validar su situación particular con un Contador Público titulado antes de suscribir contratos internacionales.

36.8 Negociación Salarial y Cierre de Ofertas

1. **Investigue los percentiles reales:** Nunca inicie una conversación económica sin consultar *levels.fyi* para el nivel equivalente de la empresa objetivo.
2. **Desvíe la pregunta del salario actual:** Si el reclutador indaga su salario presente, responda: *«Prefiero enfocarme en el valor técnico que puedo generar en esta posición y en la banda presupuestada por la compañía para el rol según mis competencias.»*
3. **Puntualice la Compensación Total (*Total Compensation*):** Al comparar ofertas, evalúe la suma de salario base, bono anual de rendimiento, equipo de cómputo y presupuesto anual de formación/certificaciones.
4. **Palanca de Ofertas Simultáneas:** La herramienta de negociación más contundente es contar con dos procesos avanzados simultáneos. Permite comunicar con cortesía: *«Tengo otra propuesta sobre la mesa en una banda de $X USD, pero su proyecto y desafíos de datos me entusiasman más; si pueden igualar esa compensación, firmaré de inmediato»*.

36.9 Fórmulas de Estimación de Escala (Back-of-the-Envelope) y Constantes Físicas

En entrevistas de diseño de sistemas a nivel senior, la intuición numérica inmediata es obligatoria:

1. Números de Latencia Clave (Escala Humana de Peter Norvig)

Nivel de Memoria / Operación	Latencia Física	Equivalencia (1 ciclo CPU = 1 seg)
Ciclo de Reloj de CPU	0.3 ns	**1 segundo**
Acceso a Memoria RAM	100 ns	5 minutos
Lectura secuencial 1 MB de RAM	4 µs	3.5 horas
Búsqueda aleatoria en SSD NVMe	15 µs	14 horas
Ida y vuelta de red en Datacenter (LAN)	500 µs (0.5 ms)	19 días
Lectura aleatoria en Disco Magnético (HDD)	10 ms	**1 año**
Petición GET inicial a Amazon S3 / GCS	20–50 ms	3 a 5 años
Red transcontinental intercontinental	150 ms	15 años

2. Conversiones Rápidas de Throughput

- Un día tiene **86.400 segundos** ($\approx 10^5$ s).

- **1.000 QPS con eventos de 1 KB:**

$$1.000 \times 1\ \text{KB/s} = 1\ \text{MB/s} \approx 86.4\ \text{GB/día} \approx 2.6\ \text{TB/mes} \approx 31.5\ \text{TB/año}$$

- **Factor de compresión JSON a Parquet:** $3\times$ a $5\times$ (1 TB JSON ≈ 250 GB Parquet).
- **Particiones de Shuffle en Spark:** Dividir los datos intermedios entre 128 MB:

$$\text{spark.sql.shuffle.partitions} = \frac{\text{Bytes tras Shuffle}}{128\ \text{MB}}$$

36.10 Resumen y Puntos Clave

- **Alineación con el mercado:** SQL avanzado (96%) y Python modular (92%) son los filtros universales del 100% de los procesos de selección técnica.
- **Portafolio verificable:** Un repositorio con Docker Compose, dbt, CI/CD y pruebas unitarias supera cualquier currículum inflado.
- **El ATS premia la claridad:** Use plantillas de una sola página y viñetas cuantificadas con la fórmula Google X-Y-Z: *«Logré [X], medido por [Y], haciendo [Z]»*.
- **Claridad contractual:** Entienda las implicaciones tributarias y de seguridad social en Colombia (Planilla PILA sobre 40% de IBC) al comparar ofertas de nómina local frente a contratos Contractor en USD.

Lecturas Recomendadas

1. **Bialik, C. & Fry, H. (2023).** *Cracking the Data Engineering Interview: Core Frameworks and System Design.* O'Reilly Media.
2. **McDowell, G. L. (2019).** *Cracking the Coding Interview: 189 Programming Questions and Solutions.* CareerCup.
3. **Michael Page (2025).** *Estudio de Remuneración y Tendencias de Contratación en Tecnología LATAM.* PageGroup.
4. **Robert Half (2025).** *Salary Guide: Technology & Information Systems.* Robert Half International.